Torneos Amateur BA

Documentación por módulo

Requerimientos · Casos de uso · Diagramas de secuencia · Diagramas de clase · Pantallas (UI)

Metodología de Sistemas · Versión del 10/11/2026

Índice

Módulo A · Cuenta y acceso

Parte 1 · Requerimientos

Parte 2 · Modelo de dominio y casos de uso, con sus diagramas y su pantalla

- Modelo de dominio del módulo (diagrama de clases)
- UC-A01 · Crear cuenta
- UC-A02 · Iniciar sesión
- UC-A03 · Cerrar sesión
- UC-A04 · Recuperar contraseña
- UC-A05 · Editar perfil
- UC-A06 · Ver avisos (campanita)

Parte 3 · Anexos y trazabilidad

Módulo B · Catálogo de torneos

Parte 1 · Requerimientos

Parte 2 · Modelo de dominio y casos de uso, con sus diagramas y su pantalla

- Modelo de dominio del módulo (diagrama de clases)
- UC-B01 · Ver el catálogo de torneos
- UC-B02 · Filtrar torneos y compartir la búsqueda
- UC-B03 · Ver la ficha pública de un torneo
- UC-B04 · Contactar al organizador
- UC-B05 · Ver la página pública del organizador
- UC-B06 · Reportar un torneo

Parte 3 · Anexos y trazabilidad

Módulo C · Equipos

Parte 1 · Requerimientos

Parte 2 · Modelo de dominio y casos de uso, con sus diagramas y su pantalla

- Modelo de dominio del módulo (diagrama de clases)
- UC-C01 · Crear equipo
- UC-C02 · Invitar jugadores al equipo
- UC-C03 · Unirse a un equipo con el link
- UC-C04 · Ver la página pública del equipo
- UC-C05 · Editar equipo
- UC-C06 · Pasar la capitanía
- UC-C07 · Sacar a un jugador del plantel
- UC-C08 · Salir del equipo
- UC-C09 · Disolver equipo
- UC-C10 · Reportar un equipo

Parte 3 · Anexos y trazabilidad

Módulo D · Partidos

Parte 1 · Requerimientos

Parte 2 · Modelo de dominio y casos de uso, con sus diagramas y su pantalla

- Modelo de dominio del módulo (diagrama de clases)
- UC-D01 · Publicar una búsqueda de rival
- UC-D02 · Editar o cancelar la búsqueda
- UC-D03 · Ver rivales compatibles
- UC-D04 · Proponer un partido
- UC-D05 · Cancelar una propuesta enviada
- UC-D06 · Aceptar o rechazar una propuesta
- UC-D07 · Coordinar cancha y hora
- UC-D08 · Confirmar asistencia
- UC-D09 · Tirar el partido
- UC-D10 · Caída de común acuerdo
- UC-D11 · Cargar el resultado
- UC-D12 · Reportar que el rival no se presentó
- UC-D13 · Calificar al rival
- UC-D14 · Ver mis partidos

Parte 3 · Anexos y trazabilidad

Módulo E · Organizador

Parte 1 · Requerimientos

Parte 2 · Modelo de dominio y casos de uso, con sus diagramas y su pantalla

- Modelo de dominio del módulo (diagrama de clases)
- UC-E01 · Publicar un torneo
- UC-E02 · Ver mis torneos (Panel de organizador)
- UC-E03 · Editar un torneo

UC-E04 · Verificar que el torneo sigue vigente
UC-E05 · Dar de baja un torneo
UC-E06 · Cargar una inscripción
UC-E07 · Dar de baja una inscripción

Parte 3 · Anexos y trazabilidad

Módulo F · Administración

Parte 1 · Requerimientos

Parte 2 · Modelo de dominio y casos de uso, con sus diagramas y su pantalla

Modelo de dominio del módulo (diagrama de clases)
UC-F01 · Acceder al Panel de administración
UC-F02 · Consultar el tablero
UC-F03 · Resolver un reporte de torneo
UC-F04 · Resolver un reporte de equipo
UC-F05 · Suspender una cuenta
UC-F06 · Buscar usuarios

Parte 3 · Anexos y trazabilidad

Módulo A · Cuenta y acceso

Primero los requerimientos del módulo completo. Después, el modelo de dominio y, para cada caso de uso, su diagrama de secuencia, su diagrama de clases y su pantalla. Al final, los anexos y la trazabilidad.

Parte 1 · Requerimientos

Versión del **04/11/2026**. Es la primera parte del módulo: sigue la sección de casos de uso, donde cada caso lleva debajo su diagrama de secuencia y su pantalla. El README tiene la matriz que los une.

Fuentes, en orden de autoridad: decisiones-casos-de-uso.md (sección «Cuenta y acceso»), casos-de-uso.md (módulo A, 03/11/2026), el glosario CONTEXT.md y el código y las migraciones del repo al día de hoy.

1. Alcance

El módulo permite **crear una cuenta, entrar y salir**, volver a lo que la persona estaba haciendo después de entrar (`next`), **editar el perfil** (nombre, foto, Zona y WhatsApp) y **ver los Avisos** en la campanita. Cada persona tiene **una sola cuenta**, con la que puede ser Jugador, Capitán y Organizador a la vez.

Fuera del módulo (tienen su propio documento):

Tema	Dónde está
Acceso del Administrador (`/admin/login`, el rol, el panel)	Módulo F — Administración
Quién publica torneos, el Panel de organizador	Módulo E — Organizador
Crear equipos, ser Capitán	Módulo C — Equipos (usa el WhatsApp del perfil)
Qué genera cada Aviso (propuestas, partidos, bajas de torneo)	Módulos D y E (el módulo A solo los **muestra**)
Bloqueo por intentos fallidos de login y confirmación de email	Los cubre Supabase (decisión del 27/09/2026)

Límite de producto: la app no manda emails ni notificaciones push. El único mail que mandaría es el de recuperar la contraseña, que todavía no existe. Los Avisos viven solo en la campanita.

2. Actores

Actor	Quién es	Qué hace en este módulo
Invitado	Persona sin sesión	Crea una cuenta, inicia sesión, pide recuperar la contraseña.
Usuario	Persona con sesión	Cierra sesión, edita su perfil, ve sus Avisos.
Sistema	La base de datos y la app	Crea el perfil con la cuenta, controla el acceso a las rutas y escribe los Avisos.

El **Administrador** es un rol distinto del Usuario que se asigna a mano en la base; el módulo A garantiza que **nunca se pueda escribir desde la app** (RF-A15).

3. Requerimientos funcionales

Estado: `Implementado` está en `main`. `Pendiente` es alcance vigente sin programar. `Parcial` tiene una parte sin cumplir, que se indica.

Crear cuenta e iniciar sesión

ID	Requerimiento	Casos de uso	Estado
RF-A01	Crear una cuenta con **email**, **contraseña** y **nombre**. El nombre es público (lo ven todos en la página del organizador), así que **no se deriva del email**: tiene de 2 a 50 caracteres.	UC-A01	Implementado
RF-A02	Al crear la cuenta se crean **en el mismo acto** el perfil (con el nombre) y la fila de contacto, vacía. Se entra directo, **sin confirmar el email**.	UC-A01	Implementado
RF-A03	Una sola cuenta por persona: no se puede crear una segunda con un email ya registrado.	UC-A01	Implementado (lo garantiza Supabase Auth)
RF-A04	Iniciar sesión con email y contraseña. Con credenciales inválidas, mostrar el error y seguir sin sesión.	UC-A02	Implementado
RF-A05	Las rutas que exigen sesión (`/crear-torneo` , `/mis-torneos` , `/perfil` , `/crear-equipo` , `/mis-equipos` , `/partidos`) redirigen a «Iniciar sesión» conservando el destino. Cada página y cada acción vuelve a verificar la sesión: el control del acceso a la ruta es una primera capa, no la única.	UC-A02	Implementado
RF-A06	Después de entrar, volver al **destino pendiente** (`next`); sin destino, al catálogo. **Solo se aceptan rutas internas**: `//sitio.com` y `/\sitio.com` empiezan con `/` pero son externas, y se rechazan.	UC-A01, UC-A02	Implementado
RF-A07	Con una sesión ya abierta, `/login` no muestra el formulario: lleva al destino.	UC-A02	Implementado
RF-A08	El **modo** de la pantalla de acceso (Iniciar sesión o Crear cuenta) vive en la URL, así que cada pantalla es un link. Llegar desde «¿Organizás torneos? Publicalo» abre **Crear cuenta** con el aviso «Para publicar un torneo necesitás una cuenta. Es gratis.» y conserva el destino al panel; con cualquier otro destino se abre Iniciar sesión, sin aviso. El link para pasar de una pantalla a la otra conserva el destino.	UC-A01	Implementado
RF-A09	**Cerrar sesión** cierra **solo la sesión del dispositivo actual** y lleva al catálogo.	UC-A03	Implementado
RF-A10	**Recuperar la contraseña**: pedir un enlace por email, abrirlo y definir una contraseña nueva. Sería el único email que manda la app.	UC-A04	**Pendiente**, a propósito (decisión del 27/09/2026: primero producto, después infraestructura de auth)

Perfil

ID	Requerimiento	Casos de uso	Estado
RF-A11	Editar el perfil: **nombre** (2 a 50 caracteres), **foto**, **Zona** (opcional, de una lista cerrada) y **WhatsApp** (opcional). Cada persona edita solo el suyo.	UC-A05	Implementado
RF-A12	**Foto** JPG, PNG o WebP de hasta 2 MB. Cada foto nueva **reemplaza** a la anterior; no hay galería. La foto anterior se retira **después** de guardar la nueva; si el guardado falla, se conserva la anterior y se retira la subida nueva.	UC-A05	Implementado

ID	Requerimiento	Casos de uso	Estado
RF-A13	El **WhatsApp** se guarda **solo con dígitos**, en formato argentino de celular: `549` + código de área sin 0 + número sin 15 (13 dígitos), que es lo que pide `wa.me`. La persona lo escribe sin el `+54 9`. El «15» se rechaza.	UC-A05	Implementado
RF-A14	El WhatsApp es **privado**: no está en la página pública ni en el perfil público. Cada persona ve solo el suyo; el capitán rival lo ve únicamente con un partido acordado.	UC-A05	Implementado
RF-A15	El **rol de administrador no se edita desde la app**: la persona edita nombre, foto, Zona y WhatsApp, nunca el rol.	UC-A05	Implementado
RF-A16	El WhatsApp es **obligatorio para crear un equipo, recibir la capitanía y buscar rival**. La app lo exige en cada una de esas acciones.	UC-A05 (dependencia)	Implementado en los módulos C y D
RF-A17	El **Capitán de un equipo no disuelto no puede dejar el WhatsApp vacío** (sí cambiarlo): el rival lo necesita para coordinar.	UC-A05	**Pendiente**: la regla está definida pero hoy ni la acción ni la base la hacen cumplir
RF-A18	Si la persona llegó al perfil desde otra acción (por ejemplo, Crear equipo), el perfil le dice «Para seguir, cargá tu WhatsApp» y, al guardar, la devuelve al destino.	UC-A05	Implementado

Avisos (campanita)

ID	Requerimiento	Casos de uso	Estado
RF-A19	Con sesión, la barra muestra la **campanita** con el número rojo de **Avisos no leídos**. Al abrirla se listan los **últimos 30**, del más nuevo al más viejo, cada uno con texto, link y fecha.	UC-A06	Implementado
RF-A20	**Abrir la campanita marca como leídos** los Avisos que se ven (no «todos»: un Aviso que llega mientras está abierta sigue contando como no leído). Mientras está abierta, los que estaban sin leer siguen resaltados.	UC-A06	Implementado
RF-A21	El Aviso **no tiene botones**: es un link a donde está el estado actual (Partidos, el Equipo). La acción se hace en esa pantalla, que valida las reglas. El Aviso es un puntero, no la fuente de verdad.	UC-A06	Implementado
RF-A22	El **texto se arma en el momento del hecho** y no cambia aunque el rival cambie de nombre: se guarda el hecho (`kind`) y la foto de sus datos (`params`). Todo Aviso que abre un plazo dice cuándo vence y qué pasa si no se responde.	UC-A06	Implementado
RF-A23	Los Avisos **no se borran ni se archivan**. No existe email ni push.	UC-A06	Implementado
RF-A24	Cada persona **lee solo sus Avisos**. Los escribe el sistema desde las funciones de cada acción, nunca el cliente; y nunca le llegan a quien hizo la acción.	UC-A06	Implementado

Barra superior

ID	Requerimiento	Casos de uso	Estado
RF-A25	**Sin sesión:** logo, «Iniciar sesión» (texto) y «Crear cuenta» (único botón destacado). **Con sesión:** logo, la campanita y el menú de usuario con Mi perfil, Mis equipos, Partidos, Mis torneos, «Administración» (solo si es Administrador) y Cerrar sesión. Ningún botón destacado permanente.	UC-A01 a UC-A03, UC-A05, UC-A06	Implementado

4. Requerimientos no funcionales

ID	Requisito	Evidencia
RNF-A01	**Control de acceso en la base (RLS).** `profiles` se lee en público y cada persona edita solo el suyo, y solo las columnas `display_name`, `avatar_url` y `zone_id`. `profile_contacts` lo lee y edita solo su dueño, y solo la columna `whatsapp`. `admins` no tiene policies: nadie la lee ni la escribe por la API.	`20260928000002_profiles.sql`
RNF-A02	**El WhatsApp no sale en ninguna lectura pública**, porque RLS filtra filas y no columnas: por eso vive en una tabla aparte.	`profile_contacts`; `organizadores/[id]/page.tsx`
RNF-A03	**Perfil y contacto nacen con la cuenta**, en la misma transacción del alta.	Trigger `on_auth_user_created` y función `handle_new_user` (`security definer`)
RNF-A04	**El destino `next` solo puede ser una ruta interna.** Se valida en la pantalla y de nuevo en la acción, porque el campo oculto se puede editar.	`safe-next-path.ts` y `safe-next-path.test.ts`
RNF-A05	**Sesión por cookies y control del acceso en dos capas.** El middleware renueva la sesión y redirige las rutas protegidas; cada página y acción la verifica de nuevo.	`src/proxy.ts`
RNF-A06	**Al cliente solo llega la `anon key`.** La `service_role key` nunca sale de un contexto de servidor controlado.	`src/utils/supabase/`
RNF-A07	**Las fotos solo se sirven desde el bucket propio** y el servidor solo borra archivos de la carpeta del propio usuario, sin confiar en la URL ni en la extensión que vengan del formulario. El bucket limita tipo y tamaño (2 MB).	`profile-avatar.ts` (`avatarSrc`, `avatarObjectPath`), `20261013000002_profile_avatars.sql`, `profile_avatars_test.sql`
RNF-A08	**Un cambio de foto no pierde la actual.** Dos pestañas no pueden reemplazar la misma foto a la vez: la segunda retira su subida y pide reintentar.	`saveProfile` (`utils/supabase/profile.ts`)
RNF-A09	**Los Avisos los escribe solo el sistema.** `notices` no tiene privilegios de escritura para la API; `notify` no es ejecutable por el cliente; `mark_notices_read` solo toca avisos propios y un id ajeno se ignora sin error, para no confirmar que existe.	`20261028000001_notices.sql`, `notices_test.sql`
RNF-A10	**Validación pura y verificable.**	`safeNextPath`, `loginScreen`, `normalizeWhatsapp`, `parseAvatar` y el texto de los Avisos, con sus `*.test.ts` (`pnpm test`)
RNF-A11	**Formularios accesibles.** Cada campo tiene su etiqueta y su `autoComplete` (`new-password` al crear la cuenta, `current-password` al entrar, para el gestor de contraseñas); los errores se anuncian (`role="alert"`); la campanita y el menú declaran `aria-expanded` y se cierran con Escape.	`login/page.tsx`, `NoticeBell.tsx`, `UserMenu.tsx`
RNF-A12	**Delegado a Supabase:** bloqueo por intentos fallidos de login, confirmación de email y política de contraseñas.	Decisión del 27/09/2026

5. Reglas de referencia

No son requerimientos nuevos: son las reglas que los de arriba concretan. El detalle está en decisiones-casos-de-uso.md, secciones «Cuenta y acceso» y «Avisos».

- **No hay «vista de jugador» ni «vista de organizador».** `/` es el catálogo para cualquiera; la app se navega de una sola forma (decidido el 07/10/2026).

- **Una sola convención para volver:** un link arriba a la izquierda con el nombre del destino. Nunca dos formas de volver en la misma pantalla.
- **Quién ve un Aviso:** los que piden una acción van solo al Capitán; los informativos, a todo el Plantel del momento del hecho. Quien está en los dos planteles recibe uno solo. El nuevo Capitán no hereda los Avisos del anterior.
- **Términos:** «Aviso» y «campanita» (nunca «notificación»); «Zona» (nunca «barrio» ni «localidad»); «Capitán» (nunca «dueño del equipo»).

Parte 2 · Modelo de dominio y casos de uso, con sus diagramas y su pantalla

Versión del **04/11/2026**. Detalla los casos de uso del módulo A de casos-de-uso.md (UC-A01 a UC-A06) con lo que hace falta para la documentación completa: los requerimientos que cubre cada uno (01-requerimientos.md), la pantalla que lo representa (03-pantallas.md) y su diagrama de secuencia (04-diagramas-secuencia).

Cómo leerlo. Cada caso expresa una meta y tiene **un actor iniciador**. **Estado** dice si está en `main`. Los mensajes entre comillas son los que ve la persona, tal como están en el código.

Mapa de casos de uso

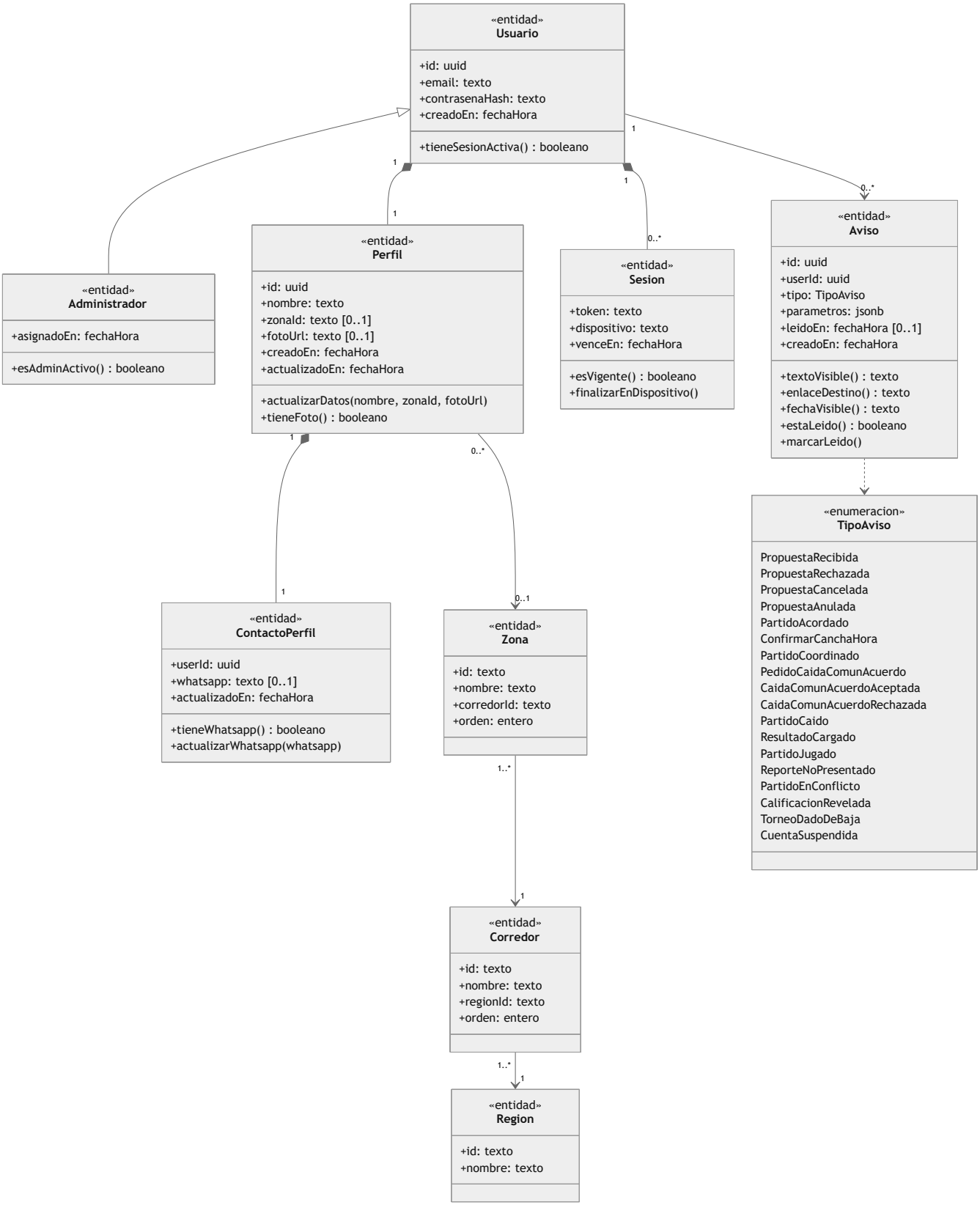
Caso	Meta del actor	Actor	Estado	ID en el PDF original
UC-A01	Crear cuenta	Invitado	Implementado	UC-01
UC-A02	Iniciar sesión	Invitado	Implementado	UC-02 y UC-05 (volver a la acción pedida)
UC-A03	Cerrar sesión	Usuario	Implementado	UC-03
UC-A04	Recuperar contraseña	Invitado	**Pendiente**	UC-04
UC-A05	Editar perfil	Usuario	**Parcial** (falta una regla)	UC-07
UC-A06	Ver avisos (campanita)	Usuario	Implementado	— (caso nuevo)

«Volver a la acción pedida después de iniciar sesión» (UC-05 del PDF) no es un caso aparte: es una regla de UC-A01 y UC-A02 (el destino `next`). El UC-06 del PDF, la elección de perfil, se retiró. La campanita (UC-A06) no existía en el PDF original.

Qué no es un caso de uso acá: ver el menú de usuario o navegar la barra no toma ninguna decisión. Va en pantallas.

Diagrama de clases · **Modelo de dominio del módulo**

Vista estructural de las entidades, relaciones de herencia/composición y enumeraciones que comparten los casos de uso `UC-A01` a `UC-A06`.



- **Privacidad por tabla (RNF-A01, RNF-A02):** `Perfil` es de lectura pública (`nombre`, `fotoUrl`, `zona`), mientras que `ContactoPerfil` (`whatsapp`) vive en una tabla separada porque RLS filtra **filas** y no columnas. Así ninguna lectura pública del perfil expone el WhatsApp.
- **Una sola cuenta y una sola navegación:** no existen cuentas separadas ni «vista de jugador / vista de organizador» (se retiró el antiguo UC-06).
- **Rol de Administrador (RF-A15):** se inserta manualmente en `public.admins` y nunca se puede escribir desde la app.
- **Avisos inmutables (RF-A22, RNF-A09):** guardan el hecho (`tipo`) y la foto de los datos del momento (`parametros`), y `textoVisible()` arma el mensaje al mostrarlo sin cambiar aunque el rival cambie de nombre.

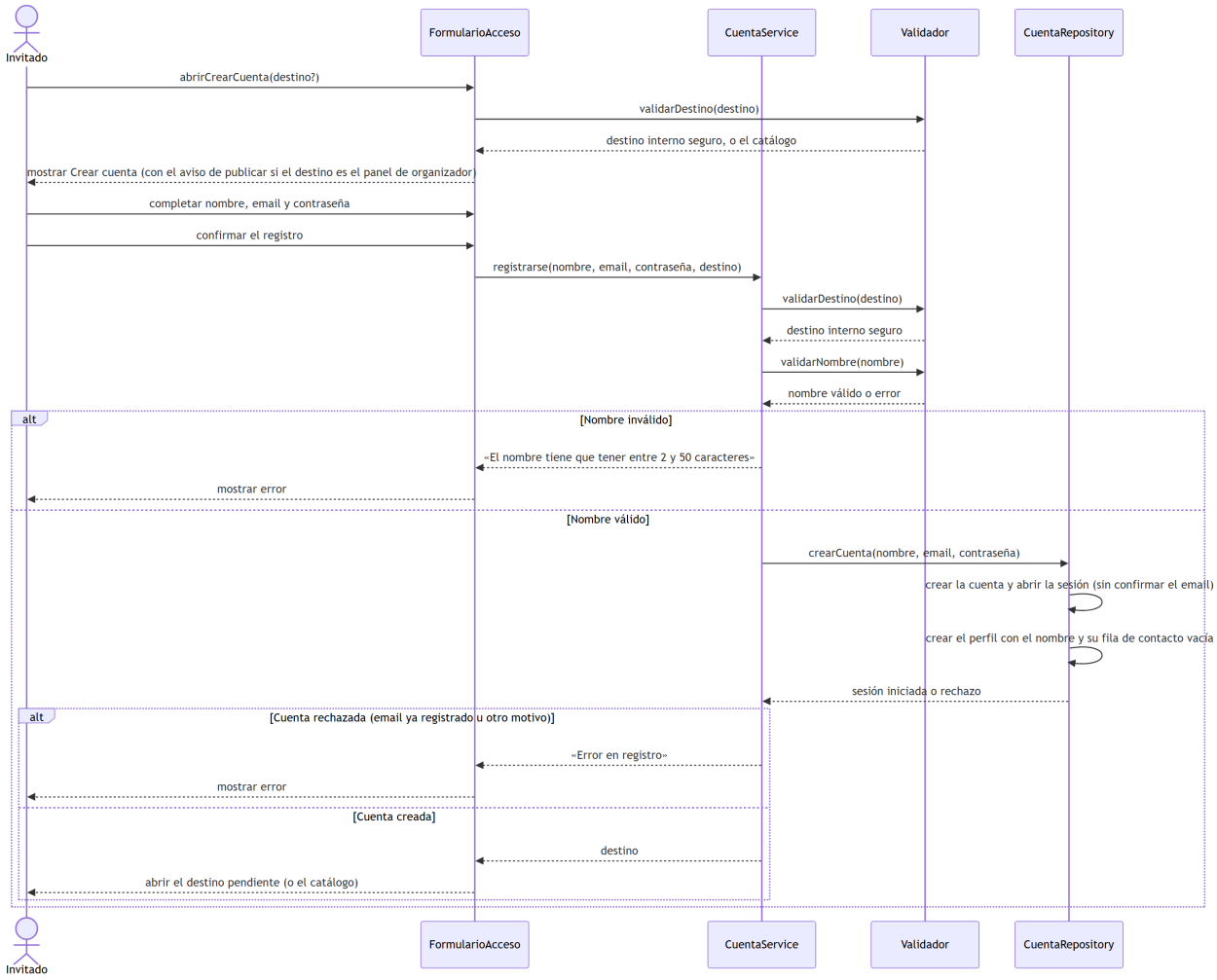
UC-A01 · Crear cuenta

- **Actor:** Invitado.
- **Requerimientos:** RF-A01 a RF-A03, RF-A06, RF-A08, RF-A25. **Pantalla:** P2 · Crear cuenta (`/login?modo=registro`). **Diagrama:** S01.
- **Precondiciones:** ninguna.
- **Postcondición:** existe el perfil con nombre; la persona está dentro.
- **Estado:** Implementado (`src/app/login` , función `signup` , trigger `handle_new_user`).

Diagrama de secuencia · **S01 · UC-A01 — Crear cuenta**

Caso de uso: UC-A01 · Pantalla: P2 · Requerimientos: RF-A01 a RF-A03, RF-A06, RF-A08

El formulario tiene solo campos de texto (nombre, email y contraseña): no hay desplegables ni listas que cargar. Lo que la pantalla sí resuelve antes de mostrarse es el **destino pendiente** (`next`).



Cómo se lee

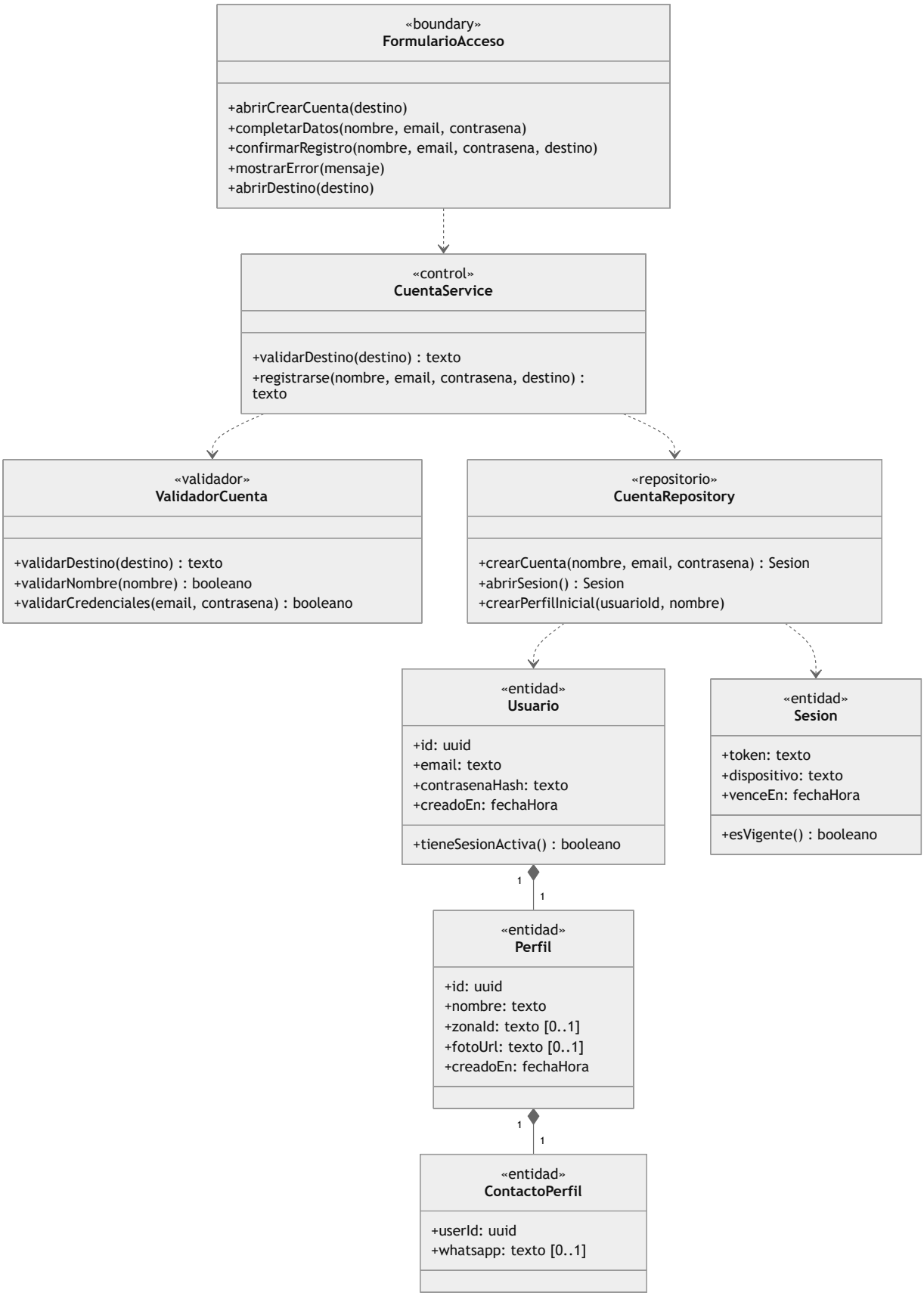
- **El destino se valida dos veces:** al abrir la pantalla y otra vez al enviar. El campo oculto `next` se puede editar, así que la acción no confía en lo que la pantalla ya revisó. Solo se aceptan rutas internas: `//sitio.com` empieza con `/` pero es externa, y se rechaza (se vuelve al catálogo).
- **Qué pantalla se abre** lo decide el destino: con `next` al panel de organizador y sin modo en la URL, se abre Crear cuenta con el aviso «Para publicar un torneo necesitás una cuenta. Es gratis.».
- **El nombre se valida antes de crear la cuenta**, para no depender del error genérico que daría la base («Database error saving new user») si el nombre no cumple la regla de 2 a 50 caracteres. El nombre viaja con la cuenta y el trigger lo usa para crear el perfil.
- **Cuenta, sesión, perfil y contacto nacen juntos:** el alta de la cuenta dispara en la base la creación del perfil y de la fila de contacto, en la misma transacción (`handle_new_user`). Nunca queda una cuenta sin perfil.
- **Se entra directo**, sin confirmar el email. Un email ya registrado y una contraseña que Supabase no acepta dan el mismo mensaje genérico, «Error en registro».
- Si ya hay una sesión abierta, abrir `/login` lleva directo al destino y el diagrama no empieza.

Correspondencia en el código

`signup` (`src/app/login/actions.ts`): `safeNextPath` → regla del nombre → `supabase.auth.signUp` con `display_name` en los metadatos → redirige al destino. El perfil lo crea el trigger `on_auth_user_created` (`20260928000002_profiles.sql`).

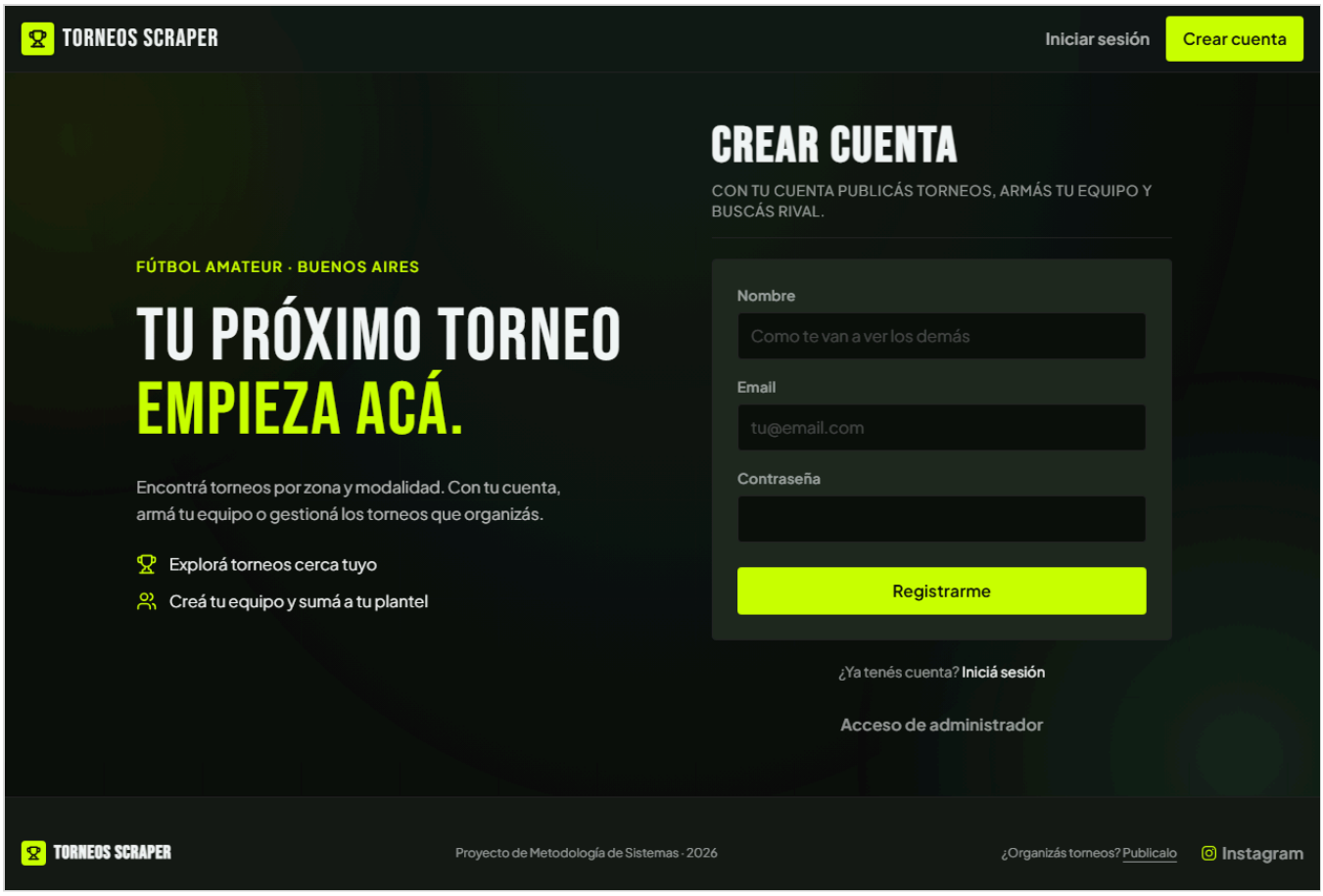
Diagrama de clases · **UC-A01 · Crear cuenta**

Caso de uso: UC-A01 · **Pantalla:** P2 · Crear cuenta (`/login?modo=registro`) · **Secuencia:** S01



> Pantalla · **P2 · Crear cuenta**

Ruta: `/login?modo=registro` · **Caso de uso:** UC-A01 · **Requerimientos:** RF-A01 a RF-A03, RF-A06, RF-A08 · **Diagrama:** S01



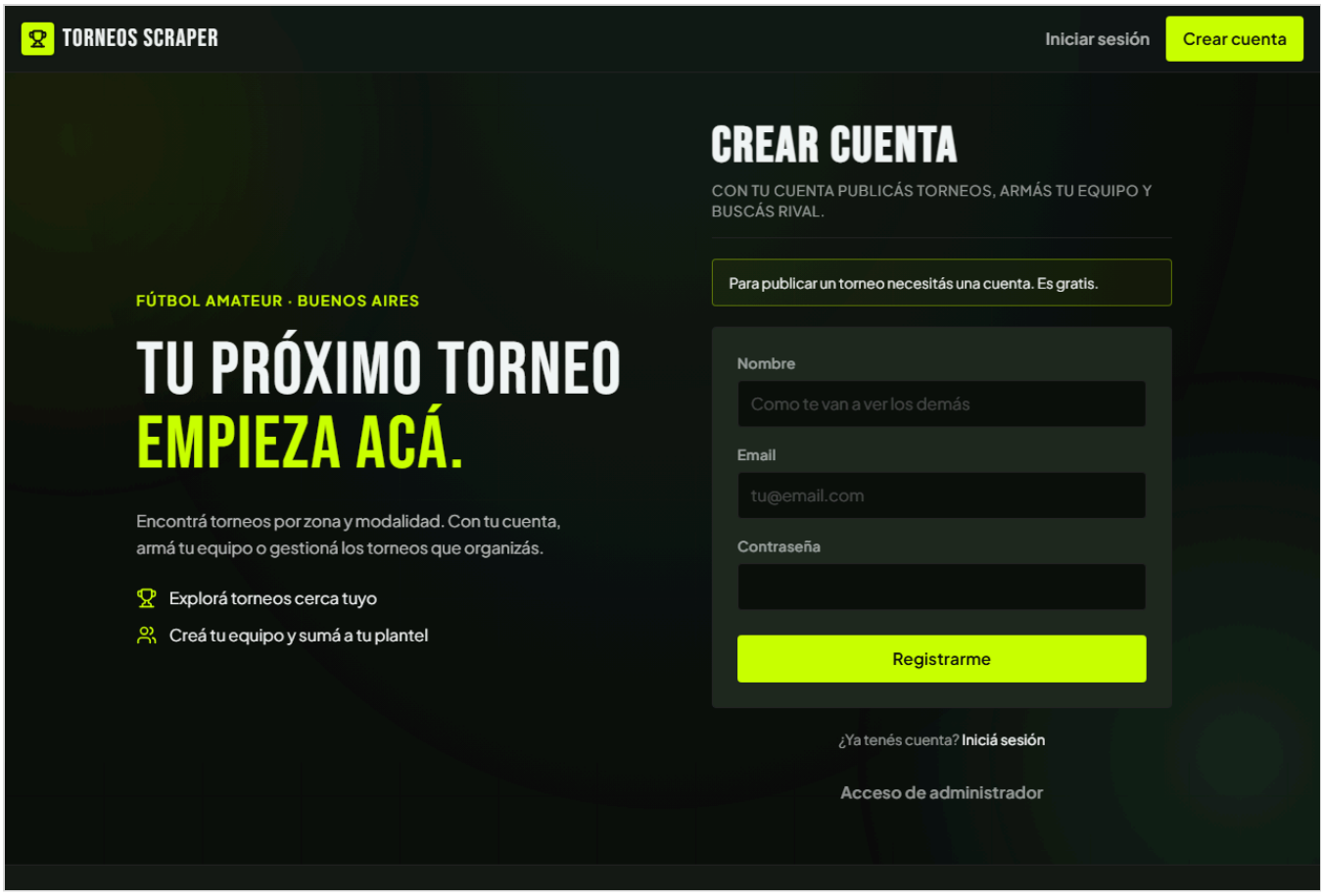
Captura ✅ vigente.

Qué muestra. El título «Crear cuenta», la bajada «Con tu cuenta publicás torneos, armás tu equipo y buscás rival» (neutra: no habla solo de torneos) y el formulario:

Campo	Tipo	Detalle	Obligatorio
Nombre	texto (2 a 50 caracteres)	Texto de ayuda «Como te van a ver los demás»: es público	Sí
Email	email		Sí
Contraseña	contraseña	El gestor de contraseñas sugiere una nueva	Sí

El botón principal es «Registrarme». Debajo: «¿Ya tenés cuenta? **Iniciá sesión**» (cambia a P1) y «Acceso de administrador».

Llegar desde «¿Organizás torneos? Publicalo» abre esta pantalla con el destino puesto en el panel de organizador y un aviso arriba del formulario:



Captura ✅ vigente. URL: `/login?next=/mis-torneos`*. Sin* `modo` *en la URL y con ese destino, la pantalla abre Crear cuenta y muestra «Para publicar un torneo necesitás una cuenta. Es gratis.». Con cualquier otro destino se abre Iniciar sesión, sin aviso.*

Estados: *error de nombre* («El nombre tiene que tener entre 2 y 50 caracteres»); *error de registro* («Error en registro», genérico: no distingue un email ya registrado de una contraseña inválida); *con sesión abierta* (se redirige al destino).

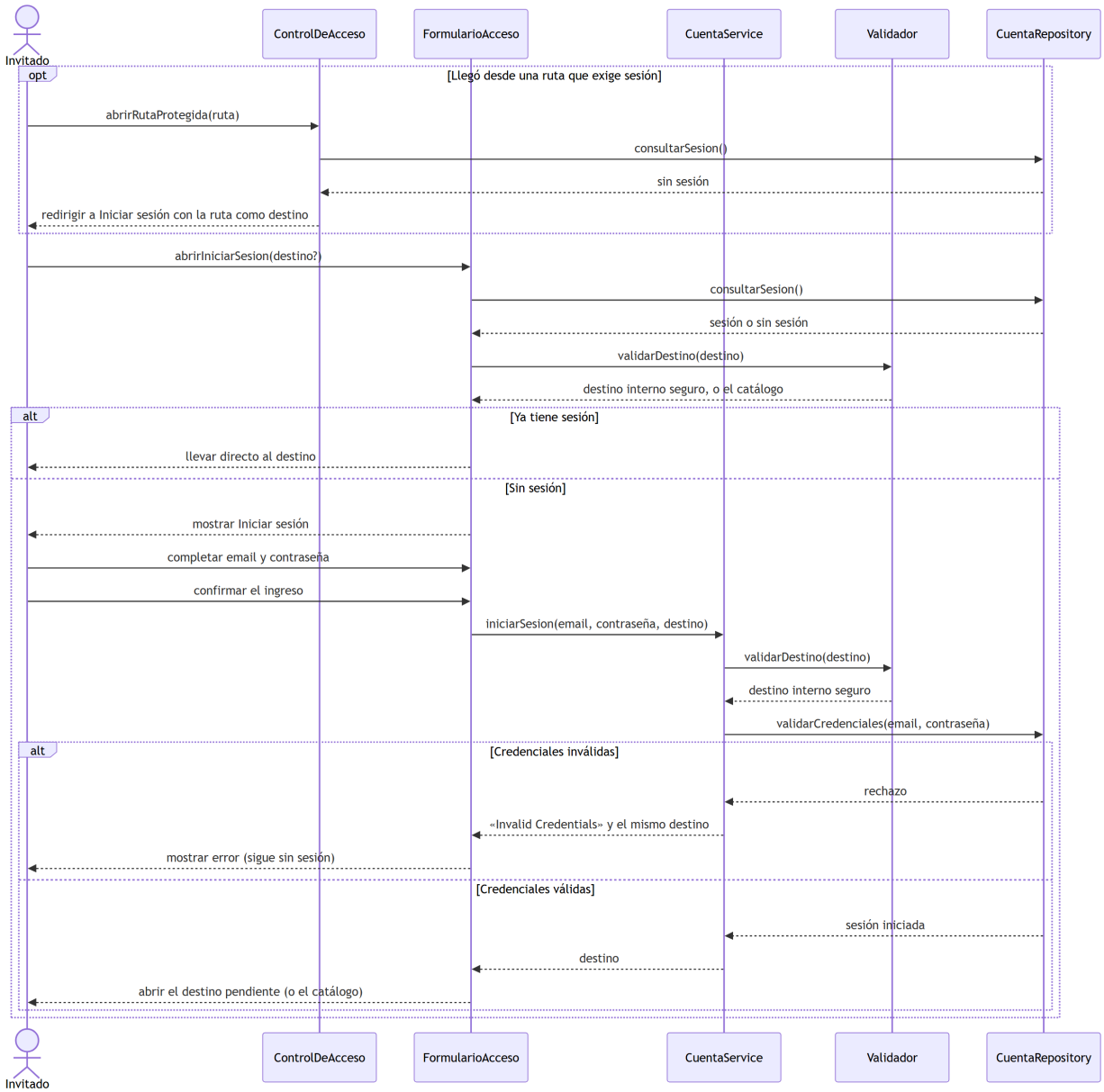
UC-A02 · Iniciar sesión

- **Actor:** Invitado.
- **Requerimientos:** RF-A04 a RF-A08, RF-A25. **Pantalla:** P1 · Iniciar sesión (`/login`). **Diagrama:** S02.
- **Precondiciones:** tiene una cuenta.
- **Postcondición:** sesión abierta en ese dispositivo.
- **Estado:** Implementado (`src/app/login` , función `login` ; `src/proxy.ts`).

Diagrama de secuencia · **S02 · UC-A02 — Iniciar sesión**

Caso de uso: UC-A02 · Pantalla: P1 · Requerimientos: RF-A04 a RF-A08

El formulario tiene solo email y contraseña: no hay desplegables. Este diagrama incluye también el caso en que la persona llega **redirigida** por intentar entrar a una ruta que exige sesión, que es la regla de «volver a la acción pedida».



Cómo se lee

- **Volver a la acción pedida.** Cuando una ruta exige sesión (`/crear-torneo`, `/mis-torneos`, `/perfil`, `/crear-equipo`, `/mis-equipos`, `/partidos`), `ControlDeAcceso` redirige a Iniciar sesión **conservando la ruta** como `next`. Al ingresar, la persona vuelve ahí. Es una primera capa «optimista»: cada página y cada acción vuelven a consultar la sesión y, si falta, mandan a `/login?next=…`.
- **El destino se valida dos veces** (al abrir y al enviar) y solo se aceptan rutas internas. Un destino externo o mal formado se reemplaza por el catálogo.
- **Con sesión abierta no hay nada que hacer en `/login`:** se sigue al destino.

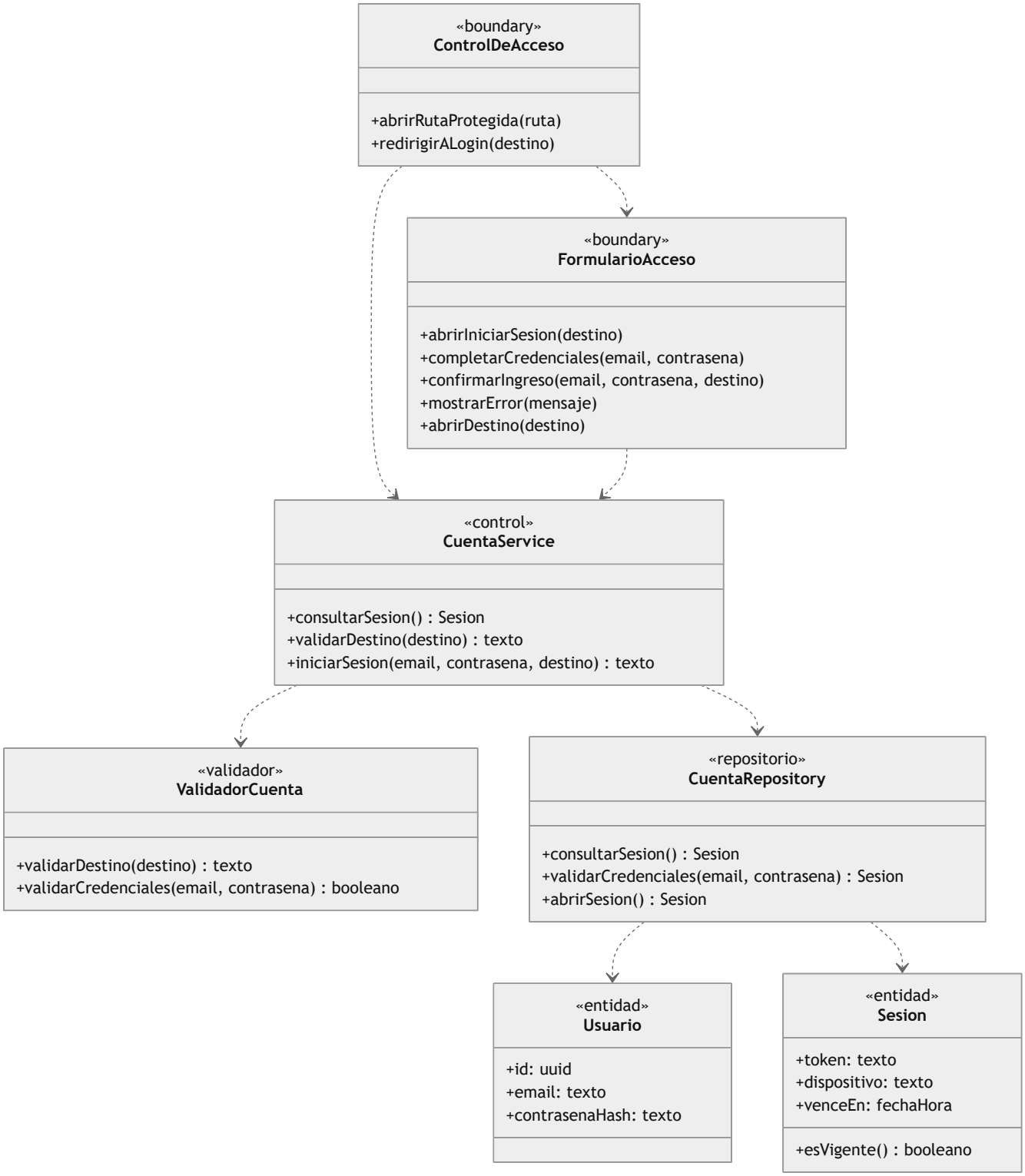
- **El error no dice qué falló.** «Invalid Credentials» no distingue un email inexistente de una contraseña incorrecta. Está en inglés en el código; el resto de la pantalla está en español.
- **El modo va explícito** al volver con error (`modo=ingreso`): sin él, un destino al panel de organizador abriría Crear cuenta en vez de volver a Iniciar sesión.
- **No hay bloqueo por intentos fallidos** en la app: lo cubre Supabase.
- `/admin` tiene su propio acceso (`/admin/login`, UC-F01); no es parte de este diagrama.

Correspondencia en el código

`login` (`src/app/login/actions.ts`): `safeNextPath` → `supabase.auth.signInWithPassword` → redirige al destino, o a `/login?modo=ingreso&error=…&next=…`. El control de acceso es `src/proxy.ts`.

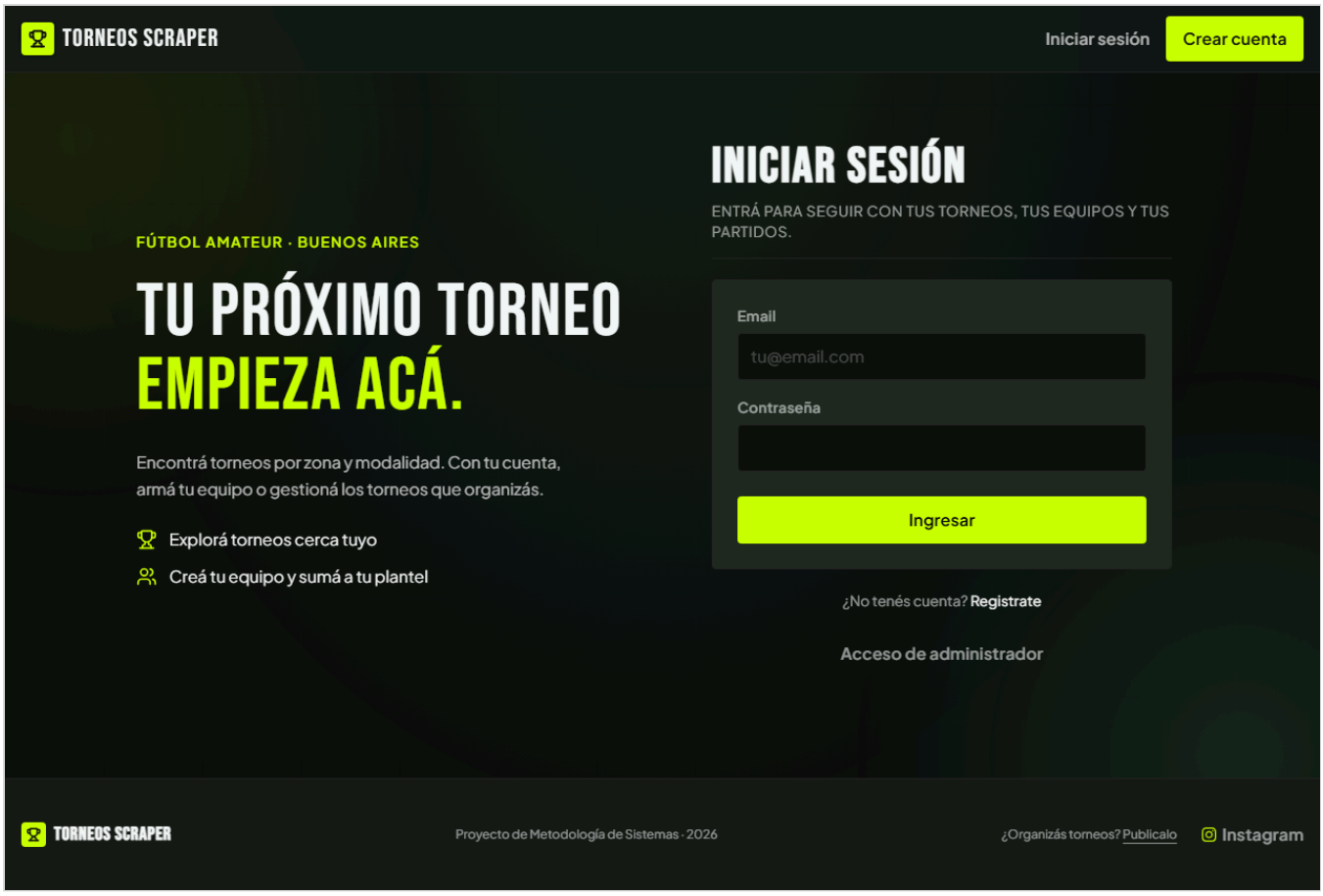
Diagrama de clases · **UC-A02 · Iniciar sesión**

Caso de uso: UC-A02 · **Pantalla:** P1 · Iniciar sesión (`/login`) · **Secuencia:** S02



Pantalla · **P1 · Iniciar sesión**

Ruta: `/login` (o `/login?modo=ingreso`) · **Caso de uso:** UC-A02 · **Requerimientos:** RF-A04 a RF-A08 · **Diagrama:** S02



Captura ✅ vigente.

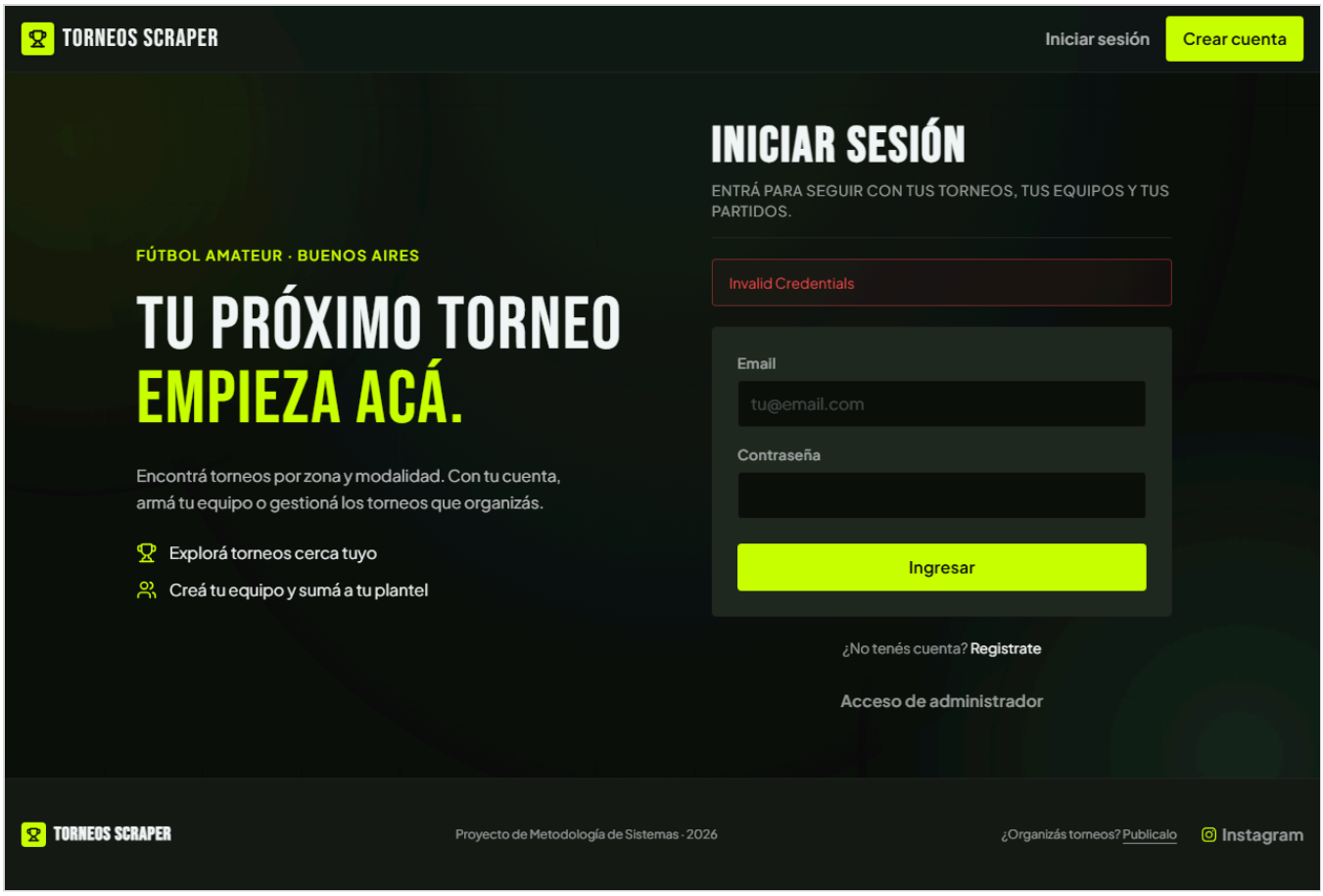
Qué muestra. A la izquierda (solo en pantalla grande), el mensaje de bienvenida «Tu próximo torneo empieza acá» con dos beneficios. A la derecha, el título «Iniciar sesión», la bajada «Entrá para seguir con tus torneos, tus equipos y tus partidos» y el formulario:

Campo	Tipo	Obligatorio
Email	email	Sí
Contraseña	contraseña	Sí

El botón principal es «Ingresar». Debajo: «¿No tenés cuenta? **Registrate**» (cambia a P2) y el link «Acceso de administrador» (`/admin/login` , UC-F01). No hay desplegables ni opciones que cargar.

Estados de la pantalla

Estado	Qué ve la persona
Normal	El formulario vacío.
Credenciales inválidas	Banda roja con el texto «Invalid Credentials» (ver la captura de abajo). El destino pendiente se conserva.
Con sesión abierta	No se ve la pantalla: se redirige al destino.
Llegó por una ruta protegida	El mismo formulario; al ingresar vuelve a esa ruta (no hay aviso).



Captura ✅ vigente del error. El mensaje está en inglés en el código (`Invalid Credentials`); el resto de la pantalla está en español.

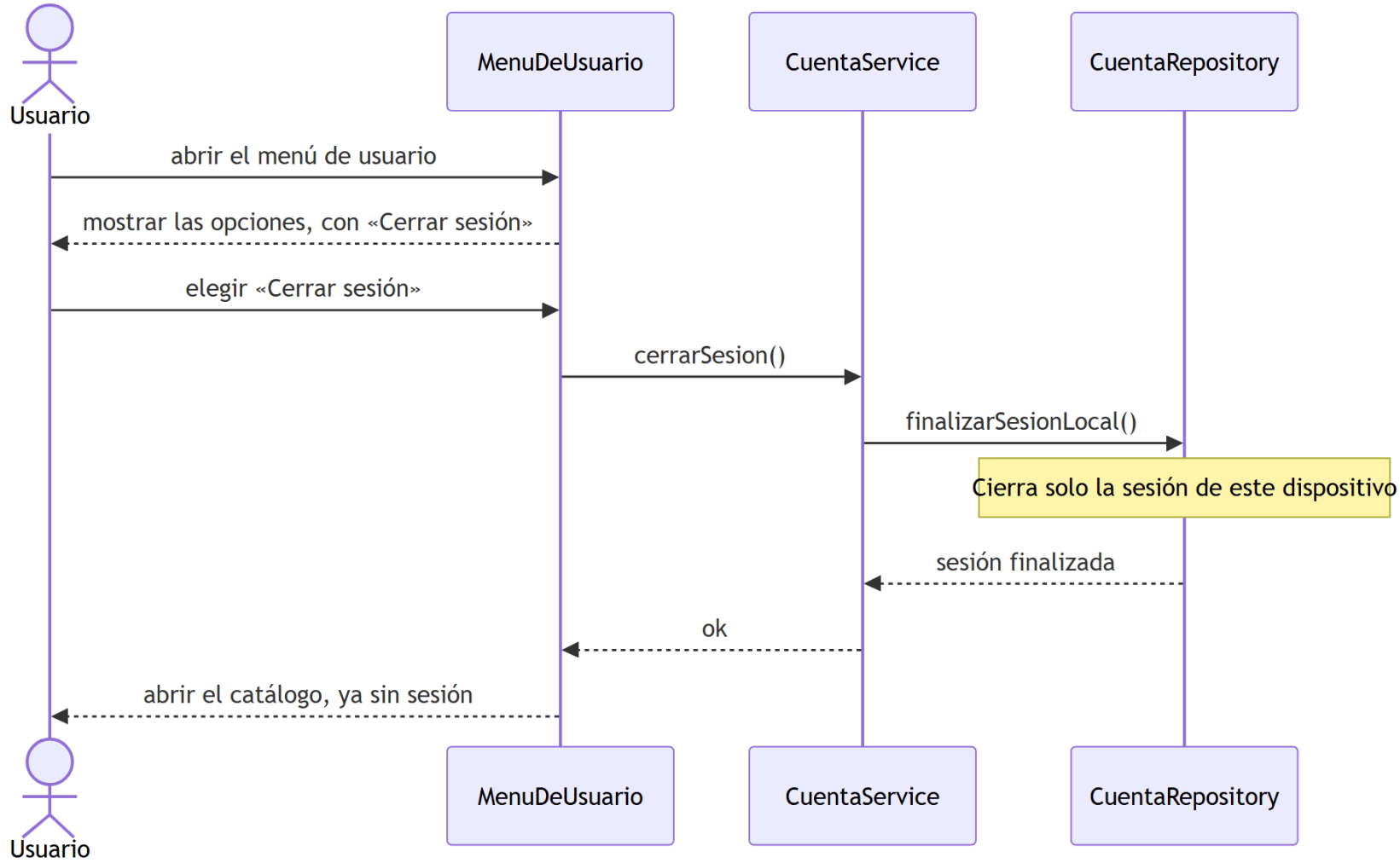
UC-A03 · Cerrar sesión

- **Actor:** Usuario.
- **Requerimientos:** RF-A09, RF-A25. **Pantalla:** P4 · Barra superior (menú de usuario, «Cerrar sesión»). **Diagrama:** S03.
- **Precondiciones:** sesión.
- **Postcondición:** el dispositivo queda sin sesión; las demás sesiones de la cuenta siguen abiertas.
- **Estado:** Implementado (función `logout`, `signOut({ scope: 'local' })`).

Diagrama de secuencia · **S03 · UC-A03 — Cerrar sesión**

Caso de uso: UC-A03 · Pantalla: P4 (menú de usuario) · Requerimientos: RF-A09, RF-A25

No hay formulario ni datos que cargar: es una sola acción.



Cómo se lee

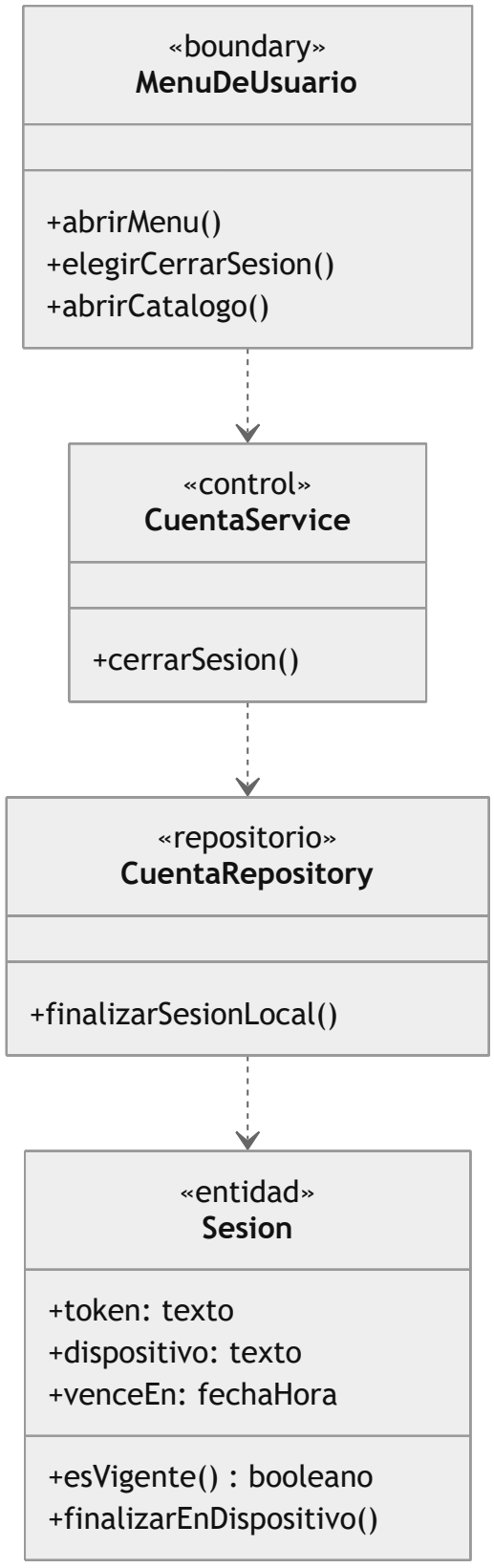
- **Solo este dispositivo.** Si se pidiera el cierre sin indicar el alcance, Supabase revocaría las sesiones de la cuenta en **todos** los dispositivos (celular, otra computadora…). Por eso la acción pide explícitamente el cierre **local**; las demás sesiones siguen abiertas.
- **Siempre termina en el catálogo**, que es público. No hay confirmación previa: cerrar sesión no destruye nada.
- Después de cerrar, la barra vuelve a mostrar «Iniciar sesión» y «Crear cuenta», y la campanita desaparece.

Correspondencia en el código

`logout` (`src/app/login/actions.ts`): `supabase.auth.signOut({ scope: 'local' })`, revalida el layout y redirige a `/`. El botón está en `UserMenu.tsx`.

Diagrama de clases · **UC-A03 · Cerrar sesión**

Caso de uso: UC-A03 · **Pantalla:** P4 · Barra superior (menú de usuario, «Cerrar sesión») · **Secuencia:** S03



Pantalla · **P4 · Barra superior**

Ruta: todas · **Casos de uso:** UC-A01, UC-A02, UC-A03, UC-A05, UC-A06 · **Requerimientos:** RF-A25 · **Diagrama:** S03 (cerrar sesión)

Es mínima: ningún botón destacado permanente.

Estado	Qué muestra
Sin sesión	El logo; «Iniciar sesión» (texto) y «**Crear cuenta**» (único botón destacado). ✅ Se ve en la parte de arriba de todas las capturas de P1 y P2.
Con sesión	El logo; la campanita (P5); y el **menú de usuario**, un botón con la inicial y el nombre de la cuenta. ⏳ Captura pendiente.

El menú de usuario se abre al tocar el botón y se cierra con un clic afuera o con Escape. Ofrece, en este orden:

1. **Mi perfil** (`/perfil`).
2. **Mis equipos** (`/mis-equipos`).
3. **Partidos** (`/partidos`).
4. **Mis torneos** (`/mis-torneos`).
5. **Administración** (`/admin`), **solo** si la cuenta es Administradora. Es solo el link: el acceso lo controlan el control de acceso y `is_admin()` .
6. **Cerrar sesión**, que cierra solo este dispositivo.

UC-A04 · Recuperar contraseña

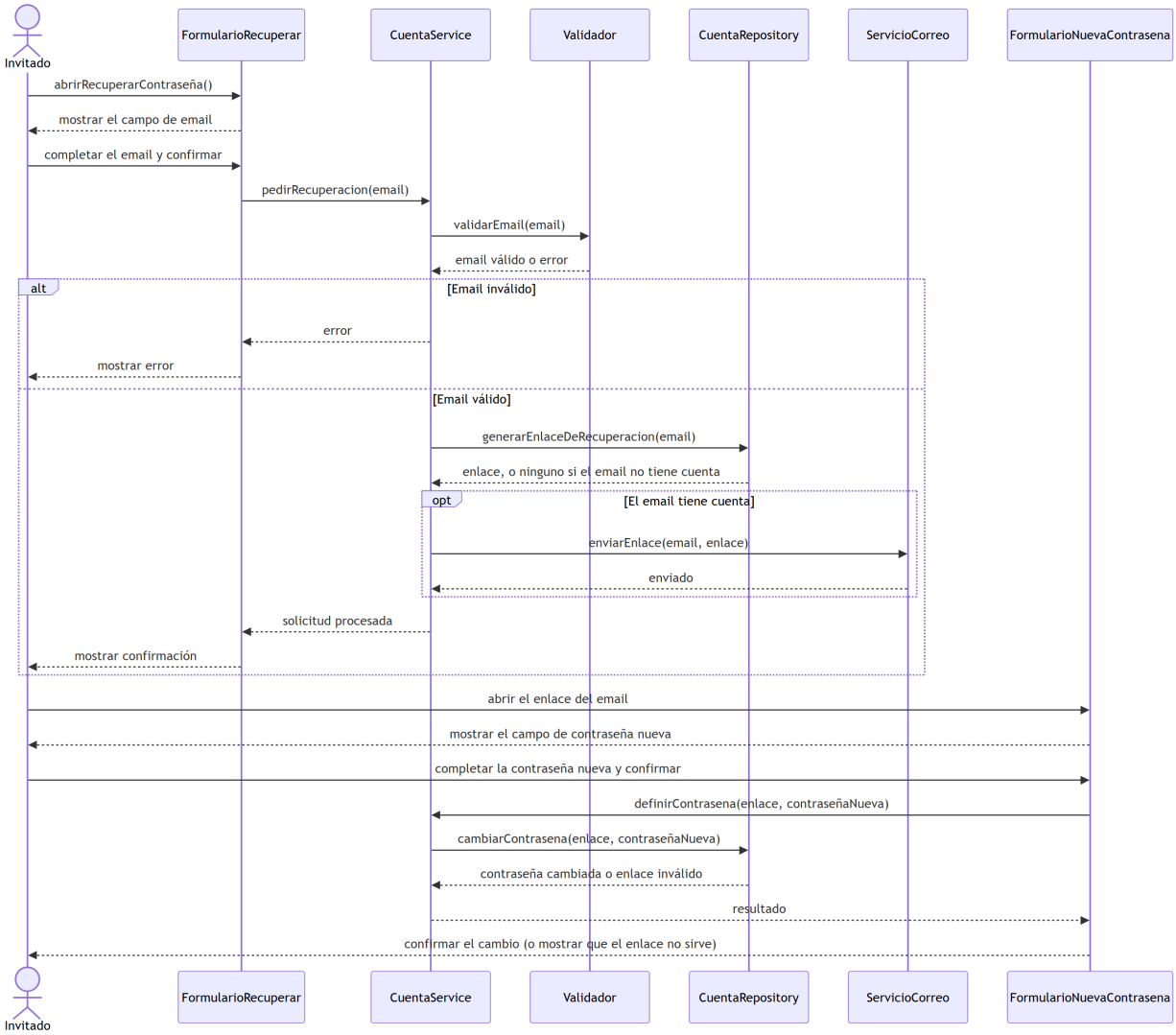
- **Actor:** Invitado.
- **Requerimientos:** RF-A10. **Pantalla:** no existe todavía. **Diagrama:** S04 (diseño objetivo).
- **Precondiciones:** tiene una cuenta.
- **Estado: Pendiente, a propósito** (decisión del 27/09/2026: primero producto, después infraestructura de auth). Todavía no hay pantalla, link ni acción.
- **Sin decidir:** qué se muestra si el email no tiene cuenta (decirlo permite averiguar quién está registrado) y si, al definir la contraseña nueva, la persona queda con la sesión abierta o debe iniciar sesión. Hay que decidirlo antes de implementar y agregar la regla a decisiones-casos-de-uso.md.

> Diagrama de secuencia · **S04 · UC-A04 — Recuperar contraseña**

Caso de uso: UC-A04 · Pantalla: no existe todavía · Requerimientos: RF-A10

> **Diseño objetivo, pendiente de implementación** (decisión del 27/09/2026: primero producto, después infraestructura de auth). Hoy no hay pantalla, link ni acción. El diagrama dibuja lo que dicen las reglas vigentes; `FormularioRecuperar`, `FormularioNuevaContrasena` y `ServicioCorreo` no existen todavía y sus nombres son provisorios.

No hay desplegables. El caso tiene dos momentos —pedir el enlace y usarlo— que son **una sola meta**, así que van en el mismo diagrama.



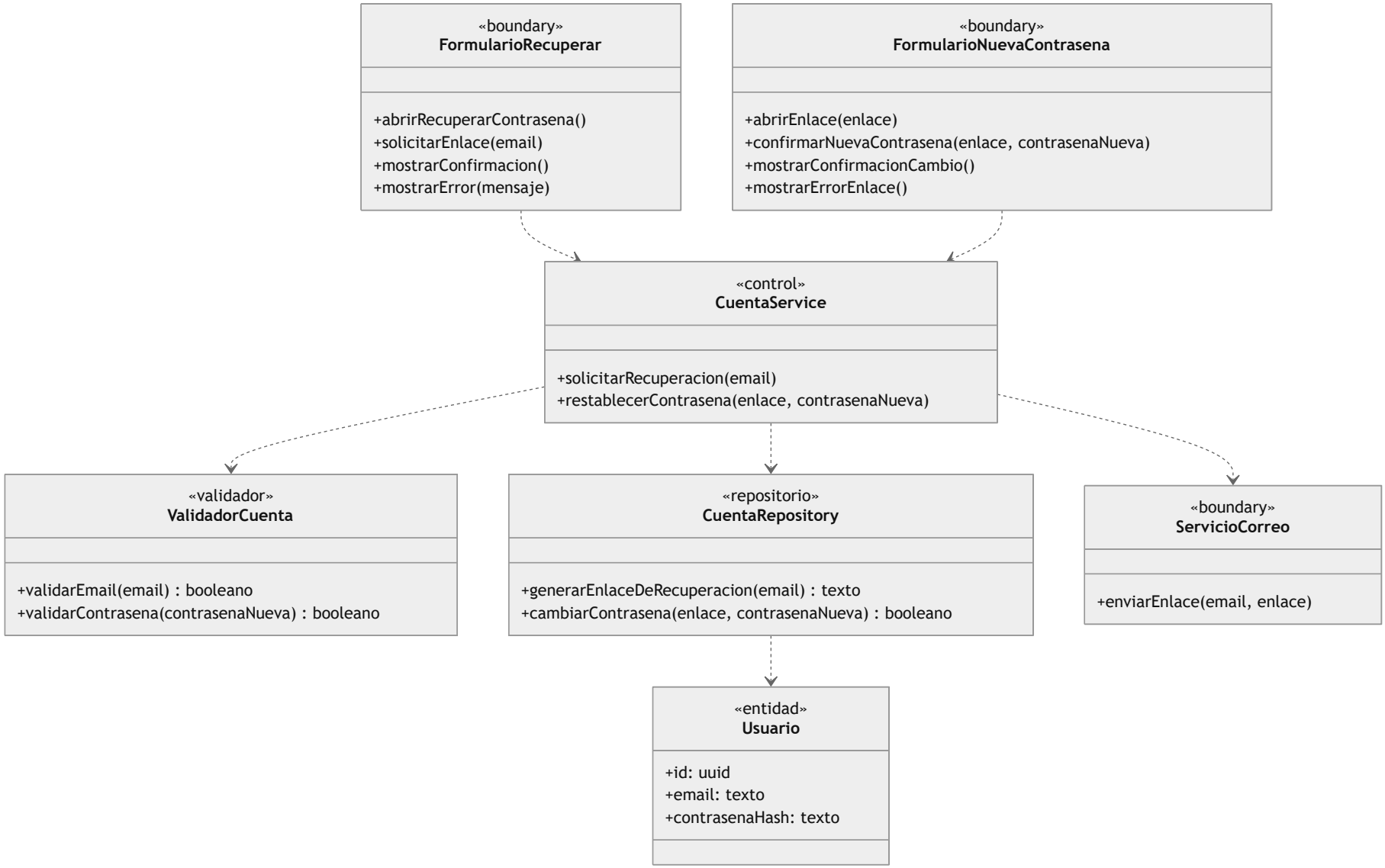
Cómo se lee

- **Es el único email que mandaría la app.** El SMTP gratuito de Supabase manda muy pocos mensajes por hora: alcanza para la entrega académica, pero con usuarios reales hace falta un servicio aparte (por ejemplo, Resend). Por eso el envío es un participante propio, `ServicioCorreo`.
- **Un enlace por email, de un solo propósito:** abrirlo lleva a definir la contraseña nueva. Un enlace inválido o vencido se muestra como tal.
- **Sin decidir, y hay que resolverlo antes de implementar** (después, agregar la regla a decisiones-casos-de-uso.md):

1. **Qué se responde si el email no tiene cuenta.** Decir «no existe» permite averiguar quién está registrado; responder siempre lo mismo lo evita. El diagrama deja el camino abierto: el envío es opcional y la confirmación es la misma.
2. **Qué pasa después de cambiar la contraseña:** si la persona queda con la sesión abierta o tiene que iniciar sesión.

Diagrama de clases · **UC-A04 · Recuperar contraseña**

Caso de uso: UC-A04 · **Pantalla:** no existe todavía · **Secuencia:** S04 (diseño objetivo)



> Pantalla · **Todavía no existe**

Este caso de uso no tiene pantalla en la app todavía; el caso y el diagrama describen el diseño objetivo.

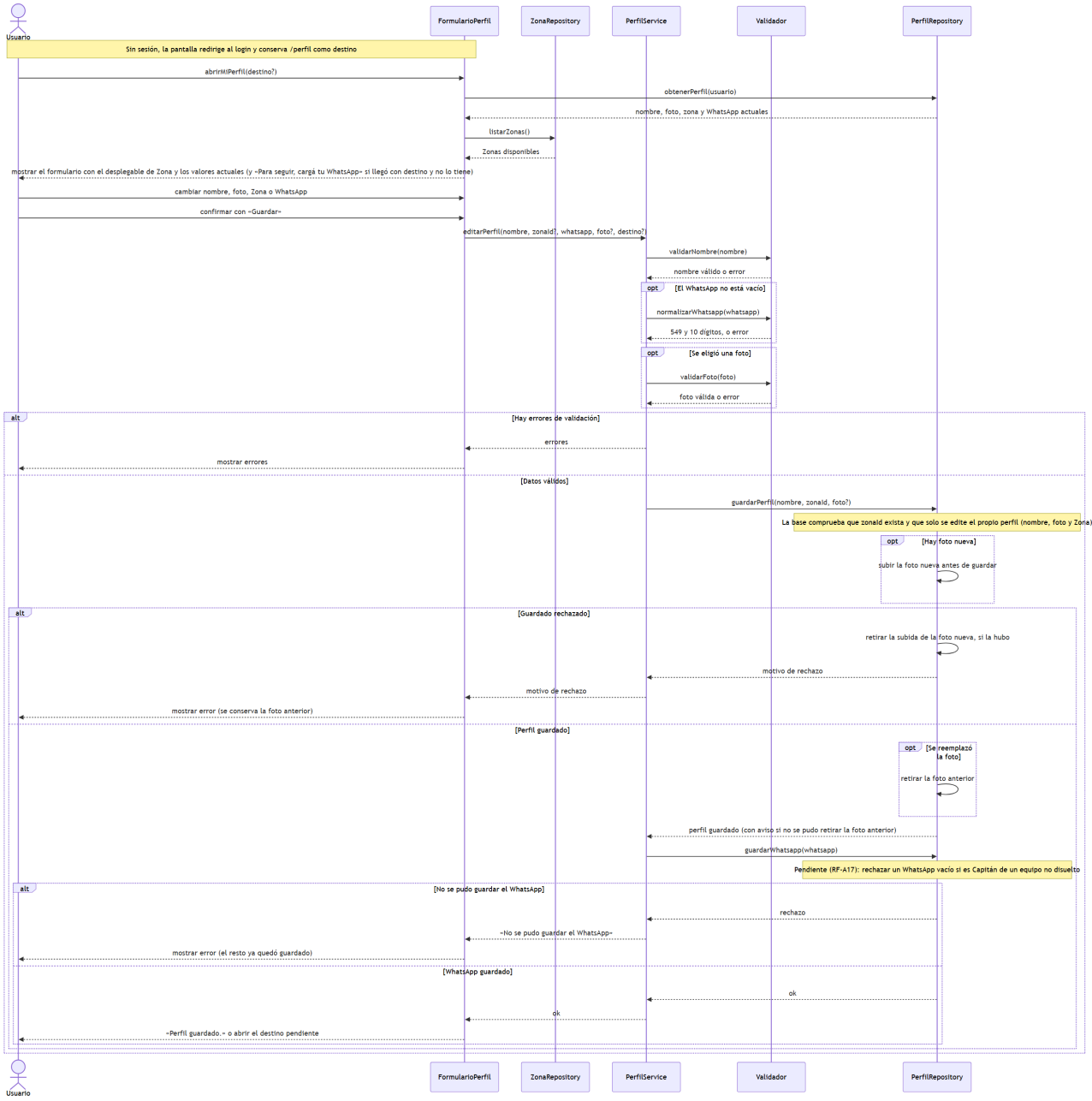
UC-A05 · Editar perfil

- **Actor:** Usuario.
- **Requerimientos:** RF-A11 a RF-A18. **Pantalla:** P3 · Mi perfil (`/perfil`). **Diagrama:** S05.
- **Precondiciones:** sesión.
- **Postcondición:** perfil actualizado; el nombre y la foto se ven en la página del organizador y en los partidos.
- **Estado: Parcial.** Todo está implementado (`src/app/perfil`) **salvo A2**: la regla del Capitán con WhatsApp vacío no se verifica en ningún lado. `casos-de-uso.md` marca el caso como «Implementado» sin esa salvedad.

Diagrama de secuencia · **S05 · UC-A05 — Editar perfil**

Caso de uso: UC-A05 · Pantalla: P3 · Requerimientos: RF-A11 a RF-A18

El formulario tiene **un solo desplegable: la Zona**. Sigue el criterio de la guía de formularios: la pantalla obtiene las Zonas de `ZonaRepository` **antes** de que la persona elija, y al enviar manda solo el `zonaId` elegido. Nombre, WhatsApp y foto son campos de texto y de archivo, sin listas que cargar; por eso no hay proveedor de opciones fijas. La carga de la Zona y el guardado van en este único diagrama.



Cómo se lee

- **Antes de elegir:** la pantalla trae los valores actuales de `PerfilRepository` y las Zonas de `ZonaRepository`, y recién entonces muestra el formulario. El desplegable ofrece además «Sin zona»: la Zona del perfil es **opcional**.
- **Al enviar:** `editarPerfil(...)` recibe solo los **valores elegidos** (`zonaId`), no la lista de Zonas.

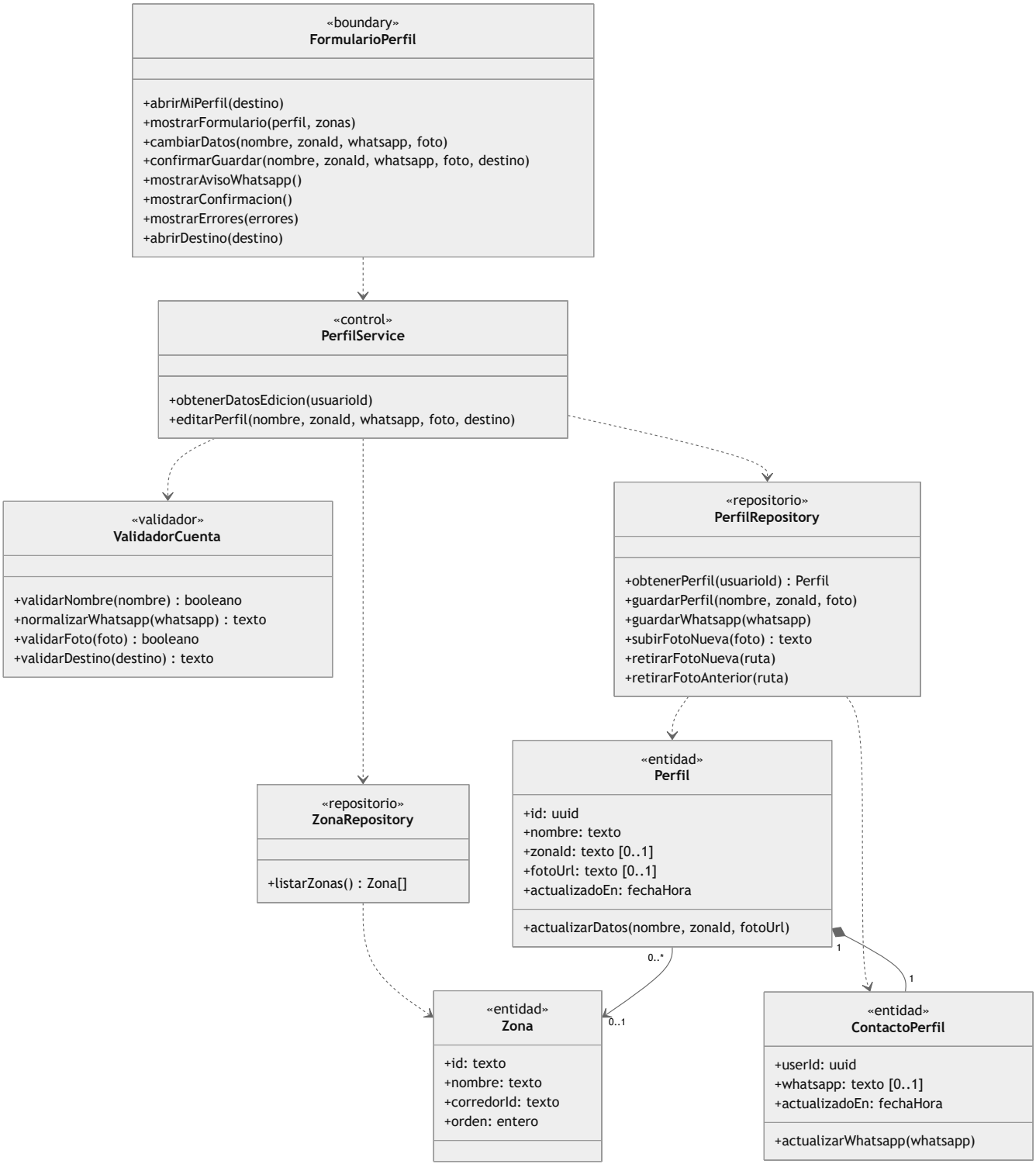
- **Se valida de nuevo al guardar.** `Validador` revisa el nombre (de 2 a 50 caracteres), el WhatsApp y la foto aunque el formulario tenga sus propias restricciones; y la base comprueba que la Zona exista (clave foránea) y que cada persona edite solo su perfil, y solo las columnas nombre, foto y Zona. El rol de administrador no está entre ellas: **no se puede editar desde la app**.
- **El WhatsApp se normaliza:** la persona escribe «11 2345 6789» y se guarda `5491123456789` (`549` + código de área sin 0 + número sin 15). Un número con «15» se rechaza, porque después del código de área no se sabe dónde termina el código. Vacío es válido.
- **La foto no se pierde si algo falla:** la foto nueva se **sube antes** de guardar; si el guardado falla, se retira esa subida y la anterior queda intacta; la anterior **solo se retira después** de guardar la URL nueva. Si dos pestañas reemplazan la foto a la vez, la segunda no se guarda.
- **Guardado en dos pasos, sin transacción:** primero nombre, Zona y foto; después el WhatsApp, que está en otra tabla (`profile_contacts`, aparte para que leer el perfil público nunca exponga el número). Si el segundo paso falla, el primero **ya quedó guardado**, y el mensaje lo dice.
- **Regla pendiente:** el Capitán de un equipo no disuelto no puede dejar el WhatsApp vacío (sí cambiarlo), porque el rival lo necesita para coordinar. **Hoy nada lo hace cumplir**: la acción no consulta los equipos y la base no tiene un trigger sobre `profile_contacts` (el único trigger de la base es el del alta de cuentas). Cuando se resuelva, la comprobación va en `guardarWhatsapp`, dentro de la base.

Correspondencia en el código

`updateProfile` (`src/app/perfil/actions.ts`): regla del nombre → `normalizeWhatsapp` → `parseAvatar` → `saveProfile` (`src/utils/supabase/profile.ts`) → `update` de `profile_contacts` → redirige al destino o a `/perfil?ok=1`. Las Zonas salen de `zones` mediante `getZoneGroups()`.

Diagrama de clases · **UC-A05 · Editar perfil**

Caso de uso: UC-A05 · **Pantalla:** P3 · Mi perfil (`/perfil`) · **Secuencia:** S05



Pantalla · **P3 · Mi perfil**

Ruta: `/perfil` · **Caso de uso:** UC-A05 · **Requerimientos:** RF-A11 a RF-A18 · **Diagrama:** S05

⏳ **Captura pendiente:** requiere una cuenta con sesión.

Qué muestra. Cabecera «Mi perfil» con el email de la cuenta, el link «Volver al catálogo» y un formulario con:

Campo	Tipo	De dónde salen las opciones	Obligatorio
Foto de perfil	archivo JPG, PNG o WebP, hasta 2 MB; muestra la foto actual	—	No
Nombre	texto (2 a 50 caracteres); «Es público: lo ven todos en tu página de organizador»	—	Sí
Zona	desplegable, agrupado por corredor, con la opción «Sin zona»	Se **consulta en la base** (`zones` , vía `getZoneGroups`)	No
WhatsApp	teléfono, con el prefijo fijo «+54 9» y el ejemplo «11 2345 6789»	—	No

Bajo el WhatsApp: «Código de área sin 0 y número sin 15. No es público: hace falta para crear un equipo o ser capitán, y solo lo ve el capitán rival cuando tienen un partido acordado». El botón es «Guardar».

Estados de la pantalla

Estado	Qué ve la persona
Normal	El formulario con los valores actuales.
Guardado	Banda lima «Perfil guardado.»
Error	Banda roja con el motivo (nombre, WhatsApp, foto, o fallo al guardar).
Llegó desde otra acción sin WhatsApp	Recuadro «Para seguir, cargá tu WhatsApp.»; al guardar, vuelve al destino.
Sin sesión	No se ve la pantalla: se redirige a `/login?next=/perfil` .

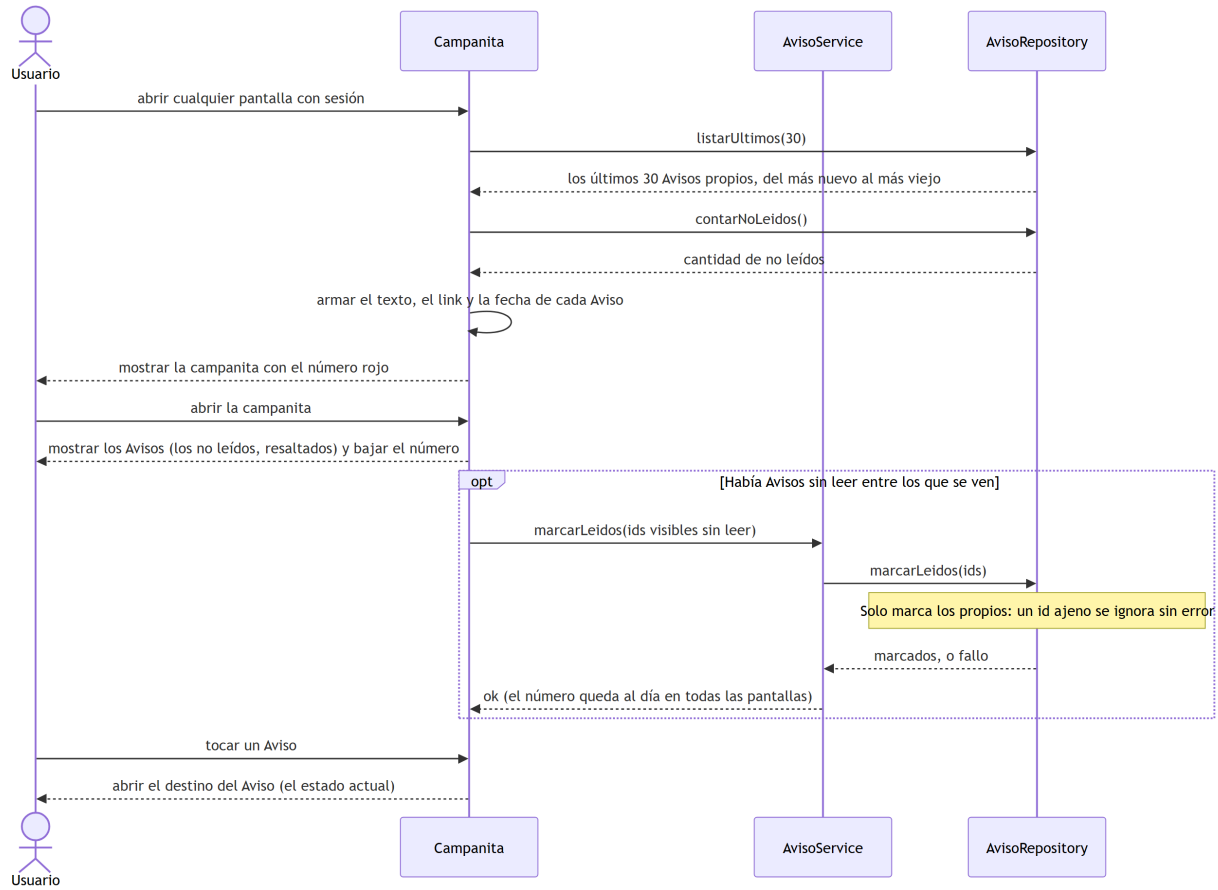
UC-A06 · Ver avisos (campanita)

- **Actor:** Usuario.
- **Requerimientos:** RF-A19 a RF-A24, RF-A25. **Pantalla:** P5 · Campanita. **Diagrama:** S06.
- **Precondiciones:** sesión.
- **Estado:** Implementado (`NoticeBell` , `src/app/avisos` , función `mark_notices_read`). La lista cerrada de disparadores está en casos-de-uso.md.

> Diagrama de secuencia · **S06 · UC-A06 — Ver avisos (campanita)**

Caso de uso: UC-A06 · Pantalla: P5 · Requerimientos: RF-A19 a RF-A24

No hay desplegables ni formulario: la campanita es una lista que se lee y se marca. **Nadie crea Avisos desde esta pantalla:** los escribe el sistema cuando ocurre una acción de otro caso de uso (por ejemplo, proponer un partido), por eso no hay un `crearAviso` en este diagrama.



Cómo se lee

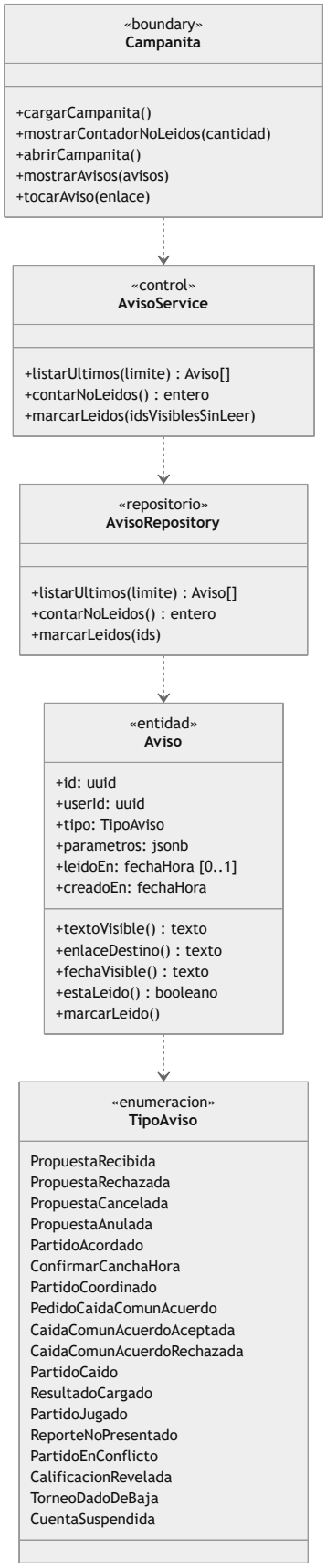
- **Qué se carga:** los **últimos 30** Avisos y, aparte, el total de no leídos (puede haber más de 30). La base devuelve solo los del propio usuario (RLS sobre `notices`).
- **El texto no se guarda, se arma al mostrar** a partir del hecho (`kind`) y la foto de sus datos del momento (`params`). Por eso un Aviso viejo no cambia si el rival cambia de nombre. Cada Aviso es un **link** al estado actual; no tiene botones: la acción se hace en la pantalla de destino, que valida las reglas.
- **Abrir la campanita marca leídos solo los que se ven**, no «todos». Si llega un Aviso con la campanita abierta, sigue contando como no leído. El número baja de inmediato en la pantalla, sin esperar a la base.
- **Si el marcado falla**, el Aviso sigue como no leído y se marca la próxima vez que se abra; no se muestra ningún error. Un id ajeno no se marca y no da error, para no confirmar que existe.
- **No hay borrado ni archivo, email ni push.** Los Avisos que abren un plazo dicen cuándo vence y qué pasa si no se responde.

Correspondencia en el código

El layout (`src/app/layout.tsx`) lee `notices` (límite 30, más el conteo de `read_at` nulo) y arma el texto con `noticeText`, `noticeLink` y `noticeDate` (`src/utils/notices.ts`). `NoticeBell` muestra la lista y llama a `markNoticesRead` (`src/app/avisos/actions.ts`), que invoca la función `mark_notices_read`.

Diagrama de clases · **UC-A06 · Ver avisos (campanita)**

Caso de uso: UC-A06 · **Pantalla:** P5 · Campanita · **Secuencia:** S06



Pantalla · **P5 · Campanita de avisos**

Ruta: todas, con sesión · **Caso de uso:** UC-A06 · **Requerimientos:** RF-A19 a RF-A24 · **Diagrama:** S06

⏳ **Captura pendiente:** requiere una cuenta con Avisos (por ejemplo, un Capitán que recibió una propuesta).

Qué muestra. Un botón con una campana. Si hay Avisos no leídos, un **círculo rojo** con el número (hasta 99; con más, «99+»). Al tocarlo se abre un panel con el título «Avisos» y una lista de hasta 30, del más nuevo al más viejo. Cada fila es un **link** con el texto del Aviso y su fecha; los que estaban sin leer tienen un borde lima a la izquierda mientras el panel está abierto. No hay botones dentro de un Aviso.

Estados: *sin Avisos* («Todavía no tenés avisos.»); *con no leídos* (número rojo); *abierta* (el número baja de inmediato y los Avisos visibles quedan leídos); *sin sesión* (la campanita no existe).

Parte 3 · Anexos y trazabilidad

Guía de pantallas: estado de las capturas y mapa

Versión del **04/11/2026**. Cada pantalla de la app que representa un caso de uso del módulo, con la ruta, el caso, los requerimientos, lo que muestra y la captura. Cada una va debajo del caso de uso que la usa por primera vez; si otro caso usa la misma pantalla, remite a esa descripción.

Estado de las capturas

Marca	Significa
✅ **Vigente**	Sacada el 04/11/2026 de `main`, con la UI actual (lima y negro), a 1280 px de ancho.
⏳ **Pendiente**	Todavía no hay captura. Al final están los pasos para sacarla.

Las imágenes están en `pantallas/`. Las de `/login` se sacaron de la app corriendo en local, sin sesión.

Mapa de pantallas

Pantalla	Ruta	Casos de uso	Sesión
P1 · Iniciar sesión	`/login`	UC-A02	No (con sesión, redirige)
P2 · Crear cuenta	`/login?modo=registro`	UC-A01	No (con sesión, redirige)
P3 · Mi perfil	`/perfil`	UC-A05	Obligatoria
P4 · Barra superior	todas	UC-A01 a A03, A05, A06	Cambia según haya sesión
P5 · Campanita de avisos	todas (con sesión)	UC-A06	Obligatoria

Iniciar sesión y Crear cuenta son **la misma página** (`/login`) con dos modos. El modo vive en la URL (`?modo=ingreso` o `?modo=registro`), no en el estado del navegador, así que cada pantalla es un link. `/perfil` exige sesión: sin ella, el control de acceso redirige a `/login?next=/perfil` .

Capturas pendientes

Para completar el módulo. Convención: PNG en `docs/cuenta-y-acceso/pantallas/` , ventana de unos 1280 px de ancho, con datos de prueba (nunca cuentas ni Avisos reales).

Archivo	Pantalla y estado	Cómo conseguirlo
`perfil.png`	P3 con los valores cargados	Con sesión, abrir `/perfil` con un perfil que tenga foto, Zona y WhatsApp.
`perfil-sin-whatsapp.png`	P3 con «Para seguir, cargá tu WhatsApp»	Con una cuenta sin WhatsApp, abrir `/perfil?next=/crear-equipo` .
`perfil-guardado.png`	P3 con «Perfil guardado.»	Guardar el perfil sin destino.
`perfil-error.png`	P3 con un error	Guardar un WhatsApp con «15» (por ejemplo, `11 15 2345 6789`).
`perfil-zona.png`	El desplegable de Zona abierto	Abrir el desplegable «Zona» para ver los corredores.
`barra-con-sesion.png`	P4 con sesión y el menú de usuario abierto	Con sesión, tocar el botón de la cuenta. Una cuenta Administradora muestra además «Administración».

Archivo	Pantalla y estado	Cómo conseguirlo
`campanita.png`	P5 abierta con Avisos	Con una cuenta que recibió Avisos, abrir la campanita.
`campanita-vacia.png`	P5 sin Avisos	Con una cuenta nueva, abrir la campanita.

Al agregar una captura, actualizar acá su marca de estado.

Diagramas de secuencia: criterios, participantes y convenciones

Versión del **04/11/2026**. Un diagrama por caso de uso, en Mermaid (GitHub los dibuja directo). Cada uno va debajo de su caso de uso, antes de su pantalla.

Diagrama	Caso de uso	Pantalla	Estado
S01 · Crear cuenta	UC-A01	P2	Implementado
S02 · Iniciar sesión	UC-A02	P1	Implementado
S03 · Cerrar sesión	UC-A03	P4	Implementado
S04 · Recuperar contraseña	UC-A04	—	**Pendiente** (diseño objetivo)
S05 · Editar perfil	UC-A05	P3	Parcial (falta una regla)
S06 · Ver avisos	UC-A06	P5	Implementado

Los PNG están dibujados sobre fondo blanco, para pegarlos en un informe. La fuente de cada diagrama es el bloque Mermaid de su archivo: si se cambia uno, hay que regenerar su PNG.

Relación con los diagramas generales. `docs/diagramas-de-secuencia/diagramas-secuencia.md` tiene diagramas de todo el proyecto con participantes genéricos (`Navegador` , `Interfaz` , `Acceso a datos`). Los de este módulo los **detallan**: cada diagrama usa objetos por responsabilidad y fuente de datos, y el de Editar perfil muestra la carga de las Zonas del desplegable (D06 no la muestra). Los generales D01 a D06 corresponden a S01, S02, S03, S04 y S05, y D05 (volver a la acción pedida) está dentro de S01 y S02.

Criterios aplicados

Son los de la guía para diagramas con formularios y desplegables.

1. **Un caso de uso por diagrama.** La carga de las opciones de un formulario y el guardado van **en el mismo diagrama**.
2. **Si el caso empieza con un formulario con desplegables**, el diagrama muestra al actor abriendo la pantalla y a la pantalla pidiendo las opciones **antes** de que el actor elija. De este módulo, solo **Editar perfil** tiene un desplegable (la Zona); Crear cuenta e Iniciar sesión solo tienen campos de texto.
3. **Un objeto por responsabilidad o fuente de datos.** Las Zonas se consultan en la base, así que las provee `ZonaRepository` . No se inventan repositorios para listas que no se guardan.
4. **Al enviar**, el método del caso de uso recibe **solo los valores elegidos** (`zonaId`), no la lista de Zonas.
5. **La validación se repite al guardar**: un valor que aparece en la pantalla no garantiza que siga siendo válido. Por eso el destino `next` se valida al abrir la pantalla y otra vez al enviar el formulario.
6. **Cada diagrama muestra solo los participantes que intervienen.**

Participantes: responsabilidad y correspondencia en el repo

Son responsabilidades conceptuales. No existen como clases TypeScript con esos nombres: la app usa Server Actions, utilidades puras y funciones de Postgres. La columna de la derecha dice dónde vive hoy cada responsabilidad.

Participante	Responsabilidad	Correspondencia en el repo
Invitado, Usuario	La persona que inicia el caso.	La sesión; el Invitado navega sin ella.
FormularioAcceso	La pantalla de acceso, en sus dos modos (Iniciar sesión y Crear cuenta).	`login/page.tsx`, con `loginScreen` y `loginHref` (`utils/login-screen.ts`).
FormularioPerfil	La pantalla de Mi perfil.	`perfil/page.tsx`.
MenuDeUsuario	El menú de la barra, con «Cerrar sesión».	`components/UserMenu.tsx`.
Campanita	La campanita de la barra y el layout que carga sus Avisos.	`components/NoticeBell.tsx` y `app/layout.tsx`.
ControlDeAcceso	Decide si una ruta exige sesión y, si falta, redirige al acceso conservando el destino.	`src/proxy.ts` (primera capa; cada página vuelve a verificar).
ZonaRepository	Entrega la lista cerrada de Zonas, que **se consulta en la base**.	`getZoneGroups()` en `ZoneSelect.tsx`: lee `zones` con sus corredores y regiones.
CuentaService	Coordina registrarse, iniciar y cerrar sesión.	Las Server Actions `signup`, `login` y `logout` (`login/actions.ts`).
PerfilService	Coordina la edición del perfil.	La Server Action `updateProfile` (`perfil/actions.ts`).
AvisoService	Coordina el marcado de Avisos como leídos.	La Server Action `markNoticesRead` (`avisos/actions.ts`).
Validador	Valida las entradas antes de escribir.	`safeNextPath`, la regla del nombre (2 a 50), `normalizeWhatsapp` y `parseAvatar` (`utils/`).
CuentaRepository	Crea cuentas y sesiones y las consulta.	Supabase Auth (`signUp`, `signInWithPassword`, `signOut`, `getUser`); el trigger `handle_new_user` crea el perfil.
PerfilRepository	Lee y escribe el perfil, el contacto y la foto.	Las tablas `profiles` y `profile_contacts` con RLS; la foto, en Storage (bucket `avatars`); `saveProfile` (`utils/supabase/profile.ts`).
AvisoRepository	Lee los Avisos propios y los marca leídos.	La tabla `notices` con RLS y la función `mark_notices_read`.
ServicioCorreo	Solo en S04: manda el email de recuperación.	**No existe todavía.**

Convenciones de dibujo

- Flecha continua (`->>`): una llamada. Flecha punteada (`-->>`): su respuesta.
- Un mensaje de un participante a sí mismo es un paso interno (por ejemplo, una verificación dentro de una función de Postgres).
- Una nota sobre un participante marca una regla que **hace cumplir la base** (o una regla pendiente).
- Entre paréntesis y en minúscula van los **valores que viajan**; entre comillas «así», los mensajes que ve la persona.
- `opt` es un paso opcional; `alt` / `else`, caminos que se excluyen.

Trazabilidad y pendientes

Documentación completa del módulo A (Cuenta y acceso) de Torneos Amateur BA, versión del **04/11/2026**. En este documento, sus piezas son la Parte 1 (requerimientos), la Parte 2 (cada caso de uso con su diagrama y su pantalla) y los anexos de esta parte.

Fuentes, en orden de autoridad: decisiones-casos-de-uso.md → casos-de-uso.md → CONTEXT.md → código y migraciones.

Matriz de trazabilidad

Caso de uso	Requerimientos	Pantalla	Diagrama	Estado
UC-A01 Crear cuenta	RF-A01 a A03, A06, A08, A25	P2 `/login?modo=registro`	S01	Implementado
UC-A02 Iniciar sesión	RF-A04 a A08, A25	P1 `/login`	S02	Implementado
UC-A03 Cerrar sesión	RF-A09, A25	P4 menú de usuario	S03	Implementado
UC-A04 Recuperar contraseña	RF-A10	— (no existe)	S04	**Pendiente**
UC-A05 Editar perfil	RF-A11 a A18	P3 `/perfil`	S05	**Parcial** (RF-A17)
UC-A06 Ver avisos	RF-A19 a A24, A25	P5 campanita	S06	Implementado

Equivalencia con el PDF original

Vigente	PDF original
UC-A01 Crear cuenta	UC-01
UC-A02 Iniciar sesión	UC-02 y UC-05 (volver a la acción pedida)
UC-A03 Cerrar sesión	UC-03
UC-A04 Recuperar contraseña	UC-04
UC-A05 Editar perfil	UC-07
UC-A06 Ver avisos	— (caso nuevo)
—	UC-06, elección de perfil: se retiró

Hallazgos y pendientes

1. **El Capitán con WhatsApp vacío no se bloquea (RF-A17).** `casos-de-uso.md` marca UC-A05 como «Implementado» y dice que un Capitán de un equipo no disuelto no puede dejar el WhatsApp vacío. No hay nada que lo haga cumplir: la acción `updateProfile` guarda `null` sin consultar los equipos, y la base solo tiene un trigger (`on_auth_user_created`). Consecuencia: un Capitán puede borrar su WhatsApp y el rival ya no tendría cómo coordinar. Por eso documenté el caso como **Parcial**. Hay que decidir dónde se comprueba (lo razonable es en la base, junto a las demás reglas de WhatsApp) y corregir el estado en `casos-de-uso.md` .
2. **Mensajes de error inconsistentes.** Iniciar sesión con datos incorrectos muestra «Invalid Credentials» (en inglés, tal cual está en el código); registrarse con un rechazo muestra «Error en registro», genérico. El resto de la app está en español y con mensajes específicos. Los documenté tal cual; conviene decidir el texto definitivo.
3. **Guardar el perfil no es atómico.** Nombre, Zona y foto se guardan en `profiles` y el WhatsApp después, en `profile_contacts` . Si el segundo paso falla, el primero ya quedó guardado (el mensaje lo dice). Es aceptable, pero queda documentado como UC-A05 A8.

4. **Recuperar contraseña (UC-A04) sigue pendiente a propósito.** Antes de implementarlo hay que decidir qué se responde si el email no tiene cuenta y si la persona queda con sesión al cambiar la contraseña.
5. **Capturas faltantes:** Mi perfil, la barra con sesión y la campanita, que requieren una cuenta con sesión. Hay una lista con los pasos en 03-pantallas.md. Las de Iniciar sesión y Crear cuenta ya están, sacadas de la app el 04/11/2026.
6. **Tests de base:** hay pgTAP para las fotos de perfil (`profile_avatars_test.sql`) y los avisos (`notices_test.sql`), y pruebas unitarias para `safeNextPath` , `loginScreen` , `normalizeWhatsapp` y `parseAvatar` . No encontré pruebas pgTAP de las policies de `profiles` y `profile_contacts` (que solo el dueño edite lo suyo y solo las columnas permitidas). La regla del proyecto es que toda policy de RLS tenga su test.
7. **Si se cambia un diagrama,** hay que regenerar su PNG (el bloque Mermaid de cada archivo es la fuente).

Diagramas de clases: trazabilidad de las clases

Clase	Estereotipo	Implementación en el repo	Estado
`ControlDeAcceso`	`«boundary»`	`src/proxy.ts`	Implementado
`FormularioAcceso`	`«boundary»`	`src/app/login/page.tsx` + `src/utils/login-screen.ts`	Implementado
`FormularioRecuperar` / `FormularioNuevaContrasena`	`«boundary»`	— (diseño objetivo de UC-A04)	**Pendiente**
`FormularioPerfil`	`«boundary»`	`src/app/perfil/page.tsx`	Implementado
`MenuDeUsuario`	`«boundary»`	`src/components/UserMenu.tsx`	Implementado
`Campanita`	`«boundary»`	`src/components/NoticeBell.tsx`	Implementada
`ServicioCorreo`	`«boundary»`	— (diseño objetivo de UC-A04)	**Pendiente**
`CuentaService`	`«control»`	Server Actions `signup` , `login` , `logout` en `src/app/login/actions.ts`	Implementado (falta UC-A04)
`PerfilService`	`«control»`	Server Action `updateProfile` en `src/app/perfil/actions.ts`	Parcial (falta RF-A17)
`AvisoService`	`«control»`	Lectura de avisos en `src/app/layout.tsx` + Server Action `markNoticesRead` en `src/app/avisos/actions.ts`	Implementado
`ValidadorCuenta`	`«validador»`	`safeNextPath` (`src/utils/safe-next-path.ts`), `normalizeWhatsapp` (`src/utils/whatsapp.ts`), `parseAvatar` (`src/utils/profile-avatar.ts`)	Implementado
`CuentaRepository`	`«repositorio»`	Supabase Auth (`signUp` , `signInWithPassword` , `signOut({ scope: 'local' })` , `getUser`) + trigger `handle_new_user`	Implementado
`ZonaRepository`	`«repositorio»`	`getZoneGroups()` en `src/components/ZoneSelect.tsx` (`public.zones` , `corridors` , `regions`)	Implementado
`PerfilRepository`	`«repositorio»`	`saveProfile` (`src/utils/supabase/profile.ts`), tablas `public.profiles` y `public.profile_contacts` , bucket `avatars`	Parcial (falta RF-A17)

Clase	Estereotipo	Implementación en el repo	Estado
`AvisoRepository`	`«repositorio»`	Tabla `public.notices` con RLS + función `public.mark_notices_read` (`20261028000001_notices.sql`)	Implementado
`Usuario` / `Administrador` / `Sesion`	`«entidad»`	`auth.users` , `public.admins` y sesiones de Supabase Auth	Implementadas
`Perfil` / `ContactoPerfil`	`«entidad»`	`public.profiles` , `public.profile_contacts` (`20260928000002_profiles.sql`)	Implementadas
`Aviso`	`«entidad»`	`public.notices` + `noticeText` , `noticeLink` , `noticeDate` (`src/utils/notices.ts`)	Implementada
`Zona` / `Corredor` / `Region`	`«entidad»`	`public.zones` , `public.corridors` , `public.regions` (`20260928000001_geography.sql`)	Implementadas

Módulo B · Catálogo de torneos

Primero los requerimientos del módulo completo. Después, el modelo de dominio y, para cada caso de uso, su diagrama de secuencia, su diagrama de clases y su pantalla. Al final, los anexos y la trazabilidad.

Parte 1 · Requerimientos

Versión del **04/11/2026**. Es la primera parte del módulo: sigue la sección de casos de uso, donde cada caso lleva debajo su diagrama de secuencia y su pantalla. El README tiene la matriz que los une.

Fuentes, en orden de autoridad: decisiones-casos-de-uso.md, casos-de-uso.md (módulo B, 03/11/2026), el glosario CONTEXT.md y el código y las migraciones del repo al día de hoy. **Donde `casos-de-uso.md` y el código no coinciden, este documento lo señala** (ver la sección «Sin decisión»).

1. Alcance

El módulo es la parte **pública** de los torneos: cualquiera, sin sesión, **encuentra torneos** en el catálogo (`/`), los **filtra**, abre la **ficha** de uno, **contacta al organizador** y mira la **página pública del organizador**. Cualquier usuario con sesión puede además **reportar** un torneo (pendiente). El flujo **termina en un deep link a WhatsApp o Instagram del organizador**.

Límite de producto: la app **nunca inscribe ni cobra**. La inscripción se arregla por fuera y la carga el organizador (módulo E). Que la ficha no tenga un botón «Anotarme» es un límite, no una función faltante.

Fuera del módulo:

Tema	Dónde está
Publicar, editar, verificar y dar de baja un torneo; cargar inscripciones	Módulo E — Organizador
Resolver un Reporte (descartar, marcar Observado, dar de baja)	Módulo F — Administración
Crear cuenta o iniciar sesión para reportar	Módulo A — Cuenta y acceso
Cómo se calcula el Estado del torneo	La base de datos (función `status`); este módulo solo lo muestra

2. Actores

Actor	Quién es	Qué hace en este módulo
Invitado	Persona sin sesión	Consulta el catálogo, filtra, abre fichas, contacta al organizador, mira la página del organizador.
Usuario	Persona con sesión	Todo lo del Invitado, y reporta un torneo.
Organizador	Usuario que publicó un torneo	Ve su propia ficha con la barra «Este torneo es tuyo · Gestionar».
Sistema	La base de datos y la app	Calcula el Estado, la antigüedad de verificación y «Últimos cupos» al momento de leer.

3. Requerimientos funcionales

Estado: `Implementado` está en `main` . `Parcial` tiene una parte sin cumplir, que se indica. `Pendiente` es alcance vigente sin programar.

Catálogo

ID	Requerimiento	Casos de uso	Estado
RF-B01	`/` es el **catálogo público**: lista los torneos **no finalizados**, ordenados por **fecha de inicio** (el que empieza antes, primero). No hay «vista de jugador» ni «de organizador».	UC-B01	Implementado
RF-B02	El listado está **paginado**.	UC-B01	**Pendiente**: hoy se traen todos los torneos de una vez (ver «Sin decisión»)
RF-B03	Cada tarjeta muestra el **flyer** (o un marcador), el nombre, la Zona con su corredor, la modalidad, el precio, el **Estado del torneo** y la **antigüedad de verificación**; y toda la tarjeta es el link a la ficha.	UC-B01	Implementado
RF-B04	**Precio:** vacío se muestra como «Consultar» y 0 como «Gratis».	UC-B01, UC-B03	Implementado
RF-B05	**Antigüedad de verificación:** desde 30 días completos sin verificar, en ámbar; desde 60, en rojo con «Sin confirmar». Un Finalizado no la muestra.	UC-B01, UC-B03	Implementado
RF-B06	**«Últimos cupos»:** etiqueta sobre un torneo **Abierto** con 2 lugares o menos. **No es un Estado.**	UC-B01	Implementado
RF-B07	Los torneos **Finalizados no aparecen en el catálogo**; su ficha sigue accesible.	UC-B01, UC-B03	Implementado

Filtros

ID	Requerimiento	Casos de uso	Estado
RF-B08	Filtrar por **modalidad** (Fútbol 5, 7, 8 u 11) y por **Estado** (inscripción abierta, en curso, inscripción cerrada). Los filtros se combinan (AND).	UC-B02	Implementado
RF-B09	Filtrar por **Zona** o por **Corredor** completo (elegir un corredor incluye todas sus Zonas).	UC-B02	**Pendiente**
RF-B10	Filtrar por **precio máximo** y por **fechas de inicio**.	UC-B02	**Pendiente** (solo figura en `casos-de-uso.md`)
RF-B11	Los filtros viven en la **URL** (`?modality=…&status=…`), nunca en estado local: una vista filtrada es siempre un **link compartible**, y cambiar un filtro vuelve a la primera página.	UC-B02	Implementado
RF-B12	Un parámetro desconocido en la URL se ignora sin romper la página.	UC-B02	**Parcial**: una clave desconocida se ignora, pero un valor inválido de `modality` o `status` no se valida antes de consultar
RF-B13	Sin resultados, mostrar un mensaje y la forma de limpiar los filtros.	UC-B02	**Parcial**: hay mensaje; limpiar es elegir «Todas las modalidades» y «Todos los estados» (no hay un botón «Limpiar»)

Ficha del torneo

ID	Requerimiento	Casos de uso	Estado
RF-B14	La **ficha pública** (`/torneos/{id}`) muestra flyer, nombre, organizador (con link a su página), modalidad, Zona, **dirección de la sede** (con link al mapa), cierre de inscripción, inicio y fin, precio, Estado, descripción, reglamento, antigüedad de verificación, cupo e inscriptos.	UC-B03	Implementado
RF-B15	La ficha muestra los **nombres de los equipos anotados**; los de una Inscripción vinculada enlazan a la página del equipo. Solo nombres: nunca contactos.	UC-B03	Implementado
RF-B16	Para el **organizador dueño** la ficha es idéntica a la de cualquiera, más una barra «Este torneo es tuyo · Gestionar».	UC-B03	Implementado
RF-B17	Un torneo **dado de baja** devuelve **404 con un link al catálogo**.	UC-B03	**Parcial**: devuelve 404, pero es la página genérica de Next, en inglés y sin link
RF-B18	Un torneo **Finalizado** se muestra **sin botones de contacto** («Este torneo ya terminó»).	UC-B03	**Pendiente**: hoy la ficha de un Finalizado muestra los botones
RF-B19	La URL de la ficha es **estable** y la página tiene **metadatos** (flyer, nombre, Zona) para la vista previa al compartir por WhatsApp.	UC-B03	**Parcial**: la URL es estable; no hay metadatos propios
RF-B20	La ficha de un torneo es **de lectura pública**, con o sin sesión.	UC-B03	Implementado

Contacto y organizador

ID	Requerimiento	Casos de uso	Estado
RF-B21	**Contactar por WhatsApp** abre `wa.me` con el número del organizador y un **mensaje sugerido** editable; **Ver Instagram** abre el perfil. Ahí termina el flujo en la app.	UC-B04	Implementado (el texto del mensaje difiere del documentado)
RF-B22	Si falta un medio de contacto, ese botón no se muestra; si faltan los dos, «El organizador no suministró datos de contacto directo».	UC-B04	Implementado
RF-B23	La **página del organizador** (`/organizadores/{id}`) muestra nombre, foto, Zona, «Organiza desde…», sus torneos vigentes y su trayectoria (finalizados), **sin ningún contacto personal**.	UC-B05	Implementado

Reportar

ID	Requerimiento	Casos de uso	Estado
RF-B24	Un Usuario con sesión puede **reportar un torneo** con un motivo («Ya no existe», «Datos incorrectos» o «No es un torneo»), **una sola vez por usuario y torneo**. Cada usuario ve solo sus reportes.	UC-B06	**Pendiente**

Captación

ID	Requerimiento	Casos de uso	Estado
RF-B25	El link «¿Organizás torneos? Publicalo» va **al pie del catálogo y en el footer**, nunca en la barra superior.	UC-B01	Implementado

4. Requerimientos no funcionales

ID	Requisito	Evidencia
RNF-B01	**Lectura pública en la base (RLS).** Los torneos se leen sin sesión; solo el organizador crea, edita y da de baja los suyos.	`20260930000001_tournaments_owner_only_rls.sql`
RNF-B02	**Los inscriptos son públicos pero solo por nombre**, a través de una vista; la tabla de inscripciones solo la lee el organizador del torneo.	Vista `tournament_inscriptions` (`20261007000000_inscriptions.sql`)
RNF-B03	**El Estado, la antigüedad y los cupos se calculan al leer**, con la hora actual: no hay tareas programadas. El Estado es una columna calculada de la base (`status`, `inscriptions_count`), que hay que pedir explícitamente.	`20260930000002_tournaments_computed_status.sql`
RNF-B04	**La antigüedad se calcula en el servidor** y la tarjeta la recibe hecha: si la calculara el navegador, el «ahora» del servidor y el del navegador podrían no coincidir al hidratar.	`TournamentList.tsx`, `tournament-verification.ts`
RNF-B05	**Una vista filtrada es un link**: los filtros se leen en el servidor desde la URL (Server Components), sin estado local.	`page.tsx`, `Filters.tsx`
RNF-B06	**El flyer solo se muestra si es del bucket propio**; una URL ajena cuenta como «sin flyer» y no rompe la página.	`tournament-flyer.ts`, `TournamentFlyer.tsx`
RNF-B07	**Privacidad del organizador.** La página pública pide solo las columnas que muestra: ni el WhatsApp del perfil ni el contacto de los torneos llegan a ella.	`organizadores/[id]/page.tsx`
RNF-B08	**Carga progresiva.** El catálogo y los filtros están en `Suspense` con esqueletos; la ficha tiene `loading.tsx`.	`page.tsx`, `torneos/[id]/loading.tsx`
RNF-B09	**Accesibilidad.** Toda la tarjeta es un solo link (el blanco para tocar es la tarjeta entera) con foco visible; los avisos de estado usan `role`.	`TournamentCard.tsx`
RNF-B10	**Lógica pura verificable:** `priceLabel`, `verificationAge`, `isLastSpots`, `splitTournaments`, `organizesSince`, `whatsappLink`, con sus `*.test.ts` (`pnpm test`).	`src/utils/`

5. Sin decisión: lo que `casos-de-uso.md` dice y el código no hace

Estas diferencias salieron de comparar `casos-de-uso.md` (marca todo el módulo B como «Implementado», salvo el reporte) con el código. Hay que decidir, caso por caso, si se programa lo que falta o se corrige el documento.

Tema	Lo que dice `casos-de-uso.md`	Lo que hace el código
Paginación	El listado está paginado (UC-B01).	`TournamentList` trae todos los torneos; el filtro borra `page` de la URL pero no existe.

Tema	Lo que dice `casos-de-uso.md`	Lo que hace el código
Filtros	Modalidad, estado, **Zona o Corredor**, **precio máximo** y **fechas de inicio** (UC-B02). `decisiones-casos-de-uso.md` define `?zona=` (una Zona o un Corredor).	Solo modalidad y estado. La rama `feat/dynamic-filters` tiene un filtro por Zona (por texto) y por rango de precio, pero **no está integrada** a `main` y es anterior a la tabla `zones`.
Finalizados en el catálogo	UC-B01 A4: «solo aparecen si se filtra explícitamente por ese estado». La regla transversal dice que el Finalizado **sale del catálogo**.	La consulta excluye siempre los Finalizados y el filtro no ofrece ese Estado: **nunca aparecen**. Coincide con la regla transversal, no con A4.
Sin resultados	«No hay torneos con esos filtros» y la opción de limpiar.	«No hay torneos registrados», con el texto «Probá cambiando las opciones en el panel lateral o limpiando los filtros». Sin botón.
Ficha de un Finalizado	Sin botones de contacto; «Este torneo ya terminó».	Muestra los botones de contacto como en cualquier otra ficha.
Torneo dado de baja	404 con un link al catálogo.	404 genérico de Next («This page could not be found.»), sin link.
Vista previa al compartir	La página tiene metadatos (flyer, nombre, Zona).	No hay `generateMetadata` en la ficha; usa los metadatos generales del sitio.
Mensaje de WhatsApp	«Hola, vi el torneo [nombre] en Torneos BA y quiero anotar a mi equipo».	«Hola, vengo de Torneos Scraper. Me interesa el torneo "[nombre]"!». También el nombre del producto difiere: la interfaz dice «Torneos Scraper» y la documentación, «Torneos BA».
Valor inválido en la URL	Se ignora ese filtro y se mantienen los válidos.	Una clave desconocida se ignora; un valor de `modality` o `status` no se valida antes de consultar.

6. Reglas de referencia

- **Estado del torneo: Abierto** (no pasó el cierre y hay lugar), **Cerrado** (pasó el cierre sin empezar, o se completó el cupo), **En curso** (entre inicio y fin), **Finalizado** (pasó el fin; sale del catálogo y no se puede dar de baja). En la interfaz: «Inscripción Abierta», «Inscripción Cerrada», «En Juego» y «Torneo Finalizado».
- **Visibilidad:** catálogo, ficha, inscriptos (solo nombres) y página del organizador son públicos; un contacto personal del organizador nunca se muestra.
- **Términos:** se dice «baja» (nunca «eliminar»); «Zona» (nunca «barrio» ni «localidad»); «Inscripción vinculada» e «Inscripción por nombre».

Parte 2 · Modelo de dominio y casos de uso, con sus diagramas y su pantalla

Versión del **04/11/2026**. Detalla los casos de uso del módulo B de casos-de-uso.md (UC-B01 a UC-B06) con los requerimientos que cubre cada uno (01-requerimientos.md), la pantalla que lo representa (03-pantallas.md) y su diagrama de secuencia (04-diagramas-secuencia).

Cómo leerlo. Cada caso expresa una meta y tiene **un actor iniciador**. **Estado** dice si está en `main`. Cuando el código **no coincide** con lo que dice `casos-de-uso.md`, la diferencia se explica en el resumen de «Sin decisión».

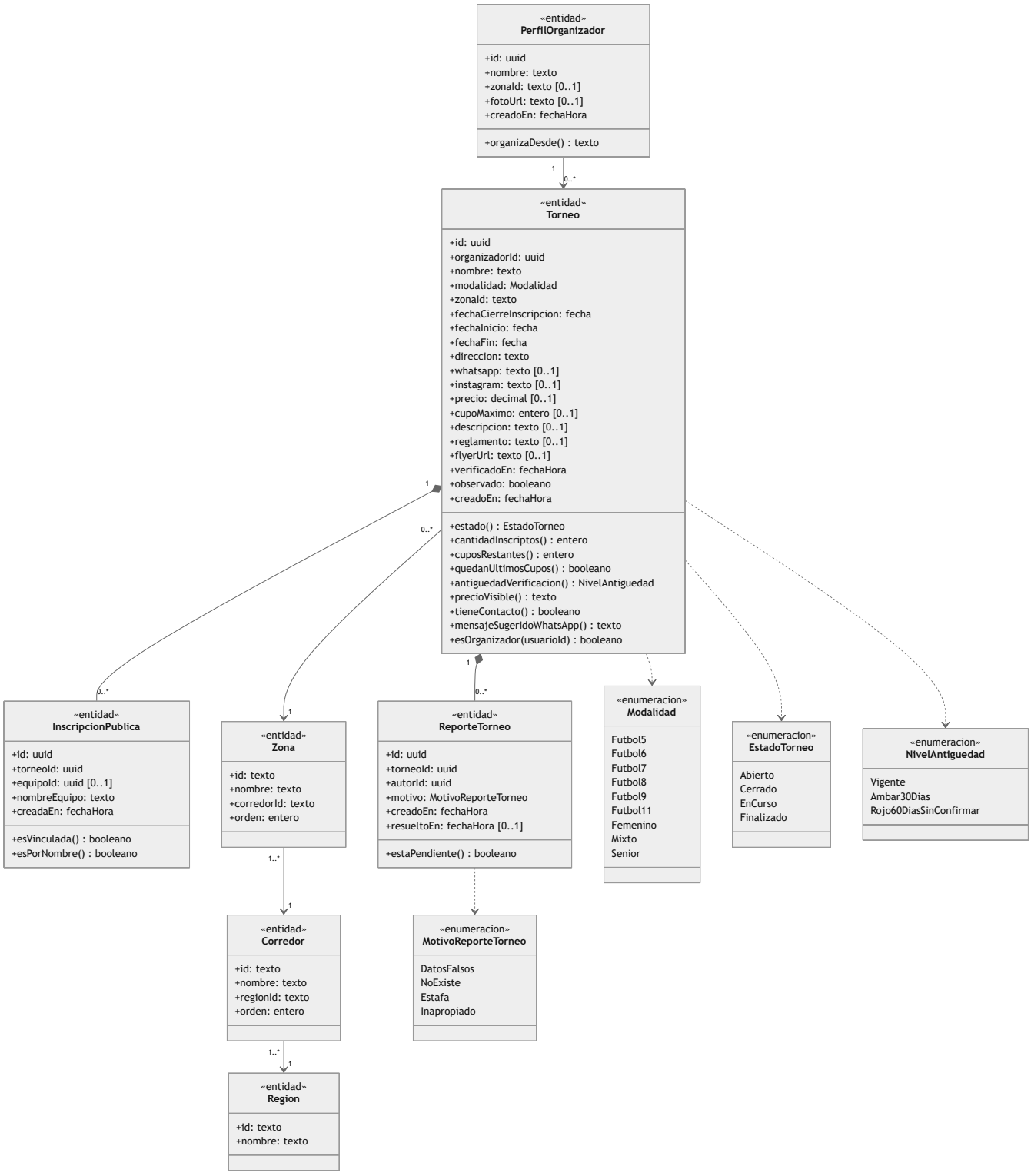
Mapa de casos de uso

Caso	Meta del actor	Actor	Estado	ID en el PDF original
UC-B01	Ver el catálogo de torneos	Invitado	**Parcial** (sin paginar)	UC-08
UC-B02	Filtrar torneos y compartir la búsqueda	Invitado	**Parcial** (solo 2 de los 5 filtros)	UC-09 y UC-10
UC-B03	Ver la ficha pública de un torneo	Invitado	**Parcial** (3 salvedades)	UC-11
UC-B04	Contactar al organizador	Invitado	Implementado	UC-12
UC-B05	Ver la página pública del organizador	Invitado	Implementado	SPEC-E04
UC-B06	Reportar un torneo	Usuario	**Pendiente**	—

Qué no es un caso de uso acá: el `loading` de la ficha, navegar del catálogo a una ficha o volver («← Volver al catálogo») no toman ninguna decisión.

Diagrama de clases · **Modelo de dominio del módulo**

Vista estructural de las entidades, relaciones de composición/asociación y enumeraciones que comparten los casos de uso `UC-B01` a `UC-B06`.



- **Todo torneo tiene exactamente un Organizador (`organizadorId`):** no existe el «torneo copiado» sin organizador (descartado el 07/10/2026).
- **Estado, cupos y antigüedad se calculan al leer (RNF-B03, RNF-B04):** `estado()` y `cantidadInscriptos()` son columnas calculadas de la base (`status` , `inscriptions_count`); «Últimos cupos»

(`quedanUltimosCupos()`) es una etiqueta sobre un torneo `Abierto` con 2 lugares o menos, no un Estado.

- **Privacidad de inscriptos y del organizador (RNF-B02, RNF-B07):** `InscripcionPublica` sale de la vista `tournament_inscriptions` (solo expone nombres de equipo y el `equipoId` si es vinculada); `PerfilOrganizador` lee solo datos públicos de `profiles`, nunca `profile_contacts`.

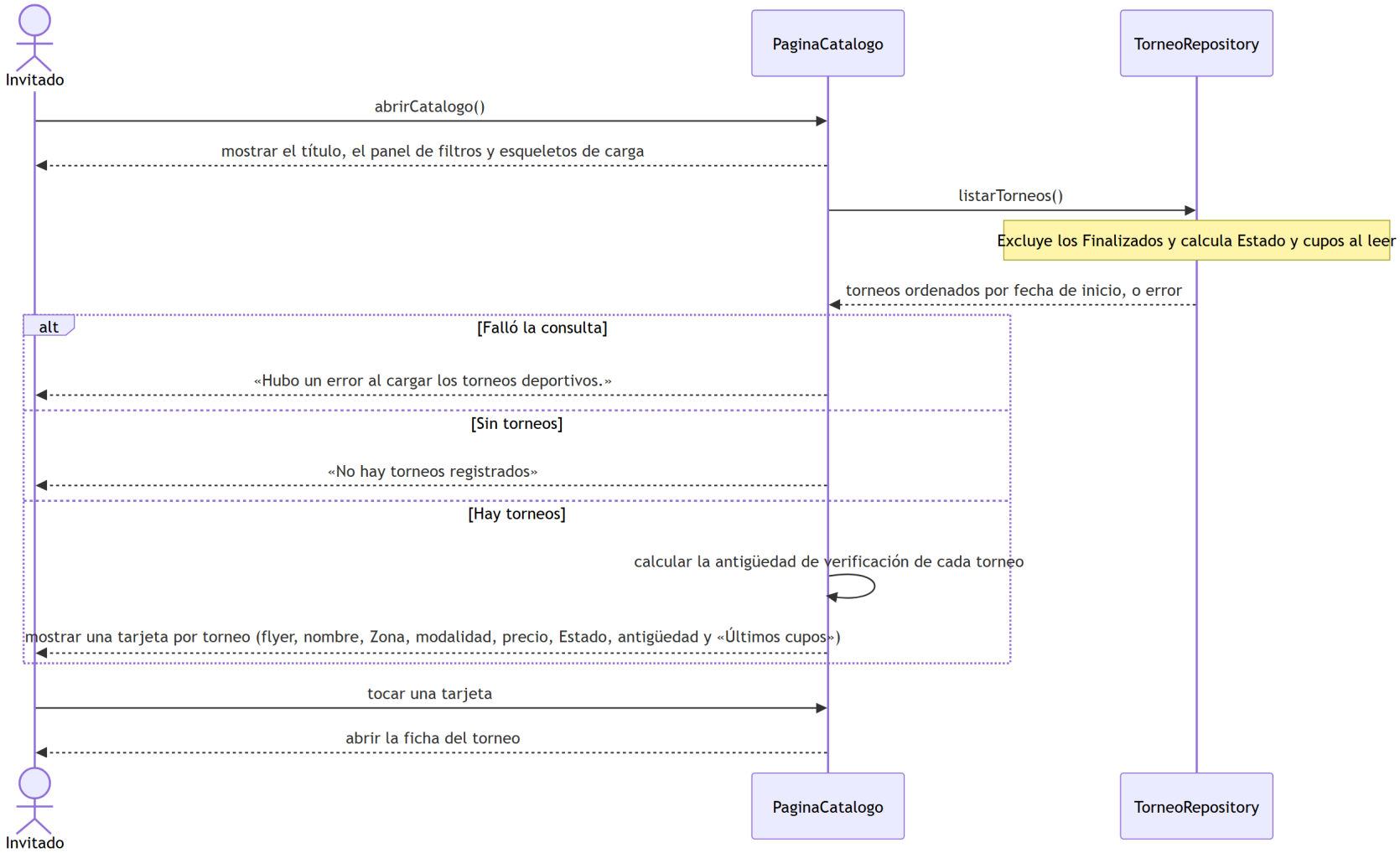
UC-B01 · Ver el catálogo de torneos

- **Actor:** Invitado (y cualquier usuario).
- **Requerimientos:** RF-B01 a RF-B07, RF-B25. **Pantalla:** P1 · Catálogo (`/`). **Diagrama:** S01.
- **Precondiciones:** ninguna. `/` es el catálogo para cualquiera.
- **Postcondición:** la persona ve los torneos vigentes y puede abrir uno.
- **Estado: Parcial** (`src/app/page.tsx`, `TournamentList`, `TournamentCard`): todo salvo la paginación.

Diagrama de secuencia · **S01 · UC-B01 — Ver el catálogo de torneos**

Caso de uso: UC-B01 · Pantalla: P1 · Requerimientos: RF-B01 a RF-B07

Es una consulta: no hay formulario. Los filtros son otro caso de uso (S02); acá se ve el catálogo sin filtros.



Cómo se lee

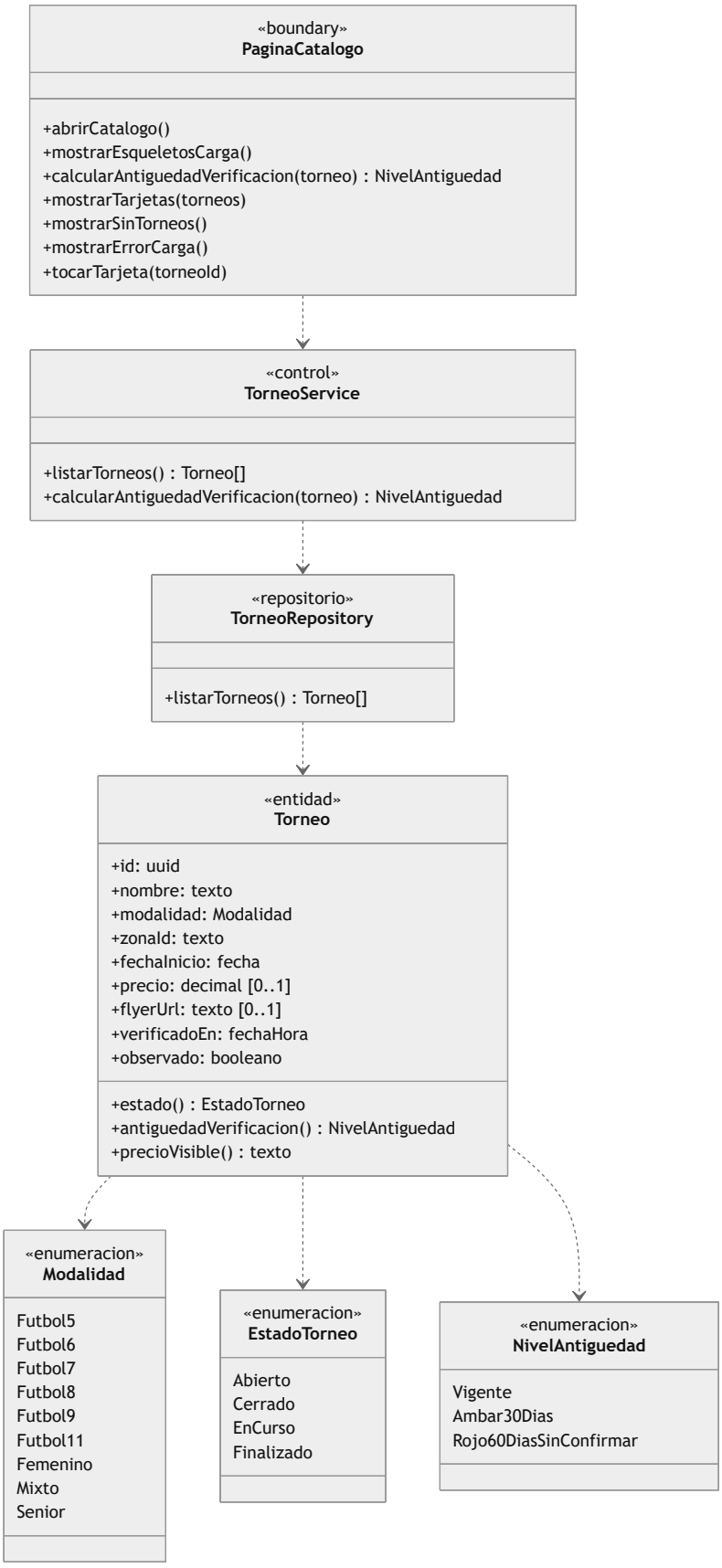
- **Lectura pública:** `listarTorneos` no necesita sesión; la base permite leer los torneos a cualquiera (RLS).
- **Estado y cupos los calcula la base al leer**, con la hora actual: nadie los elige y no hay tareas programadas. Por eso se piden explícitamente (`status` , `inscriptions_count`). «Últimos cupos» sale de ahí: torneo **Abierto** con 2 lugares o menos.
- **Los Finalizados nunca salen** en el catálogo; su ficha sigue accesible por link. (`casos-de-uso.md` A4 dice otra cosa: ver «Sin decisión».)
- **La antigüedad se calcula en el servidor** y la tarjeta la recibe hecha, para que el «ahora» del servidor y el del navegador no difieran al hidratar. Desde 30 días es ámbar; desde 60, rojo con «Sin confirmar».
- **Sin paginación:** hoy `listarTorneos` devuelve todos los torneos. `casos-de-uso.md` dice que está paginado.
- El panel de filtros y el listado cargan por separado (`Suspense`): el panel aparece primero.

Correspondencia en el código

`Home` (`src/app/page.tsx`) renderiza `Filters` y `TournamentList` ; `TournamentList` hace `from('tournaments').select('*, status, inscriptions_count, zones(...)').neq('status','finalizado').order('start_date')` .

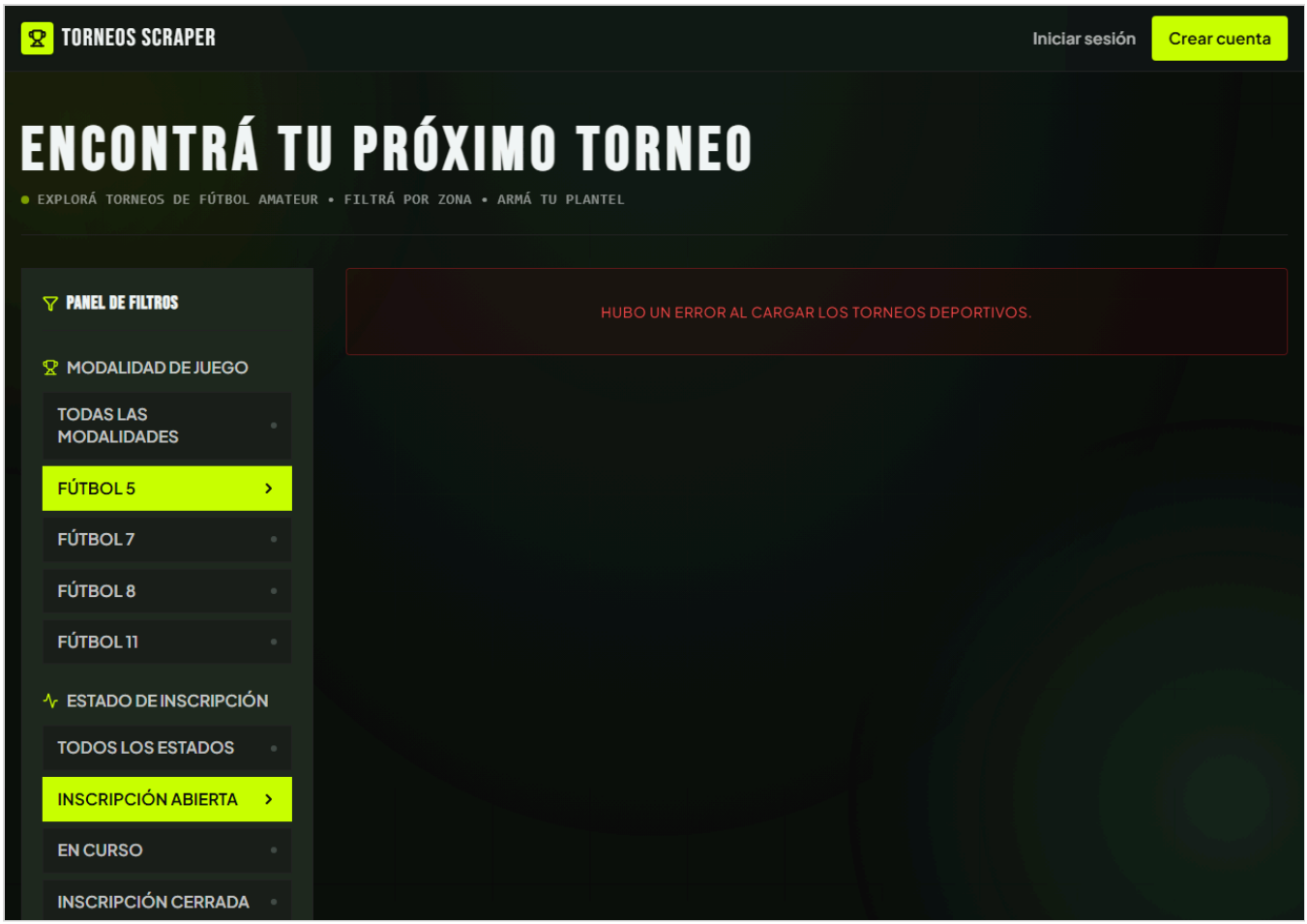
Diagrama de clases · **UC-B01 · Ver el catálogo de torneos**

Caso de uso: UC-B01 · **Pantalla:** P1 · Catálogo (`/`) · **Secuencia:** S01



> Pantalla · **P1 · Catálogo**

Ruta: `/` · **Casos de uso:** UC-B01, UC-B02 · **Requerimientos:** RF-B01 a RF-B13, RF-B25 · **Diagramas:** S01, S02



*Captura ✅ vigente. URL: `/?modality=futbol_5&status=abierto`. Muestra el panel de filtros con «Fútbol 5» e «Inscripción abierta» activos y, a la derecha, el **estado de error de carga** («Hubo un error al cargar los torneos deportivos.») porque la base no respondía. Con la base disponible, ahí van las tarjetas.*

Qué muestra. El título «Encontrá tu próximo torneo» y dos columnas:

- **Panel de filtros** (izquierda, fijo al desplazarse en pantalla grande), con dos grupos de botones:

Grupo	Opciones	De dónde salen las opciones
Modalidad de juego	Todas las modalidades · Fútbol 5 · Fútbol 7 · Fútbol 8 · Fútbol 11	**Lista fija del código** (`SPORT_MODALITIES`)
Estado de inscripción	Todos los estados · Inscripción abierta · En curso · Inscripción cerrada	**Lista fija del código** (en `Filters.tsx`)

  La opción activa se resalta en lima con una flecha. Elegir una opción reescribe la URL.

- **Listado** (derecha): una **tarjeta** por torneo, en una grilla de 1 a 3 columnas. Cada tarjeta es un link entero a la ficha y muestra: el flyer (o un marcador) con el **Estado** y, si corresponde, «Últimos cupos»; un número de orden y el nombre; la Zona con su corredor y la modalidad; la **antigüedad de verificación** (ámbar o rojo desde los 30 y 60 días) y el **precio** («Consultar» o «Gratis»); y el botón «Ficha». Los torneos cerrados se ven en escala de grises.
- **Pie:** «¿Organizás torneos? Publicalo» (lleva al panel; sin sesión, a Crear cuenta con el aviso).

Estados de la pantalla

Estado	Qué ve la persona
Con torneos	La grilla de tarjetas.
Cargando	Esqueletos animados del panel y de seis tarjetas.
Sin resultados	«No hay torneos registrados … Probá cambiando las opciones en el panel lateral o limpiando los filtros de búsqueda».
Error de carga	«Hubo un error al cargar los torneos deportivos.» ✅ (captura de arriba)

> ⏳ **Pendiente:** la grilla de tarjetas con datos reales, y el estado «Sin resultados». Tampoco hay paginación, ni filtros de Zona, precio o fechas (ver «Sin decisión»).

UC-B02 · Filtrar torneos y compartir la búsqueda

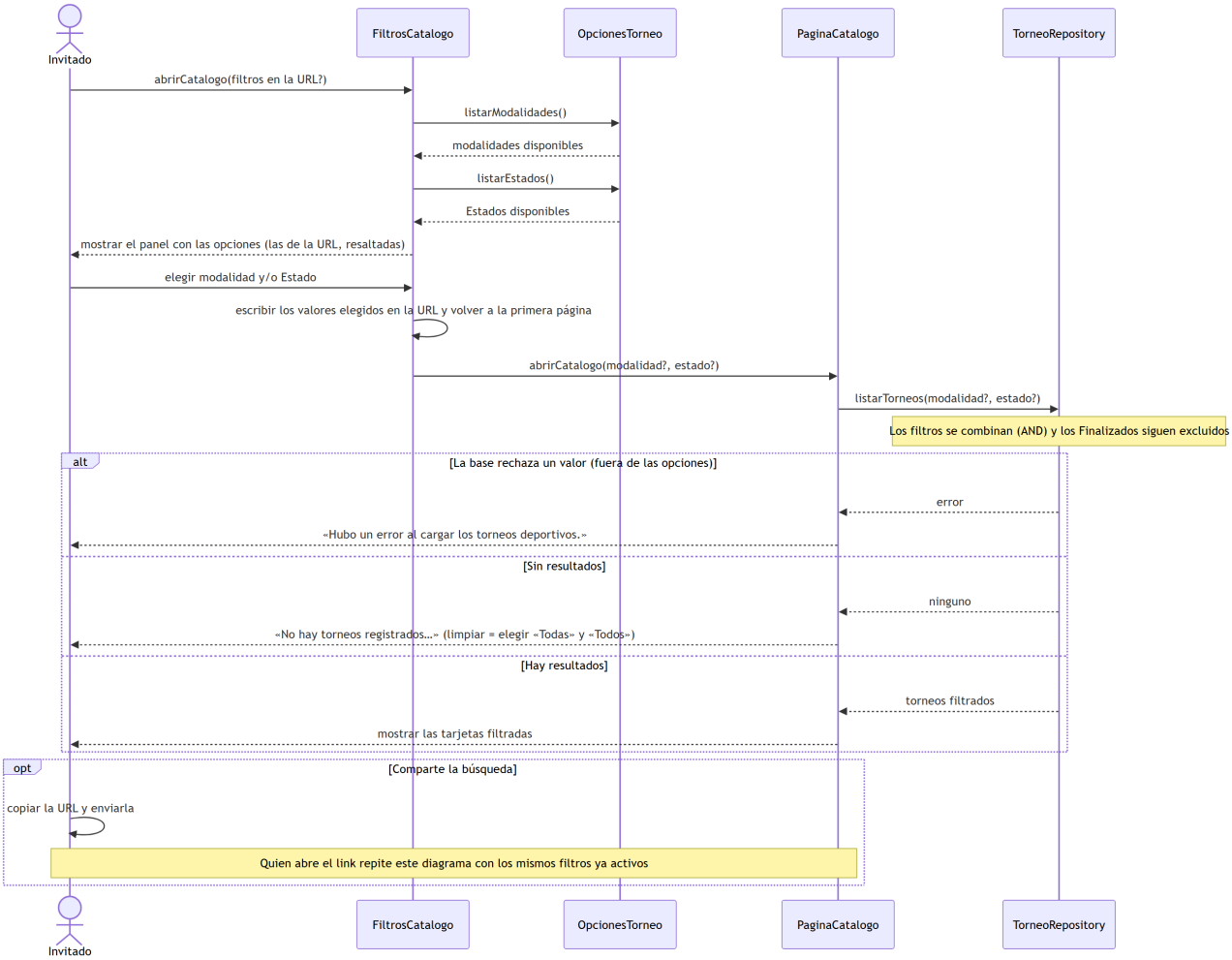
- **Actor:** Invitado.
- **Requerimientos:** RF-B08 a RF-B13. **Pantalla:** P1 · Catálogo (panel de filtros). **Diagrama:** S02.
- **Precondiciones:** está en el catálogo.
- **Postcondición:** el catálogo muestra solo los torneos que cumplen los filtros; la URL los refleja.
- **Estado: Parcial** (`Filters.tsx`): modalidad y estado. Faltan Zona o Corredor, precio máximo y fechas.

> Diagrama de secuencia · **S02 · UC-B02 — Filtrar torneos y compartir la búsqueda**

Caso de uso: UC-B02 · Pantalla: P1 (panel de filtros) · Requerimientos: RF-B08, RF-B11 a RF-B13

El caso empieza con **dos grupos de opciones** (modalidad y Estado). Siguiendo la guía de formularios, el panel obtiene las opciones **antes** de que la persona elija. Las dos listas son **fijas del código** (no se guardan en la base), así que las provee `OpcionesTorneo` y no un repositorio. La carga de las opciones y el filtrado van en el mismo diagrama.

> **Alcance real.** Hoy el panel filtra solo por **modalidad** y **Estado**. Zona o Corredor, precio máximo y fechas de inicio figuran en `casos-de-uso.md` pero **no están en** `main`: cuando entren, la Zona se consultará en la base (un `ZonaRepository`, como en Editar perfil) y agregará su lista a este diagrama.



Cómo se lee

- **Antes de elegir:** el panel muestra las opciones de `OpcionesTorneo` y resalta las que ya están en la URL. «Todas las modalidades» y «Todos los estados» quitan ese filtro.
- **Al elegir**, solo viajan los **valores elegidos** (`modalidad`, `estado`), nunca las listas. El panel los escribe en la URL y **vuelve a la primera página** (borra `page`).
- **Los filtros viven en la URL**, no en estado local: una vista filtrada es siempre un **link compartible**. Por eso compartir no necesita un paso aparte: quien abre el link repite la consulta.
- **La validación se repite al consultar… en teoría.** Un valor que el panel ofrece puede no seguir siendo válido, porque la URL se puede editar a mano. Hoy el servidor **no valida** `modality` y `status` antes de consultar: un

valor fuera de la lista lo rechaza la base y la persona ve el error de carga, en lugar de que se ignore ese filtro como dice `casos-de-uso.md`. Una **clave** desconocida (`?foo=1`) sí se ignora.

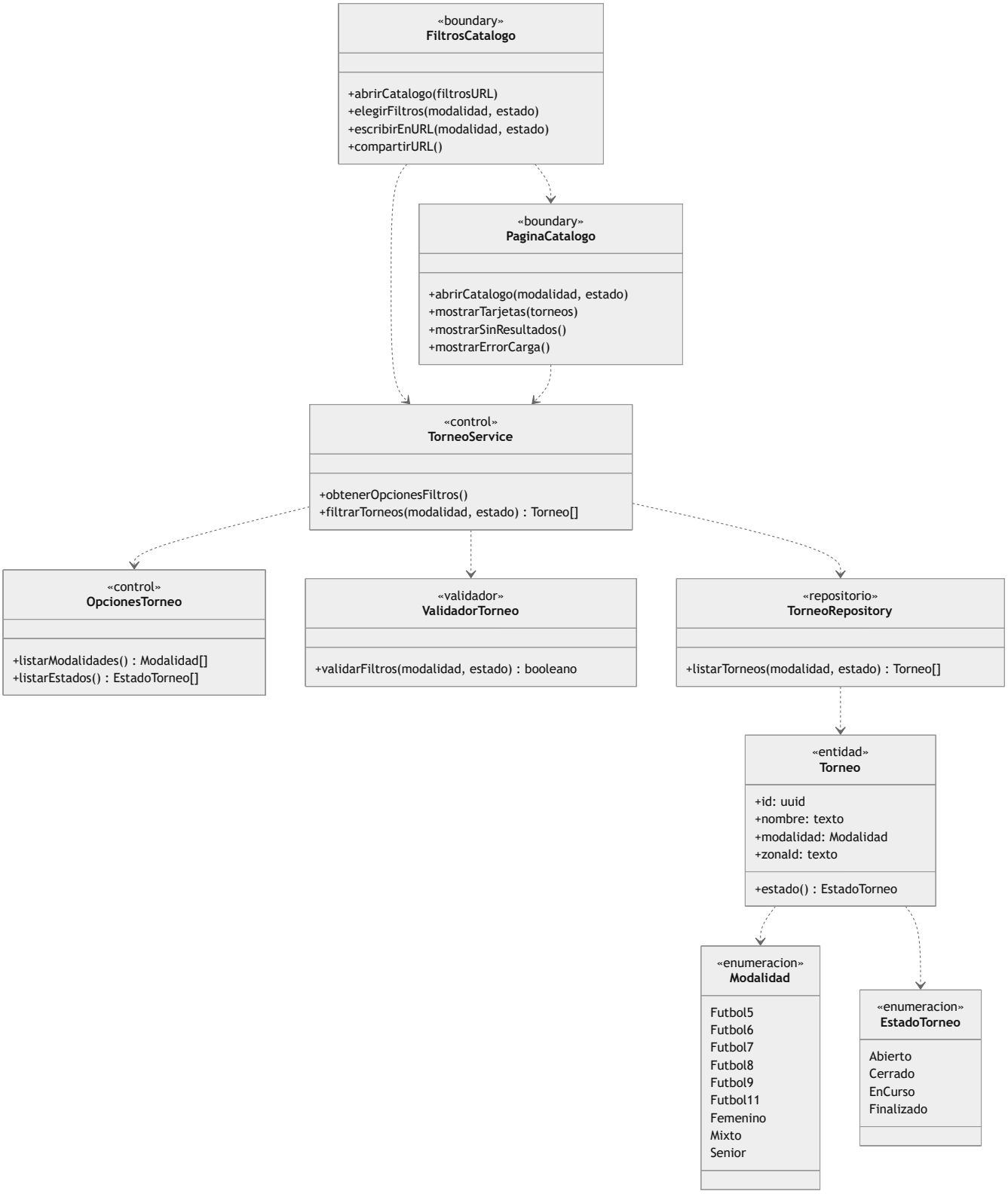
- Las dos listas son del código: si algún día modalidad o Estado se guardaran en una tabla, `OpcionesTorneo` se reemplazaría por un repositorio.

Correspondencia en el código

`Filters` (`src/components/Filters.tsx`) lee `useSearchParams`, arma la URL con `router.push` y borra `page`; `TournamentList` aplica `.eq('sport_modality', …)` y `.eq('status', …)`.

Diagrama de clases · **UC-B02 · Filtrar torneos y compartir la búsqueda**

Caso de uso: UC-B02 · **Pantalla:** P1 · Catálogo (panel de filtros) · **Secuencia:** S02



Pantalla · **P1 · Catálogo**

Es la misma pantalla que se describe, con su captura, en P1 · Catálogo, debajo de UC-B01 · Ver el catálogo de torneos. Este caso usa la parte que indica su ficha («Pantalla»).

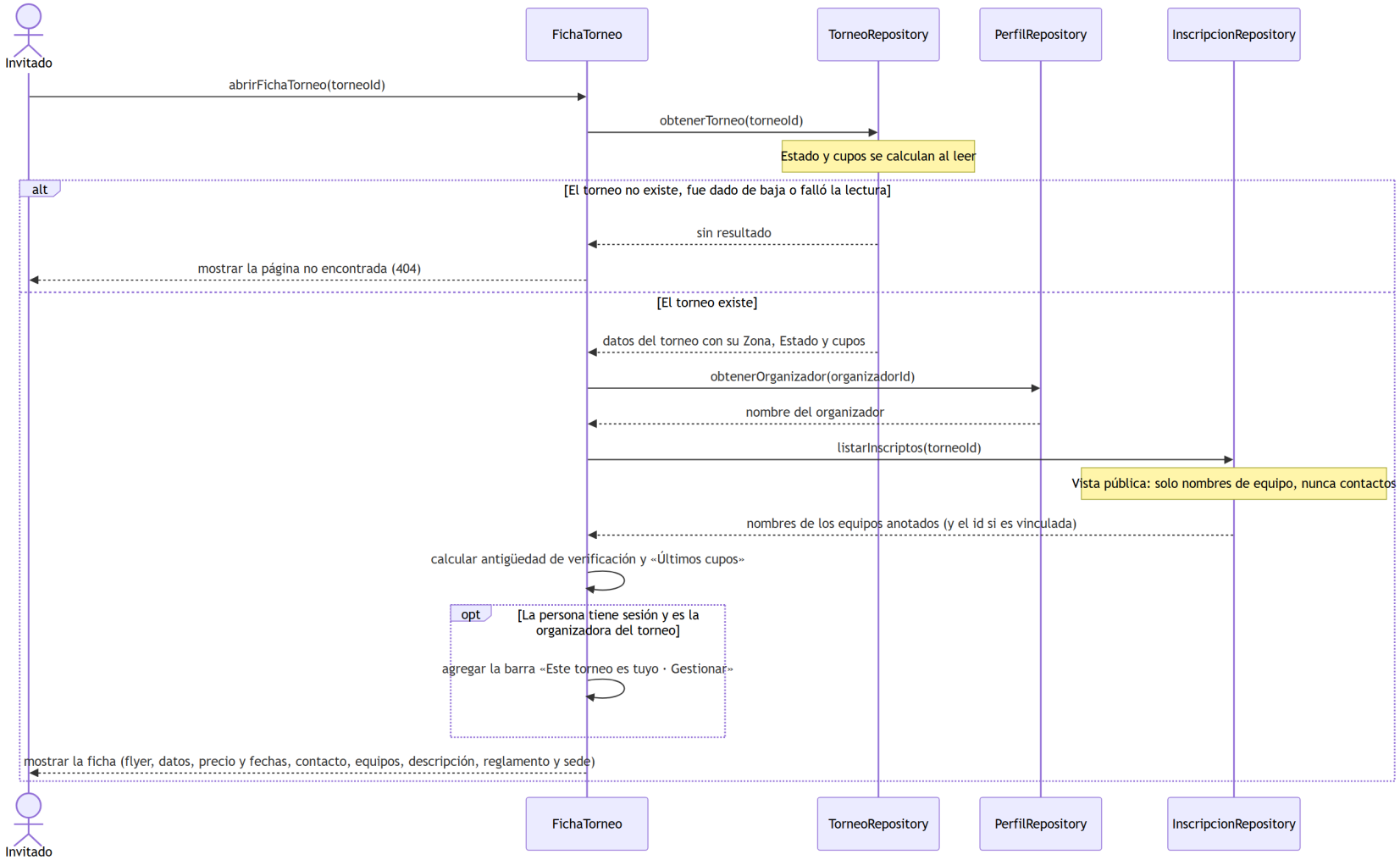
UC-B03 · Ver la ficha pública de un torneo

- **Actor:** Invitado.
- **Requerimientos:** RF-B14 a RF-B20. **Pantalla:** P2 · Ficha del torneo (`/torneos/{id}`). **Diagrama:** S03.
- **Precondiciones:** el torneo existe y no fue dado de baja.
- **Postcondición:** la persona conoce las condiciones del torneo y puede contactar al organizador.
- **Estado: Parcial** (`src/app/torneos/[id]`): funciona, pero con las salvedades A1, A2 y A4.

Diagrama de secuencia · **S03 · UC-B03 — Ver la ficha pública de un torneo**

Caso de uso: UC-B03 · Pantalla: P2 · Requerimientos: RF-B14 a RF-B20

Es una consulta: no hay formulario ni desplegables. Los botones de contacto son el caso siguiente (S04).



Cómo se lee

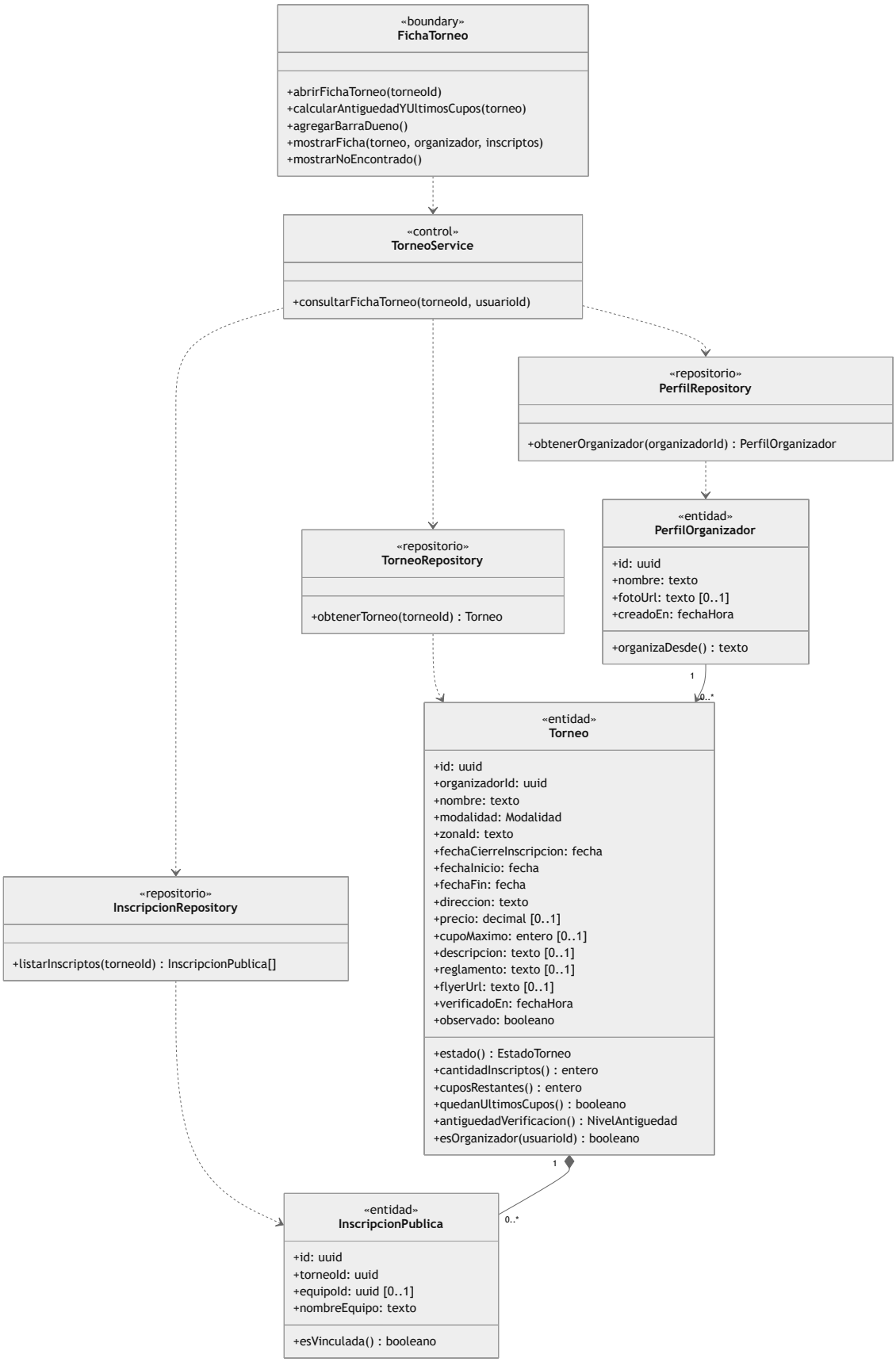
- **Qué se muestra:** todo lo del torneo más su organizador (con link a su página) y los **nombres** de los equipos anotados. Los inscriptos salen de una **vista pública** que expone solo nombres: la tabla de inscripciones solo la lee el organizador.
- **Los equipos de una Inscripción vinculada son un link** a su página (`team_id`); los de una Inscripción por nombre son solo texto.
- **404 cubre tres casos con el mismo resultado:** el torneo no existe, se dio de baja (su fila se borra) o la lectura falló. Hoy es la página 404 genérica de Next, sin link al catálogo (`casos-de-uso.md` pide uno).
- **Un torneo Finalizado se muestra** (su ficha sigue siendo pública). `casos-de-uso.md` dice que sin botones de contacto; hoy los muestra.
- **Para el dueño**, la ficha es idéntica a la de cualquiera salvo la barra «Este torneo es tuyo · Gestionar». Eso se decide comparando la sesión con el organizador del torneo; un Invitado nunca la ve.
- **Metadatos:** la ficha no define los suyos (flyer, nombre, Zona) para la vista previa de WhatsApp; usa los del sitio.

Correspondencia en el código

`TournamentDetail` (`src/app/torneos/[id]/page.tsx`): `from('tournaments').select('*, status, inscriptions_count, zones(...)')`, `from('profiles')` para el organizador, `from('tournament_inscriptions')` para los anotados, y `notFound()` si no hay torneo.

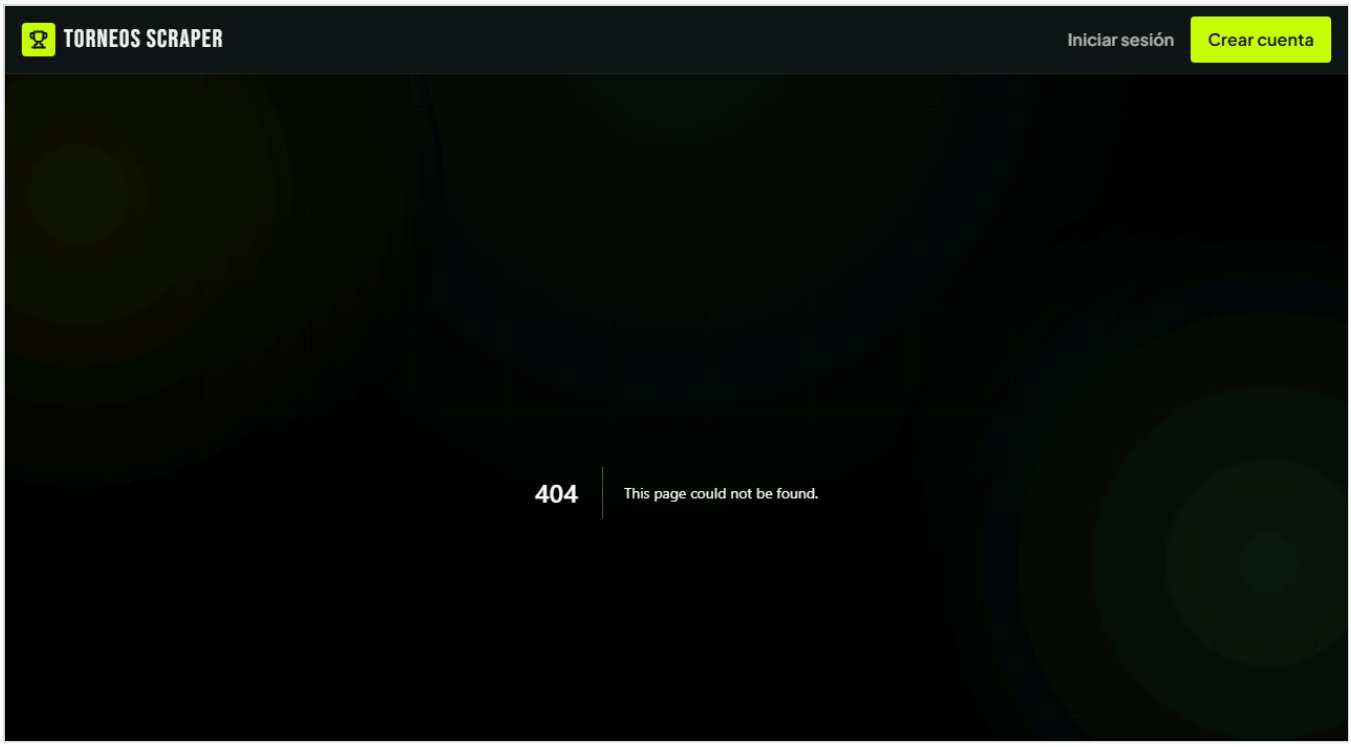
Diagrama de clases · **UC-B03 · Ver la ficha pública de un torneo**

Caso de uso: UC-B03 · **Pantalla:** P2 · Ficha del torneo (`/torneos/{id}`) · **Secuencia:** S03



Pantalla · **P2 · Ficha del torneo**

Ruta: `/torneos/{id}` · **Casos de uso:** UC-B03, UC-B04, UC-B06 (pendiente) · **Requerimientos:** RF-B14 a RF-B22, RF-B24 · **Diagramas:** S03, S04, S06



*Captura ✅ vigente del **404** de una ficha (`/torneos/00000000-0000-0000-0000-000000000000`). Es la página genérica de Next, en inglés y sin link al catálogo: ver UC-B03 A1.*

Qué muestra. Con flyer, dos columnas (el flyer a la izquierda en proporción 4:5, entero y sin recortar; la información a la derecha). Sin flyer, una sola columna. En orden:

1. **Barra de dueño** (solo para el organizador del torneo): «Este torneo es tuyo · Gestionar».
2. Link «← Volver al catálogo».
3. **Estado** y «Últimos cupos» (sobre el flyer, o arriba del título si no hay flyer).
4. **Título y chips:** Zona con corredor, modalidad, organizador (link a su página, P3) y antigüedad de verificación.
5. **Precio y fechas:** precio de inscripción, cierre de inscripción, fecha de inicio, fecha de finalización y cupo (o cantidad de inscriptos).
6. **Contacto:** «Contactar por WhatsApp» (botón principal) y «Ver Instagram». Es donde termina el flujo en la app. Sin ninguno: «El organizador no suministró datos de contacto directo».
7. **Equipos anotados:** una lista numerada de nombres; los vinculados son links a la página del equipo.
8. **Sobre el torneo** (descripción), **Reglamento** y **Sede** (la dirección, con link al mapa).

Con flyer, el orden es: título, precio y fechas, contacto, equipos anotados, descripción, reglamento y sede. Sin flyer, el contacto sube justo después de saber cuánto sale, cuándo y dónde se juega.

Mensajes extra solo para el dueño (recién publicado): «Publicado. Así lo ve la gente» y, si falló la subida, «No pudimos subir el flyer. Probá de nuevo desde Gestionar».

Estados: *normal*; *no existe o dado de baja* (404, captura); *finalizado* (hoy con los botones de contacto); *con y sin flyer*.

⏳ **Pendiente:** la ficha con datos reales, con y sin flyer, y la de un torneo finalizado. El botón «Reportar» no existe todavía.

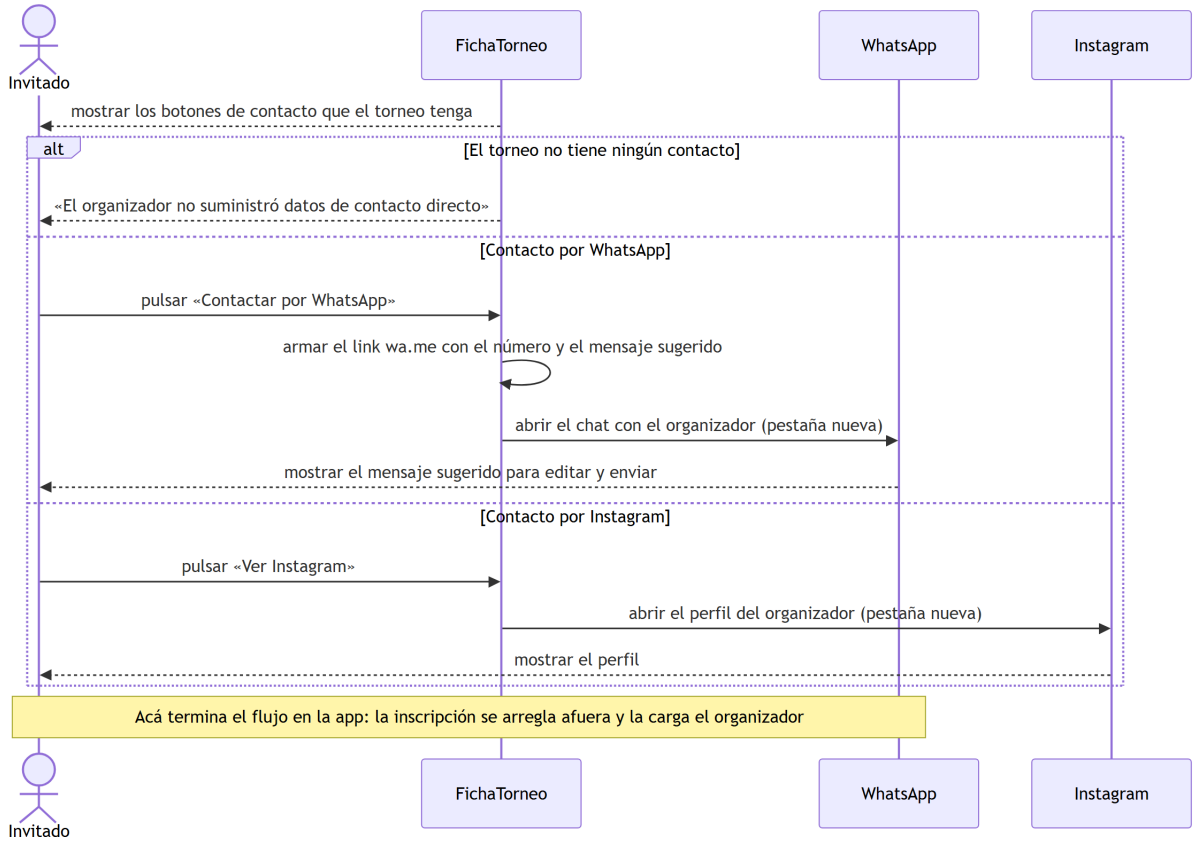
UC-B04 · Contactar al organizador

- **Actor:** Invitado.
- **Requerimientos:** RF-B21, RF-B22. **Pantalla:** P2 · Ficha del torneo (botones de contacto). **Diagrama:** S04.
- **Precondiciones:** el torneo tiene al menos un medio de contacto.
- **Postcondición:** la persona está en WhatsApp o Instagram, fuera de la app.
- **Estado:** Implementado.

Diagrama de secuencia · **S04 · UC-B04 — Contactar al organizador**

Caso de uso: UC-B04 · Pantalla: P2 (botones de contacto) · Requerimientos: RF-B21, RF-B22

No hay formulario ni desplegables. **Aquí termina el flujo de la app:** la inscripción se arregla por fuera y la carga el organizador (UC-E06). `WhatsApp` e `Instagram` son sistemas externos.



Cómo se lee

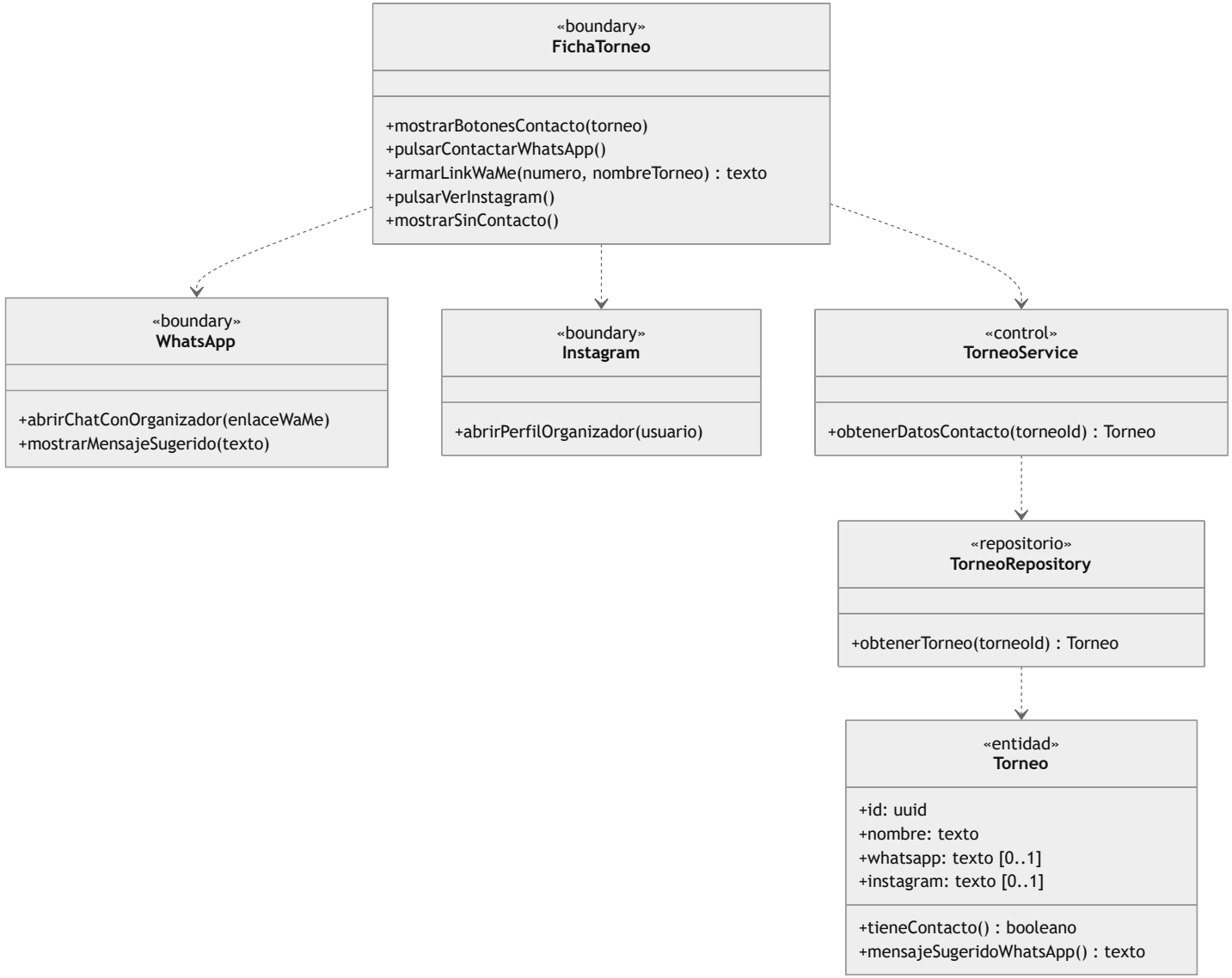
- **Cada medio es independiente:** si falta uno, su botón no se muestra; si hay ambos, se muestran los dos. Al publicar un torneo es obligatorio al menos un contacto, así que el mensaje «sin datos» solo puede aparecer en torneos viejos.
- **El link a WhatsApp se arma en la pantalla**, sin llamar al servidor: `https://wa.me/<número>?text=<mensaje>`. El número está guardado solo con dígitos, en formato argentino de celular (`549…`), que es lo que pide `wa.me`. La persona puede editar el mensaje antes de enviarlo.
- **El mensaje sugerido hoy** es «Hola, vengo de Torneos Scraper. Me interesa el torneo "[nombre]"!». `casos-de-uso.md` documenta otro texto.
- **Instagram:** se abre `instagram.com/<usuario>` (se quita la `@` si la tiene).
- **La app no inscribe ni cobra.** No hay un «Anotar mi equipo»: eso se descartó y es un límite del producto.

Correspondencia en el código

Los botones están en `TournamentDetail` (`src/app/torneos/[id]/page.tsx`); `whatsappLink` está en `src/utils/whatsapp.ts`. Ambos links usan `target="_blank"` y `rel="noopener noreferrer"`.

Diagrama de clases · **UC-B04 · Contactar al organizador**

Caso de uso: UC-B04 · **Pantalla:** P2 · Ficha del torneo (botones de contacto) · **Secuencia:** S04



Pantalla · **P2 · Ficha del torneo**

Es la misma pantalla que se describe, con su captura, en P2 · Ficha del torneo, debajo de UC-B03 · Ver la ficha pública de un torneo. Este caso usa la parte que indica su ficha («Pantalla»).

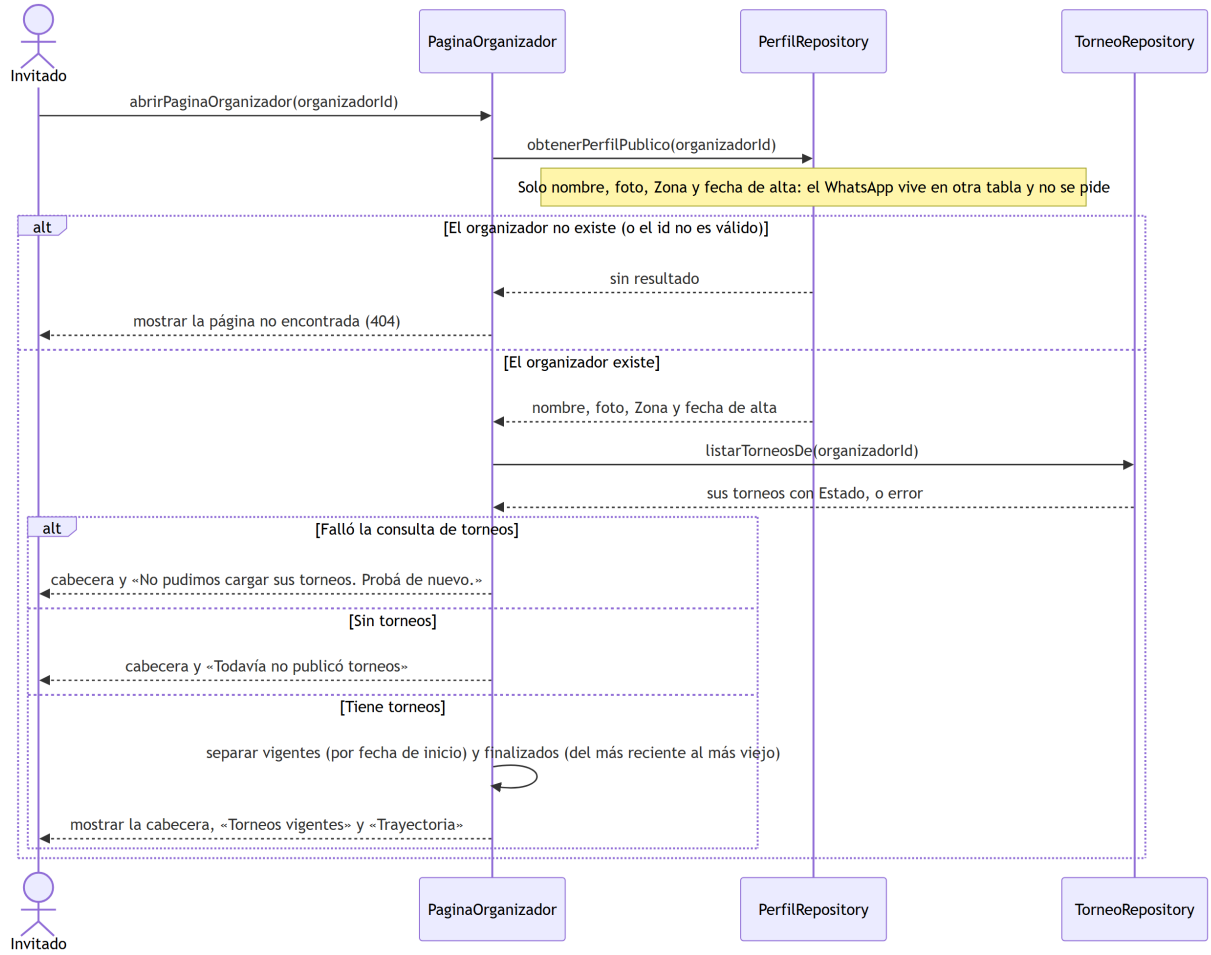
UC-B05 · Ver la página pública del organizador

- **Actor:** Invitado.
- **Requerimientos:** RF-B23. **Pantalla:** P3 · Página del organizador (`/organizadores/{id}`). **Diagrama:** S05.
- **Precondiciones:** el organizador existe.
- **Postcondición:** la persona ve la trayectoria del organizador.
- **Estado:** Implementado (`src/app/organizadores/[id]`).

> Diagrama de secuencia · **S05 · UC-B05 — Ver la página pública del organizador**

Caso de uso: UC-B05 · Pantalla: P3 · Requerimientos: RF-B23

Es una consulta: no hay formulario. La página es pública y **no pide ningún contacto**.



Cómo se lee

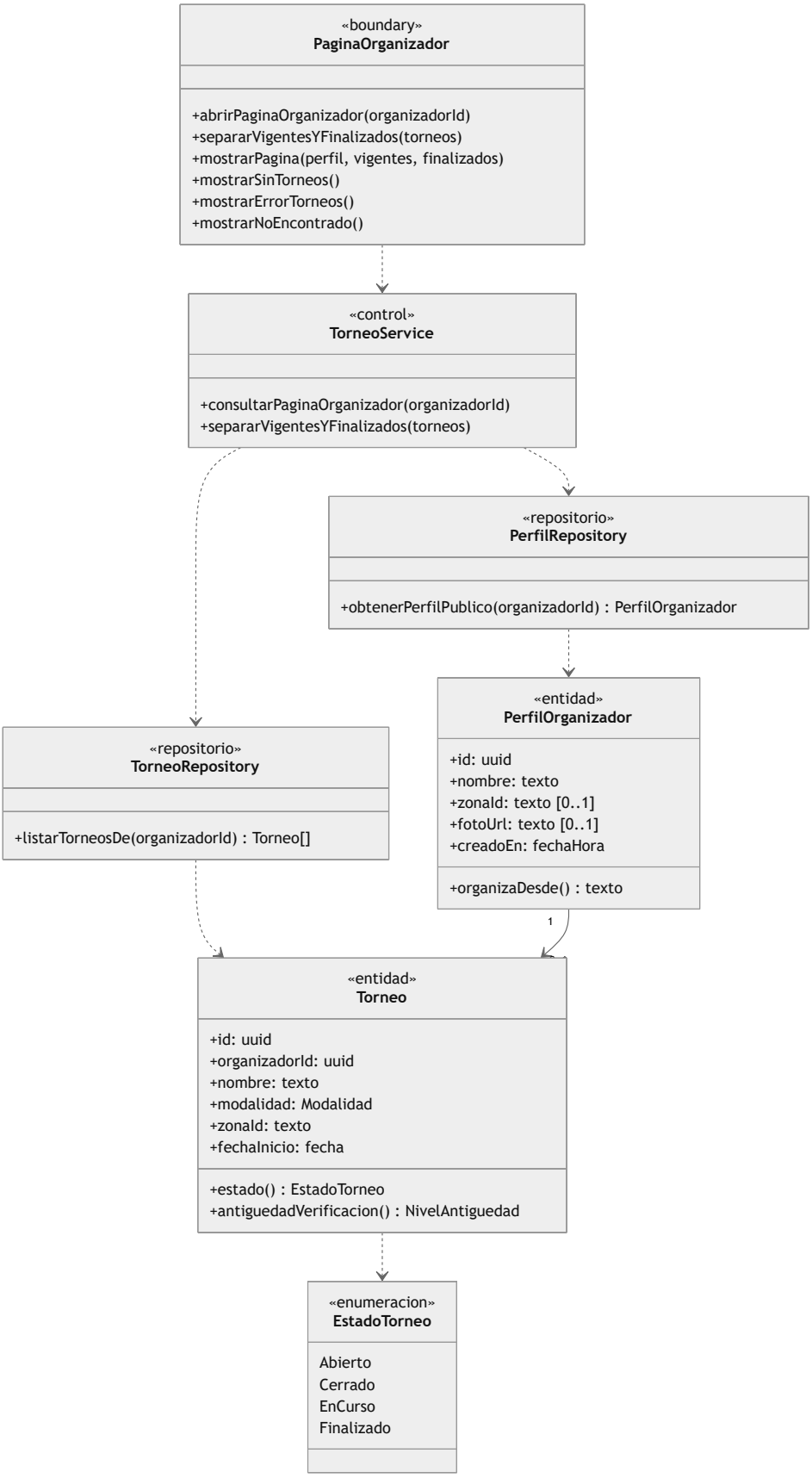
- **Privacidad:** la página pide solo las columnas que muestra. El WhatsApp del perfil vive en `profile_contacts` y **nunca llega** a esta página, ni tampoco el contacto de cada torneo: el contacto es siempre el de la ficha del torneo.
- **«Organiza desde [mes y año]»** sale de la fecha de alta de la cuenta, con el mes contado en hora de Buenos Aires.
- **Trayectoria:** los torneos finalizados quedan en la página como historia del organizador. Por eso un Finalizado no se puede dar de baja.
- Cada torneo de la lista es un link a su ficha (S03).

Correspondencia en el código

`OrganizerPage` (`src/app/organizadores/[id]/page.tsx`): `from('profiles')` y `from('tournaments').eq('user_id', id)`; `splitTournaments` y `organizesSince` están en `src/utils/organizer-page.ts`.

Diagrama de clases · **UC-B05 · Ver la página pública del organizador**

Caso de uso: UC-B05 · **Pantalla:** P3 · Página del organizador (`/organizadores/{id}`) · **Secuencia:** S05



> Pantalla · **P3 · Página del organizador**

Ruta: `/organizadores/{id}` · **Caso de uso:** UC-B05 · **Requerimientos:** RF-B23 · **Diagrama:** S05

> ⏳ **Captura pendiente:** requiere datos de la base.

Qué muestra. «← Volver al catálogo»; la cabecera con la foto (o la inicial en lima), el **nombre**, la Zona con su corredor y «Organiza desde [mes y año]»; y debajo dos secciones, **«Torneos vigentes (N)»** y **«Trayectoria (N)»** (los finalizados, más apagados). Cada torneo es un link con su Estado, nombre, Zona, modalidad y fechas. No se muestra ningún contacto.

Estados: *sin torneos* («Todavía no publicó torneos»); *error al cargar los torneos* («No pudimos cargar sus torneos. Probá de nuevo.»); *no existe* (404).

UC-B06 · Reportar un torneo

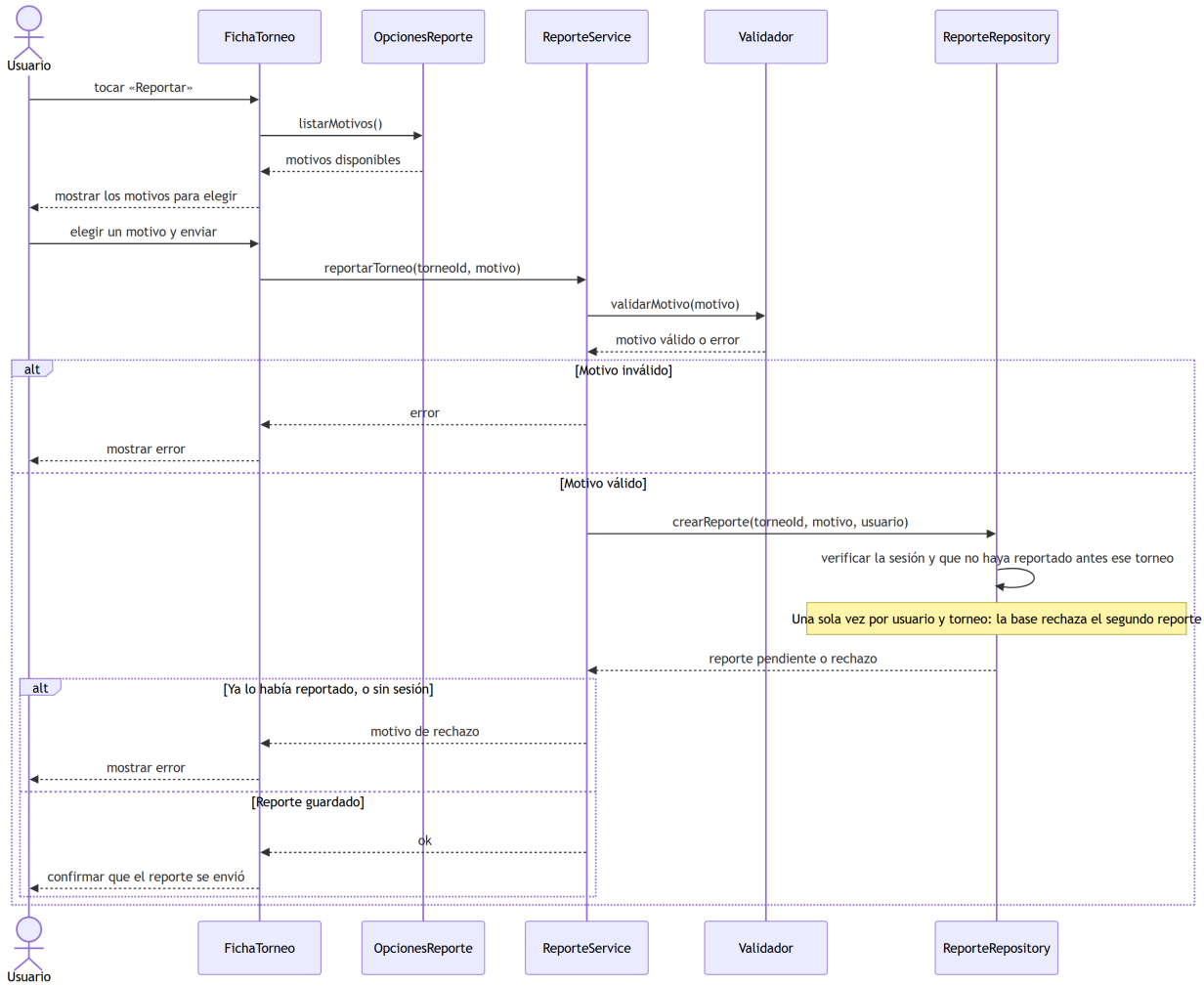
- **Actor:** Usuario.
- **Requerimientos:** RF-B24. **Pantalla:** P2 · Ficha del torneo (botón «Reportar», todavía no existe). **Diagrama:** S06 (diseño objetivo).
- **Precondiciones:** sesión; no reportó antes ese torneo.
- **Postcondición:** el reporte queda pendiente en el panel de administración. Cada usuario ve solo los suyos.
- **Estado: Pendiente.** No hay tabla, función ni botón.

> Diagrama de secuencia · **S06 · UC-B06 — Reportar un torneo**

Caso de uso: UC-B06 · Pantalla: P2 (botón «Reportar», todavía no existe) · Requerimientos: RF-B24

> **Diseño objetivo, pendiente de implementación.** Hoy no hay tabla, función ni botón de reportes de torneos. El diagrama dibuja lo que dicen las reglas vigentes; `OpcionesReporte`, `ReporteService`, `Validador` y `ReporteRepository` no existen todavía y sus nombres son provisorios.

El caso empieza con una elección entre tres motivos. Siguiendo la guía de formularios, la pantalla obtiene la lista de motivos **antes** de que la persona elija. Los motivos son una **lista fija del código**, así que los provee un proveedor de opciones y no un repositorio.

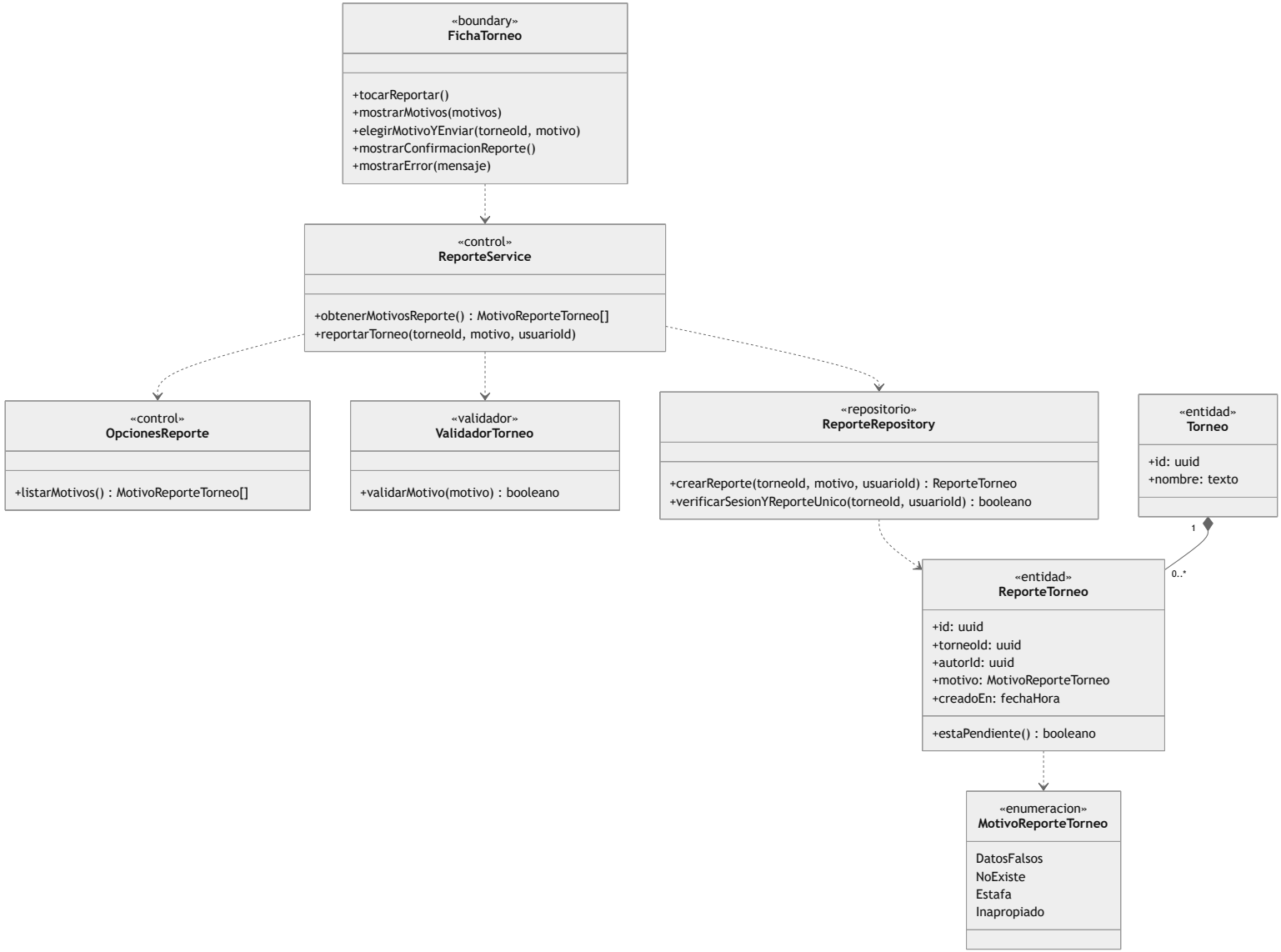


Cómo se lee

- **Motivos** (lista fija): «Ya no existe», «Datos incorrectos» y «No es un torneo».
- **Una sola vez por usuario y torneo.** No alcanza con ocultar el botón: la base lo hace cumplir al guardar, porque la pantalla pudo quedar desactualizada o el pedido pudo armarse a mano.
- **Privacidad:** cada usuario ve solo sus reportes; el administrador ve todos.
- **Destino:** el reporte queda pendiente en el panel de administración, donde el administrador elige entre descartarlo, marcar el torneo como **Observado** o darlo de baja (UC-F03).

Diagrama de clases · **UC-B06 · Reportar un torneo**

Caso de uso: UC-B06 · **Pantalla:** P2 · Ficha del torneo (botón «Reportar», todavía no existe) · **Secuencia:** S06 (diseño objetivo)



Pantalla · **P2 · Ficha del torneo**

Es la misma pantalla que se describe, con su captura, en P2 · Ficha del torneo, debajo de UC-B03 · Ver la ficha pública de un torneo. Este caso usa la parte que indica su ficha («Pantalla»).

Parte 3 · Anexos y trazabilidad

Guía de pantallas: estado de las capturas y mapa

Versión del **04/11/2026**. Cada pantalla pública de los torneos, con su ruta, el caso, los requerimientos, lo que muestra y la captura. Cada una va debajo del caso de uso que la usa por primera vez; si otro caso usa la misma pantalla, remite a esa descripción.

Estado de las capturas

Marca	Significa
✅ **Vigente**	Sacada el 04/11/2026 de `main`, con la UI actual, a 1280 px de ancho.
⏳ **Pendiente**	Todavía no hay captura. Al final están los pasos para sacarla.

Las capturas están en `pantallas/`. **La base de datos de desarrollo no respondía** cuando se sacaron, así que las dos capturas vigentes muestran estados que la app produce justamente cuando la base falla: el catálogo en su estado de error (con el panel de filtros completo y activo) y el 404 de una ficha. Las que muestran torneos reales están pendientes.

Mapa de pantallas

Pantalla	Ruta	Casos de uso	Sesión
P1 · Catálogo	`/` (y `/?modality=…&status=…`)	UC-B01, UC-B02	No
P2 · Ficha del torneo	`/torneos/{id}`	UC-B03, UC-B04, UC-B06	No (el dueño ve una barra extra)
P3 · Página del organizador	`/organizadores/{id}`	UC-B05	No

Las tres son **públicas**: ninguna está en la lista de rutas protegidas.

Capturas pendientes

Convención: PNG en `docs/catalogo-de-torneos/pantallas/`, ventana de unos 1280 px de ancho, con datos de prueba. Hace falta la base de desarrollo disponible.

Archivo	Pantalla y estado	Cómo conseguirlo
`catalogo.png`	P1 con tarjetas	Abrir `/` con varios torneos, de distintos Estados y con y sin flyer.
`catalogo-sin-resultados.png`	P1 sin resultados	Combinar filtros que no coincidan con ningún torneo.
`catalogo-movil.png`	P1 en el celular	Ancho de ~390 px.
`ficha-con-flyer.png`	P2 con flyer	Un torneo con flyer, con cupo y con equipos anotados.
`ficha-sin-flyer.png`	P2 sin flyer	Un torneo sin flyer.
`ficha-dueno.png`	P2 vista del dueño	Abrir la ficha propia con sesión: se ve la barra «Este torneo es tuyo».
`ficha-finalizada.png`	P2 de un torneo finalizado	Para documentar que hoy muestra los botones de contacto.
`organizador.png`	P3	Un organizador con torneos vigentes y finalizados.

Diagramas de secuencia: criterios, participantes y convenciones

Versión del **04/11/2026**. Un diagrama por caso de uso, en Mermaid (GitHub los dibuja directo). Cada uno va debajo de su caso de uso, antes de su pantalla.

Diagrama	Caso de uso	Pantalla	Estado
S01 · Ver el catálogo	UC-B01	P1	Parcial (sin paginar)
S02 · Filtrar torneos	UC-B02	P1	Parcial (2 de 5 filtros)
S03 · Ver la ficha	UC-B03	P2	Parcial
S04 · Contactar al organizador	UC-B04	P2	Implementado
S05 · Ver al organizador	UC-B05	P3	Implementado
S06 · Reportar un torneo	UC-B06	P2	**Pendiente** (diseño objetivo)

Los PNG están dibujados sobre fondo blanco, para pegarlos en un informe. La fuente de cada diagrama es el bloque Mermaid de su archivo: si se cambia uno, hay que regenerar su PNG.

Relación con los diagramas generales. `docs/diagramas-de-secuencia/diagramas-secuencia.md` tiene diagramas de todo el proyecto con participantes genéricos. Los de este módulo los **detallan**: D07 a D11 corresponden a S01 a S04 (D08 y D09, filtrar y compartir, están juntos en S02), y D29 (página del organizador) a S05. S06 no tiene equivalente.

Criterios aplicados

Son los de la guía para diagramas con formularios y desplegables.

1. **Un caso de uso por diagrama.** Cargar las opciones de un filtro y filtrar van **en el mismo diagrama**.
2. **Si el caso empieza con un formulario o filtros con opciones**, el diagrama muestra al actor abriendo la pantalla y a la pantalla pidiendo las opciones **antes** de que el actor elija. Acá, el panel de filtros (S02) y los motivos del reporte (S06).
3. **Un objeto por responsabilidad o fuente de datos.** Si la lista se consulta en la base, es un repositorio; si es una lista fija del código, es un **proveedor de opciones**. Las modalidades y los Estados del filtro son listas fijas del código, así que las provee `OpcionesTorneo`; no se inventa un repositorio para ellas.
4. **Al filtrar**, se pasan **solo los valores elegidos** (`modalidad`, `estado`), nunca las listas.
5. **La validación se repite al consultar.** Un valor que aparece en el panel no garantiza que siga siendo válido en la URL: cualquiera puede editarla. Hoy el servidor **no** lo valida antes de consultar (ver S02).
6. **Cada diagrama muestra solo los participantes que intervienen.**

Participantes: responsabilidad y correspondencia en el repo

Son responsabilidades conceptuales. No existen como clases TypeScript con esos nombres: la app usa Server Components, utilidades puras y consultas a la base.

Participante	Responsabilidad	Correspondencia en el repo
Invitado, Usuario	La persona que inicia el caso.	La sesión; el Invitado navega sin ella.
PaginaCatalogo	La portada con el listado de tarjetas.	`app/page.tsx`, `TournamentList.tsx`, `TournamentCard.tsx`.
FiltrosCatalogo	El panel de filtros, que escribe los filtros en la URL.	`components/Filters.tsx`.

Participante	Responsabilidad	Correspondencia en el repo
FichaTorneo	La ficha pública de un torneo.	`app/torneos/[id]/page.tsx`.
PaginaOrganizador	La página pública del organizador.	`app/organizadores/[id]/page.tsx`.
OpcionesTorneo	Entrega las modalidades y los Estados del filtro: **listas fijas del código**.	`SPORT_MODALITIES` (`utils/modality.ts`) y la lista de Estados de `Filters.tsx`.
OpcionesReporte	Solo en S06: los motivos de reporte, lista fija.	**No existe todavía.**
TorneoRepository	Lee torneos, con su Estado y cupos calculados por la base.	Tabla `tournaments` con RLS y columnas calculadas `status` e `inscriptions_count`; la Zona sale de `zones`.
InscripcionRepository	Entrega los nombres de los equipos anotados.	Vista pública `tournament_inscriptions`.
PerfilRepository	Entrega el nombre, la foto y la Zona del organizador.	Tabla `profiles` (lectura pública).
ReporteService, Validador, ReporteRepository	Solo en S06: validar y guardar el reporte.	**No existen todavía.**
WhatsApp, Instagram	Sistemas externos adonde termina el flujo.	`wa.me` (`whatsappLink`) e `instagram.com`.

Convenciones de dibujo

- Flecha continua (`->>`): una llamada. Flecha punteada (`-->>`): su respuesta.
- Un mensaje de un participante a sí mismo es un paso interno.
- Una nota sobre un participante marca una regla que **hace cumplir la base** o una diferencia con la documentación.
- Entre paréntesis y en minúscula van los **valores que viajan**; entre comillas «así», los mensajes que ve la persona.
- `opt` es un paso opcional; `alt` / `else`, caminos que se excluyen.

Trazabilidad y pendientes

Documentación completa del módulo B (Catálogo de torneos, público) de Torneos Amateur BA, versión del **04/11/2026**. En este documento, sus piezas son la Parte 1 (requerimientos), la Parte 2 (cada caso de uso con su diagrama y su pantalla) y los anexos de esta parte.

Fuentes, en orden de autoridad: decisiones-casos-de-uso.md → casos-de-uso.md → CONTEXT.md → código y migraciones.

Matriz de trazabilidad

Caso de uso	Requerimientos	Pantalla	Diagrama	Estado
UC-B01 Ver el catálogo	RF-B01 a B07, B25	P1 `/`	S01	**Parcial** (sin paginar)
UC-B02 Filtrar y compartir	RF-B08 a B13	P1 panel de filtros	S02	**Parcial** (2 de 5 filtros)
UC-B03 Ver la ficha	RF-B14 a B20	P2 `/torneos/{id}`	S03	**Parcial**
UC-B04 Contactar al organizador	RF-B21, B22	P2	S04	Implementado

Caso de uso	Requerimientos	Pantalla	Diagrama	Estado
UC-B05 Ver al organizador	RF-B23	P3 `/organizadores/{id}`	S05	Implementado
UC-B06 Reportar un torneo	RF-B24	P2	S06	**Pendiente**

Equivalencia con el PDF original

Vigente	PDF original
UC-B01 Ver el catálogo	UC-08
UC-B02 Filtrar y compartir	UC-09 (filtrar) y UC-10 (compartir)
UC-B03 Ver la ficha	UC-11
UC-B04 Contactar al organizador	UC-12
UC-B05 Ver al organizador	SPEC-E04
UC-B06 Reportar un torneo	— (caso nuevo)

Hallazgos y pendientes

`casos-de-uso.md` **marca el módulo B como «Implementado», salvo el reporte. Al compararlo con el código, hay nueve diferencias.** Están detalladas en «Sin decisión». Las más importantes:

1. **El catálogo solo filtra por modalidad y estado.** Zona o Corredor, precio máximo y fechas de inicio **no existen en** `main`. `decisiones-casos-de-uso.md` incluso define el parámetro `?zona=` (que acepta una Zona o un Corredor completo). La rama `feat/dynamic-filters` tiene un filtro por Zona y por precio, pero **no está integrada** y es anterior a la tabla `zones`.
2. **No hay paginación**: se traen todos los torneos de una vez. Con el catálogo creciendo, es la primera deuda de rendimiento.
3. **La ficha de un torneo Finalizado muestra los botones de contacto**, aunque `casos-de-uso.md` pide «Este torneo ya terminó» sin botones.
4. **El 404 de una ficha es el genérico de Next**: en inglés, sin link al catálogo. Se comprobó con una captura.
5. **La ficha no tiene metadatos propios** para la vista previa de WhatsApp.
6. **El texto del mensaje de WhatsApp y el nombre del producto difieren** de lo documentado: la interfaz dice «Torneos Scraper»; la documentación, «Torneos BA».
7. **Un valor inválido en un filtro de la URL no se ignora**: lo rechaza la base y se ve el error de carga.
8. `casos-de-uso.md` **se contradice** sobre los Finalizados: UC-B01 A4 dice que aparecen si se filtra por ese Estado; la regla transversal dice que salen del catálogo. El código hace lo segundo.
9. **El reporte de torneos (UC-B06)** sigue pendiente: sin tabla, función ni botón.

Capturas: hay dos vigentes (el catálogo con filtros activos en su estado de error de carga, y el 404 de la ficha). La base de desarrollo no respondía, así que **faltan las que muestran torneos reales**: la lista en 03-pantallas.md.

Si se cambia un diagrama, hay que regenerar su PNG (el bloque Mermaid de cada archivo es la fuente).

Diagramas de clases: trazabilidad de las clases

Clase	Estereotipo	Implementación en el repo	Estado
`PaginaCatalogo`	`«boundary»`	`src/app/page.tsx` + `src/components/TournamentList.tsx` + `src/components/TournamentCard.tsx`	Parcial (faltan Zona, precio «Gratis»/«Consultar», antigüedad y Observado en tarjeta)
`FiltrosCatalogo`	`«boundary»`	`src/components/Filters.tsx`	Parcial (faltan filtros de Zona, Estado y Precio; hoy tiene `city` / `maxPrice`)
`FichaTorneo`	`«boundary»`	`src/app/torneos/[id]/page.tsx` + `src/app/torneos/[id]/loading.tsx`	Implementada (falta botón «Reportar»)
`PaginaOrganizador`	`«boundary»`	`src/app/organizadores/[id]/page.tsx`	Implementada
`WhatsApp` / `Instagram`	`«boundary»`	`whatsappLink` (`src/utils/whatsapp.ts`), `instagramUrl` (`src/utils/instagram.ts`)	Implementados
`TorneoService` / `ReporteService` / `OpcionesTorneo` / `OpcionesReporte`	`«control»`	Consultas RSC en `TournamentList`, `torneos/[id]/page.tsx`, `organizadores/[id]/page.tsx`, `SPORT_MODALITIES` (`src/utils/modality.ts`), `STATUS_FILTERS` (`src/utils/tournament-status.ts`)	Implementado (`ReporteService` y `OpcionesReporte` pendientes)
`ValidadorTorneo`	`«validador»`	Validación de parámetros de búsqueda y motivos de reporte	Parcial
`TorneoRepository`	`«repositorio»`	Vista `public.tournaments_with_status` (`20261028000010_tournaments_with_status.sql`)	Implementado
`PerfilRepository`	`«repositorio»`	Tabla `public.profiles` (lectura pública)	Implementado
`InscripcionRepository`	`«repositorio»`	Vista `public.public_inscriptions` (`20261001000000_inscriptions.sql`, `20261028000002_teams.sql`)	Implementado
`ReporteRepository`	`«repositorio»`	Tabla de reportes de torneo con unicidad `(tournament_id, reporter_id)`	**Pendiente**
`Torneo` / `InscripcionPublica` / `PerfilOrganizador`	`«entidad»`	`tournaments_with_status`, `public_inscriptions`, `profiles` + `tournament-status.ts` y `tournament-verification.ts`	Implementadas

Módulo C · Equipos

Primero los requerimientos del módulo completo. Después, el modelo de dominio y, para cada caso de uso, su diagrama de secuencia, su diagrama de clases y su pantalla. Al final, los anexos y la trazabilidad.

Parte 1 · Requerimientos

Versión del **04/11/2026**. Es la primera parte del módulo: sigue la sección de casos de uso, donde cada caso lleva debajo su diagrama de secuencia y su pantalla. El README tiene la matriz que los une.

Fuentes, en orden de autoridad: decisiones-casos-de-uso.md (reglas vigentes), casos-de-uso.md (módulo C, 03/11/2026), el glosario CONTEXT.md y el código y las migraciones del repo al día de hoy. Reemplaza a la especificación del 28/10/2026 (`modulo-equipos-especificacion.pdf`), que usaba otra numeración y daba por pendiente lo que hoy ya está.

1. Alcance

El módulo permite **crear y gestionar un Equipo**: formar su **Plantel** con un link de invitación, editar sus datos, pasar la capitanía, sacar jugadores o salir, y **disolverlo**. Cada Equipo tiene una **página pública** con sus datos, los torneos en los que tiene una **Inscripción vinculada** y su historial de Partidos. El Plantel lo ven solo sus integrantes. Un **Equipo disuelto** conserva su página y su historia, pero ya no puede operar.

Fuera del módulo (tienen su propio documento):

Tema	Dónde está
Buscar rival, propuestas, partidos, resultados, faltas y calificaciones	Módulo D — Partidos
Cargar o dar de baja una inscripción	Módulo E — Organizador
Suspender una cuenta, resolver reportes	Módulo F — Administración
Editar el WhatsApp del perfil	Módulo A — Cuenta (UC-A05)

Fuera del alcance vigente: la grilla de disponibilidad del equipo (mañana/tarde/noche) se eliminó el 28/09/2026; no existe en la página ni en la base.

Límite de producto: la app nunca inscribe ni cobra. Un Equipo no "se anota" a un torneo: lo anota el organizador desde su panel (UC-E06).

2. Actores

Una cuenta puede cumplir roles distintos según su relación con cada Equipo. **Jugador y Capitán son roles respecto de un Equipo, no cuentas distintas.**

Actor	Quién es	Qué hace en este módulo
Invitado	Persona sin sesión	Consulta la página pública de un Equipo.
Usuario	Persona con sesión	Crea Equipos, acepta invitaciones, reporta equipos con los que jugó.
Jugador	Usuario que integra un Plantel	Ve el Plantel y los partidos del Equipo, y sale si no es Capitán. No opera.
Capitán	Único miembro que gestiona un Equipo activo	Edita, invita, saca jugadores, pasa la capitanía y disuelve.
Sistema	La base de datos y la app	Valida y calcula al momento de leer; no hay tareas programadas.

3. Requerimientos funcionales

Estado: `Implementado` está en `main`. `Parcial` tiene una parte pendiente que se indica. `Pendiente` es alcance vigente sin programar. **ID anterior** es el de la especificación del 28/10 (`RF-D##`).

Crear y datos del Equipo

ID	Requerimiento	Casos de uso	Estado	ID anterior
RF-C01	Crear un Equipo con **nombre** (2 a 50 caracteres), **Zona**, **modalidad habitual** (Fútbol 5, 7, 8 u 11) y **nivel** (recreativo, intermedio o competitivo). Zona, modalidad y nivel se eligen de listas cerradas.	UC-C01	Implementado	D01
RF-C02	**Escudo** opcional, JPG, PNG o WebP de hasta 2 MB; se puede subir, reemplazar o quitar. Si falla el guardado del escudo al crear, el Equipo igual queda creado.	UC-C01, UC-C05	Implementado	D02
RF-C03	Permitir **nombres de Equipo repetidos**; públicamente se distinguen por la Zona.	UC-C01	Implementado	D03
RF-C04	Exigir **sesión** y **WhatsApp cargado en el perfil** para crear un Equipo.	UC-C01	Implementado	D04
RF-C05	Máximo **3 capitanías** de Equipos no disueltos por persona, tanto al crear como al recibir la capitanía.	UC-C01, UC-C06	Implementado	D05
RF-C06	Crear el Equipo y su Capitán inicial en **una sola transacción**: nunca queda un Equipo sin Capitán.	UC-C01	Implementado	D06
RF-C16	Solo el **Capitán de un Equipo activo** edita nombre, escudo, Zona, modalidad y nivel. Cambiar la Zona o el nivel cambia en el acto cómo aparece el Equipo en los rivales compatibles.	UC-C05	Implementado	D14

Invitar y formar el Plantel

ID	Requerimiento	Casos de uso	Estado	ID anterior
RF-C07	Link de invitación **válido 7 días**; **regenerarlo invalida el anterior**. Hay un solo link vigente por Equipo.	UC-C02	Implementado	D07
RF-C08	Unirse al Plantel **pulsando «Unirme al equipo»**, sin aprobación del Capitán. Abrir el link no suma a nadie (los chats y navegadores precargan links). Si ya integra el Plantel, lo lleva al Equipo sin error.	UC-C03	Implementado	D08
RF-C09	**Mensaje único** para un link inexistente, vencido, regenerado o de un Equipo disuelto: «El link de invitación no es válido o venció».	UC-C03	Implementado	D09
RF-C10	Si el Usuario no tiene sesión, conservar el destino de la invitación al iniciar sesión o crear cuenta.	UC-C03	Implementado	D10

Página pública y Plantel

ID	Requerimiento	Casos de uso	Estado	ID anterior
RF-C11	Página pública `/equipos/{id}`, accesible **sin sesión**, con escudo, nombre, Zona (y su corredor), modalidad, nivel y cantidad de jugadores. Si el Equipo está disuelto, lo marca.	UC-C04	Implementado	D19

ID	Requerimiento	Casos de uso	Estado	ID anterior
RF-C12	La página lista los **torneos con Inscripción vinculada** al Equipo, con su estado.	UC-C04	Implementado	D19
RF-C13	**Historial público**: contador de partidos jugados, faltas y conflictos (con la cantidad de rivales distintos), promedio de estrellas (solo calificaciones reveladas) y los **últimos 5 partidos cerrados**, sin cancha ni hora.	UC-C04	Implementado	D19, UC-30
RF-C14	El **Plantel** (nombres) lo ven solo sus integrantes. El WhatsApp de los jugadores nunca está en la página.	UC-C04	Implementado	D20
RF-C15	**Mis equipos** lista los Equipos del Usuario, activos primero y disueltos después, y marca en cuáles es Capitán.	— (pantalla)	Implementado	D21

Gestionar el Plantel

ID	Requerimiento	Casos de uso	Estado	ID anterior
RF-C17	**Pasar la capitanía** a un integrante del Plantel que tenga WhatsApp, no esté suspendido y capitanee menos de 3 Equipos. El Capitán anterior queda como Jugador.	UC-C06	**Parcial**: la regla de la Suspensión espera al módulo de administración (UC-F05)	D11
RF-C18	El Capitán puede **sacar** a cualquier otro jugador del Plantel.	UC-C07	Implementado	D13
RF-C19	Cualquier Jugador que **no sea Capitán** puede **salir** del Equipo.	UC-C08	Implementado	D13
RF-C20	El Capitán **no puede salir ni sacarse a sí mismo**: primero tiene que pasar la capitanía.	UC-C06, UC-C07, UC-C08	Implementado	D12
RF-C21	Cambiar el Capitán o el Plantel **nunca se bloquea** por partidos en curso: las búsquedas, propuestas y partidos son del Equipo, y el Capitán nuevo opera todo lo pendiente. El Capitán anterior deja de ver las propuestas y sigue viendo los partidos; quien sale del Plantel deja de ver los partidos y conserva los Avisos que ya recibió.	UC-C06, UC-C07, UC-C08	Implementado	D22

Disolver

ID	Requerimiento	Casos de uso	Estado	ID anterior
RF-C22	**Disolver** con confirmación explícita. El Equipo **no se borra**: conserva su página, su Plantel, su escudo y su historia. **No se reactiva**: si vuelven, crean un Equipo nuevo. Deja de contar para el tope de 3 capitanías.	UC-C09	Implementado	D15
RF-C23	**No se puede disolver** un Equipo con un partido por jugarse o esperando resultado. El mensaje indica que primero lo tire o cierre el resultado.	UC-C09	Implementado	D16
RF-C24	Al disolver: las **invitaciones dejan de valer** y se **cancelan las búsquedas de rival activas** con sus propuestas pendientes (enviadas y recibidas); el otro lado recibe el Aviso «Se canceló tu propuesta».	UC-C09	Implementado	D17
RF-C25	Un **Equipo disuelto** no admite nuevas incorporaciones, ediciones, invitaciones, salidas, expulsiones, pase de capitanía ni inscripciones. Un partido Jugado se sigue pudiendo calificar hasta que venzan los 7 días.	UC-C02 a UC-C09	Implementado	D18

Pendientes del alcance vigente

ID	Requerimiento	Casos de uso	Estado	ID anterior
RF-C26	**Reportar un Equipo** (motivos: «Violencia o agresiones», «Cuenta falsa», «Abuso de calificaciones»), solo con sesión y solo si el Usuario jugó al menos un partido contra el reportado.	UC-C10	Pendiente	—
RF-C27	La **Suspensión** de una cuenta impide crear Equipos y recibir la capitanía; un suspendido puede sumarse a un Plantel. Mientras dure, su Equipo no puede cambiar de Capitán.	UC-C01, UC-C03, UC-C06	Pendiente (UC-F05); hoy la base no la verifica	—

Sin decisión

Tema	Situación
Catálogo público de Equipos (`/equipos` , con búsqueda por nombre y filtros por Zona, modalidad, nivel y estado). La especificación del 28/10 lo daba como requerimiento (RF-D23).	Existe solo en la rama `franco-equipos` (commit `5564c51`). **No está en `main` ni en `casos-de-uso.md` .** Hay que decidir con el equipo si entra al alcance antes de documentarlo como caso de uso.

4. Requerimientos no funcionales

ID	Requisito	Evidencia
RNF-C01	**Control de acceso en la base (RLS).** `teams` se lee en público y solo el Capitán edita su Equipo activo, y solo las columnas `name` , `zone_id` , `modality` , `level` y `crest_url` . `team_members` no tiene policies de escritura: el Plantel cambia únicamente por funciones.	`20260928000004_teams.sql`
RNF-C02	**Un solo Capitán por Equipo.**	Índice único parcial `team_members_one_captain` ; las funciones conservan al Capitán.
RNF-C03	**El límite de 3 capitanías se respeta aun con altas simultáneas.**	`create_team` y `transfer_captaincy` bloquean el perfil (`for update`) antes de contar.
RNF-C04	**Privacidad del Plantel y de los contactos.**	Policy de `team_members` solo para integrantes; el WhatsApp vive en `profile_contacts` , fuera de la ficha pública.
RNF-C05	**Token de invitación no adivinable ni listable.**	16 bytes aleatorios en hexadecimal (`gen_random_bytes(16)`); `team_invites` tiene RLS sin policies: solo se toca desde funciones.
RNF-C06	**Alta atómica.**	`create_team` inserta el Equipo y el Capitán en una transacción.
RNF-C07	**Gestión y disolución se serializan por Equipo.** Una invitación o transferencia concurrente no puede modificar un Equipo que otra transacción acaba de disolver.	`20261014000003_team_management_locks.sql` (`for update` sobre `teams`)
RNF-C08	**Errores claros y confirmación de acciones destructivas.** Pasar la capitanía, sacar, salir y disolver piden confirmación; disolver pide además tildar «Entiendo que no podré reactivar este equipo».	`ConfirmButton` , `equipos/[id]/page.tsx`
RNF-C09	**URL pública estable** por Equipo, abierta sin sesión (`/equipos/{id}` no está protegida a propósito).	`src/proxy.ts` , integracion-rama-equipos.md

ID	Requisito	Evidencia
RNF-C10	**Validación pura y verificable.**	`parseTeamForm` y `parseCrest`, con `team-form.test.ts` y `team-crest.test.ts` (`pnpm test`).
RNF-C11	**Los escudos solo se sirven desde el bucket propio** y el servidor solo borra archivos de la carpeta del propio Equipo.	`team-crest.ts` (`crestSrc`, `crestObjectPath`), `20261014000002_team_crests.sql`, `team_crests_test.sql`
RNF-C12	**El historial público nunca expone cancha ni hora**, aunque los partidos solo los lea su plantel.	Funciones `security definer` y vistas `team_record` y `team_match_history` (`20261028000012_team_history.sql`), `team_history_test.sql`
RNF-C13	**Las transiciones del Plantel van por funciones de Postgres**, no por `update` directo: `create_team`, `join_team`, `leave_team`, `remove_team_member`, `transfer_captaincy`, `regenerate_team_invite`, `dissolve_team`. Las ejecuta solo `authenticated`.	Migraciones de `teams`, `team_invites`, `team_roster_changes`, `team_management_locks`

5. Reglas de negocio de referencia

No son requerimientos nuevos: son las reglas que los requerimientos de arriba concretan. El detalle está en decisiones-casos-de-uso.md, sección «Equipos».

- **Plantel:** sin tope de jugadores por ahora.
- **Quién es capitán:** es una marca dentro del Plantel (`team_members.is_captain`), no una columna de `teams`; así queda tan privado como el resto del Plantel.
- **WhatsApp del Capitán:** obligatorio para crear un Equipo o recibir la capitanía; el Capitán de un Equipo no disuelto no puede dejarlo vacío en el perfil (sí cambiarlo).
- **Suspensión vs. Bloqueo por faltas:** la Suspensión recae sobre una cuenta y la decide el administrador; el Bloqueo por faltas recae sobre un Equipo y es automático. No se confunden.
- **Términos:** se dice «disolver» y «Equipo disuelto» (nunca «eliminar» ni «borrar»); «Capitán» (nunca «dueño» ni «admin del equipo»); «Zona» (nunca «barrio» ni «localidad»).

Parte 2 · Modelo de dominio y casos de uso, con sus diagramas y su pantalla

Versión del **04/11/2026**. Detalla los casos de uso del módulo C de casos-de-uso.md (UC-C01 a UC-C10) con lo que hace falta para la documentación completa: los requerimientos que cubre cada uno (01-requerimientos.md), la pantalla que lo representa (03-pantallas.md) y su diagrama de secuencia (04-diagramas-secuencia).

Cómo leerlo. Cada caso expresa una meta y tiene **un actor iniciador**. **Estado** dice si está en `main`. Los términos en negrita son los del glosario (CONTEXT.md). Los mensajes entre comillas son los que ve la persona.

Mapa de casos de uso

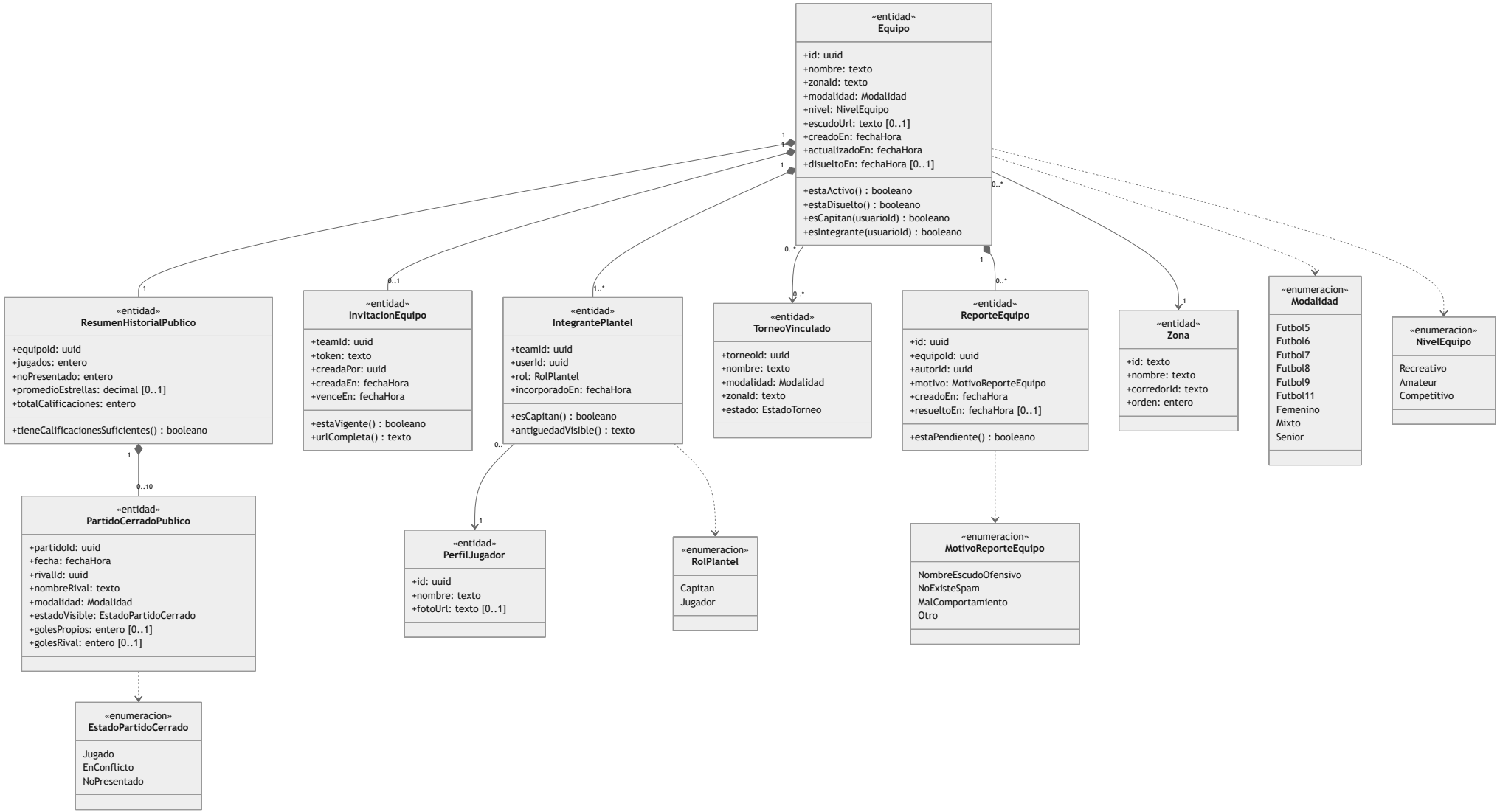
Caso	Meta del actor	Actor	Estado	ID anterior (PDF 28/10)
UC-C01	Crear equipo	Usuario	Implementado	UC-25
UC-C02	Invitar jugadores al equipo	Capitán	Implementado	UC-27
UC-C03	Unirse a un equipo con el link	Usuario	Implementado	UC-28
UC-C04	Ver la página pública del equipo	Invitado	Implementado	UC-29 y UC-30
UC-C05	Editar equipo	Capitán	Implementado	«Editar Equipo» (sin número)
UC-C06	Pasar la capitanía	Capitán	Implementado	UC-26
UC-C07	Sacar a un jugador del plantel	Capitán	Implementado	«Sacar a un Jugador» (sin número)
UC-C08	Salir del equipo	Jugador	Implementado	«Salir del Equipo» (sin número)
UC-C09	Disolver equipo	Capitán	Implementado	«Disolver Equipo» (sin número)
UC-C10	Reportar un equipo	Usuario	**Pendiente**	—

La numeración vigente es la de `casos-de-uso.md` (módulo C). El PDF del 28/10 usaba «módulo D» y los números UC-25 a UC-30; el historial público (antes UC-30, «pendiente») hoy está implementado y forma parte de UC-C04, porque no hay un actor ni una meta distintos de «ver la página del equipo».

Qué no es un caso de uso acá: abrir **Mis equipos** (`/mis-equipos`) no toma ninguna decisión: es la lista desde la que se llega a los casos. Está en pantallas y en RF-C15.

Diagrama de clases · **Modelo de dominio del módulo**

Vista estructural de las entidades, relaciones de composición/asociación y enumeraciones que comparten los casos de uso `UC-C01` a `UC-C10`.



- **Quién es Capitán (RNF-C02, RNF-C04):** la capitanía es una marca dentro del Plantel (`IntegrantePlantel.esCapitan` con índice único parcial `team_members_one_captain`), no una columna pública de `teams` . Así el Plantel y quién es el Capitán solo los ven los integrantes del Equipo.
- **Límite de 3 capitanías y WhatsApp obligatorio (RF-C04, RF-C05, RNF-C03):** una persona puede capitanear hasta 3 Equipos activos (no disueltos) y debe tener su WhatsApp cargado en `ContactoPerfil` .
- **Disolución sin borrado (RF-C22 a RF-C25):** un `Equipo` disuelto (`disueltoEn` no nulo) conserva su página pública, su Plantel, su escudo y su historia, pero no puede operar ni reactivarse.

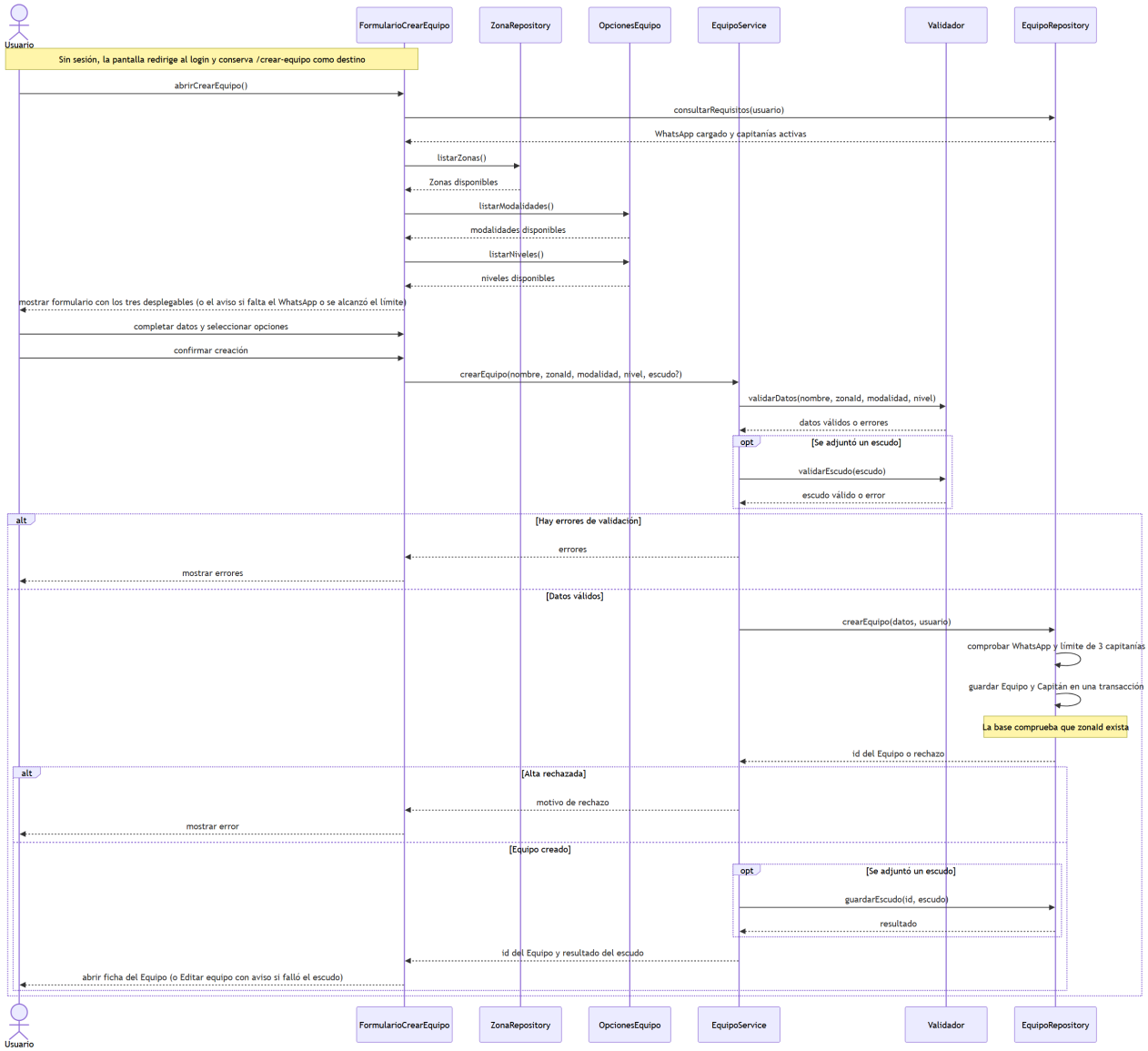
UC-C01 · Crear equipo

- **Actor:** Usuario.
- **Requerimientos:** RF-C01 a RF-C06, RF-C27. **Pantalla:** P1 · Crear equipo (`/crear-equipo`). **Diagrama:** S01.
- **Precondiciones:** sesión; **WhatsApp cargado en el perfil**; menos de 3 equipos como Capitán (los disueltos no cuentan); sin **Suspensión** (regla vigente, hoy la base no la verifica).
- **Postcondición:** Equipo activo, con el creador como único integrante y Capitán.
- **Estado:** Implementado (`src/app/crear-equipo` , función `create_team`).

Diagrama de secuencia · **S01 · UC-C01 — Crear equipo**

Caso de uso: UC-C01 · Pantalla: P1 · Requerimientos: RF-C01 a RF-C06

El formulario carga las opciones de sus tres desplegables **antes** de permitir que el Usuario envíe el alta. `ZonaRepository` representa la consulta de la lista cerrada de Zonas; `OpcionesEquipo` provee las listas fijas de modalidades y niveles. Son responsabilidades conceptuales, no clases con esos nombres (ver participantes).



Cómo se lee

- **Antes de elegir:** la pantalla obtiene las Zonas de `ZonaRepository` y las modalidades y los niveles de `OpcionesEquipo`, y recién entonces muestra el formulario. `consultarRequisitos` es lo que le permite mostrar de entrada el aviso de WhatsApp o de límite de capitanías; es un adelanto, no una garantía.
- **Al enviar:** `crearEquipo(...)` recibe solo los **valores elegidos** (`zonaId`, `modalidad`, `nivel`), no las listas.
- **Se valida de nuevo al guardar.** `Validador` revisa nombre, Zona, modalidad y nivel aunque la pantalla solo ofrezca opciones válidas. Después, la base vuelve a comprobar WhatsApp y límite, y que `zonaId` exista (clave foránea): una Zona que desapareció entre la carga del formulario y el envío hace fallar el alta.
- **El Equipo y su Capitán se guardan juntos**, en una transacción (`create_team`): nunca queda un Equipo sin Capitán.

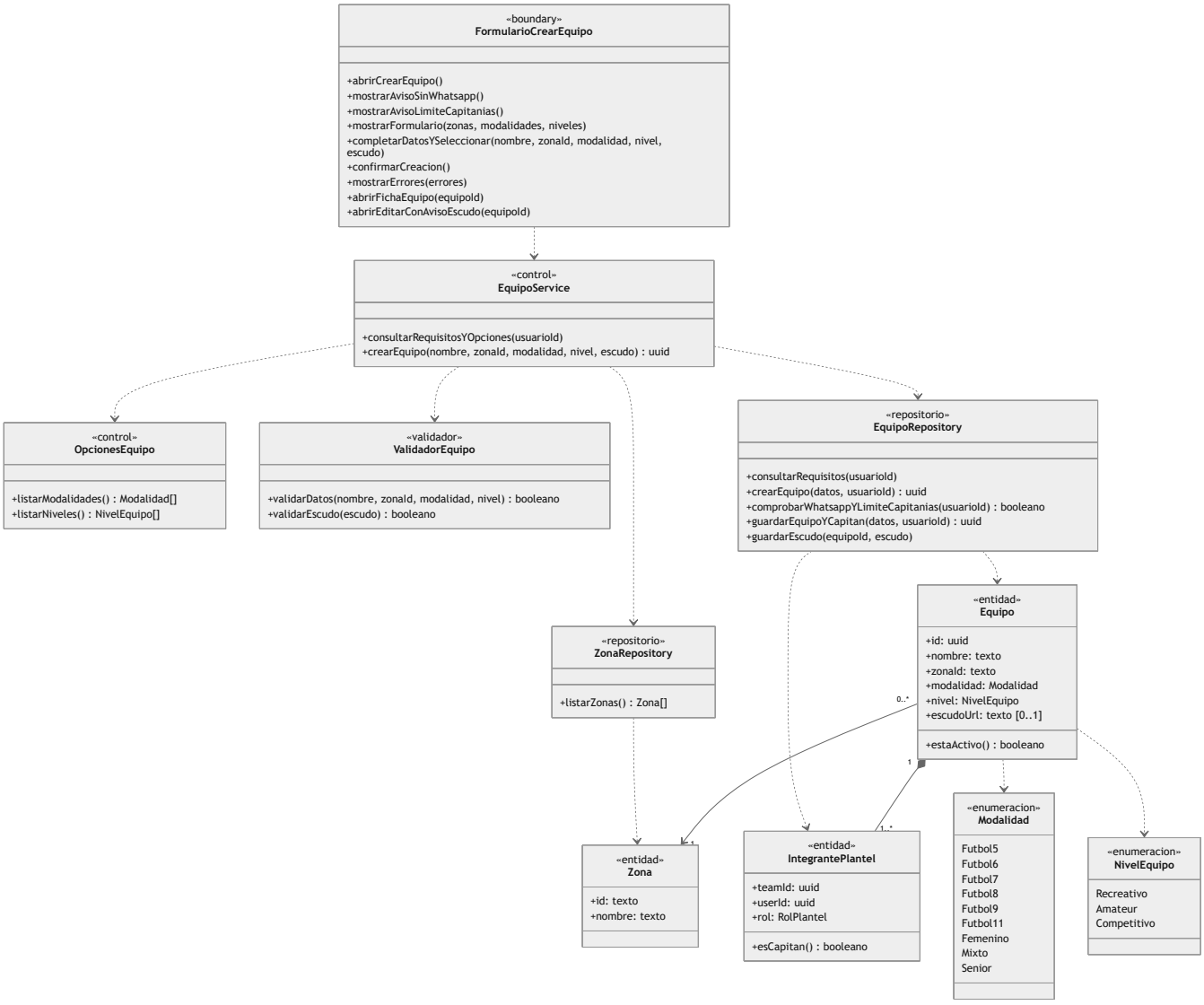
- **El escudo se guarda después del alta**, no dentro de ella. Si falla, el Equipo ya existe y la persona llega a «Editar equipo» con el aviso «El equipo se creó. …» para reintentar (UC-C01 A7).

Correspondencia en el código

`createTeam` (`src/app/crear-equipo/actions.ts`) hace: `parseTeamForm` → `parseCrest` → RPC `create_team` → `saveTeam` si hubo escudo. Las Zonas salen de `zones` mediante `getZoneGroups()` ; modalidad y nivel, de `SPORT_MODALITIES` y `TEAM_LEVELS` .

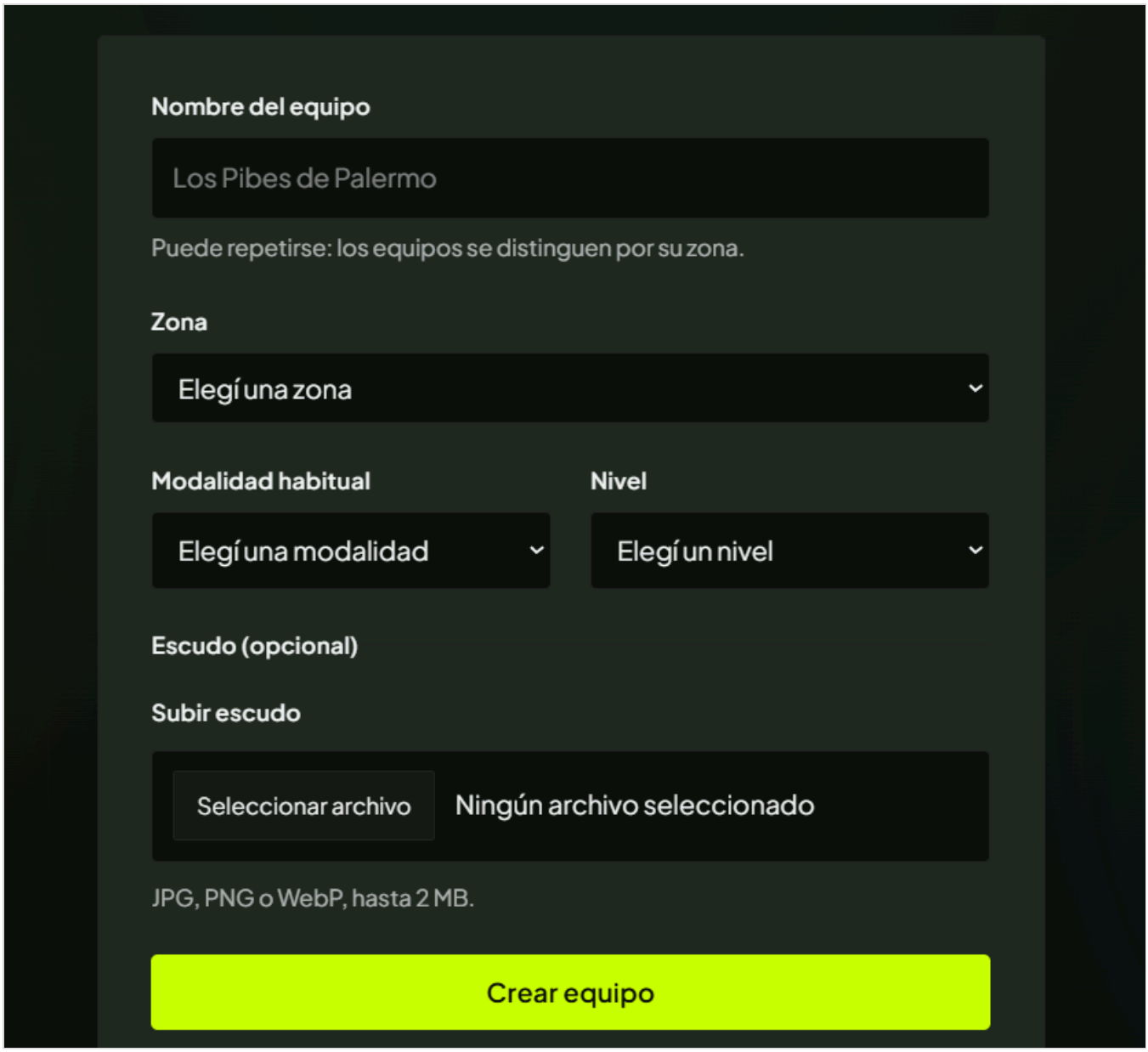
Diagrama de clases · **UC-C01 · Crear equipo**

Caso de uso: UC-C01 · **Pantalla:** P1 · Crear equipo (`/crear-equipo`) · **Secuencia:** S01



Pantalla · **P1 · Crear equipo**

Ruta: `/crear-equipo` · **Caso de uso:** UC-C01 · **Requerimientos:** RF-C01 a RF-C06 · **Diagrama:** S01



Captura ✅ vigente del formulario. Falta la cabecera de la página («Crear equipo», «Vas a quedar como capitán…»): ver pendientes.

Qué muestra. Un formulario con:

Campo	Tipo	De dónde salen las opciones	Obligatorio
Nombre del equipo	texto (2 a 50 caracteres)	—	Sí
Zona	desplegable, agrupado por corredor	Se **consulta en la base** (`zones` , vía `getZoneGroups`)	Sí
Modalidad habitual	desplegable	Lista **fija del código** (`SPORT_MODALITIES` : Fútbol 5, 7, 8 y 11)	Sí
Nivel	desplegable	Lista **fija del código** (`TEAM_LEVELS` : Recreativo, Intermedio, Competitivo)	Sí
Escudo (opcional)	archivo JPG, PNG o WebP, hasta 2 MB	—	No

Debajo del nombre, el texto «Puede repetirse: los equipos se distinguen por su zona» (RF-C03). El botón principal es «Crear equipo».

Estados de la pantalla

Estado	Qué ve la persona
Normal	El formulario habilitado.
Sin WhatsApp en el perfil	Aviso ámbar «Para crear un equipo necesitás tener tu WhatsApp en el perfil…» con el botón «Cargar mi WhatsApp»; el formulario se ve **deshabilitado**.
Ya capitanea 3 equipos	En lugar del formulario: «Ya sos capitán de 3 equipos, que es el máximo. Para crear otro, pasá la capitanía de alguno o disolvelo».
Error al enviar	Banda roja arriba del formulario con el motivo.
Sin sesión	No se ve la pantalla: se redirige a `/login?next=/crear-equipo`.

Los avisos de WhatsApp y límite anticipan lo que `create_team` comprueba al guardar; **la base es la que decide**.

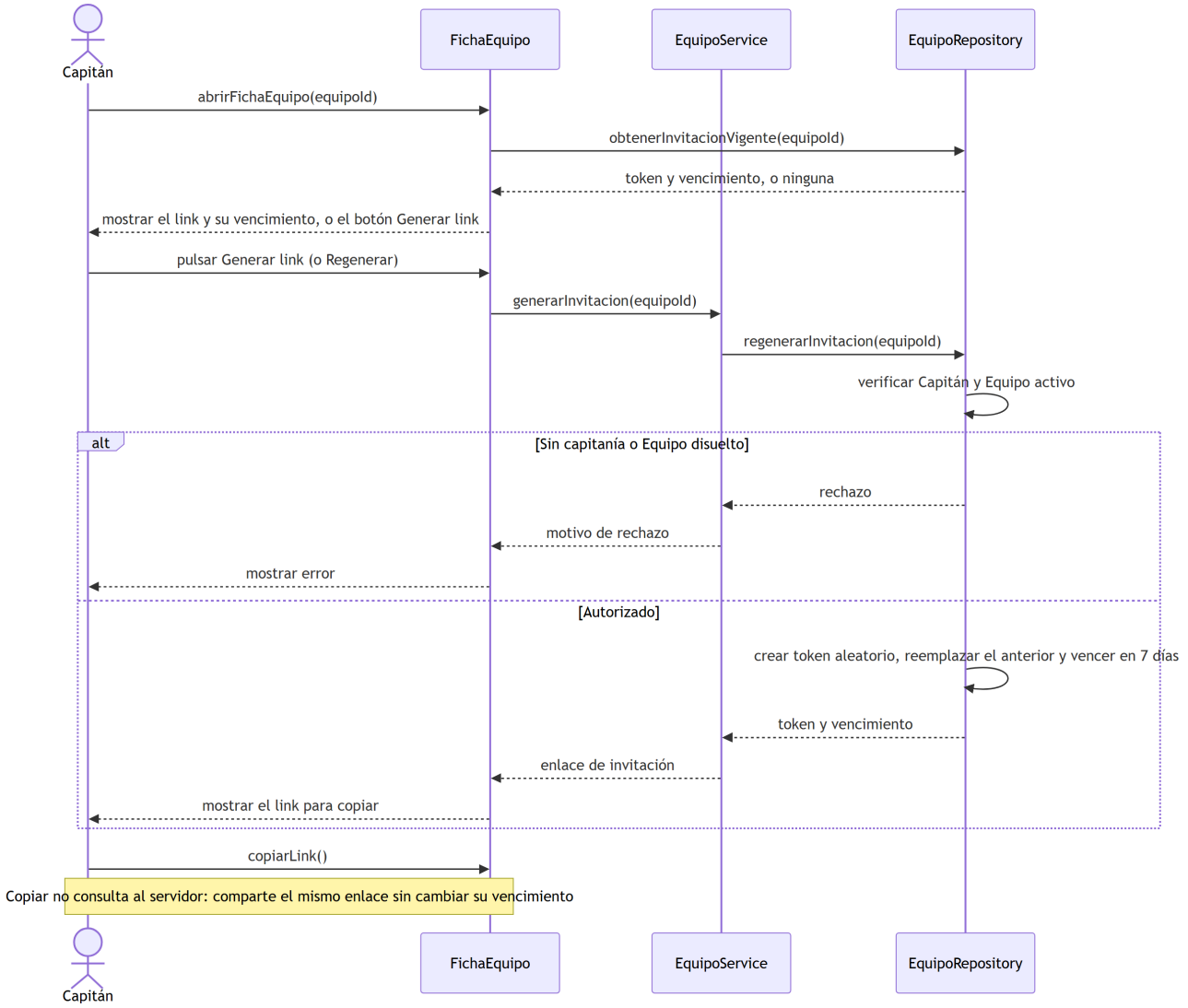
UC-C02 · Invitar jugadores al equipo

- **Actor:** Capitán.
- **Requerimientos:** RF-C07, RF-C25. **Pantalla:** P4 · Equipo, vista del Capitán (sección «Invitar jugadores»). **Diagrama:** S02.
- **Precondiciones:** Equipo no disuelto; el actor es su Capitán.
- **Postcondición:** un único link vigente por Equipo.
- **Estado:** Implementado (`regenerateInvite` , `get_team_invite` , `CopyLinkButton`).

Diagrama de secuencia · **S02 · UC-C02 — Invitar jugadores al equipo**

Caso de uso: UC-C02 · Pantalla: P4 (sección «Invitar jugadores») · Requerimientos: RF-C07, RF-C25

No hay desplegables: la pantalla solo muestra el link vigente o el botón para generarlo.



Cómo se lee

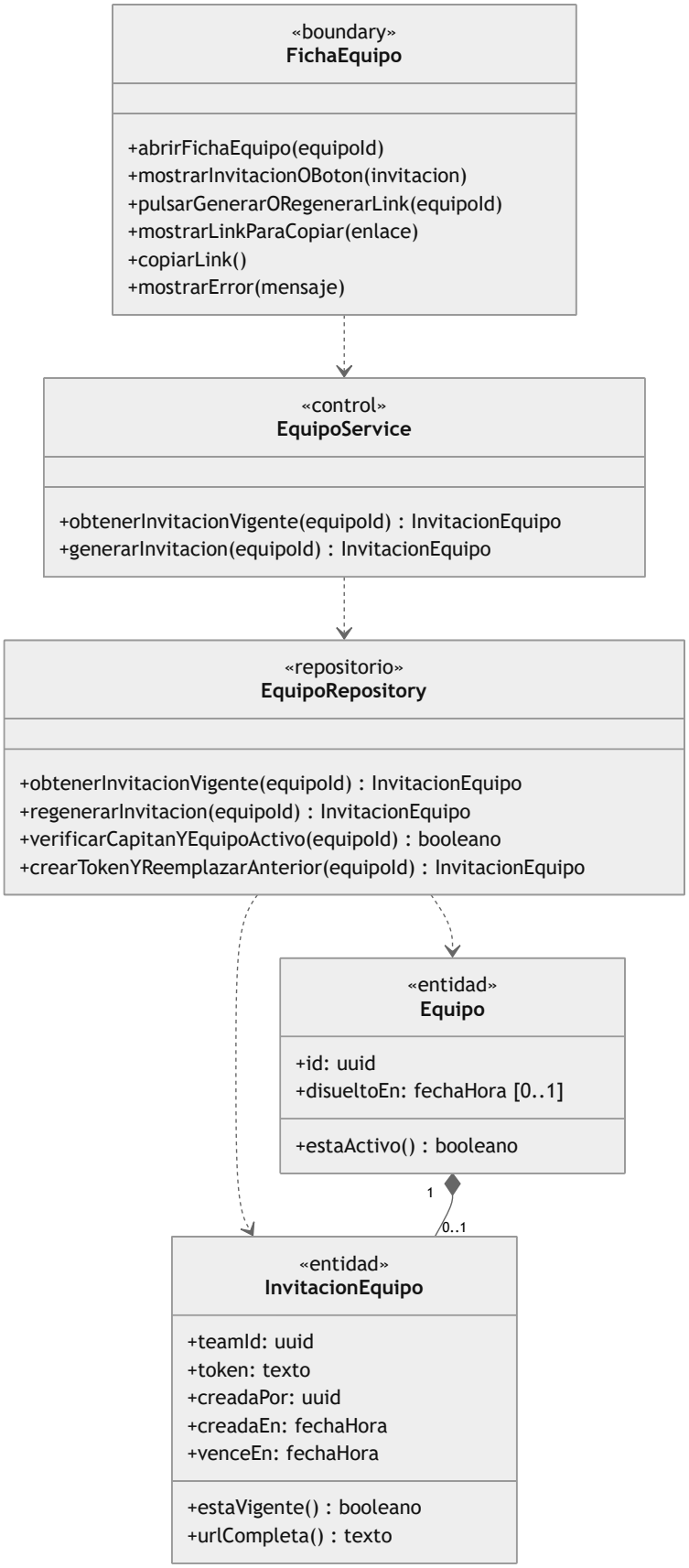
- **Generar y regenerar son la misma operación** (`regenerarInvitacion`): hay una sola fila de invitación por Equipo y regenerarla pisa el token, con lo que **el link anterior deja de valer**. Sin link vigente el botón dice «Generar link»; con uno, «Regenerar».
- **Quién puede:** solo el Capitán de un Equipo activo. La pantalla no ofrece la sección a nadie más, pero la base lo verifica igual.
- **El token** son 16 bytes aleatorios en hexadecimal y la tabla no se puede leer directo: solo se toca desde funciones (RNF-C05).
- Pasados los 7 días, `obtenerInvitacionVigente` no devuelve nada y la pantalla vuelve a ofrecer «Generar link».

Correspondencia en el código

`regenerateInvite` (`src/app/equipos/[id]/actions.ts`) llama a la RPC `regenerate_team_invite`; la página lee el link con `get_team_invite` y `CopyLinkButton` copia al portapapeles.

Diagrama de clases · **UC-C02 · Invitar jugadores al equipo**

Caso de uso: UC-C02 · **Pantalla:** P4 · Equipo, vista del Capitán (sección «Invitar jugadores») · **Secuencia:** S02



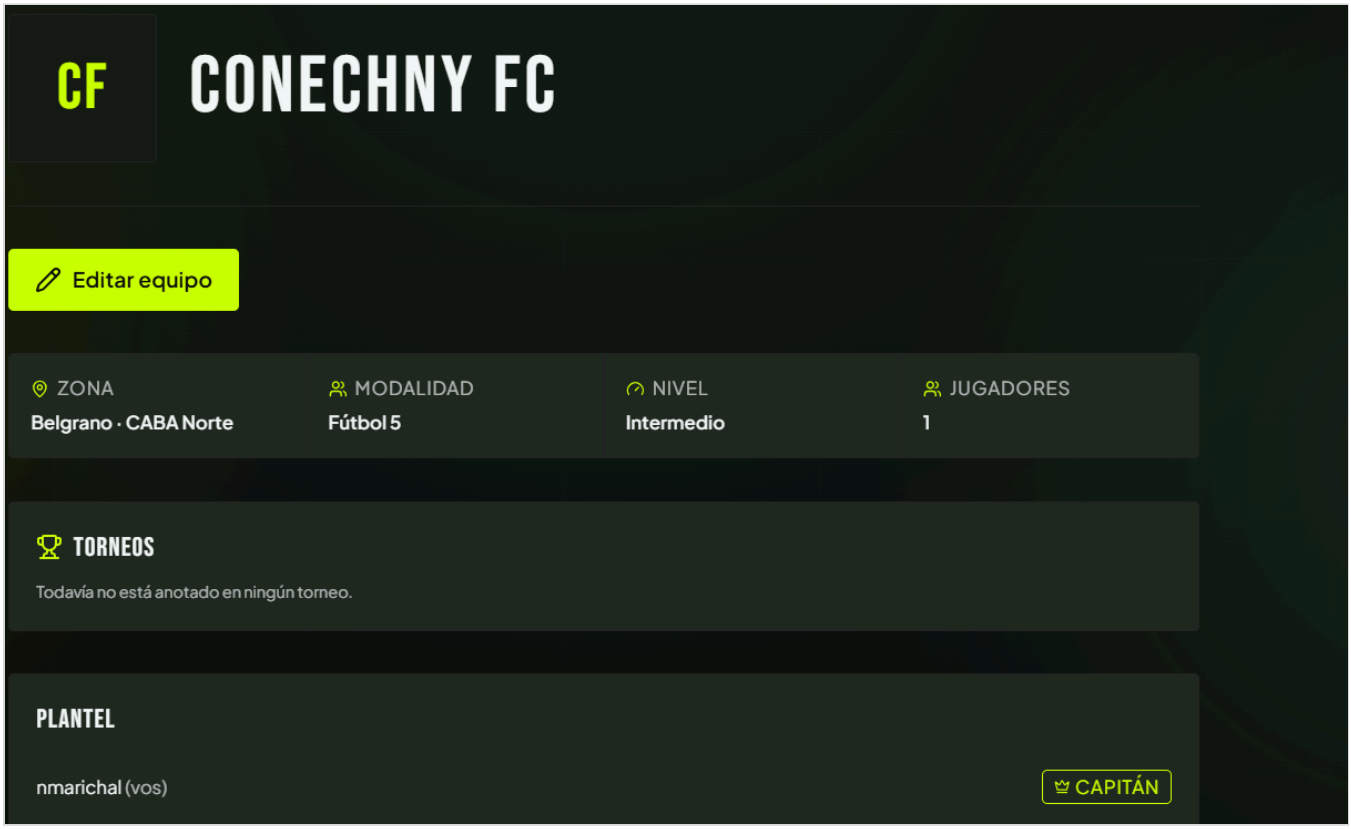
> Pantalla · **P4 · Equipo, vista del Jugador y del Capitán**

Ruta: `/equipos/{id}` · **Casos de uso:** UC-C02, UC-C04 (A1), UC-C06, UC-C07, UC-C08, UC-C09 · **Requerimientos:** RF-C07, RF-C14, RF-C17 a RF-C25 · **Diagramas:** S02, S06, S07, S08, S09

Es **la misma página que P3**: cuando quien la mira integra el Plantel, la página agrega secciones. Qué se agrega depende del rol.

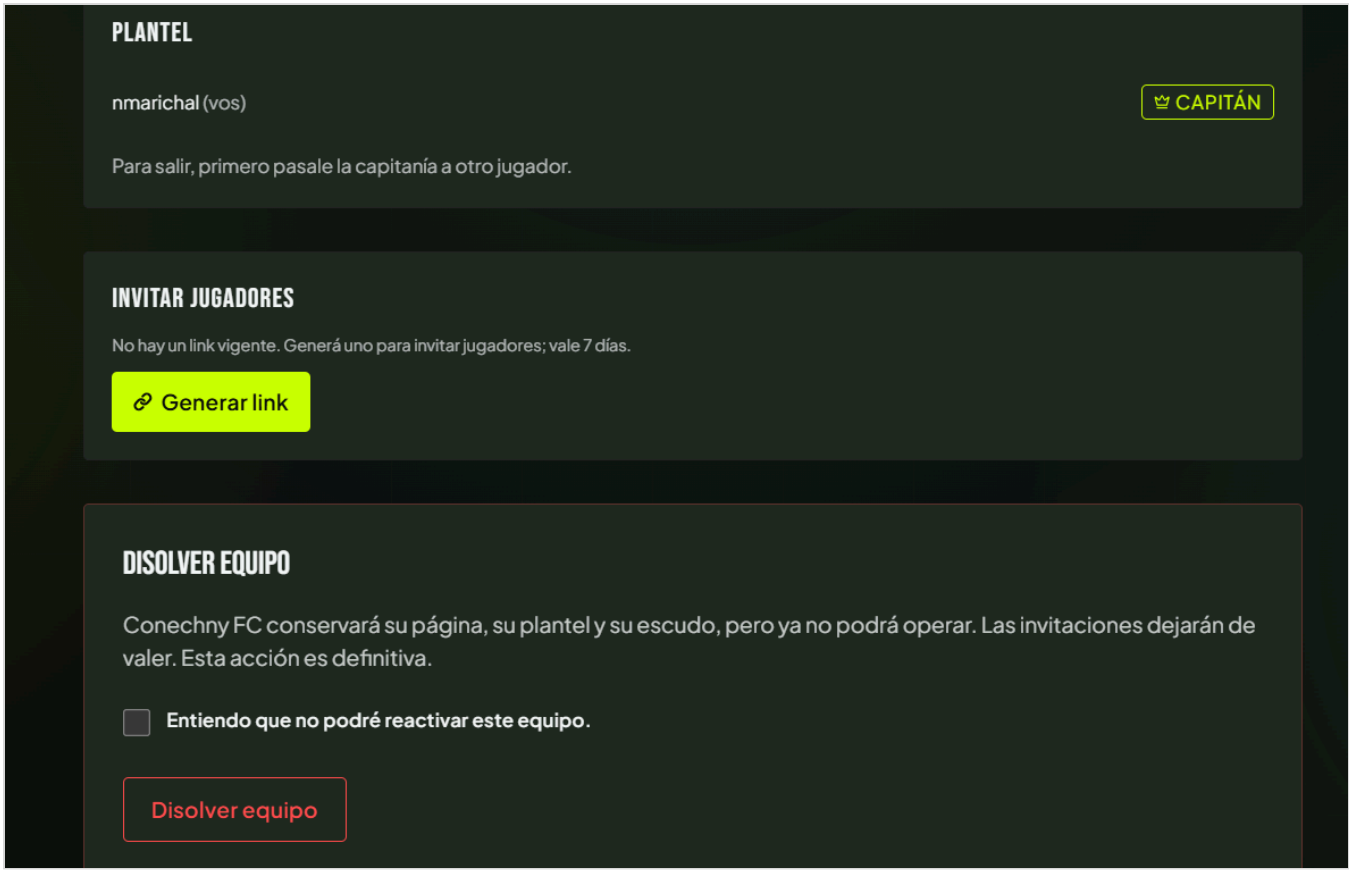
Elemento	Jugador (no Capitán)	Capitán de Equipo activo	Equipo disuelto
Sección **Plantel** (nombres, marca CAPITÁN, «(vos)»)	Sí	Sí	Sí (se conserva)
«Editar equipo»	No	Sí	No
«Hacer capitán» y «Sacar» en cada fila de otro jugador	No	Sí	No
«Salir del equipo»	Sí	No: «Para salir, primero pasale la capitanía a otro jugador»	No
Sección **Invitar jugadores** (UC-C02)	No	Sí	No
Sección **Disolver equipo** (UC-C09)	No	Sí	No
Link de vuelta	«Mis equipos»	«Mis equipos»	«Mis equipos»

Vista del Capitán: datos y Plantel



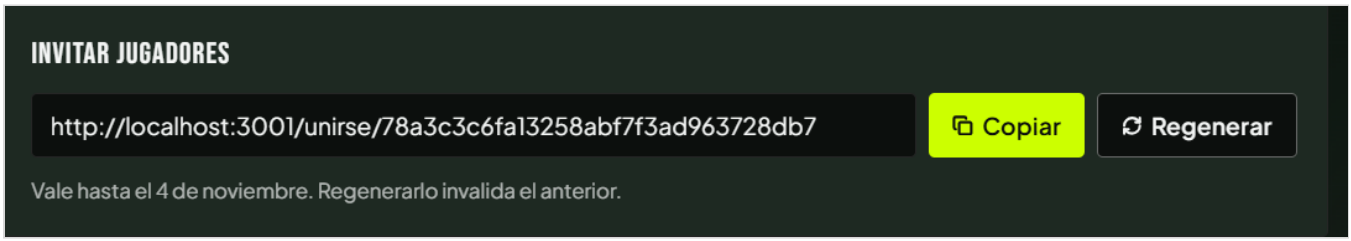
*Captura ⚠️ **desactualizada**: es del 28/10 y no tiene la sección «Partidos» (entre los datos y «Torneos»), que se agregó con el historial público. El resto coincide. Reemplazar por una captura nueva (ver pendientes).*

Vista del Capitán: invitar y disolver



Captura ✅ vigente. El Capitán es el único integrante: no hay filas con «Hacer capitán» ni «Sacar», y se ve «Para salir, primero pasale la capitanía a otro jugador». Sin link vigente, «Invitar jugadores» ofrece «Generar link». En «Disolver equipo», el botón exige tildar antes «Entiendo que no podré reactivar este equipo».

Vista del Capitán: link de invitación vigente



Captura ✅ vigente. Con un link vigente se ve el enlace, «Copiar» y «Regenerar», y el texto «Vale hasta el 4 de noviembre. Regenerarlo invalida el anterior». El token de la captura es un ejemplo de un entorno local.

Confirmaciones. «Hacer capitán», «Sacar», «Salir del equipo» y «Disolver equipo» piden confirmación con un cuadro del navegador (`window.confirm`) antes de enviar: «¿Pasarle la capitanía a {nombre}? Vas a quedar como jugador», «¿Sacar a {nombre} del equipo?», «¿Salir del equipo?» y «¿Disolver "{nombre}"? La página y la historia se conservan. No se puede deshacer».

Mensajes de la página: error (recuadro con el motivo que devolvió la base), «Los cambios del equipo se guardaron» y «El equipo quedó disuelto. Su página y su historia se conservan».

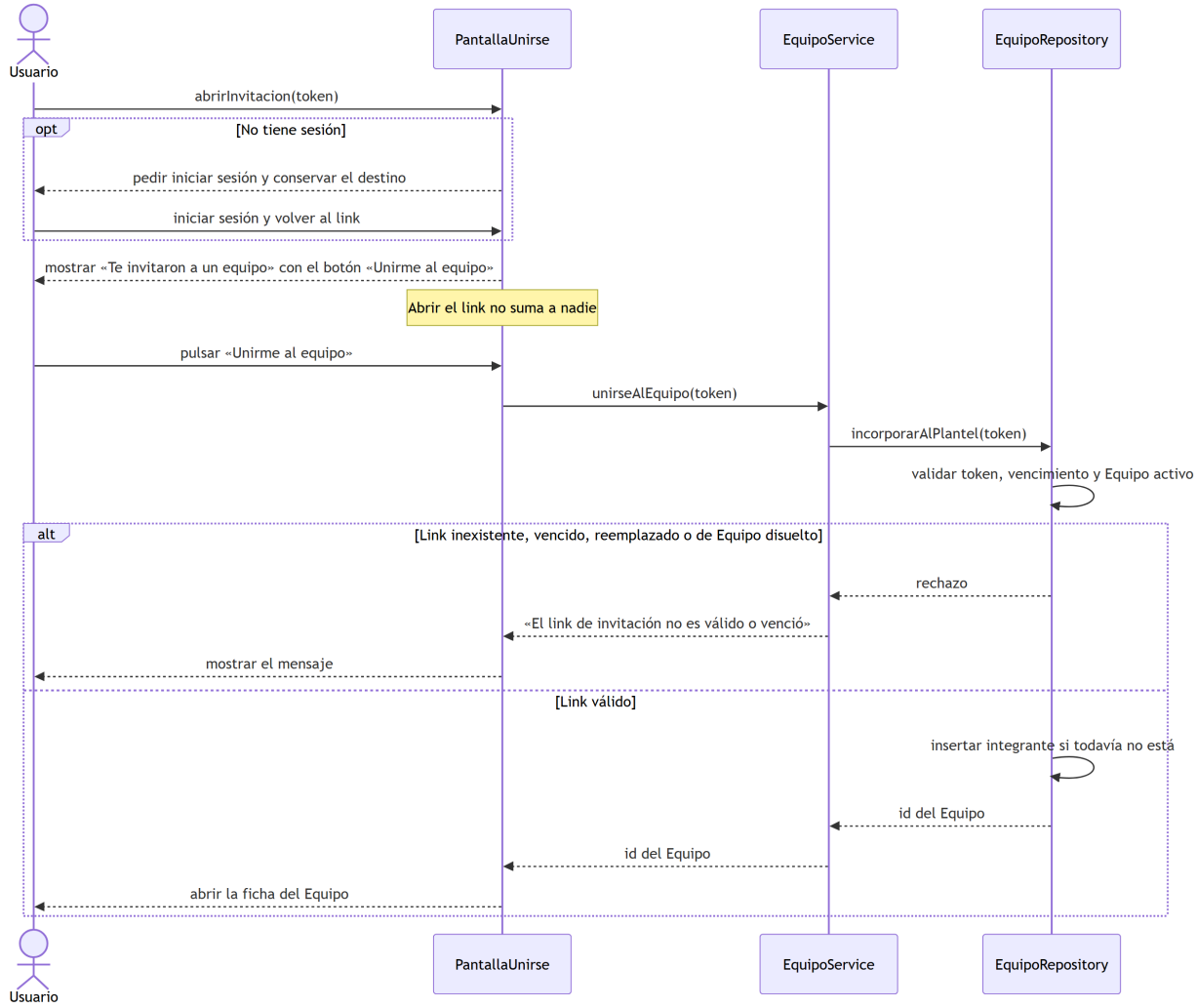
UC-C03 · Unirse a un equipo con el link

- **Actor:** Usuario.
- **Requerimientos:** RF-C08, RF-C09, RF-C10, RF-C27. **Pantalla:** P6 · Unirse al equipo (`/unirse/{token}`). **Diagrama:** S03.
- **Precondiciones:** tiene un link vigente.
- **Postcondición:** el Jugador ve el Plantel y los partidos del Equipo, sin poder operar.
- **Estado:** Implementado (`src/app/unirse/[token]` , función `join_team`).

Diagrama de secuencia · **S03 · UC-C03 — Unirse a un equipo con el link**

Caso de uso: UC-C03 · Pantalla: P6 · Requerimientos: RF-C08, RF-C09, RF-C10

No hay desplegables. La pantalla tiene un único botón.



Cómo se lee

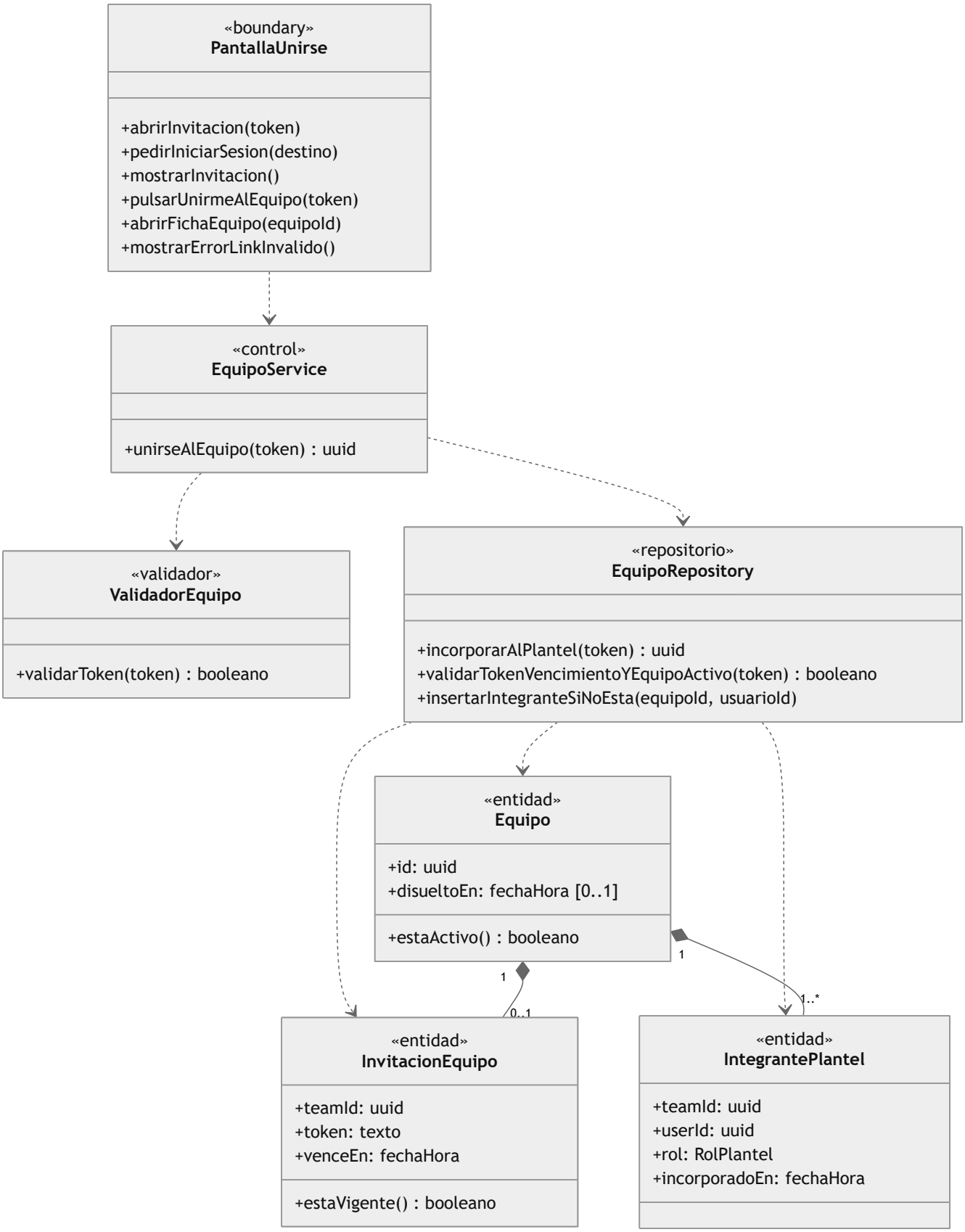
- **Unirse es un botón, no un efecto de abrir la página.** Las aplicaciones de chat y los navegadores precargan los links; si abrir `/unirse/{token}` sumara al Plantel, una vista previa de WhatsApp incorporaría gente sin que lo sepa. Por eso el paso 1 solo muestra la pantalla y el alta ocurre en el paso 2.
- **Un solo mensaje** para link inexistente, vencido, regenerado o de un Equipo disuelto. No se revela cuál fue la causa.
- **Si ya integra el Plantel**, `incorporarAlPlantel` no falla ni duplica: lo lleva a la ficha del Equipo.
- **Sin aprobación del Capitán**: quien tiene un link vigente entra directo.
- **El destino se conserva** al iniciar sesión (`/login?next=/unirse/{token}`), así que después del login la persona vuelve a esta pantalla.
- Un Usuario suspendido puede sumarse; lo que no podrá es recibir la capitanía (UC-C06).

Correspondencia en el código

`UnirsePage` redirige al login sin sesión y muestra el botón; `joinTeam` (`src/app/unirse/[token]/actions.ts`) llama a la RPC `join_team`, que valida el link y hace `insert … on conflict do nothing`.

Diagrama de clases · **UC-C03 · Unirse a un equipo con el link**

Caso de uso: UC-C03 · **Pantalla:** P6 · Unirse al equipo (`/unirse/{token}`) · **Secuencia:** S03



> Pantalla · **P6 · Unirse al equipo**

Ruta: `/unirse/{token}` · **Caso de uso:** UC-C03 · **Requerimientos:** RF-C08 a RF-C10 · **Diagrama:** S03

> ⏳ **Captura pendiente.**

Qué muestra. Una tarjeta centrada con el ícono de invitación, el título **«Te invitaron a un equipo»**, el texto «Al unirte vas a formar parte del plantel y el capitán te va a ver en él» y el botón **«Unirme al equipo»**. Unirse es un botón y **no un efecto de abrir la página**: los chats y los navegadores precargan los links y eso nunca debe sumar a alguien a un Plantel.

Estados: *error* («El link de invitación no es válido o venció», con el mismo texto para un link inexistente, vencido, regenerado o de un Equipo disuelto). *Sin sesión:* no se ve la pantalla; se redirige a `/login?next=/unirse/{token}`.

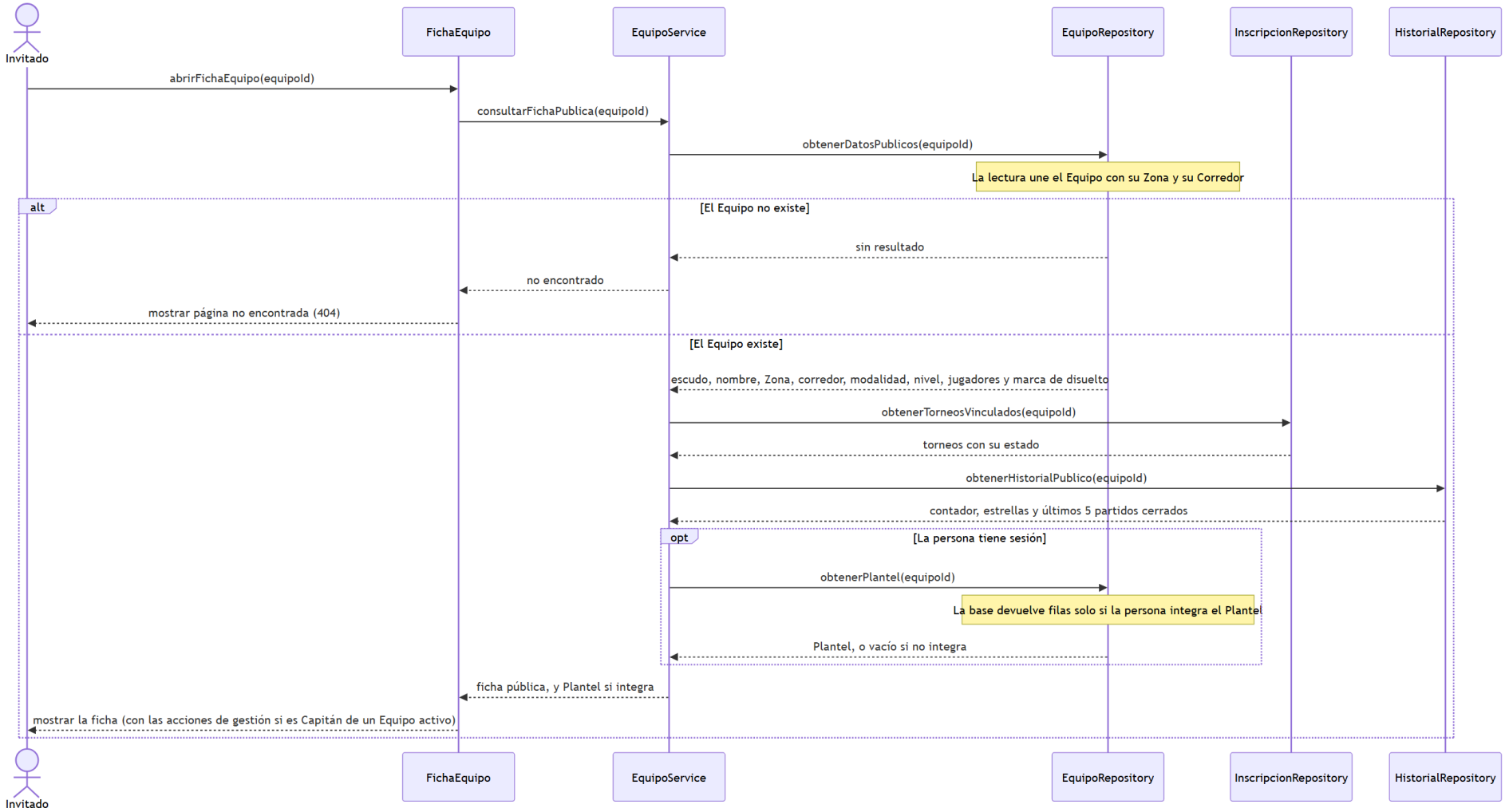
UC-C04 · Ver la página pública del equipo

- **Actor:** Invitado (también cualquier Usuario).
- **Requerimientos:** RF-C11 a RF-C14, RF-C25. **Pantalla:** P3 · Equipo, vista pública y P4 (`/equipos/{id}`). **Diagrama:** S04.
- **Precondiciones:** el Equipo existe. No hace falta sesión.
- **Privacidad:** el Plantel y los contactos no son públicos; el WhatsApp no aparece nunca en esta página.
- **Estado:** Implementado (`src/app/equipos/[id]` , vistas `team_record` y `team_match_history`).

Diagrama de secuencia · **S04 · UC-C04 — Ver la página pública del equipo**

Caso de uso: UC-C04 · Pantallas: P3 y P4 · Requerimientos: RF-C11 a RF-C14

No hay desplegables ni envío de datos: es una consulta. El actor es un Invitado; si quien mira tiene sesión, puede sumarse el Plantel.



Cómo se lee

- **Lo público no necesita sesión:** datos del Equipo, torneos vinculados e historial. La página `/equipos/{id}` no está protegida a propósito.
- **El Plantel es privado.** La pantalla no decide quién lo ve: pide las filas y **la base responde vacío** si quien pregunta no integra el Equipo (RLS sobre `team_members`). Un resultado vacío significa «no sos integrante» (o «sin sesión»).
- **El historial no expone cancha ni hora.** Los partidos solo los lee su plantel; el historial público lo arman funciones que devuelven solo lo que se puede mostrar (RNF-C12).
- **Las acciones de gestión** (UC-C02, C05 a C09) aparecen según el rol que resulta de ese Plantel: Capitán de un Equipo activo, Jugador, o ninguno. Un Equipo disuelto no ofrece acciones.
- **Torneos vinculados:** son datos del módulo de torneos (vista pública `tournament_inscriptions`). Un torneo dado de baja se lleva sus inscripciones, así que acá solo quedan los vigentes.

Correspondencia en el código

`EquipoPage` (`src/app/equipos/[id]/page.tsx`) lee `teams` (con `zones` y `corridors`), `team_members` (filtrado por RLS), `tournament_inscriptions` + `tournaments`, y las vistas `team_record` y `team_match_history`. Sin resultado de `teams`, llama a `notFound()`.

Diagrama de clases · **UC-C04 · Ver la página pública del equipo**

Caso de uso: UC-C04 · **Pantalla:** P3 · Equipo, vista pública y P4 (`/equipos/{id}`) · **Secuencia:** S04



Pantalla · **P3 · Equipo, vista pública**

Ruta: `/equipos/{id}` · **Casos de uso:** UC-C04, UC-C10 (pendiente) · **Requerimientos:** RF-C11 a RF-C14 · **Diagrama:** S04

> ⏳ **Captura pendiente:** la vista de un Invitado, sin sesión, con la sección «Partidos». La captura de la ficha del Capitán de abajo (P4) es la más cercana, pero es anterior a esa sección.

Qué muestra a cualquier persona, con o sin sesión:

1. **Cabecera:** escudo, nombre y, si está disuelto, la marca «Equipo disuelto».
2. **Datos:** Zona (con su corredor), Modalidad, Nivel y Jugadores.
3. **Partidos:** el contador («Jugó N · N faltas · N conflictos…»), las estrellas y los últimos 5 partidos cerrados, cada uno con fecha, «vs» el rival (link a su página) y el desenlace. Sin partidos: «Todavía no tiene partidos cerrados».
4. **Torneos:** los torneos con Inscripción vinculada, con fecha y estado. Sin torneos: «Todavía no está anotado en ningún torneo».

No muestra el Plantel ni ningún contacto. Para un Invitado el link de vuelta dice «Volver al catálogo».

Estados: *Equipo disuelto* (marca visible, sin acciones); *no existe* (404).

> Pantalla · **P4 · Equipo, vista del Jugador y del Capitán**

Es la misma pantalla que se describe, con su captura, en P4 · Equipo, vista del Jugador y del Capitán, debajo de UC-C02 · Invitar jugadores al equipo. Este caso usa la parte que indica su ficha («Pantalla»).

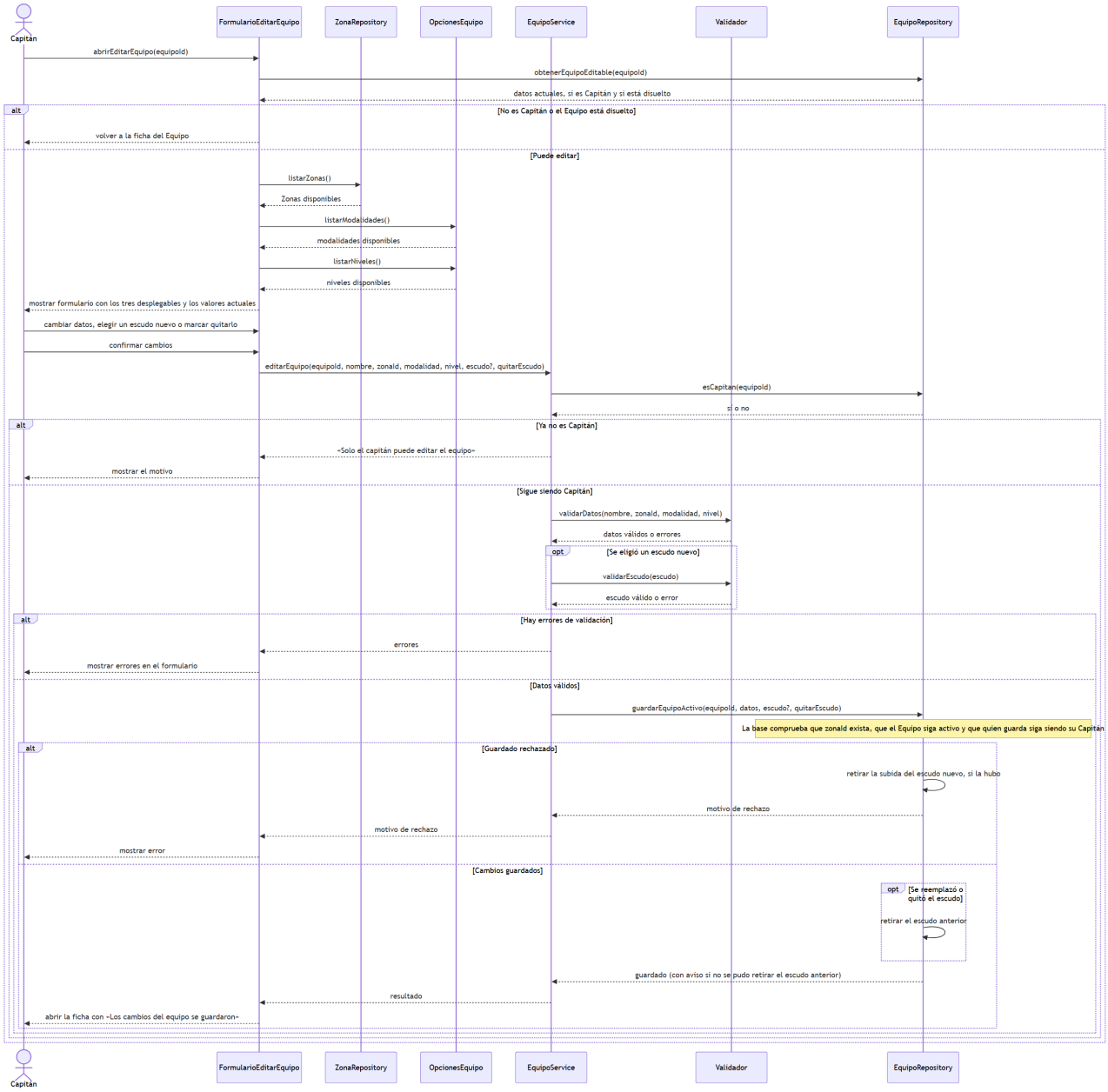
UC-C05 · Editar equipo

- **Actor:** Capitán.
- **Requerimientos:** RF-C01, RF-C02, RF-C16, RF-C25. **Pantalla:** P5 · Editar equipo (`/equipos/{id}/editar`). **Diagrama:** S05.
- **Precondiciones:** Equipo no disuelto; el actor es su Capitán.
- **Postcondición:** ficha actualizada.
- **Estado:** Implementado (`updateTeam` , `saveTeam`).

Diagrama de secuencia · **S05 · UC-C05 — Editar equipo**

Caso de uso: UC-C05 · Pantalla: P5 · Requerimientos: RF-C01, RF-C02, RF-C16

Es el mismo formulario que S01: comparte los tres desplegables, así que sigue el mismo criterio. La carga de las opciones y el guardado van en este único diagrama. La diferencia es que el formulario llega con los **valores actuales** y que antes de guardar se **vuelve a comprobar que quien edita sigue siendo el Capitán**.



Cómo se lee

- **Mismo patrón que crear:** la pantalla obtiene las Zonas de `ZonaRepository` y las modalidades y niveles de `OpcionesEquipo` antes de mostrar el formulario, y al enviar manda solo los **valores elegidos**. Los valores actuales del Equipo vienen de `obtenerEquipoEditable`.
- **Quién puede editar se comprueba dos veces:** al abrir (si no es Capitán o el Equipo está disuelto, no hay formulario) y al guardar (`esCapitan`), porque pudo dejar de serlo mientras tenía el formulario abierto. Además, la base solo deja actualizar a un Capitán de un Equipo activo.
- **Se valida de nuevo al guardar**, igual que en el alta, y la base vuelve a comprobar que la Zona exista.

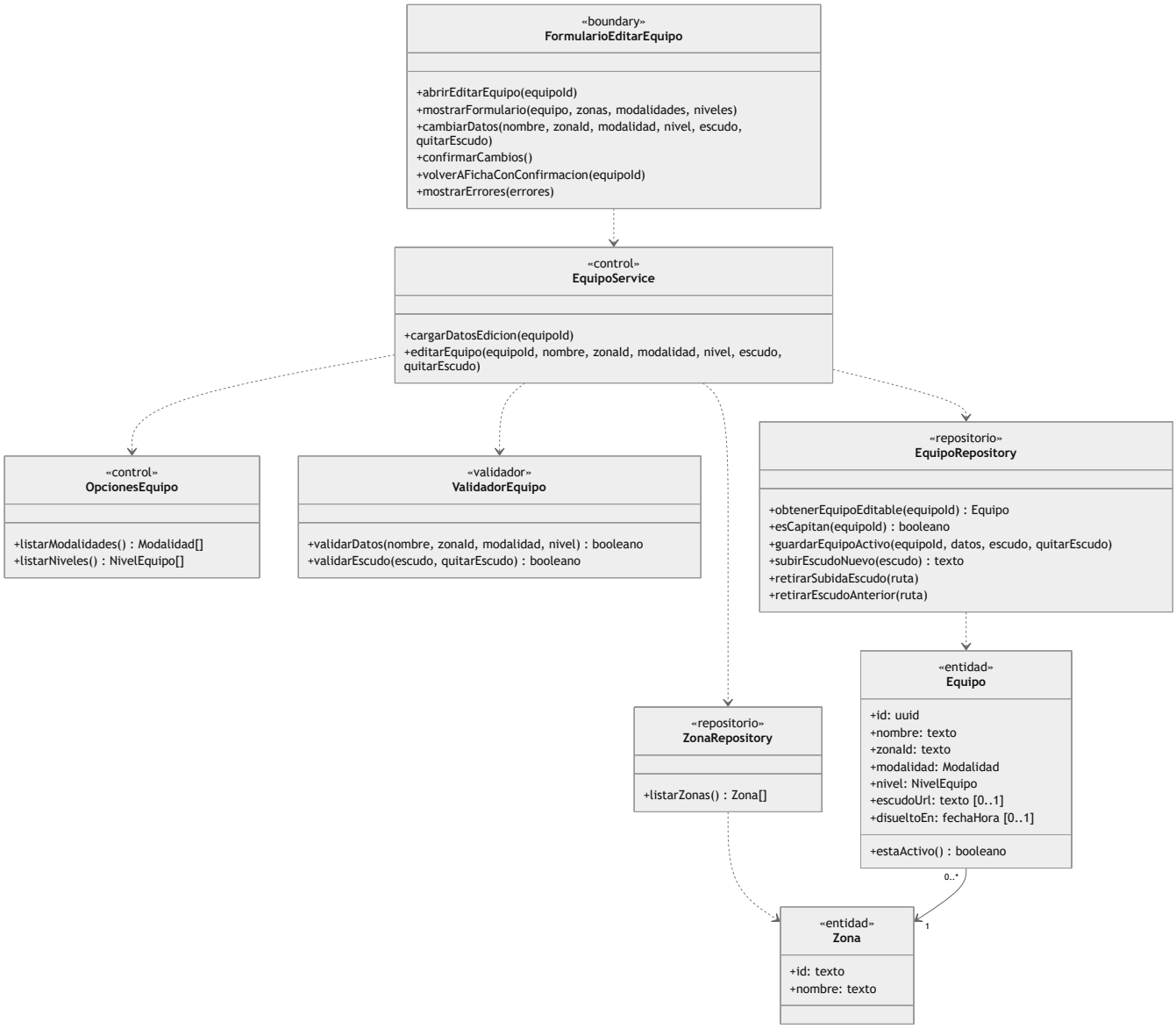
- **El escudo se maneja con cuidado de no perder el actual:** el archivo nuevo se **sube antes** de guardar; si el guardado falla se retira esa subida y el escudo anterior queda intacto; el escudo anterior **solo se retira después** de guardar. Si ese último retiro falla, la edición ya está hecha y se avisa.
- **Dejar el archivo vacío conserva el escudo actual.** Reemplazar y quitar a la vez se rechaza.
- **Cambios concurrentes:** cuando se cambia el escudo, el guardado exige que siga siendo el que se leyó, así que reemplazarlo desde otra pestaña no se pisa en silencio. Si el Equipo se disolvió o la persona dejó de ser Capitán, la actualización no encuentra ninguna fila y se rechaza.

Correspondencia en el código

`updateTeam` (`src/app/equipos/[id]/actions.ts`): RPC `is_team_captain` → `parseTeamForm` → `parseCrest` → `saveTeam` (`src/utils/supabase/team.ts`). La página es `equipos/[id]/editar/page.tsx` , con `TeamForm` .

Diagrama de clases · **UC-C05 · Editar equipo**

Caso de uso: UC-C05 · **Pantalla:** P5 · Editar equipo (`/equipos/{id}/editar`) · **Secuencia:** S05



Pantalla · **P5 · Editar equipo**

Ruta: `/equipos/{id}/editar` · **Caso de uso:** UC-C05 · **Requerimientos:** RF-C01, RF-C02, RF-C16 · **Diagrama:** S05

> ⏳ **Captura pendiente.**

Qué muestra. Cabecera «Editar equipo — Actualizá los datos de {nombre}» y **el mismo formulario que P1**, con los valores actuales cargados y el botón «Guardar cambios». Los tres desplegables (Zona, Modalidad habitual, Nivel) tienen el mismo origen que en P1. La sección del escudo agrega:

- el escudo actual;
- «Reemplazar escudo» (si ya tiene uno) en vez de «Subir escudo», y «Si no elegís otro, conservás el escudo actual»;
- la casilla «Quitar el escudo actual» (solo si tiene escudo).

Quién la ve: solo el Capitán de un Equipo activo. Cualquier otra persona vuelve a la página del Equipo; sin sesión, al login.

Estados: *error* (banda roja con el motivo, por ejemplo «El equipo se creó. No se pudo subir el escudo…» cuando se llega desde UC-C01 A7).

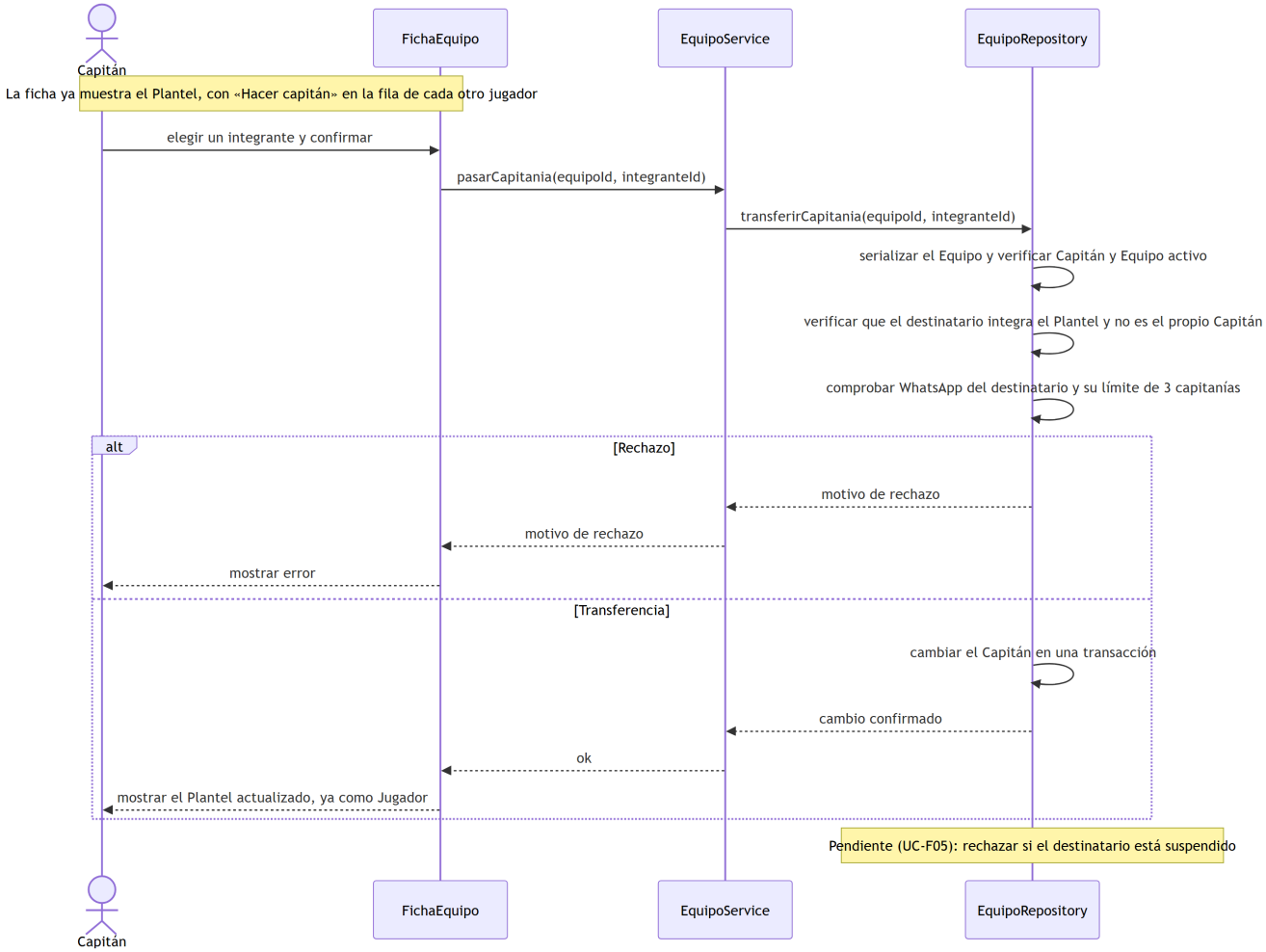
UC-C06 · Pasar la capitanía

- **Actor:** Capitán.
- **Requerimientos:** RF-C05, RF-C17, RF-C20, RF-C21, RF-C27. **Pantalla:** P4 (sección «Plantel»). **Diagrama:** S06.
- **Precondiciones:** Equipo no disuelto; el destinatario está en el Plantel, tiene WhatsApp en el perfil, no está suspendido y capitanea menos de 3 equipos; el Capitán actual no está suspendido.
- **Postcondición:** exactamente un Capitán vigente.
- **Estado:** Implementado (`transferCaptaincy` , función `transfer_captaincy`). La verificación de la Suspensión del destinatario está pendiente (UC-F05).

Diagrama de secuencia · **S06 · UC-C06 — Pasar la capitanía**

Caso de uso: UC-C06 · Pantalla: P4 (sección «Plantel») · Requerimientos: RF-C05, RF-C17, RF-C20, RF-C21

No hay desplegables: el jugador se elige con el botón «Hacer capitán» de su fila en el Plantel, que ya cargó S04.



Cómo se lee

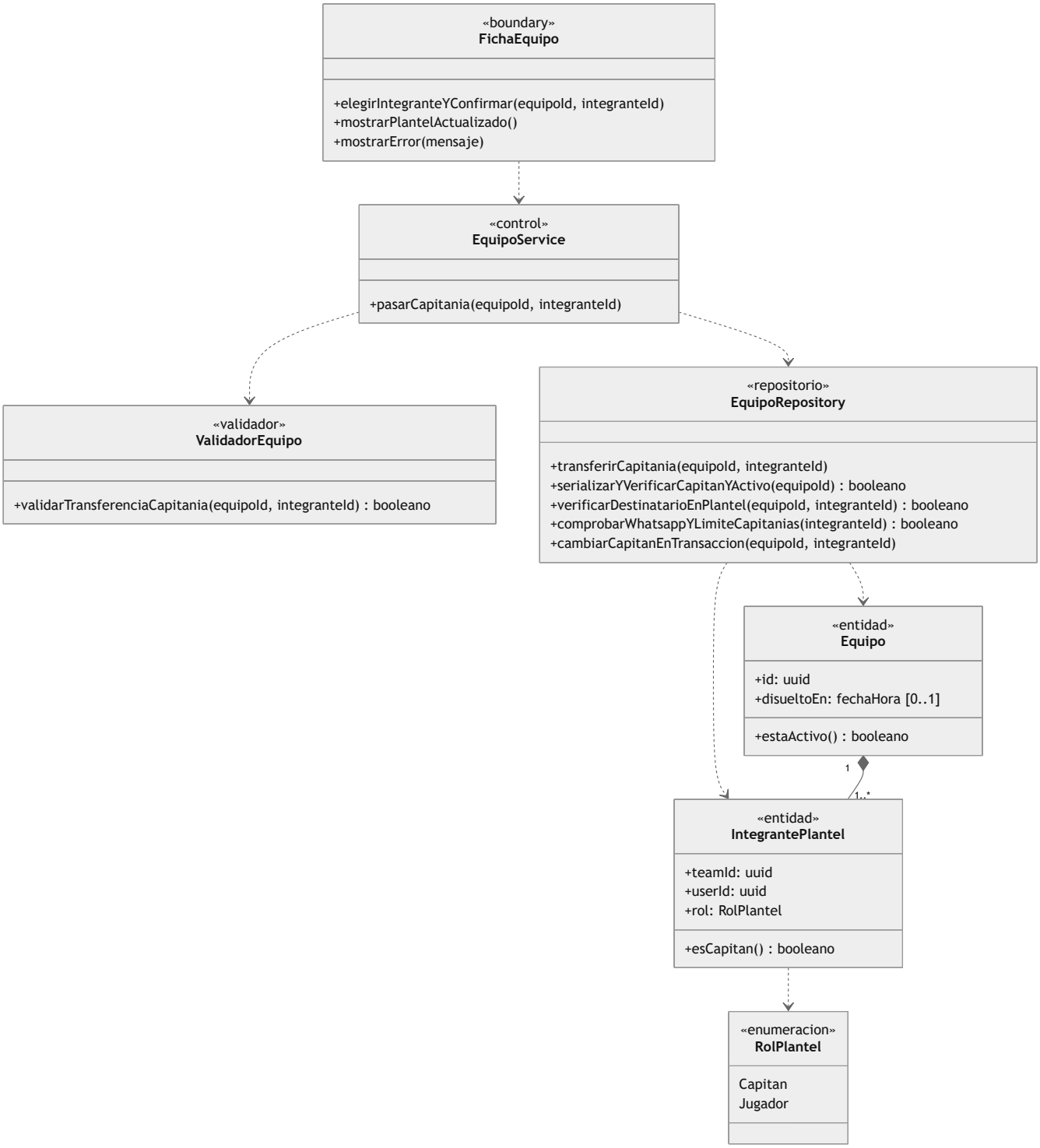
- **Un solo Capitán por Equipo, siempre.** El cambio se hace en una transacción y primero se le quita la marca al Capitán actual, porque el índice único de «un capitán por equipo» se evalúa en cada sentencia.
- **El límite de 3 capitanías se respeta aunque haya altas simultáneas:** la función bloquea el perfil del destinatario antes de contar, igual que `create_team`.
- **Nunca se bloquea por partidos en curso.** Las búsquedas, propuestas y partidos son del Equipo: el Capitán nuevo opera todo lo pendiente. El anterior deja de ver las propuestas y sigue viendo los partidos. El Capitán nuevo no hereda los Avisos del anterior.
- **Regla pendiente:** un usuario suspendido no puede recibir la capitanía. Está definida pero la base todavía no la verifica, porque la Suspensión se implementa con el módulo de administración (UC-F05).

Correspondencia en el código

`transferCaptaincy` (`src/app/equipos/[id]/actions.ts`) llama a la RPC `transfer_captaincy`. Los mensajes de rechazo llegan al usuario porque la función usa `raise exception` (código `P0001`).

Diagrama de clases · **UC-C06 · Pasar la capitanía**

Caso de uso: UC-C06 · **Pantalla:** P4 (sección «Plantel») · **Secuencia:** S06



Pantalla · **P4 · Equipo, vista del Jugador y del Capitán**

Es la misma pantalla que se describe, con su captura, en P4 · Equipo, vista del Jugador y del Capitán, debajo de UC-C02 · Invitar jugadores al equipo. Este caso usa la parte que indica su ficha («Pantalla»).

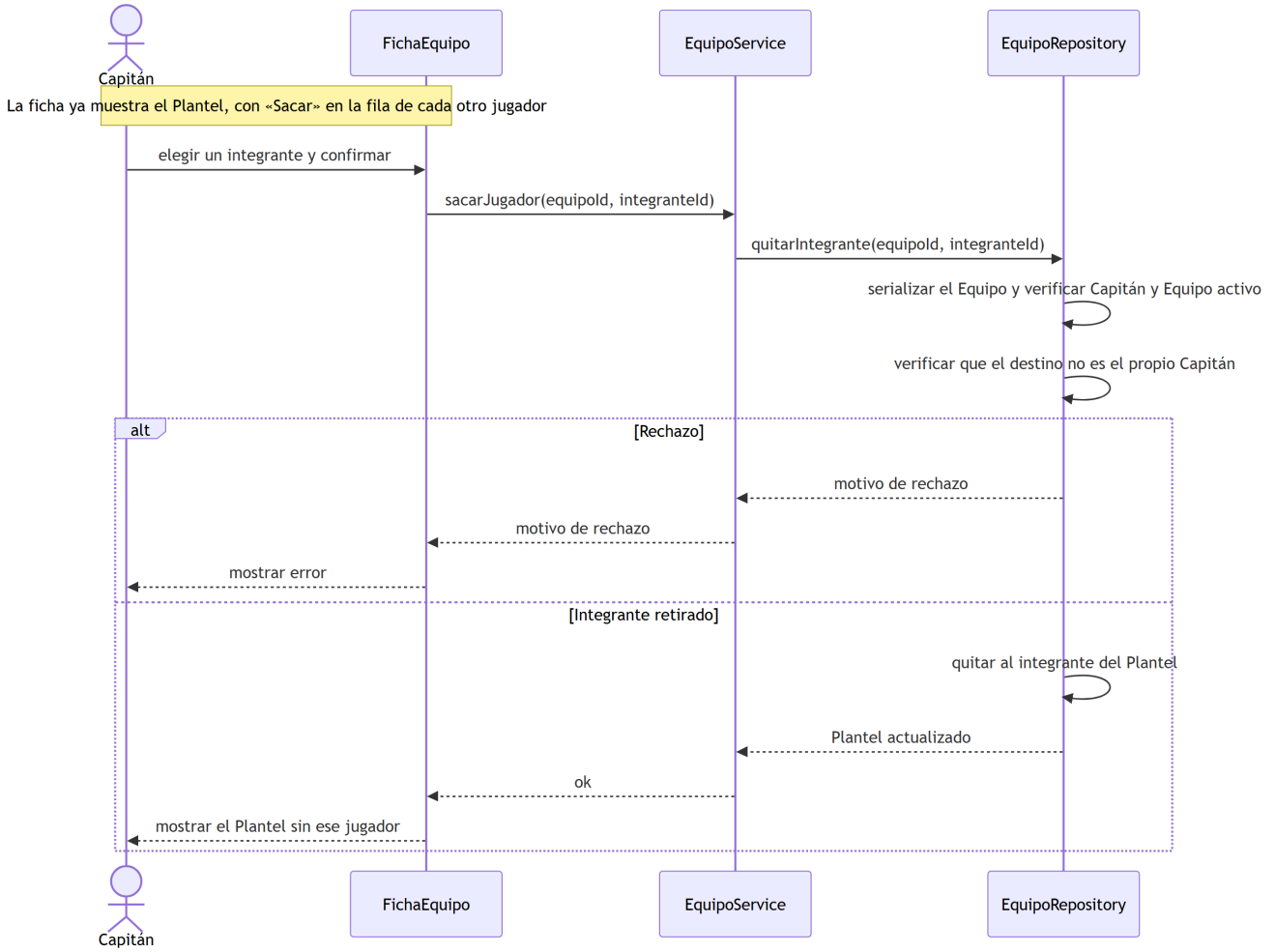
UC-C07 · Sacar a un jugador del plantel

- **Actor:** Capitán.
- **Requerimientos:** RF-C18, RF-C20, RF-C21, RF-C25. **Pantalla:** P4 (sección «Plantel»). **Diagrama:** S07.
- **Precondiciones:** Equipo no disuelto; el actor es su Capitán.
- **Postcondición:** Plantel actualizado.
- **Estado:** Implementado (`removeMember` , función `remove_team_member`).

Diagrama de secuencia · **S07 · UC-C07 — Sacar a un jugador del plantel**

Caso de uso: UC-C07 · Pantalla: P4 (sección «Plantel») · Requerimientos: RF-C18, RF-C20, RF-C21

No hay desplegables: el jugador se elige con el botón «Sacar» de su fila en el Plantel.



Cómo se lee

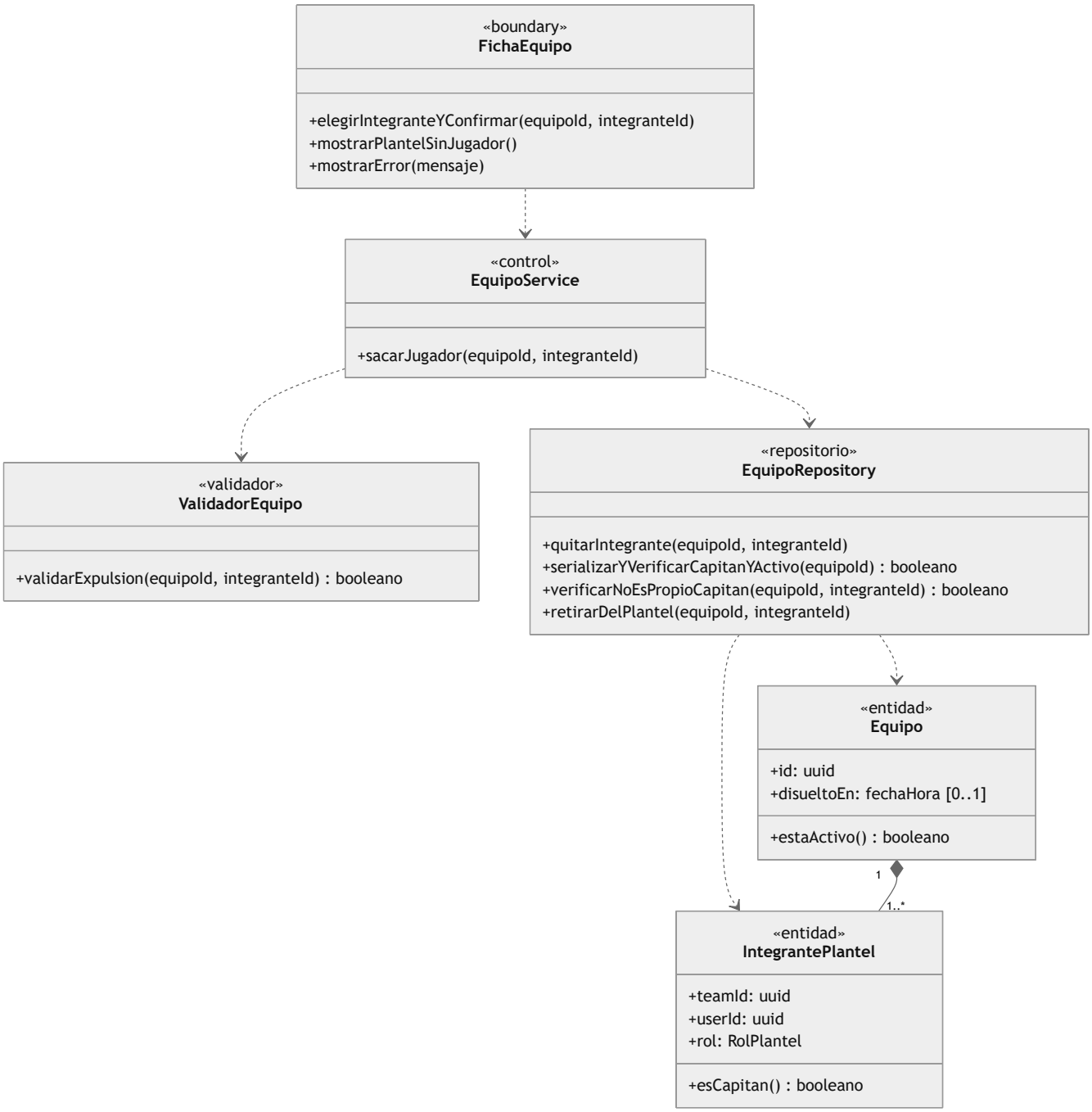
- **El Capitán no puede sacarse a sí mismo:** si quiere irse, primero pasa la capitanía (S06). Su propia fila no tiene el botón; la base lo rechaza igual («El capitán no puede sacarse a sí mismo: pasá la capitanía primero»).
- **Si el jugador ya no está** (otro lo sacó o salió mientras tanto), la base responde «Ese jugador no está en el equipo».
- **Efectos para quien sale:** deja de ver los partidos del Equipo y **conserva los Avisos** que ya recibió. Nunca se bloquea por partidos en curso.
- Un Equipo disuelto no admite expulsiones.

Correspondencia en el código

`removeMember` (`src/app/equipos/[id]/actions.ts`) llama a la RPC `remove_team_member`.

Diagrama de clases · **UC-C07 · Sacar a un jugador del plantel**

Caso de uso: UC-C07 · **Pantalla:** P4 (sección «Plantel») · **Secuencia:** S07



Pantalla · **P4 · Equipo, vista del Jugador y del Capitán**

Es la misma pantalla que se describe, con su captura, en P4 · Equipo, vista del Jugador y del Capitán, debajo de UC-C02 · Invitar jugadores al equipo. Este caso usa la parte que indica su ficha («Pantalla»).

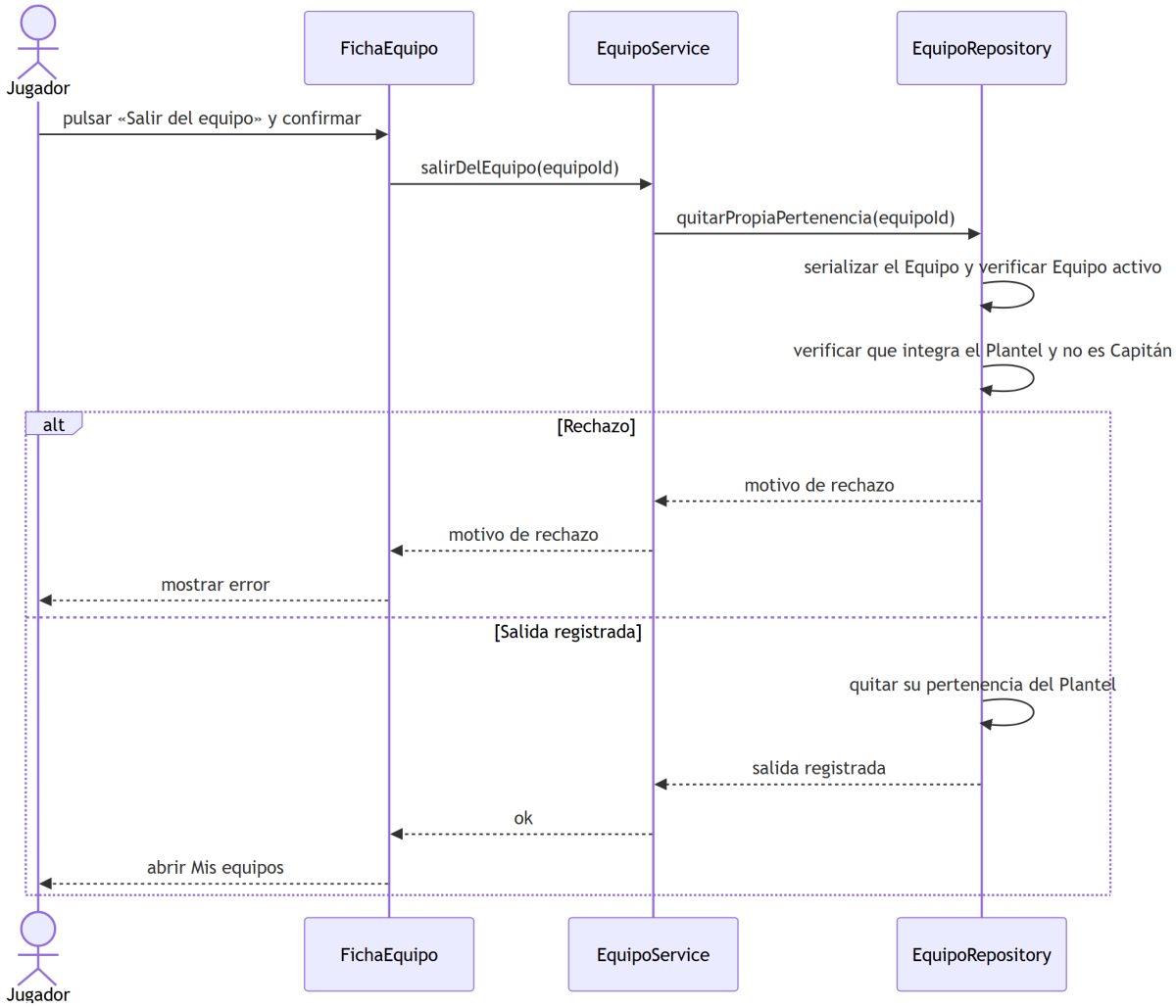
UC-C08 · Salir del equipo

- **Actor:** Jugador.
- **Requerimientos:** RF-C19, RF-C20, RF-C21, RF-C25. **Pantalla:** P4 (botón «Salir del equipo»). **Diagrama:** S08.
- **Precondiciones:** integra el Plantel y no es su Capitán; Equipo no disuelto.
- **Postcondición:** el Jugador deja de integrar el Equipo; conserva los Avisos que ya recibió.
- **Estado:** Implementado (`leaveTeam` , función `leave_team`).

Diagrama de secuencia · **S08 · UC-C08 — Salir del equipo**

Caso de uso: UC-C08 · Pantalla: P4 (botón «Salir del equipo») · Requerimientos: RF-C19, RF-C20, RF-C21

No hay desplegables ni datos que cargar: la salida es siempre la del propio actor.



Cómo se lee

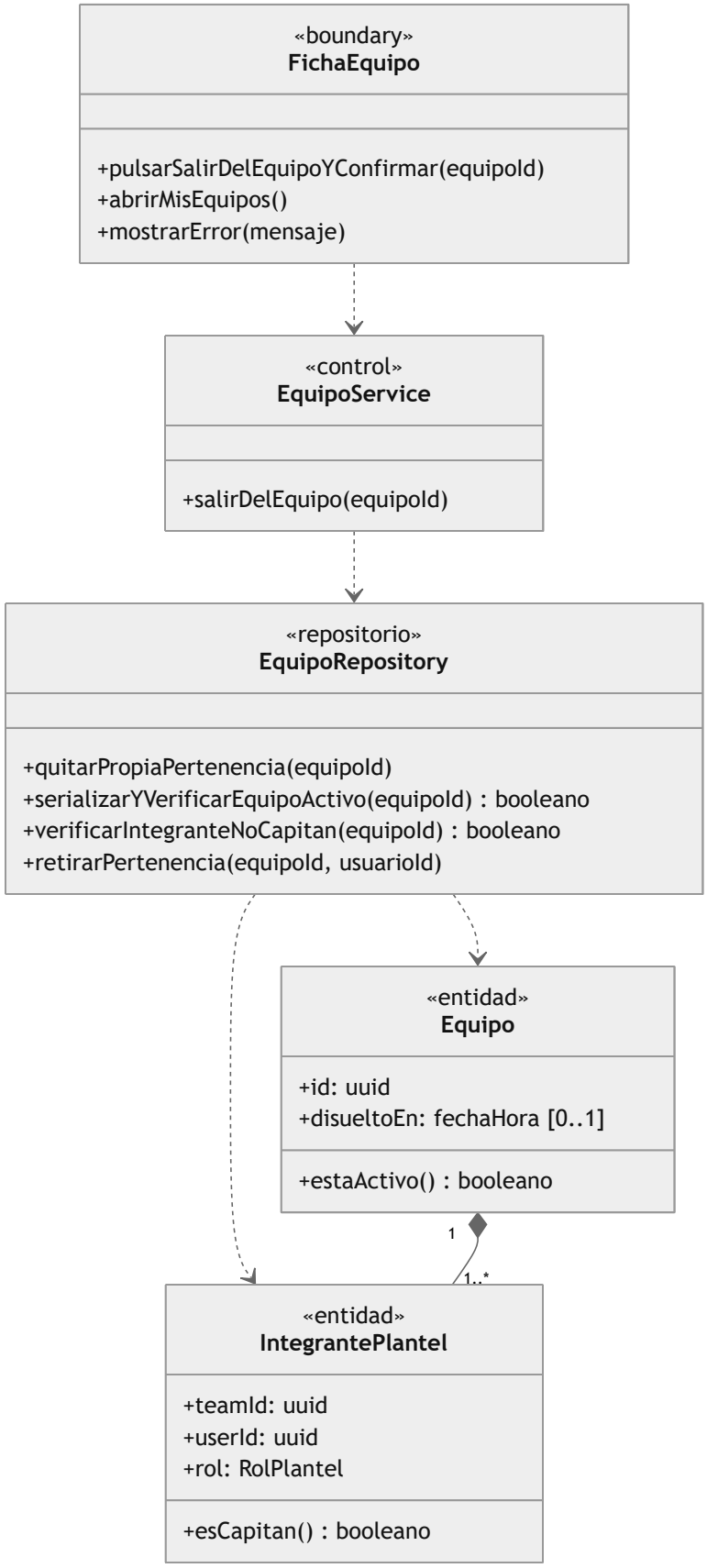
- **Quién puede salir:** cualquier Jugador **que no sea Capitán**. Al Capitán la página no le ofrece el botón y le dice «Para salir, primero pasale la capitanía a otro jugador»; si igual lo intentara, la base responde «Pasá la capitanía antes de salir del equipo» (S06).
- **Un Equipo disuelto no admite salidas** («El equipo está disuelto»).
- **Efectos:** la persona deja de ver los partidos del Equipo y conserva los Avisos que ya recibió. Nunca se bloquea por partidos en curso.
- No hay confirmación del Capitán: la salida es unilateral.

Correspondencia en el código

`leaveTeam` (`src/app/equipos/[id]/actions.ts`) llama a la RPC `leave_team` y redirige a `/mis-equipos`.

Diagrama de clases · **UC-C08 · Salir del equipo**

Caso de uso: UC-C08 · **Pantalla:** P4 (botón «Salir del equipo») · **Secuencia:** S08



> Pantalla · **P4 · Equipo, vista del Jugador y del Capitán**

Es la misma pantalla que se describe, con su captura, en P4 · Equipo, vista del Jugador y del Capitán, debajo de UC-C02 · Invitar jugadores al equipo. Este caso usa la parte que indica su ficha («Pantalla»).

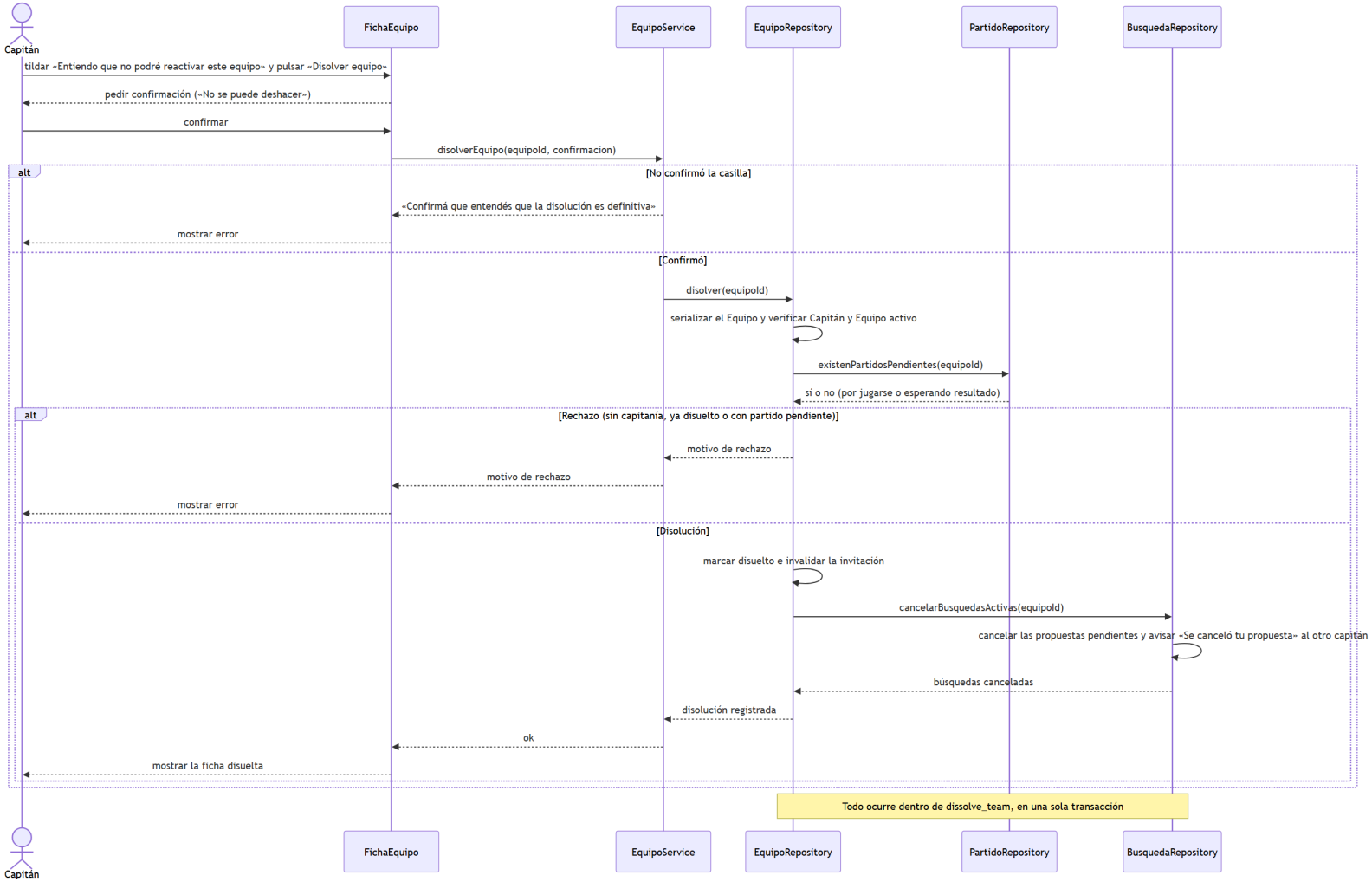
UC-C09 · Disolver equipo

- **Actor:** Capitán.
- **Requerimientos:** RF-C22 a RF-C25. **Pantalla:** P4 (sección «Disolver equipo»). **Diagrama:** S09.
- **Precondiciones:** Equipo no disuelto; el actor es su Capitán; el Equipo **no tiene ningún partido por jugarse ni esperando resultado**.
- **Postcondición:** Equipo disuelto, **sin reactivación**. Si vuelven, crean un Equipo nuevo. Deja de contar para el tope de 3 capitanías.
- **Estado:** Implementado (`dissolveTeam`, función `dissolve_team`).

Diagrama de secuencia · **S09 · UC-C09 — Disolver equipo**

Caso de uso: UC-C09 · Pantalla: P4 (sección «Disolver equipo») · Requerimientos: RF-C22 a RF-C25

No hay desplegables. La confirmación es una casilla y un cuadro de confirmación.



Cómo se lee

- **Doble confirmación:** la casilla es obligatoria en el formulario y, además, el servidor vuelve a comprobarla; después el navegador pide una última confirmación («No se puede deshacer»).
- **Disolver no borra.** El Equipo queda marcado como disuelto: conserva su página, su Plantel, su escudo y su historia. **No se reactiva**: si vuelven, crean un Equipo nuevo.
- **Se bloquea con un partido sin terminar** (por jugarse o esperando resultado): «Tenés un partido pendiente: tiralo o cerrá el resultado antes de disolver». Así el rival no queda esperando una respuesta que nunca llega, y un Equipo acusado de no presentarse no puede escaparse disolviéndose.
- **Cancela las búsquedas de rival** como si el Capitán las cancelara una por una. Las propuestas pendientes (enviadas y recibidas) caen y el capitán del otro lado recibe el Aviso «Se canceló tu propuesta». La disolución no tiene motivo ni Aviso propios.
- **No toca la calificación:** los partidos Jugados se siguen pudiendo calificar hasta que venzan los 7 días.
- **Todo o nada:** las comprobaciones y los cambios van en una sola función de Postgres, así que un fallo a mitad de camino no deja un Equipo disuelto con búsquedas activas.

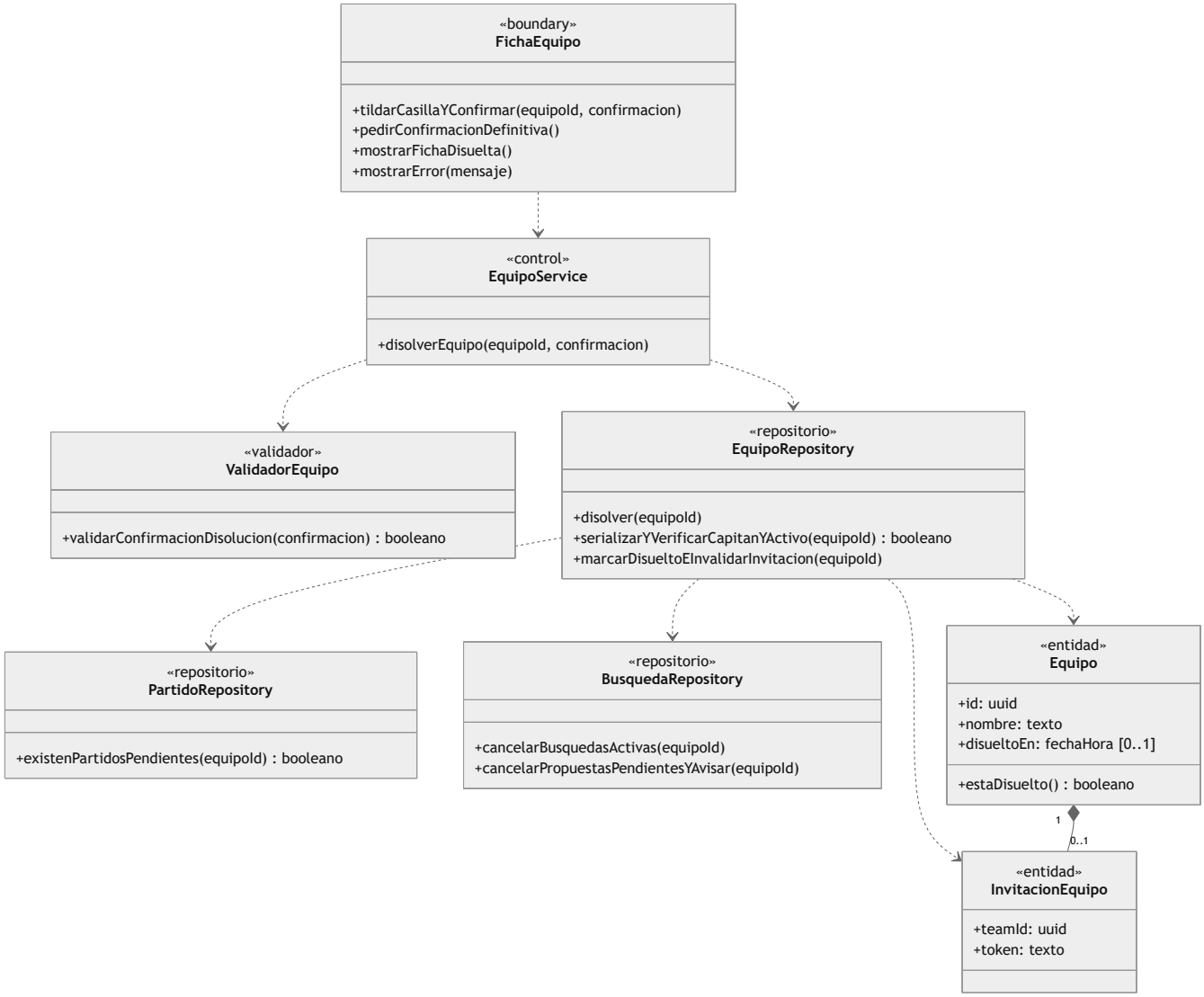
`PartidoRepository` y `BusquedaRepository` son responsabilidades de otro módulo (Partidos) que la disolución consulta; el módulo Equipos no las implementa.

Correspondencia en el código

`dissolveTeam` (`src/app/equipos/[id]/actions.ts`) comprueba la casilla `confirm_dissolve` y llama a la RPC `dissolve_team` (versión de `20261028000006_matches.sql`, que agrega el bloqueo por partidos y la cancelación de búsquedas). Probado en `supabase/tests/teams_management_test.sql`.

Diagrama de clases · **UC-C09 · Disolver equipo**

Caso de uso: UC-C09 · **Pantalla:** P4 (sección «Disolver equipo») · **Secuencia:** S09



Pantalla · **P4 · Equipo, vista del Jugador y del Capitán**

Es la misma pantalla que se describe, con su captura, en P4 · Equipo, vista del Jugador y del Capitán, debajo de UC-C02 · Invitar jugadores al equipo. Este caso usa la parte que indica su ficha («Pantalla»).

UC-C10 · Reportar un equipo

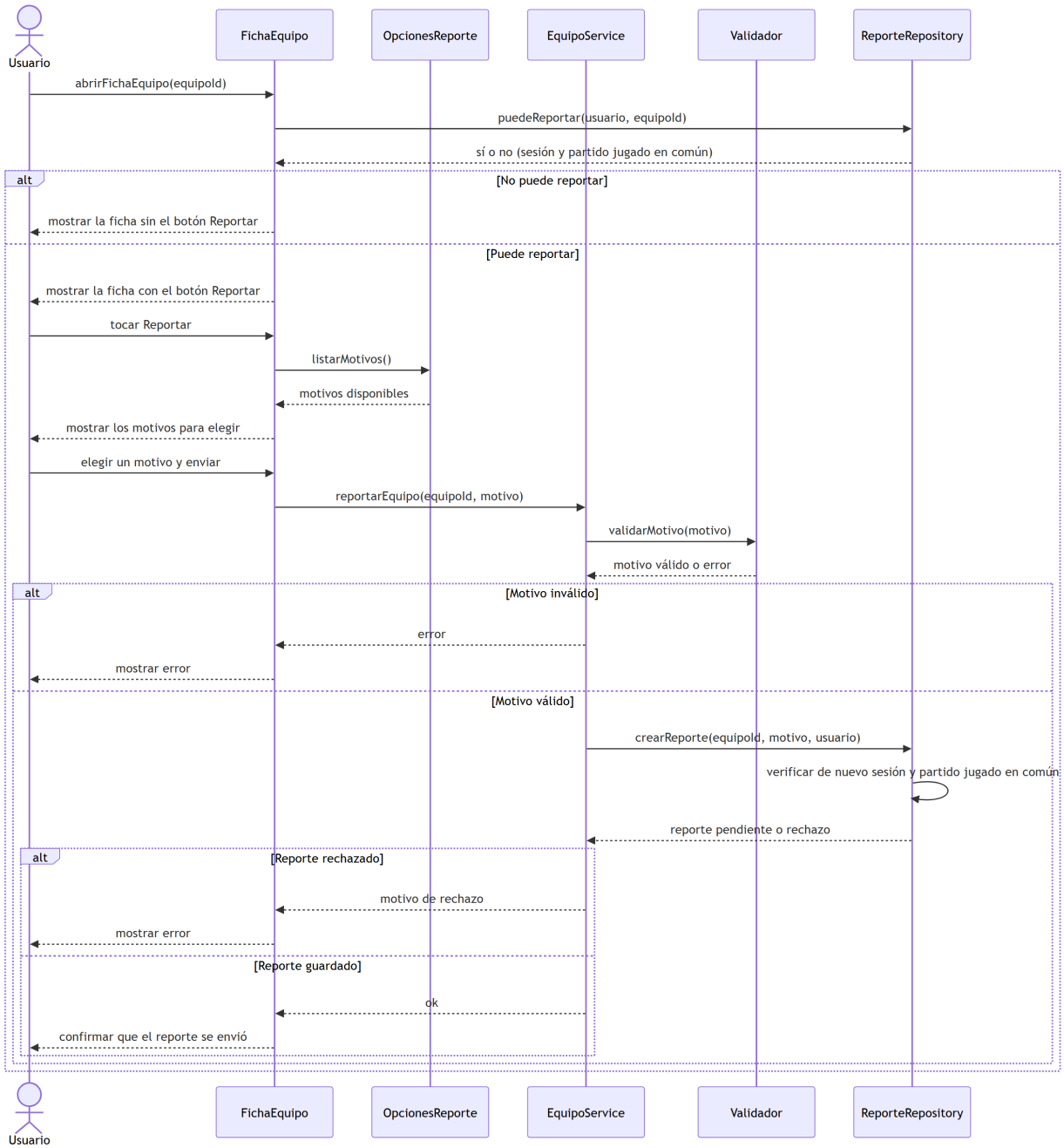
- **Actor:** Usuario (Capitán o Jugador de un Equipo que jugó al menos un partido contra el reportado).
- **Requerimientos:** RF-C26. **Pantalla:** P3/P4 (botón «Reportar», todavía no existe). **Diagrama:** S10 (diseño objetivo).
- **Precondiciones:** sesión; **un partido Jugado en común** con el Equipo reportado.
- **Postcondición:** el reporte queda pendiente en el panel de administración.
- **Estado: Pendiente.** Todavía no hay tabla, función ni pantalla. Queda por decidir si un Usuario puede reportar más de una vez al mismo Equipo (para los torneos, UC-B06, es una sola vez por Usuario y torneo; para los Equipos no está definido).

> Diagrama de secuencia · **S10 · UC-C10 — Reportar un equipo**

Caso de uso: UC-C10 · Pantalla: P3 (botón «Reportar», todavía no existe) · Requerimientos: RF-C26

> **Diseño objetivo, pendiente de implementación.** Hoy no hay tabla, función ni pantalla de reportes de equipos. El diagrama dibuja lo que dicen las reglas vigentes; los participantes `OpcionesReporte` y `ReporteRepository` no existen todavía y sus nombres son provisorios.

El caso empieza con una elección entre tres motivos. Siguiendo la guía de formularios, la pantalla obtiene la lista de motivos **antes** de que la persona elija. Los motivos son una **lista fija del código** (no se guardan en la base), así que los provee un proveedor de opciones y no un repositorio.



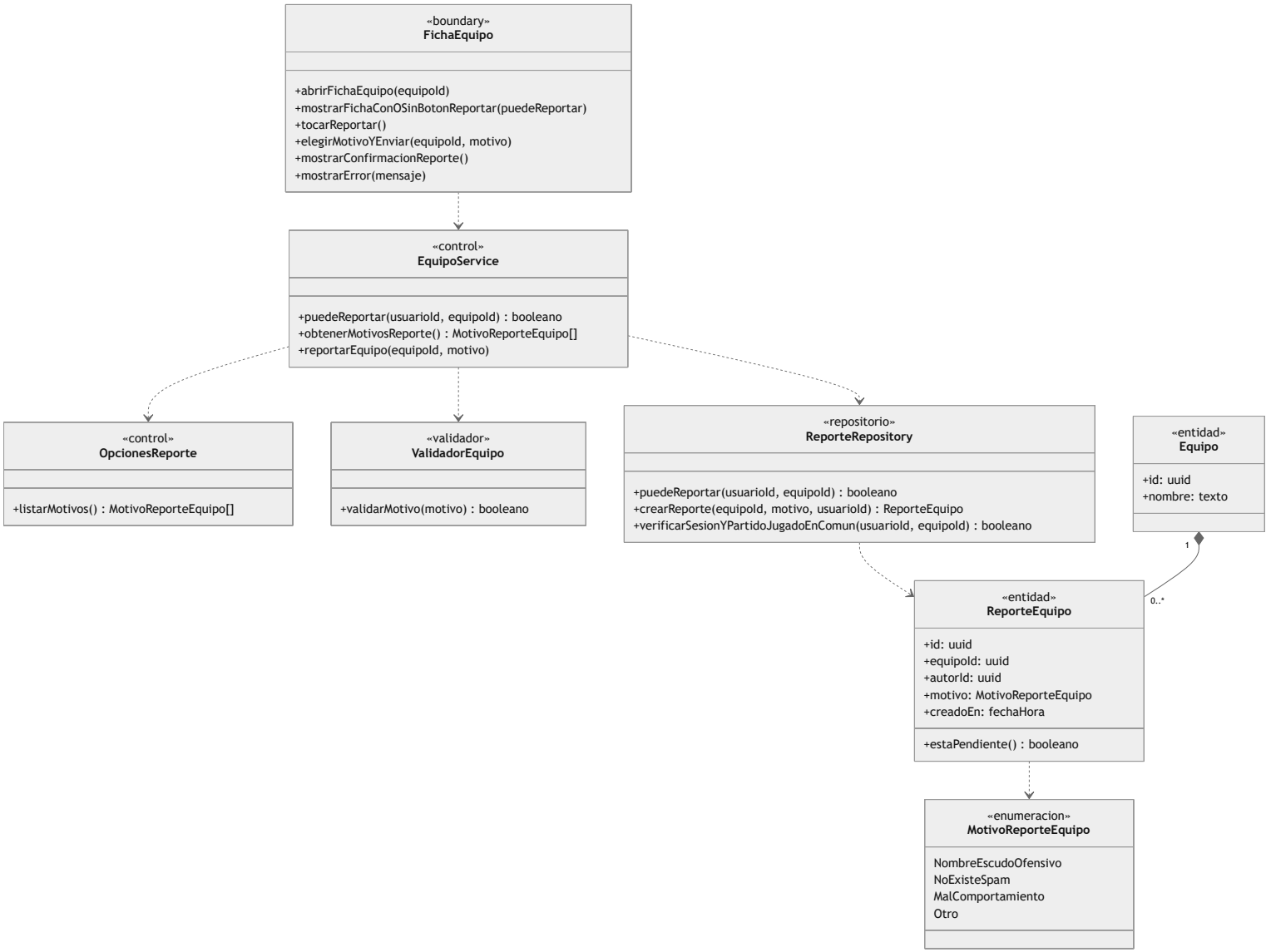
Cómo se lee

- **Quién ve el botón:** solo una persona con sesión que **jugó al menos un partido contra el Equipo reportado**. Que el botón aparezca no alcanza: `crearReporte` vuelve a verificarlo al guardar.

- **Motivos** (lista fija): «Violencia o agresiones», «Cuenta falsa» y «Abuso de calificaciones».
- **Destino:** el reporte queda pendiente para el administrador (UC-F04), que puede descartarlo o suspender al **Capitán**, porque el Capitán responde por la conducta del Plantel.
- **Sin decidir:** si una misma persona puede reportar más de una vez al mismo Equipo. En los torneos (UC-B06) es una sola vez por Usuario y torneo; para los Equipos no está definido. Hay que decidirlo antes de implementar y, entonces, agregar la regla a decisiones-casos-de-uso.md.

Diagrama de clases · **UC-C10 · Reportar un equipo**

Caso de uso: UC-C10 · **Pantalla:** P3/P4 (botón «Reportar», todavía no existe) · **Secuencia:** S10 (diseño objetivo)



Pantalla · **P3 · Equipo, vista pública**

Es la misma pantalla que se describe, con su captura, en P3 · Equipo, vista pública, debajo de UC-C04 · Ver la página pública del equipo. Este caso usa la parte que indica su ficha («Pantalla»).

Pantalla · **P4 · Equipo, vista del Jugador y del Capitán**

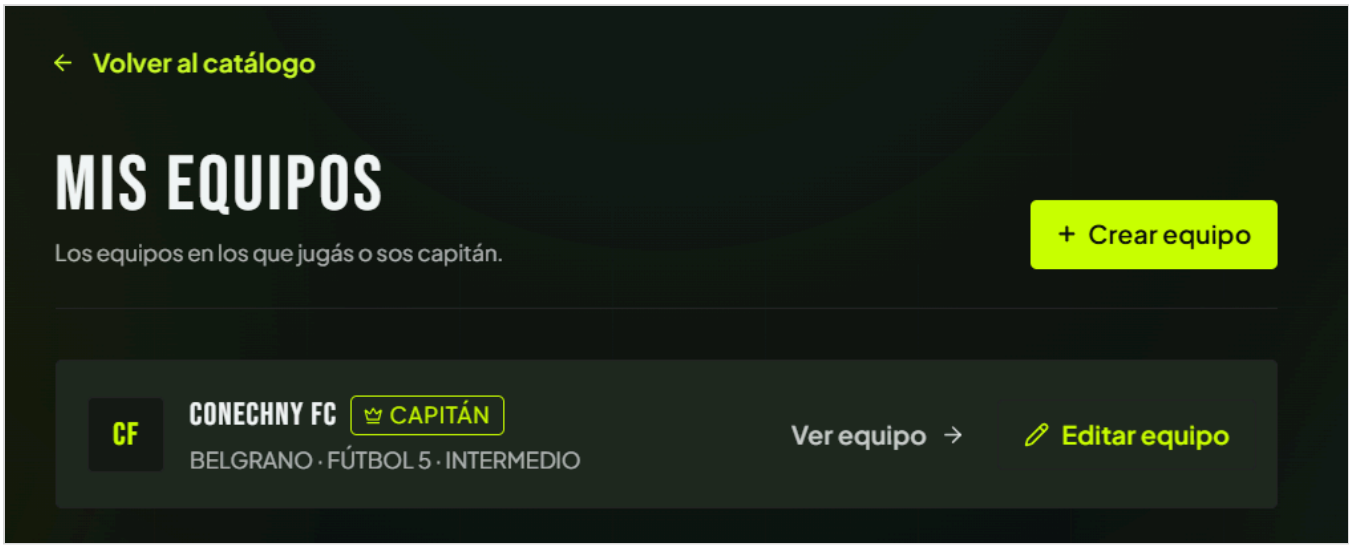
Es la misma pantalla que se describe, con su captura, en P4 · Equipo, vista del Jugador y del Capitán, debajo de UC-C02 · Invitar jugadores al equipo. Este caso usa la parte que indica su ficha («Pantalla»).

Pantallas sin caso de uso propio

Pantallas del módulo que ningún caso de uso cita como suya (por ejemplo, la lista desde la que se llega a los casos).

Pantalla · **P2 · Mis equipos**

Ruta: `/mis-equipos` · **Casos de uso:** punto de entrada a UC-C01, UC-C04 y UC-C05 · **Requerimientos:** RF-C15 · **Diagrama:** no tiene (no toma ninguna decisión)



Captura ✅ vigente.

Qué muestra. Una fila por Equipo que el Usuario integra: escudo (o las iniciales), nombre, la marca **CAPITÁN** si lo es, la marca **DISUELTO** si corresponde, y «Zona · Modalidad · Nivel». Cada fila ofrece «Ver equipo» y, solo al Capitán de un Equipo activo, «Editar equipo». Arriba a la derecha, «Crear equipo».

Orden: activos primero, disueltos después (conservan su historia, así que siguen en la lista).

Estados: *vacío* («Todavía no estás en ningún equipo. Creá uno, o pedile al capitán de tu equipo el link de invitación»); *error de carga* («No pudimos cargar tus equipos. Probá de nuevo»).

Parte 3 · Anexos y trazabilidad

Guía de pantallas: estado de las capturas y mapa

Versión del **04/11/2026**. Cada pantalla de la app que representa un caso de uso del módulo, con la ruta, el caso, los requerimientos, lo que muestra y la captura. Cada una va debajo del caso de uso que la usa por primera vez; si otro caso usa la misma pantalla, remite a esa descripción.

Estado de las capturas

Marca	Significa
✅ **Vigente**	Coincide con el código de `main` del 04/11/2026.
⚠️ **Desactualizada**	Es de la especificación del 28/10; falta algo que se agregó después. Se indica qué.
⏳ **Pendiente**	Todavía no hay captura. Al final están los pasos para sacarla.

Las imágenes están en `pantallas/`. Todas son de un entorno local con la UI lima y negro del sistema de diseño actual.

Mapa de pantallas

Pantalla	Ruta	Casos de uso	Sesión
P1 · Crear equipo	`/crear-equipo`	UC-C01	Obligatoria
P2 · Mis equipos	`/mis-equipos`	acceso a UC-C01, C04, C05	Obligatoria
P3 · Equipo, vista pública	`/equipos/{id}`	UC-C04, UC-C10	No
P4 · Equipo, vista del Jugador y del Capitán	`/equipos/{id}`	UC-C02, C04, C06, C07, C08, C09	Integrante del Plantel
P5 · Editar equipo	`/equipos/{id}/editar`	UC-C05	Capitán
P6 · Unirse al equipo	`/unirse/{token}`	UC-C03	Obligatoria

`/crear-equipo`, `/mis-equipos` y `/equipos/{id}/editar` exigen sesión (el middleware `src/proxy.ts` y cada página redirigen a `/login?next=…`). `/equipos/{id}` **no está protegida a propósito**: es pública. `/unirse/{token}` maneja su propio redirect al login para conservar el token.

Capturas pendientes

Para completar los casos. Convención: PNG en `docs/equipos/pantallas/`, ventana de escritorio de unos 1280 px de ancho, sin la barra del navegador, con datos de prueba (nunca cuentas reales). Hacen falta estas cuentas y estados:

Archivo	Pantalla y estado	Cómo conseguirlo
`crear-equipo-pagina.png`	P1 completa, con la cabecera	Con sesión y WhatsApp cargado, abrir `/crear-equipo` y capturar la página entera.
`crear-equipo-sin-whatsapp.png`	P1 con el aviso ámbar y el formulario deshabilitado	Con una cuenta **sin** WhatsApp en el perfil.
`crear-equipo-limite.png`	P1 con «Ya sos capitán de 3 equipos…»	Con una cuenta que capitanea 3 Equipos activos.

Archivo	Pantalla y estado	Cómo conseguirlo
`equipo-publico.png`	P3 sin sesión, con «Partidos» y «Torneos»	Abrir `/equipos/{id}` en una ventana privada, de un Equipo con partidos cerrados.
`equipo-disuelto.png`	P3/P4 con «Equipo disuelto»	Un Equipo disuelto, abierto con y sin sesión.
`equipo-capitan-datos.png`	P4, vista del Capitán, **reemplaza** la desactualizada	El Capitán de un Equipo con varios integrantes: se ven «Partidos», «Torneos» y «Plantel» con «Hacer capitán» y «Sacar».
`equipo-jugador.png`	P4, vista del Jugador	Una cuenta que integra el Plantel sin ser Capitán: se ve «Salir del equipo».
`editar-equipo.png`	P5	Como Capitán, «Editar equipo»; mejor con un escudo cargado para que se vean «Reemplazar» y «Quitar».
`unirse-equipo.png`	P6	Con sesión, abrir un link de invitación vigente **sin** pulsar el botón.
`unirse-equipo-error.png`	P6 con el mensaje de link inválido	Abrir `/unirse/cualquier-cosa` con sesión y pulsar «Unirme al equipo».
`disolver-confirmacion.png`	Cuadro de confirmación de disolver	Tildar la casilla y pulsar «Disolver equipo» (sin aceptar).

Al agregar una captura nueva, actualizar acá su marca de estado y, si reemplaza a una vieja, borrar la vieja.

Diagramas de secuencia: criterios, participantes y convenciones

Versión del **04/11/2026**. Un diagrama por caso de uso, en Mermaid (GitHub los dibuja directo). Cada uno va debajo de su caso de uso, antes de su pantalla.

Diagrama	Caso de uso	Pantalla	Estado
S01 · Crear equipo	UC-C01	P1	Implementado
S02 · Invitar jugadores	UC-C02	P4	Implementado
S03 · Unirse al equipo	UC-C03	P6	Implementado
S04 · Ver la página del equipo	UC-C04	P3 y P4	Implementado
S05 · Editar equipo	UC-C05	P5	Implementado
S06 · Pasar la capitanía	UC-C06	P4	Implementado (Suspensión pendiente)
S07 · Sacar a un jugador	UC-C07	P4	Implementado
S08 · Salir del equipo	UC-C08	P4	Implementado
S09 · Disolver equipo	UC-C09	P4	Implementado
S10 · Reportar un equipo	UC-C10	P3	**Pendiente** (diseño objetivo)

Los PNG están dibujados sobre fondo blanco, para pegarlos en un informe. La fuente de cada diagrama es el bloque Mermaid de su archivo: si se cambia uno, hay que regenerar su PNG.

Criterios aplicados

Son los de la guía para diagramas con formularios y desplegables.

1. **Un caso de uso por diagrama.** Se separan casos distintos, no las etapas de uno solo: la carga de las opciones de un formulario y el alta o la edición van **en el mismo diagrama**.

2. **Si el caso empieza con un formulario**, el diagrama muestra al actor abriendo la pantalla y a la pantalla pidiendo las opciones de cada desplegable **antes** de que el actor elija. Recién después, la pantalla completa que recibe la persona.
3. **Un objeto por responsabilidad o fuente de datos**, no uno por desplegable. Si la lista se consulta en la base, es un repositorio con nombre específico (`ZonaRepository`). Si es una lista fija del código, es un proveedor de opciones (`OpcionesEquipo`). No se inventan repositorios para listas que no se guardan en la base.
4. **Al enviar**, el método del caso de uso recibe **solo los valores elegidos** (`zonaId`, `modalidad`, `nivel`), nunca las listas completas.
5. **La validación se repite al guardar.** Que un valor aparezca en un desplegable no garantiza que siga siendo válido cuando se guarda: la Zona pudo dejar de existir, o el cliente pudo mandar otro valor.
6. **Cada diagrama muestra solo los participantes que intervienen.**

Participantes: responsabilidad y correspondencia en el repo

Son responsabilidades conceptuales. No existen como clases TypeScript con esos nombres: la app usa Server Actions, utilidades puras y funciones de Postgres. La columna de la derecha dice dónde vive hoy cada responsabilidad.

Participante	Responsabilidad	Correspondencia en el repo
Usuario, Invitado, Jugador, Capitán	La persona que inicia el caso. Es un rol respecto del Equipo, no una cuenta distinta.	La sesión y su relación con el Equipo (`team_members`). El Invitado navega sin sesión.
FormularioCrearEquipo, FormularioEditarEquipo	La pantalla: carga las opciones, muestra el formulario, recoge y envía la selección.	`crear-equipo/page.tsx` y `equipos/[id]/editar/page.tsx`, con el componente `TeamForm`.
FichaEquipo	La página del Equipo y sus botones de gestión.	`equipos/[id]/page.tsx`.
PantallaUnirse	La pantalla de invitación con el botón «Unirme al equipo».	`unirse/[token]/page.tsx`.
ZonaRepository	Entrega la lista cerrada de Zonas, que **se consulta en la base**.	`getZoneGroups()` en `ZoneSelect.tsx`: lee `zones` con sus corredores y regiones.
OpcionesEquipo	Entrega las modalidades y los niveles: **listas fijas del código**, sin consulta a la base.	`SPORT_MODALITIES` (`utils/modality.ts`) y `TEAM_LEVELS` (`utils/teams.ts`).
EquipoService	Coordina el caso de uso: valida, llama al repositorio y arma la respuesta.	Las Server Actions: `crear-equipo/actions.ts`, `equipos/[id]/actions.ts` y `unirse/[token]/actions.ts`.
Validador	Valida las entradas antes de escribir.	`parseTeamForm` (`utils/team-form.ts`) y `parseCrest` (`utils/team-crest.ts`).

Participante	Responsabilidad	Correspondencia en el repo
EquipoRepository	Lee y escribe el Equipo, el Plantel y las invitaciones.	Las tablas `teams`, `team_members` y `team_invites`, tocadas por consultas con RLS y por las funciones de Postgres `create_team`, `join_team`, `leave_team`, `remove_team_member`, `transfer_captaincy`, `regenerate_team_invite`, `get_team_invite`, `dissolve_team`; y `saveTeam` (`utils/supabase/team.ts`) para los datos y el escudo (Storage, bucket `team-crests`).
InscripcionRepository	Entrega los torneos donde el Equipo tiene una Inscripción vinculada.	La vista pública `tournament_inscriptions` más `tournaments`. Datos del módulo E.
HistorialRepository	Entrega el historial público del Equipo.	Las vistas `team_record` y `team_match_history`. Datos del módulo de Partidos.
PartidoRepository, **BusquedaRepository**	Solo en S09: los partidos y las búsquedas de rival del Equipo.	`matches` y `match_phase`; `rival_searches`, `search_slots` y `cancel_proposals`. Datos del módulo de Partidos.
OpcionesReporte, **ReporteRepository**	Solo en S10: los motivos de reporte y el guardado del reporte.	**No existen todavía.**

Zonas de la base y listas del código. Esa diferencia es la que justifica tener un repositorio para las Zonas y un proveedor de opciones para modalidad y nivel. Si alguna vez la modalidad o el nivel pasaran a guardarse en una tabla, `OpcionesEquipo` se reemplazaría por un repositorio.

Convenciones de dibujo

- Flecha continua (`->>`): una llamada. Flecha punteada (`-->>`): su respuesta.
- Un mensaje de un participante a sí mismo es un paso interno (por ejemplo, una verificación dentro de una función de Postgres).
- Una nota sobre un participante marca una regla que **hace cumplir la base**, no la pantalla.
- Entre paréntesis y en minúscula van los **valores que viajan**; entre comillas «así», los mensajes que ve la persona.
- `opt` es un paso opcional; `alt` / `else`, caminos que se excluyen.

Trazabilidad y pendientes

Documentación completa del módulo de Equipos de Torneos Amateur BA, versión del **04/11/2026**. En este documento, sus piezas son la Parte 1 (requerimientos), la Parte 2 (cada caso de uso con su diagrama y su pantalla) y los anexos de esta parte.

Fuentes, en orden de autoridad: decisiones-casos-de-uso.md → casos-de-uso.md → CONTEXT.md → código y migraciones. Reemplaza a la especificación del 28/10/2026 (`docs/especificaciones-y-requerimientos/modulo-equipos-especificacion.pdf`).

Matriz de trazabilidad

Cada caso de uso con lo que lo cubre en los cuatro documentos.

Caso de uso	Requerimientos	Pantalla	Diagrama	Estado
UC-C01 Crear equipo	RF-C01 a C06, C27	P1 `/crear-equipo`	S01	Implementado
UC-C02 Invitar jugadores	RF-C07, C25	P4 `/equipos/{id}`	S02	Implementado
UC-C03 Unirse con el link	RF-C08 a C10, C27	P6 `/unirse/{token}`	S03	Implementado
UC-C04 Ver la página del equipo	RF-C11 a C14, C25	P3 y P4	S04	Implementado
UC-C05 Editar equipo	RF-C01, C02, C16, C25	P5 `/equipos/{id}/editar`	S05	Implementado
UC-C06 Pasar la capitanía	RF-C05, C17, C20, C21, C27	P4	S06	Implementado (Suspensión pendiente)
UC-C07 Sacar a un jugador	RF-C18, C20, C21, C25	P4	S07	Implementado
UC-C08 Salir del equipo	RF-C19, C20, C21, C25	P4	S08	Implementado
UC-C09 Disolver equipo	RF-C22 a C25	P4	S09	Implementado
UC-C10 Reportar un equipo	RF-C26	P3 (botón por crear)	S10	**Pendiente**
— (pantalla, sin decisión)	RF-C15	P2 `/mis-equipos`	—	Implementado

Equivalencia con la numeración anterior

La especificación del 28/10 usaba «módulo D» y otros números. La numeración vigente es la de `casos-de-uso.md` (módulo C).

Vigente	Anterior (PDF 28/10)
UC-C01 Crear equipo	UC-25
UC-C02 Invitar jugadores	UC-27
UC-C03 Unirse con el link	UC-28
UC-C04 Ver la página del equipo	UC-29 y UC-30 (historial)
UC-C05 Editar equipo	«Editar Equipo» (sin número)
UC-C06 Pasar la capitanía	UC-26
UC-C07 Sacar a un jugador	«Sacar a un Jugador» (sin número)
UC-C08 Salir del equipo	«Salir del Equipo» (sin número)
UC-C09 Disolver equipo	«Disolver Equipo» (sin número)
UC-C10 Reportar un equipo	—
RF-C01 a C06	RF-D01 a D06
RF-C07 a C10	RF-D07 a D10
RF-C11 a C14	RF-D19 y D20
RF-C15	RF-D21
RF-C16	RF-D14
RF-C17	RF-D11

Vigente	Anterior (PDF 28/10)
RF-C18 y C19	RF-D13
RF-C20	RF-D12
RF-C21	RF-D22
RF-C22 a C25	RF-D15 a D18

Qué cambió respecto del PDF del 28/10

- **Lo que el PDF daba por pendiente hoy está implementado:** el historial público de Partidos (antes UC-30), el bloqueo de la disolución con partidos sin terminar y la cancelación de búsquedas y propuestas al disolver.
- **El catálogo `/equipos` no está en `main`** (era RF-D23). Existe solo en la rama `franco-equipos` y no figura en `casos-de-uso.md`. Queda en «Sin decisión» en los requerimientos.
- **Reportar un equipo** (UC-C10) y la **Suspensión** son alcance vigente que todavía no está programado.
- **Los diagramas se rehicieron** con los objetos de los formularios (`ZonaRepository` y `OpcionesEquipo`) y la validación al guardar, según la guía de diagramas con desplegables. El de Crear equipo coincide con el de la rama `codex/diagrama-crear-equipo`, más la consulta de requisitos de la pantalla y el manejo del escudo.

Pendientes y decisiones abiertas

1. **Capturas faltantes** (ficha pública, vista del Jugador, editar, unirse, estados de error) y una **desactualizada** (ficha del Capitán, sin la sección «Partidos»). Está la lista con los pasos en 03-pantallas.md.
2. **Catálogo público de Equipos:** decidir si entra al alcance.
3. **Reportar un equipo:** decidir si una persona puede reportar más de una vez al mismo Equipo.
4. **Suspensión:** hoy la base no la verifica al crear un Equipo ni al recibir la capitanía (depende de UC-F05).
5. **Tests de base:** `teams_management_test.sql` cubre disolver, pasar la capitanía, invitación inválida y `create_team`; no encontré pruebas pgTAP dedicadas a `leave_team`, `remove_team_member` ni al límite de 3 capitanías. La regla del proyecto es que cada policy o función de RLS nueva tenga su test.
6. **Si se cambia un diagrama,** hay que regenerar su PNG (el bloque Mermaid de cada archivo es la fuente).

Diagramas de clases: trazabilidad de las clases

Clase	Estereotipo	Implementación en el repo	Estado
`FormularioCrearEquipo`	«boundary»	`src/app/equipos/crear/page.tsx` + `src/components/TeamForm.tsx`	Implementada
`FichaEquipo`	«boundary»	`src/app/equipos/[id]/page.tsx` + `InviteLinkBox.tsx`, `MemberActions.tsx`, `LeaveTeamForm.tsx`, `DissolveTeamForm.tsx`	Implementada (falta botón «Reportar» de UC-C10)
`FormularioEditarEquipo`	«boundary»	`src/app/equipos/[id]/editar/page.tsx` + `src/components/TeamForm.tsx`	Implementada
`PantallaUnirse`	«boundary»	`src/app/equipos/unirse/[token]/page.tsx`	Implementada

Clase	Estereotipo	Implementación en el repo	Estado
`EquipoService` / `OpcionesEquipo` / `OpcionesReporte`	`«control»`	Server Actions en `src/app/equipos/crear/actions.ts`, `src/app/equipos/[id]/actions.ts`, `src/app/equipos/[id]/editar/actions.ts`, `src/app/equipos/unirse/[token]/actions.ts`, `SPORT_MODALITIES` (`src/utils/modality.ts`), `TEAM_LEVELS` (`src/utils/teams.ts`)	Implementados (faltan `reportarEquipo` y `OpcionesReporte`)
`ValidadorEquipo`	`«validador»`	`parseTeamForm`, `parseCrest`, `parseRemoveCrest`, `parseInviteToken`, `parseMemberTarget`, `parseDissolveConfirmation` (`src/utils/teams.ts`)	Implementado
`ZonaRepository`	`«repositorio»`	`getZoneGroups()` en `src/components/ZoneSelect.tsx` (`public.zones`)	Implementado
`EquipoRepository`	`«repositorio»`	Tablas `public.teams`, `team_members`, `team_invites`; funciones `create_team`, `regenerate_team_invite`, `join_team`, `transfer_captaincy`, `remove_team_member`, `leave_team`, `dissolve_team`; bucket `crests` (`src/utils/supabase/crest.ts`)	Implementado
`InscripcionRepository`	`«repositorio»`	Vista `public.public_inscriptions` filtrada por `team_id`	Implementado
`HistorialRepository`	`«repositorio»`	Funciones `public.team_public_record` y `public.team_recent_matches` (`20261028000017_team_public_history.sql`)	Implementado
`PartidoRepository` / `BusquedaRepository`	`«repositorio»`	Verificación y cancelaciones invocadas dentro de `public.dissolve_team` (`20261028000015_dissolve_team_matches.sql`)	Implementados
`ReporteRepository`	`«repositorio»`	Tabla de reportes de equipo	**Pendiente**
`Equipo` / `IntegrantePlantel` / `InvitacionEquipo`	`«entidad»`	`public.teams`, `public.team_members`, `public.team_invites` (`20261028000002_teams.sql`)	Implementadas
`ResumenHistorialPublico` / `PartidoCerradoPublico`	`«entidad»`	Retorno de `team_public_record` y `team_recent_matches`	Implementadas

Módulo D · Partidos

Primero los requerimientos del módulo completo. Después, el modelo de dominio y, para cada caso de uso, su diagrama de secuencia, su diagrama de clases y su pantalla. Al final, los anexos y la trazabilidad.

Parte 1 · Requerimientos

Versión del **04/11/2026**. Es la primera parte del módulo: sigue la sección de casos de uso, donde cada caso lleva debajo su diagrama de secuencia y su pantalla. El README tiene la matriz que los une.

Fuentes, en orden de autoridad: decisiones-casos-de-uso.md (sección «Partidos»), diseno-partidos.md (cómo se construye), ADR 0002, casos-de-uso.md (módulo D, 03/11/2026), el glosario CONTEXT.md y el código y las migraciones del repo al día de hoy.

> **No confundir los prefijos.** Los `RF-D##` de este módulo son los del módulo D (Partidos). La especificación vieja del módulo Equipos (28/10/2026) usaba `RF-D##` para los suyos, cuando llamaba «módulo D» a Equipos; esos hoy son los `RF-C##` de Equipos.

1. Alcance

El módulo permite que los **equipos busquen rival y jueguen**: el **Capitán** publica cuándo puede jugar su equipo, la app muestra los **rivales compatibles**, se **proponen** y **aceptan** partidos, los capitanes **coordinan** cancha y hora, **confirman** asistencia, **tiran** el partido si hace falta, **cargan el resultado**, **califican** al rival y todo queda en un **historial público** con presentismo.

Todo el módulo lo opera solo el Capitán. El Plantel ve los partidos de sus equipos sin botones. Las búsquedas y las propuestas las ve solo el Capitán. Si otro jugador tiene que encargarse, se pasa la capitanía.

Límites de producto:

- La app **no inscribe ni cobra**, y **no negocia**: los capitanes arreglan cancha y hora **por WhatsApp**; la app **registra lo acordado**.
- **No hay árbitro.** Si un equipo dice «no vinieron» y el otro dice «fuimos», el partido queda **en conflicto**, sin falta para nadie. El administrador no interviene en partidos (ADR 0002).
- **No hay tareas programadas.** Nada ocurre «a las 48 h»: ocurre cuando alguien mira la pantalla. Los plazos y los estados se calculan al leer.

Fuera del módulo:

Tema	Dónde está
Crear y gestionar equipos, pasar la capitanía	Módulo C — Equipos
La campanita (cómo se muestran los Avisos)	Módulo A — Cuenta y acceso (UC-A06)
Suspender una cuenta, que cancela búsquedas y tira partidos	Módulo F — Administración (UC-F05)

2. Actores

Actor	Quién es	Qué hace en este módulo
Capitán	Único miembro de un equipo que lo gestiona	Opera todo: busca rival, propone, acepta, coordina, confirma, tira, carga el resultado y califica.
Jugador	Integrante del Plantel	Ve los partidos de su equipo, sin botones.
Sistema	La base de datos y la app	Calcula fases, plazos, faltas y bloqueos al leer, y reparte los Avisos.

3. Requerimientos funcionales

Estado: `Implementado` está en `main`. `Parcial` tiene una parte sin cumplir, que se indica. `Pendiente` es alcance vigente sin programar.

Búsqueda de rival

ID	Requerimiento	Casos de uso	Estado
RF-D01	**Solo el Capitán opera** el módulo. El Plantel solo mira los partidos; las búsquedas y las propuestas las ve solo el Capitán.	Todos	Implementado
RF-D02	**Publicar una búsqueda** por equipo y modalidad (la modalidad habitual del equipo viene cargada y se puede cambiar). Hay **una sola búsqueda activa por equipo y modalidad**. Requiere: equipo no disuelto, Capitán con WhatsApp en el perfil, equipo sin **Bloqueo por faltas** y Capitán no suspendido.	UC-D01	**Parcial**: la Suspensión no se verifica todavía
RF-D03	**Franjas:** un día de los **próximos 14**, con un **Turno** (Mañana 9–12, Tarde 14–18, Noche 20–23) o «Otro horario» con dos horas enteras. Va de **8 a 24** sin cruzar la medianoche, dura **al menos 1 h** y, si es hoy, empieza en una hora que no pasó. «**Repetir la semana que viene**» crea la misma franja 7 días después; si cae fuera de los 14 días, se rechaza entera.	UC-D01	Implementado
RF-D04	Se **rechaza una franja que choca** con un partido acordado del equipo o con otra franja de la misma búsqueda. Entre búsquedas de **distinta modalidad** del mismo equipo sí se pueden superponer.	UC-D01	Implementado
RF-D05	La búsqueda guarda **solo la modalidad y las franjas**: la cercanía y el nivel salen de la **Zona y el nivel actuales del equipo**, así que cambiarlos en el equipo se refleja en el acto.	UC-D01, UC-D03	Implementado
RF-D06	La búsqueda está **activa** mientras le quede alguna franja por empezar y **vence sola** al pasar la última. Publicarla no genera ningún Aviso. Que nazca un partido no la cierra.	UC-D01, UC-D02	Implementado
RF-D07	**Editar la búsqueda:** agregar horarios o sacar uno con la ×; la modalidad no se cambia. Sacar una franja **cancela sus propuestas pendientes** (enviadas y recibidas) con el Aviso «Se canceló tu propuesta»; sacar la última cancela la búsqueda.	UC-D02	Implementado
RF-D08	**Cancelar la búsqueda** cancela todas sus propuestas pendientes, con el mismo Aviso. **Los partidos que ya nacieron de ella siguen en pie.**	UC-D02	Implementado
RF-D09	**Rivales compatibles:** misma modalidad y al menos **1 h en común el mismo día**; los niveles a dos escalones (recreativo contra competitivo) **no se cruzan**; **nunca dos equipos del mismo Capitán**; una **franja ocupada** o ya empezada se ignora.	UC-D03	Implementado
RF-D10	**Orden de los compatibles:** primero la cercanía (Zona > Corredor > Región), después el nivel y por último el **presentismo** (faltas en 90 días, de menos a más; sin historial cuenta 0). Los equipos con 2 faltas en 90 días van al final igual. Desempate: el horario en común más temprano.	UC-D03	Implementado

Propuestas

ID	Requerimiento	Casos de uso	Estado
RF-D11	**Proponer un partido** con el **horario en común**, que calcula la app (nadie elige un rango) y **sin mensaje**. El Capitán rival recibe el Aviso «Te propusieron un partido». **Sin límite** de propuestas pendientes por equipo; no se puede mandar otra al mismo rival para el mismo horario mientras la anterior está pendiente; nunca entre dos equipos del mismo Capitán.	UC-D04	Implementado
RF-D12	La propuesta **vence a las 72 h o cuando empieza su rango**, lo que ocurra primero. **Vencer no avisa.**	UC-D04	Implementado
RF-D13	**Cancelar** una propuesta enviada la cierra **sin Aviso**: el rival deja de verla al entrar. Se puede volver a proponer. Solo avisan las cancelaciones en cascada (sacar una franja, cancelar la búsqueda, disolver, bloqueo).	UC-D05	Implementado
RF-D14	**Aceptar** una propuesta crea el **Partido** con ese rango, **A coordinar**. Los dos planteles reciben «Partido acordado». Cada Capitán ve el **WhatsApp del otro** (siempre el del Capitán actual) mientras el partido esté por jugarse. Las franjas de los dos equipos que se superponen quedan **ocupadas** enteras.	UC-D06	Implementado
RF-D15	Al nacer un partido se **anulan** las propuestas pendientes de los dos equipos que se superponen con ese horario, con el Aviso «Se anuló tu propuesta: ya no tienen ese horario» al Capitán del otro lado. Si la anulada era entre estos mismos dos equipos (la que venía en sentido contrario), **no hay Aviso**: ya les llegó «Partido acordado».	UC-D06	Implementado
RF-D16	**Rechazar** una propuesta la cierra y avisa «Rechazaron tu propuesta» a quien la mandó.	UC-D06	Implementado
RF-D17	**Nunca dos partidos superpuestos:** si al aceptar alguno de los dos equipos ya tiene un partido en ese horario, la aceptación se rechaza.	UC-D06	Implementado
RF-D18	Si un equipo está **bloqueado por faltas**, la propuesta figura **cancelada, sin botones**, con la nota «el equipo está bloqueado por faltas», hasta cuando vencería: es la única forma de que el rival se entere, porque el bloqueo no avisa.	UC-D06	Implementado

Coordinación, asistencia y caída

ID	Requerimiento	Casos de uso	Estado
RF-D19	**Coordinar cancha y hora:** uno de los dos Capitanes carga la **cancha** (texto libre obligatorio, de 1 a 120 caracteres; se muestra con «Ver en el mapa») y la **hora exacta** (de a 15 min, desde el inicio del rango incluido hasta el fin excluido). El otro **confirma** o **corrige**; al confirmar, el partido queda **Coordinado**. Solo se puede **antes del inicio del rango**. Coordinar es una ayuda, no un requisito.	UC-D07	Implementado
RF-D20	**Cambiar después de Coordinado:** cualquiera vuelve a cargar cancha y hora; el partido deja de estar Coordinado, le toca confirmar al otro y mientras tanto la **Hora de inicio** vuelve a ser el inicio del rango. Los «Vamos» ya dados no se borran. Para cambiar de día, se tira y se propone de nuevo.	UC-D07	Implementado
RF-D21	**Confirmar asistencia:** desde **24 h** antes de la Hora de inicio, cada Capitán ve «¿Van?» con **Vamos** y **No vamos**. Si los dos dicen «Vamos», el partido queda **Confirmado** (sin Aviso, a propósito). Desde 12 h antes, si el rival no confirmó, se muestra «El rival todavía no confirmó». Si nadie confirma, el partido sigue en pie. «No vamos» abre el formulario de tirar el partido.	UC-D08	Implementado

ID	Requerimiento	Casos de uso	Estado
RF-D22	**Tirar el partido** antes de la Hora de inicio, con un **motivo** obligatorio de 1 a 140 caracteres, que queda **visible en el historial público** (nadie lo modera). Con **menos de 24 h**, suma una **Falta** al equipo que lo tiró, desde que lo tiró. Las franjas que ocupaba se liberan y el partido entra al historial desde que se tira. Aviso «Se cayó el partido» a los dos planteles.	UC-D09	Implementado
RF-D23	**Caída de común acuerdo:** con menos de 24 h, el Capitán puede pedir tirarlo de común acuerdo (con su motivo). Si el rival acepta antes de la Hora de inicio, el partido se cae **sin falta para nadie**. Si rechaza o no responde, el partido sigue en pie y el que pidió puede tirarlo solo, con la falta. **No se vuelve a pedir.** Un cambio de Capitán en el medio no lo bloquea: el nuevo responde.	UC-D10	Implementado

Resultado, calificación y mis partidos

ID	Requerimiento	Casos de uso	Estado
RF-D24	**Cargar el resultado** una vez pasada la Hora de inicio, como un **marcador** (de 0 a 99 por lado, el equipo propio a la izquierda). El rival recibe «Cargaron el resultado: confirmá o corregí»; si confirma, el partido queda **Jugado** y los dos planteles reciben «Jugado».	UC-D11	Implementado
RF-D25	**Plazos de 48 h**, calculados al leer, contados desde la última carga de cualquiera (o desde el fin del rango, si nadie cargó): si el rival no hace nada, Jugado con el resultado cargado (sin Aviso); con resultados distintos, cada corrección da 48 h al otro y, si siguen sin coincidir al vencer, **Jugado sin resultado**; si nadie carga, **Jugado sin resultado**, sin falta para nadie. Un partido Jugado, en conflicto o con falta queda **cerrado** y no se corrige más.	UC-D11	Implementado
RF-D26	**«No vinieron»:** pasada la hora de inicio, quien no cargó resultado puede reportar que el rival no se presentó. El acusado recibe «Dicen que no fueron: respondé» y tiene 48 h. Si no responde, el partido cierra como **no presentado** y el ausente suma una **Falta**, firme al vencer el plazo (sin Aviso).	UC-D12	Implementado
RF-D27	**Conflicto:** si el acusado responde «fuimos» o carga un resultado, o si los dos reportan «no vinieron», el partido queda **en conflicto**: sin falta para nadie, los dos equipos suman un **Conflicto** visible en su página con las dos versiones, y los dos planteles reciben «Partido en conflicto». Cargar un resultado equivale a decir «fuimos». **Nadie arbitra.**	UC-D11, UC-D12	Implementado
RF-D28	**Calificar al rival** en un partido **Jugado** (con o sin resultado), hasta 7 días después: de **1 a 5 estrellas** y un comentario opcional de hasta 500 caracteres, **privado**. Una calificación por equipo y partido; no se edita. Un partido caído, en conflicto o con falta no se califica. Un equipo disuelto se sigue calificando hasta vencer los 7 días.	UC-D13	Implementado
RF-D29	**Revelación cruzada** (para que nadie califique en represalia): cada Capitán ve la calificación que le pusieron recién cuando los dos calificaron o pasaron 7 días. Quien calificó primero recibe «Ya podés ver tu calificación» (revelar por vencimiento no avisa). El promedio público cuenta solo las calificaciones reveladas.	UC-D13	Implementado
RF-D30	**Ver mis partidos** en `/partidos`: las propuestas recibidas (solo Capitán), el próximo partido en grande, los partidos esperando resultado, «Más adelante» y el historial plegado. El Jugador ve todo sin botones. El WhatsApp del Capitán rival lo ve solo el Capitán. **Esta pantalla es la fuente de verdad:** todo lo pendiente se ve acá, haya o no Aviso.	UC-D14	Implementado

Reglas calculadas por el sistema

ID	Requerimiento	Casos de uso	Estado
RF-D31	**Presentismo y faltas:** una Falta cuenta por tirar con menos de 24 h (desde que se tiró) o por no presentarse (al vencer las 48 h). Cuentan las de los últimos 90 días.	UC-D03, UC-D09, UC-D12	Implementado
RF-D32	**Bloqueo por faltas:** 3 faltas en 90 días bloquean al equipo 30 días, desde la tercera: no puede publicar búsquedas ni proponer, y se cancelan sus búsquedas y propuestas pendientes. Los partidos acordados siguen. **No avisa.**	UC-D01, UC-D06	Implementado
RF-D33	**Los Conflictos solo se muestran** (contador y cantidad de rivales distintos); nunca penalizan.	UC-D12	Implementado
RF-D34	**Los Avisos de Partidos** son una lista cerrada de disparadores; los que piden acción van solo al Capitán y los informativos a todo el Plantel del momento; nunca a quien hizo la acción. Los detalles están en casos-de-uso.md.	Todos	Implementado
RF-D35	**Cambios de Capitán o de Plantel con partidos en curso:** nunca se bloquean; las búsquedas, propuestas y partidos son del equipo.	UC-D10	Implementado

4. Requerimientos no funcionales

ID	Requisito	Evidencia
RNF-D01	**Se guardan hechos con su hora, nunca estados.** El estado sale de funciones puras que reciben la hora actual (`match_phase` , `match_deadline` , `proposal_status`). **Sin tareas programadas.**	diseno-partidos.md, `20261028000006_matches.sql`
RNF-D02	**El cálculo vive solo en Postgres:** las funciones validan contra la misma función que lee la pantalla; TypeScript solo traduce a texto.	`src/utils/match-display.ts`
RNF-D03	**Ninguna tabla de Partidos se escribe desde el cliente.** No hay policies de insert, update ni delete: todo pasa por funciones (`save_rival_search` , `send_proposal` , `respond_proposal` , `coordinate_match` , `confirm_going` , `drop_match` , `request_mutual_drop` , `answer_mutual_drop` , `report_result` , `report_no_show` , `reply_went` , `rate_rival`). RLS no puede expresar «cada capitán escribe solo su propia confirmación».	`20261028000003` a `20261028000014`
RNF-D04	**Visibilidad por tabla** (RLS): búsquedas y franjas, solo el Capitán dueño; propuestas, los Capitanes de los dos equipos; partidos, el Plantel de los dos equipos; calificaciones, nadie por la API (el comentario lo lee el administrador).	`diseno-partidos.md` , tests pgTAP
RNF-D05	**El historial público nunca expone cancha ni hora.** Lo arman funciones `security definer` y dos vistas (`team_match_history` , `team_record`).	`20261028000012_team_history.sql` , `team_history_test.sql`
RNF-D06	**El WhatsApp del Capitán rival** lo entrega una función `security definer` , solo al Capitán actual de un equipo con ese partido acordado y mientras esté por jugarse.	`rival_captain_whatsapp`

ID	Requisito	Evidencia
RNF-D07	**«Nunca dos partidos superpuestos»** se hace cumplir bloqueando las filas de `teams` involucradas (`for update` , ordenadas por id) y consultando si hay un partido no caído superpuesto. No hay restricción de exclusión sobre partidos porque obligaría a duplicar un «activo» guardado.	`respond_proposal` , `save_rival_search`
RNF-D08	**Las validaciones se repiten:** TypeScript las hace para dar un mensaje claro; **la función de la base es la que manda**. Los ids que llegan del cliente no son de confiar.	`rival-search-form.ts` , `match-forms.ts` , `actions.ts`
RNF-D09	**Zona horaria fija:** los rangos se arman en `America/Argentina/Buenos_Aires` (UTC-3, sin horario de verano) a partir de fecha y horas enteras.	`save_rival_search` , `match-forms.ts`
RNF-D10	**Los plazos se pueden probar sin esperar:** un script de escenarios versionado arma los casos que no se pueden esperar (72 h, 48 h, 7 días, faltas en 90 días), con la `service_role key` **local**, sin pasar por las funciones.	`scripts/qa/partidos.ts`
RNF-D11	**Pruebas:** pgTAP para cada frontera de fase y plazo (23:59 h contra 24 h, 48 h justas, 72 h), cada función con sus rechazos y cada policy y vista; y pruebas unitarias de la validación y el texto.	`rival_searches_test.sql` , `proposals_test.sql` , `matches_test.sql` , `match_coordination_test.sql` , `match_drops_test.sql` , `match_results_test.sql` , `ratings_test.sql` , `team_faults_test.sql` , `team_history_test.sql` , `compatible_rivals_test.sql` , `notices_test.sql` ; `match-forms.test.ts` , `rival-search-form.test.ts` , `match-display.test.ts` , `proposals.test.ts`
RNF-D12	**Cada vista es una URL:** las dos pantallas son pestañas que son links (`/partidos` , `/partidos/buscar`); la búsqueda guarda el equipo y la modalidad en la URL.	`PartidosTabs.tsx`

5. Sin decisión y diferencias con la documentación

Tema	Situación
«Copiar horarios de la última búsqueda»	`decisiones-casos-de-uso.md` dice que reemplaza a la grilla de disponibilidad eliminada. **No existe en el código** y `casos-de-uso.md` no lo menciona. Hay que decidir si se programa o si se quita esa frase de las decisiones.
Suspensión	UC-D01 pide «Capitán no suspendido», pero la Suspensión no existe todavía (UC-F05). Hoy no se verifica en ninguna función de Partidos.
Leer las calificaciones y sus comentarios	Los lee el administrador al revisar un equipo reportado (UC-F04), pero la tabla no la lee nadie por la API: falta definir cómo accede el backoffice.
Bloqueo por faltas	Es automático y no avisa; el rival se entera por la propuesta cancelada. Está implementado; se documenta aquí porque no tiene pantalla propia.
El motivo de tirar un partido no se modera	Riesgo aceptado (27/10/2026): queda visible en el historial público tal cual lo escribió el Capitán.

6. Reglas de referencia

- **Fases de un partido:** por jugarse (con las marcas **A coordinar**, **Coordinado** y **Confirmado**, que son datos y no fases), esperando resultado, **Jugado** (con o sin resultado), **caído**, **en conflicto** y **no presentado**. Las cuatro últimas son los «cerrados» del historial.

- **Hora de inicio:** la hora exacta coordinada, o el inicio del rango. Antes: solo tirar el partido. Después: solo resultado o «no vinieron».
- **Términos:** un partido **se cae** (nunca «se suspende»); se dice «tirar el partido»; «Aviso» y «campanita»; «Franja» es el término del dominio y **«horario»** en la interfaz.

Parte 2 · Modelo de dominio y casos de uso, con sus diagramas y su pantalla

Versión del **04/11/2026**. Detalla los casos de uso del módulo D de casos-de-uso.md (UC-D01 a UC-D14) con los requerimientos que cubre cada uno (01-requerimientos.md), la pantalla que lo representa (03-pantallas.md) y su diagrama de secuencia (04-diagramas-secuencia).

Cómo leerlo. Cada caso expresa una meta y tiene **un actor iniciador**. **Estado** dice si está en `main`. Los mensajes entre comillas son los que ve la persona. Todo el módulo lo opera **solo el Capitán**; el Plantel ve los partidos sin botones. Las dos pantallas son `/partidos` (lo que tenés) y `/partidos/buscar` (lo que querés).

Mapa de casos de uso

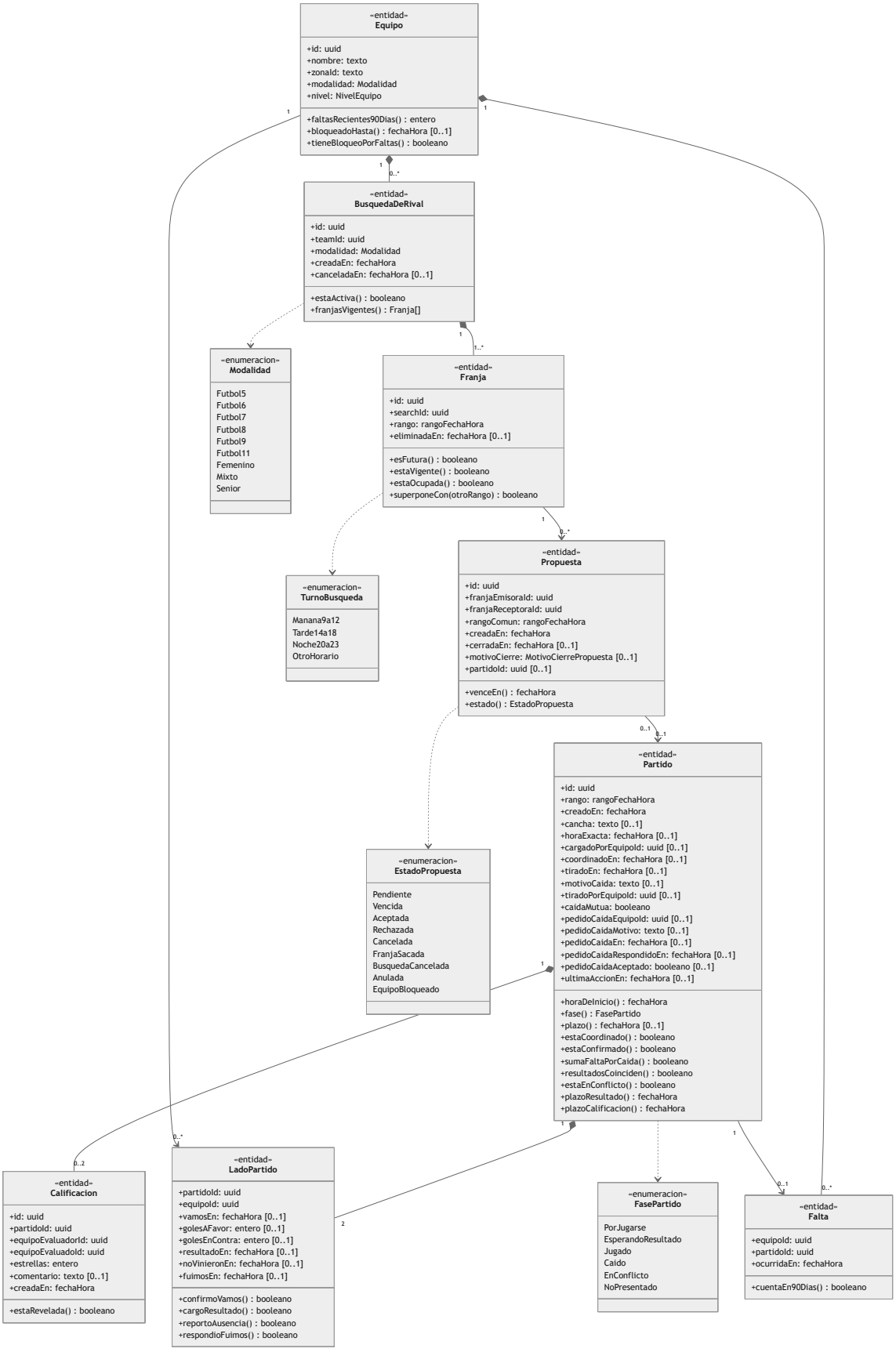
Caso	Meta del actor	Actor	Estado	ID en el PDF original
UC-D01	Publicar una búsqueda de rival	Capitán	**Parcial** (Suspensión)	UC-13
UC-D02	Editar o cancelar la búsqueda	Capitán	Implementado	—
UC-D03	Ver rivales compatibles	Capitán	Implementado	UC-14
UC-D04	Proponer un partido	Capitán	Implementado	UC-15
UC-D05	Cancelar una propuesta enviada	Capitán	Implementado	UC-17
UC-D06	Aceptar o rechazar una propuesta	Capitán	Implementado	UC-16
UC-D07	Coordinar cancha y hora	Capitán	Implementado	UC-18
UC-D08	Confirmar asistencia	Capitán	Implementado	UC-19
UC-D09	Tirar el partido	Capitán	Implementado	UC-20
UC-D10	Caída de común acuerdo	Capitán	Implementado	—
UC-D11	Cargar el resultado	Capitán	Implementado	—
UC-D12	Reportar que el rival no se presentó	Capitán	Implementado	—
UC-D13	Calificar al rival	Capitán	Implementado	—
UC-D14	Ver mis partidos	Capitán o Jugador	Implementado	—

Los diagramas generales de `diagramas-secuencia.md` (D12 a D19) cubren los ocho primeros casos del PDF original (UC-13 a UC-20); el resto (editar, común acuerdo, resultado, «no vinieron», calificar y mis partidos) figura como «para otra tanda» y no tiene correspondencia exacta en los documentos del repo.

Qué no es un caso de uso acá: abrir una pestaña, desplegar «Historial» o «Más adelante», o los avisos de estado que la app calcula sola (vencimientos, faltas, bloqueo) no son casos: están en las reglas transversales.

Diagrama de clases · **Modelo de dominio del módulo**

Vista estructural de las entidades, relaciones de composición/asociación y enumeraciones involucradas en la **Búsqueda de rival**, el cruce de **Franjas**, las **Propuestas**, la máquina de estados del **Partido**, el presentismo (**Faltas** y **Bloqueo por faltas**) y la **Calificación** con revelación cruzada (`UC-D01` a `UC-D14`).


«entidad»
Equipo
+id: uuid
+nombre: texto
+zonaId: texto
+modalidad: Modalidad
+nivel: NivelEquipo
+faltasRecientes90Dias() : entero
+bloqueadoHasta() : fechaHora [0..1]
+tieneBloqueoPorFaltas() : booleano
«entidad»
BusquedaDeRival
+id: uuid
+teamId: uuid
+modalidad: Modalidad
+creadaEn: fechaHora
+canceladaEn: fechaHora [0..1]
+estaActiva() : booleano
+franjasVigentes() : Franja[]
«enumeracion»
Modalidad
Futbol5
Futbol6
Futbol7
Futbol8
Futbol9
Futbol11
Femenino
Mixto
Senior
«entidad»
Franja
+id: uuid
+searchId: uuid
+rango: rangoFechaHora
+eliminadaEn: fechaHora [0..1]
+esFutura() : booleano
+estaVigente() : booleano
+estaOcupada() : booleano
+superponeCon(otroRango) : booleano
«enumeracion»
TurnoBusqueda
Manana9a12
Tarde14a18
Noche20a23
OtroHorario
«entidad»
Propuesta
+id: uuid
+franjaEmisoraId: uuid
+franjaReceptoraId: uuid
+rangoComun: rangoFechaHora
+creadaEn: fechaHora
+cerradaEn: fechaHora [0..1]
+motivoCierre: MotivoCierrePropuesta [0..1]
+partidoId: uuid [0..1]
+venceEn() : fechaHora
+estado() : EstadoPropuesta
«enumeracion»
EstadoPropuesta
Pendiente
Vencida
Aceptada
Rechazada
Cancelada
FranjaSacada
BusquedaCancelada
Anulada
EquipoBloqueado
«entidad»
Partido
+id: uuid
+rango: rangoFechaHora
+creadoEn: fechaHora
+cancha: texto [0..1]
+horaExacta: fechaHora [0..1]
+cargadoPorEquipoId: uuid [0..1]
+coordinadoEn: fechaHora [0..1]
+tiradoEn: fechaHora [0..1]
+motivoCaida: texto [0..1]
+tiradoPorEquipoId: uuid [0..1]
+caidaMutua: booleano
+pedidoCaidaEquipoId: uuid [0..1]
+pedidoCaidaMotivo: texto [0..1]
+pedidoCaidaEn: fechaHora [0..1]
+pedidoCaidaRespondidoEn: fechaHora [0..1]
+pedidoCaidaAceptado: booleano [0..1]
+ultimaAccionEn: fechaHora [0..1]
+horaDeInicio() : fechaHora
+fase() : FasePartido
+plazo() : fechaHora [0..1]
+estaCoordinado() : booleano
+estaConfirmado() : booleano
+sumaFaltaPorCaida() : booleano
+resultadosCoinciden() : booleano
+estaEnConflicto() : booleano
+plazoResultado() : fechaHora
+plazoCalificacion() : fechaHora
«entidad»
Calificacion
+id: uuid
+partidoId: uuid
+equipoEvaluadorId: uuid
+equipoEvaluadoId: uuid
+estrellas: entero
+comentario: texto [0..1]
+creadaEn: fechaHora
+estaRevelada() : booleano
«entidad»
LadoPartido
+partidoId: uuid
+equipoId: uuid
+vamosEn: fechaHora [0..1]
+golesAFavor: entero [0..1]
+golesEnContra: entero [0..1]
+resultadoEn: fechaHora [0..1]
+noVinieronEn: fechaHora [0..1]
+fuimosEn: fechaHora [0..1]
+confirmoVamos() : booleano
+cargoResultado() : booleano
+reportoAusencia() : booleano
+respondioFuimos() : booleano
«enumeracion»
FasePartido
PorJugarse
EsperandoResultado
Jugado
Caido
EnConflicto
NoPresentado
«entidad»
Falta
+equipoId: uuid
+partidoId: uuid
+ocurridaEn: fechaHora
+cuentaEn90Dias() : booleano

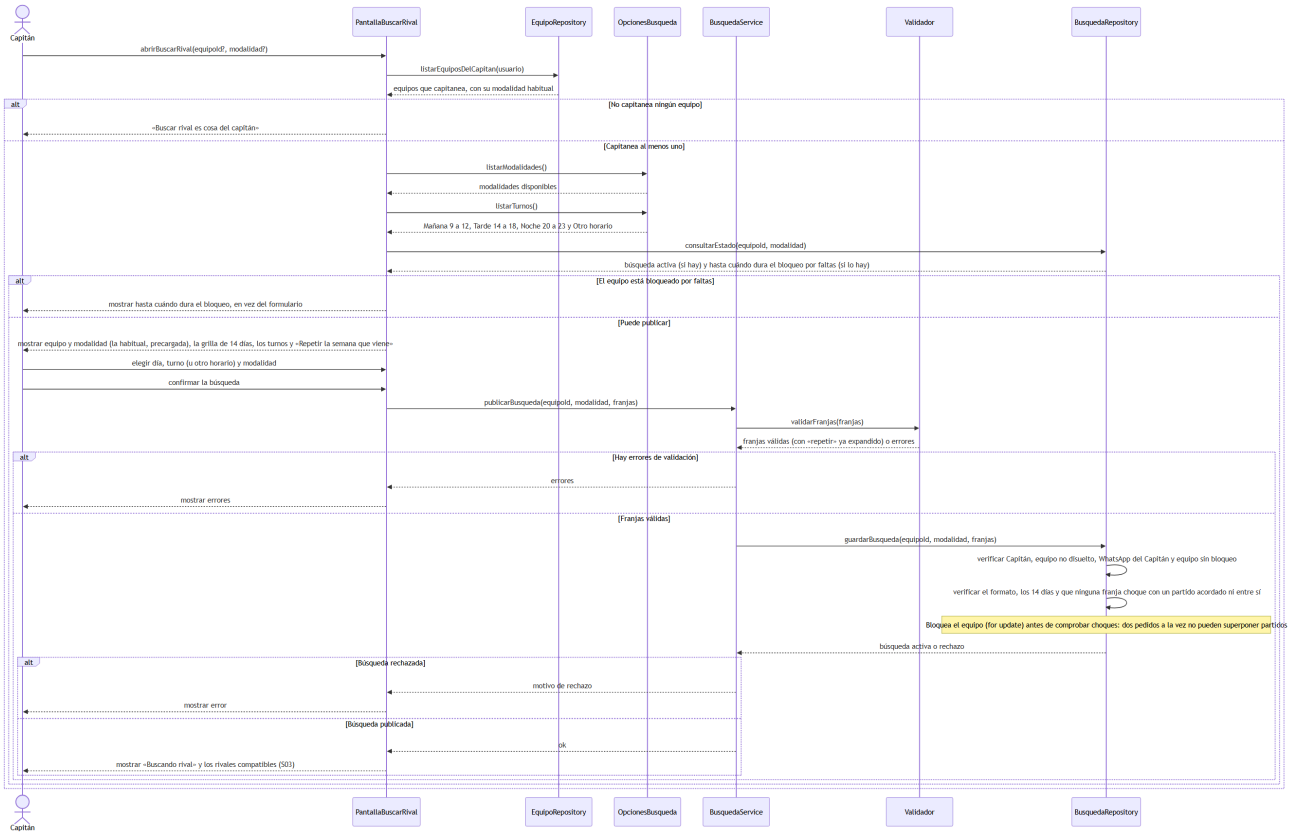
UC-D01 · Publicar una búsqueda de rival

- **Actor:** Capitán.
- **Requerimientos:** RF-D01 a RF-D06, RF-D32. **Pantalla:** P2 · Buscar rival (`/partidos/buscar`). **Diagrama:** S01.
- **Precondiciones:** equipo no disuelto; Capitán con WhatsApp en el perfil y no suspendido (hoy no se verifica); equipo sin **Bloqueo por faltas**; sin otra búsqueda activa del equipo en esa modalidad.
- **Postcondición:** búsqueda activa mientras le quede alguna franja por empezar; vence sola al pasar la última.
- **Estado: Parcial** (`save_rival_search`): todo salvo la verificación de la Suspensión.

Diagrama de secuencia · **S01 · UC-D01 — Publicar una búsqueda de rival**

Caso de uso: UC-D01 · Pantalla: P2 · Requerimientos: RF-D02 a RF-D06, RF-D32

El formulario tiene **tres elecciones con opciones**: el **equipo** (si el Capitán capitanea más de uno), la **modalidad** y el **turno**. Siguiendo la guía de formularios, la pantalla obtiene las opciones **antes** de que el Capitán elija: los equipos, que se consultan en la base, de `EquipoRepository`; las modalidades y los turnos, que son listas fijas del código, de `OpcionesBusqueda`. Al enviar viajan solo los valores elegidos. La carga de las opciones y la publicación van en el mismo diagrama.



Cómo se lee

- **Antes de elegir:** la pantalla obtiene los equipos de `EquipoRepository` y las modalidades y los turnos de `OpcionesBusqueda`. El equipo solo se pregunta si el Capitán capitanea más de uno; la modalidad llega con la habitual del equipo.
- **Al enviar,** `publicarBusqueda(...)` recibe solo los **valores elegidos** (`equipoId`, `modalidad`, `franjas`), no las listas. Las franjas se arman en el servidor a partir de fecha y horas enteras, en hora de Buenos Aires.
- **Se valida dos veces, y manda la base.** `Validador` revisa horas enteras, inicio menor que fin, dentro de 14 días, sin pisarse y el atajo «repetir»; la función de la base **repite todo** (formato, 14 días, 8 a 24, al menos 1 h, hoy en una hora que no pasó, choques) y además verifica Capitán, WhatsApp y bloqueo.
- **Una franja que choca con un partido acordado se rechaza**; entre búsquedas de **distinta modalidad** del mismo equipo se pueden superponer.
- **La búsqueda guarda solo modalidad y franjas.** La cercanía y el nivel salen de la Zona y el nivel **actuales** del equipo, así que si el equipo los cambia, se refleja en el acto.
- **Publicar no genera ningún Aviso.** La búsqueda queda activa mientras le quede una franja por empezar y vence sola.
- **Suspensión (pendiente):** un Capitán suspendido no puede buscar rival; hoy no se verifica (UC-F05).

- **«Copiar horarios de la última búsqueda»** figura en las decisiones, pero no existe en el código (ver «Sin decisión»).

Correspondencia en el código

`saveRivalSearch` (`src/app/partidos/actions.ts`): `parseRivalSearchForm` → RPC `save_rival_search(p_team_id, p_modality, p_slots)` . La pantalla es `app/partidos/buscar/page.tsx` con el componente `SlotPicker` .

Diagrama de clases · **UC-D01 · Publicar una búsqueda de rival**

Caso de uso: UC-D01 · **Pantalla:** P2 · Buscar rival (`/partidos/buscar`) · **Secuencia:** S01



> Pantalla · **P2 · Buscar rival**

Ruta: `/partidos/buscar` · **Casos de uso:** UC-D01 a UC-D05 · **Requerimientos:** RF-D02 a RF-D13 · **Diagramas:** S01 a S05

> ⏳ **Captura pendiente.**

Qué muestra: tres pasos en una sola columna.

1. **El equipo y la modalidad.** El equipo solo si el Capitán capitanea más de uno; la modalidad como chips (en el celular, plegados en una línea «Los Pibes · Fútbol 5 · Cambiar»). La modalidad viene cargada con la habitual del equipo.
2. **¿Cuándo pueden jugar?** Sin búsqueda activa, el formulario de la primera franja: una **grilla de 14 días** para tocar un día, un **Turno** (Mañana 9–12, Tarde 14–18, Noche 20–23) u «Otro horario» con dos horas enteras, y la casilla «**Repetir la semana que viene**». Con una búsqueda activa, el título **«Buscando rival»**, cada franja como una tarjeta con el bloque del día y una **×**, el botón «**Agregar otro horario**» que abre el formulario y el link «**Cancelar la búsqueda**».
3. **Rivales compatibles.** Una tarjeta por franja en común con cada rival compatible, con el horario en común y el botón «**Proponer partido**». Una propuesta ya mandada muestra «Propuesta enviada» y «Cancelar»; si la última ya no está pendiente, una nota con lo que pasó («Tu propuesta venció», «Rechazaron tu propuesta»).

Opciones de los formularios:

Elección	De dónde salen las opciones
Equipo	Se **consultan en la base**: los equipos que el usuario capitanea
Modalidad	Lista **fija del código** (`SPORT_MODALITIES` : Fútbol 5, 7, 8 y 11)
Día	Los próximos 14 días, que calcula la app
Turno	Lista **fija del código** (Mañana, Tarde, Noche y «Otro horario»)

Estados de la pantalla: *sin búsqueda* (formulario de la primera franja); *buscando* (tarjetas de franjas y compatibles); *sin compatibles* («Ningún equipo de la zona publicó esos horarios. Con más horarios, más chances de partido.»); *equipo bloqueado* (en vez del formulario, hasta cuándo dura el bloqueo); *no es Capitán* («Buscar rival es cosa del capitán», con el texto «Si capitaneás un equipo, publicás cuándo pueden jugar y te mostramos rivales de tu zona»); *sin sesión* (redirige al login).

> En la interfaz, la **franja se llama «horario»** («Agregar otro horario», «¿Cuándo pueden jugar?»): «franja» es el término del dominio, en el código y en los documentos.

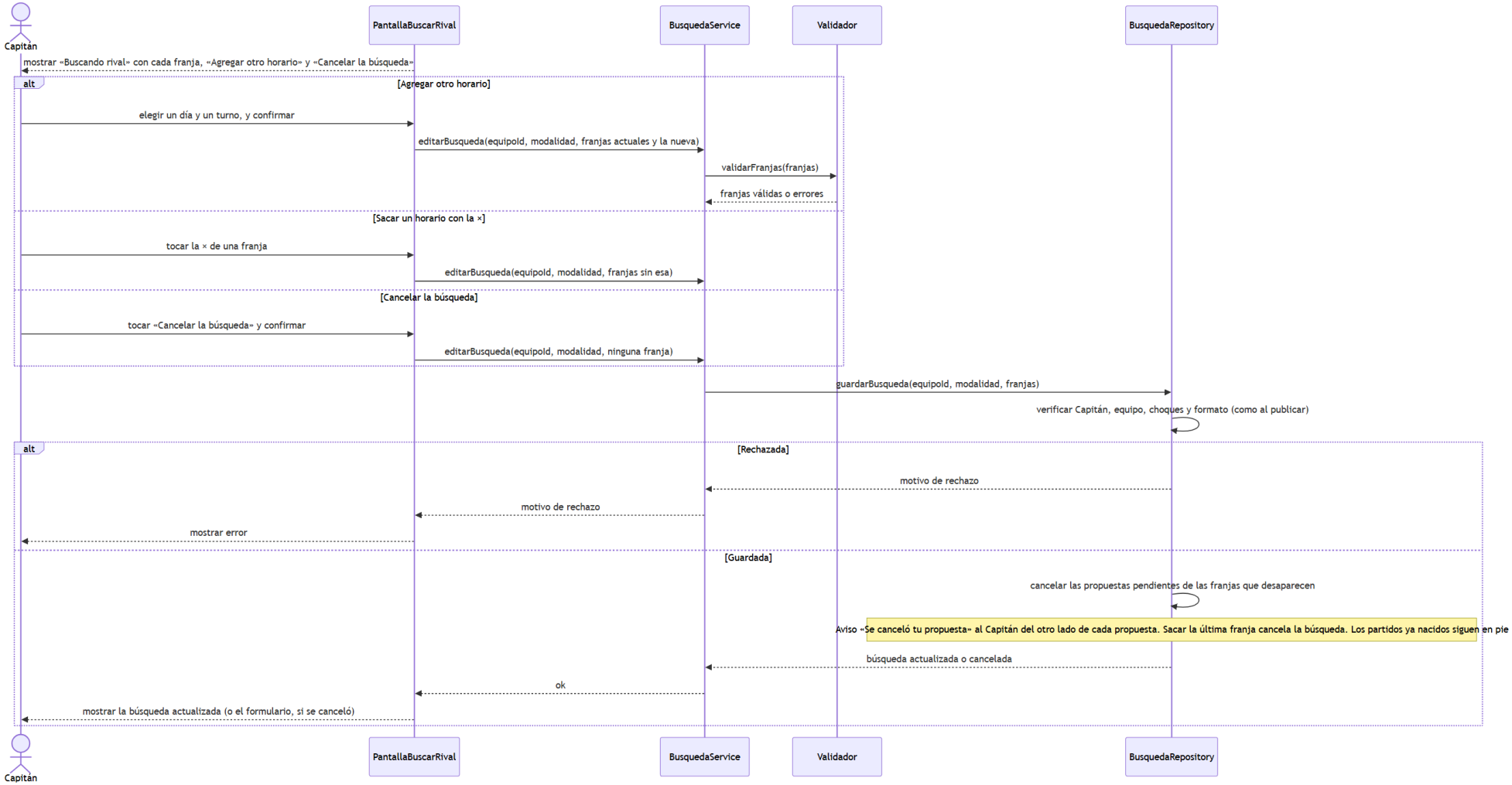
UC-D02 · Editar o cancelar la búsqueda

- **Actor:** Capitán.
- **Requerimientos:** RF-D07, RF-D08. **Pantalla:** P2 · Buscar rival. **Diagrama:** S02.
- **Precondiciones:** tiene una búsqueda activa.
- **Estado:** Implementado (`save_rival_search` con el conjunto completo de franjas; vacío = cancelar).

Diagrama de secuencia · **S02 · UC-D02 — Editar o cancelar la búsqueda**

Caso de uso: UC-D02 · Pantalla: P2 · Requerimientos: RF-D07, RF-D08

Agregar un horario usa el mismo formulario de S01 (los mismos turnos y días). Este diagrama muestra lo que cambia: la función recibe **el conjunto completo de franjas** que debe quedar, y las que desaparecen **cancelan sus propuestas**.



Cómo se lee

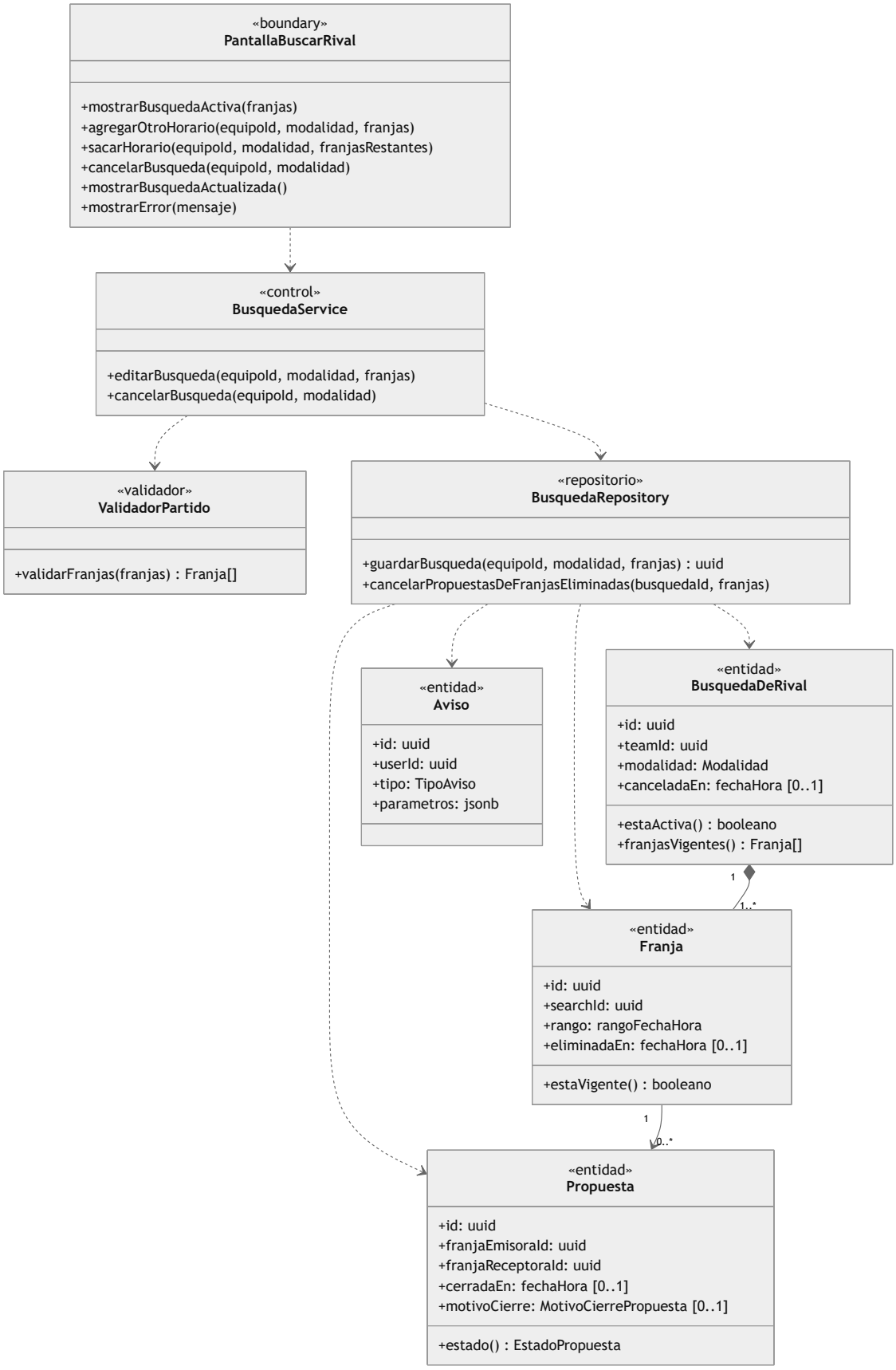
- **Una sola función para las tres cosas.** `save_rival_search` recibe el **conjunto completo de franjas**: agregar es mandar una más, sacar es mandar una menos, y **cancelar es mandar ninguna**. La modalidad no se cambia: es otra búsqueda.
- **Las propuestas siguen a las franjas.** Sacar una franja cancela sus propuestas pendientes (enviadas y recibidas), con el Aviso «Se canceló tu propuesta» al Capitán del otro lado. Cancelar la búsqueda cancela todas.
- **Los partidos que ya nacieron de la búsqueda siguen en pie.** Que nazca un partido no cierra la búsqueda: sigue con sus otras franjas.
- **La validación se repite en la base**, igual que al publicar.

Correspondencia en el código

Misma Server Action que S01, `saveRivalSearch`; la base distingue crear, editar y cancelar según el conjunto de franjas que recibe.

Diagrama de clases · **UC-D02 · Editar o cancelar la búsqueda**

Caso de uso: UC-D02 · **Pantalla:** P2 · Buscar rival · **Secuencia:** S02



> Pantalla · **P2 · Buscar rival**

Es la misma pantalla que se describe, con su captura, en P2 · Buscar rival, debajo de UC-D01 · Publicar una búsqueda de rival. Este caso usa la parte que indica su ficha («Pantalla»).

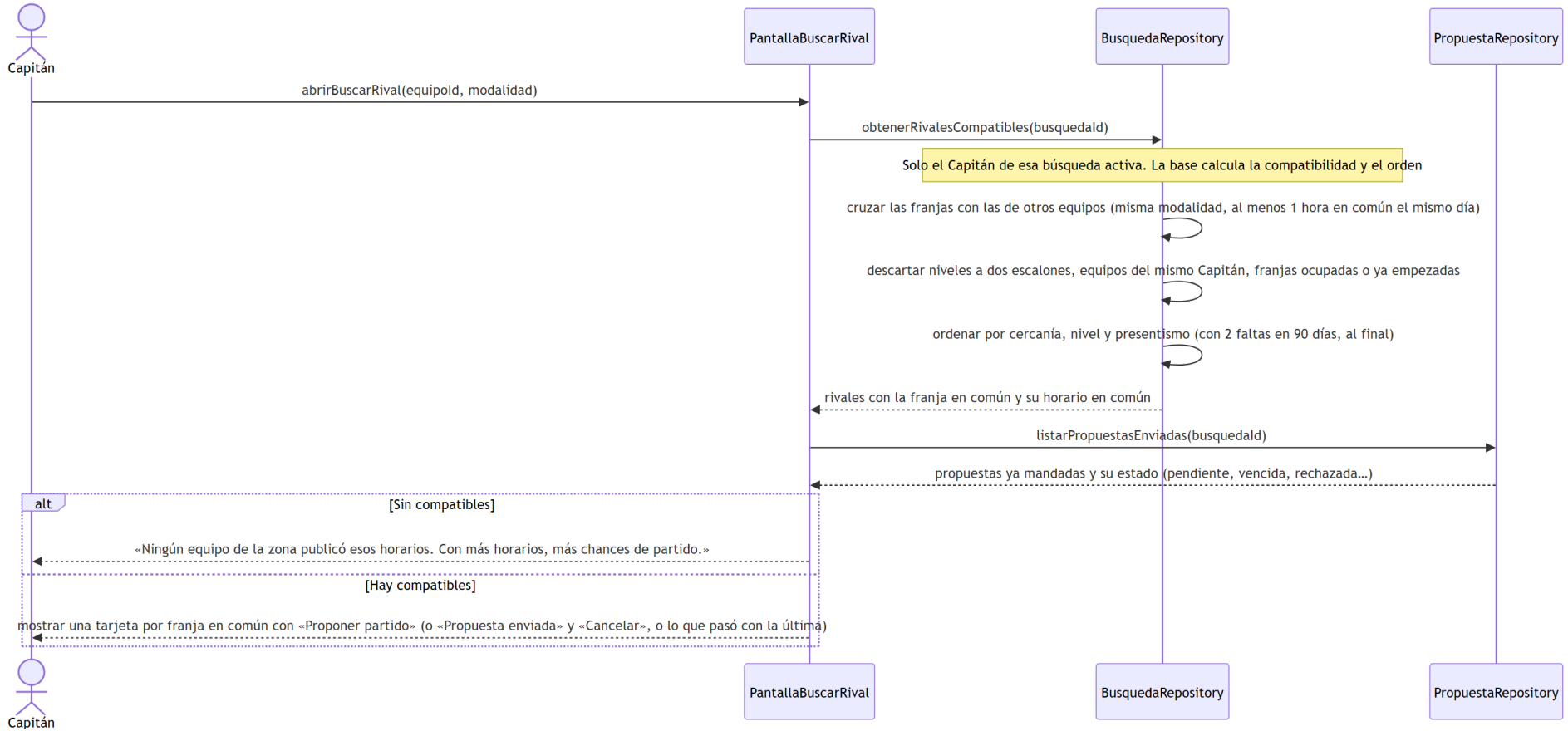
UC-D03 · Ver rivales compatibles

- **Actor:** Capitán.
- **Requerimientos:** RF-D09, RF-D10, RF-D31. **Pantalla:** P2 · Buscar rival (sección «Rivales compatibles»). **Diagrama:** S03.
- **Precondiciones:** búsqueda activa.
- **Orden:** cercanía (Zona > Corredor > Región), después nivel, después **Presentismo**; los equipos con 2 faltas en 90 días van al final; empate: el horario en común más temprano.
- **Estado:** Implementado (`compatible_rivals`).

Diagrama de secuencia · **S03 · UC-D03 — Ver rivales compatibles**

Caso de uso: UC-D03 · Pantalla: P2 · Requerimientos: RF-D09, RF-D10, RF-D31

Es una consulta: no hay formulario ni desplegables.



Cómo se lee

- **La calcula la base, no la pantalla:** `compatible_rivals` es una función `security definer` que responde solo al Capitán de esa búsqueda activa y devuelve el equipo, la franja y el horario en común, **ya ordenados**.

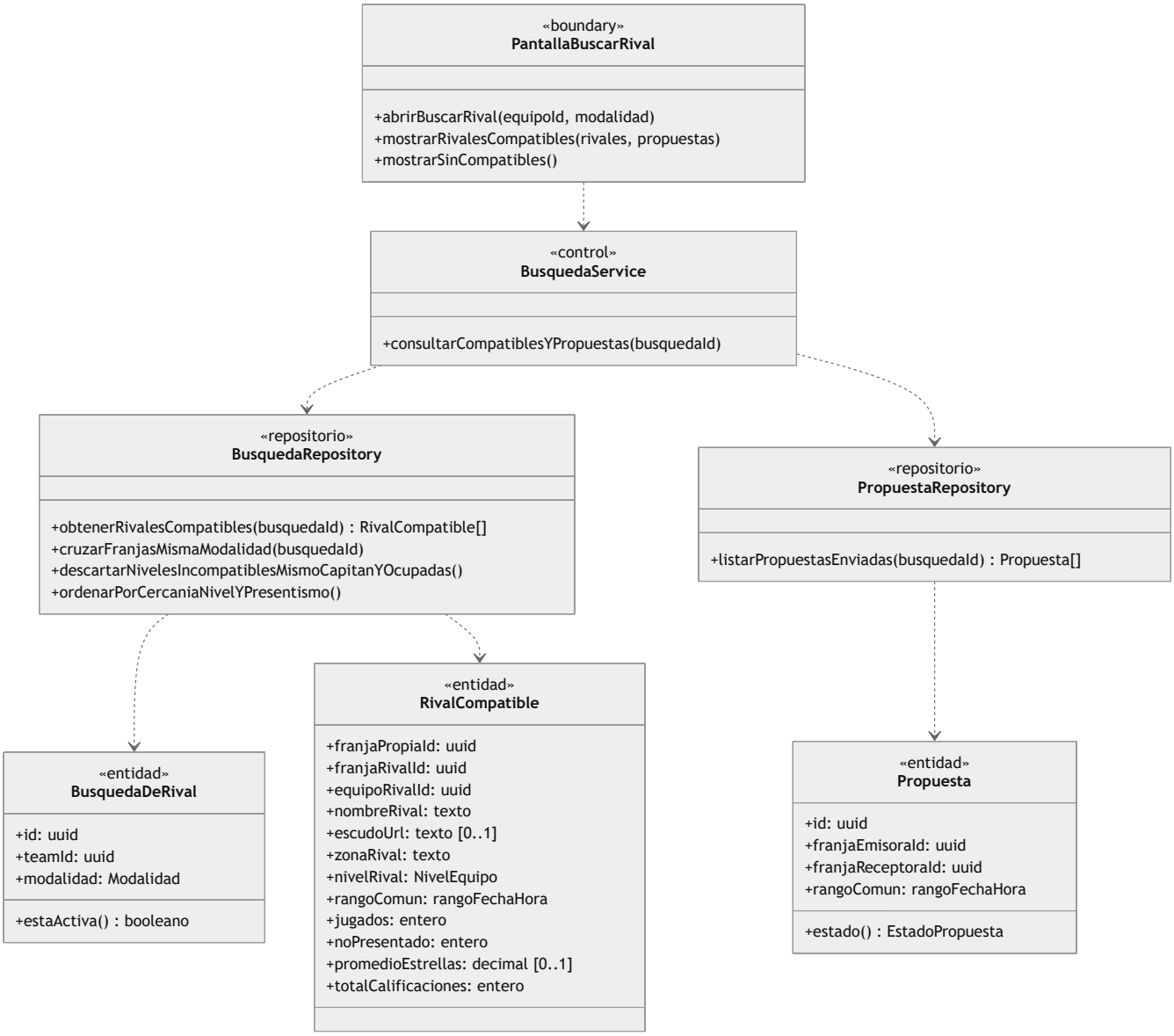
- **Compatible significa:** misma modalidad y al menos 1 h en común el mismo día; los niveles con dos escalones de diferencia (recreativo contra competitivo) **no se cruzan**; **nunca dos equipos del mismo Capitán** (si no, una persona cargaría los dos resultados y las dos calificaciones); una **franja ocupada** (superpuesta con un partido del equipo) o ya empezada se ignora.
- **Orden:** primero la cercanía (Zona, luego Corredor, luego Región), después el nivel y por último el **presentismo** (faltas en los últimos 90 días, de menos a más; sin historial cuenta 0). Los equipos con **2 faltas en 90 días** van al final igual. Empate: el horario en común más temprano primero.
- **La Zona y el nivel son los actuales del equipo:** la búsqueda no los guarda.
- **Cada tarjeta recuerda lo ya hecho:** si ya se le mandó una propuesta por ese par de franjas, se ve «Propuesta enviada»; si la última se cerró, una nota dice qué pasó y se puede volver a proponer.

Correspondencia en el código

`app/partidos/buscar/page.tsx` llama a la función `compatible_rivals(p_search_id)` y lee `proposals` de la búsqueda. Probado en `supabase/tests/compatible_rivals_test.sql`.

Diagrama de clases · **UC-D03 · Ver rivales compatibles**

Caso de uso: UC-D03 · **Pantalla:** P2 · Buscar rival (sección «Rivales compatibles») · **Secuencia:** S03



Pantalla · **P2 · Buscar rival**

Es la misma pantalla que se describe, con su captura, en P2 · Buscar rival, debajo de UC-D01 · Publicar una búsqueda de rival. Este caso usa la parte que indica su ficha («Pantalla»).

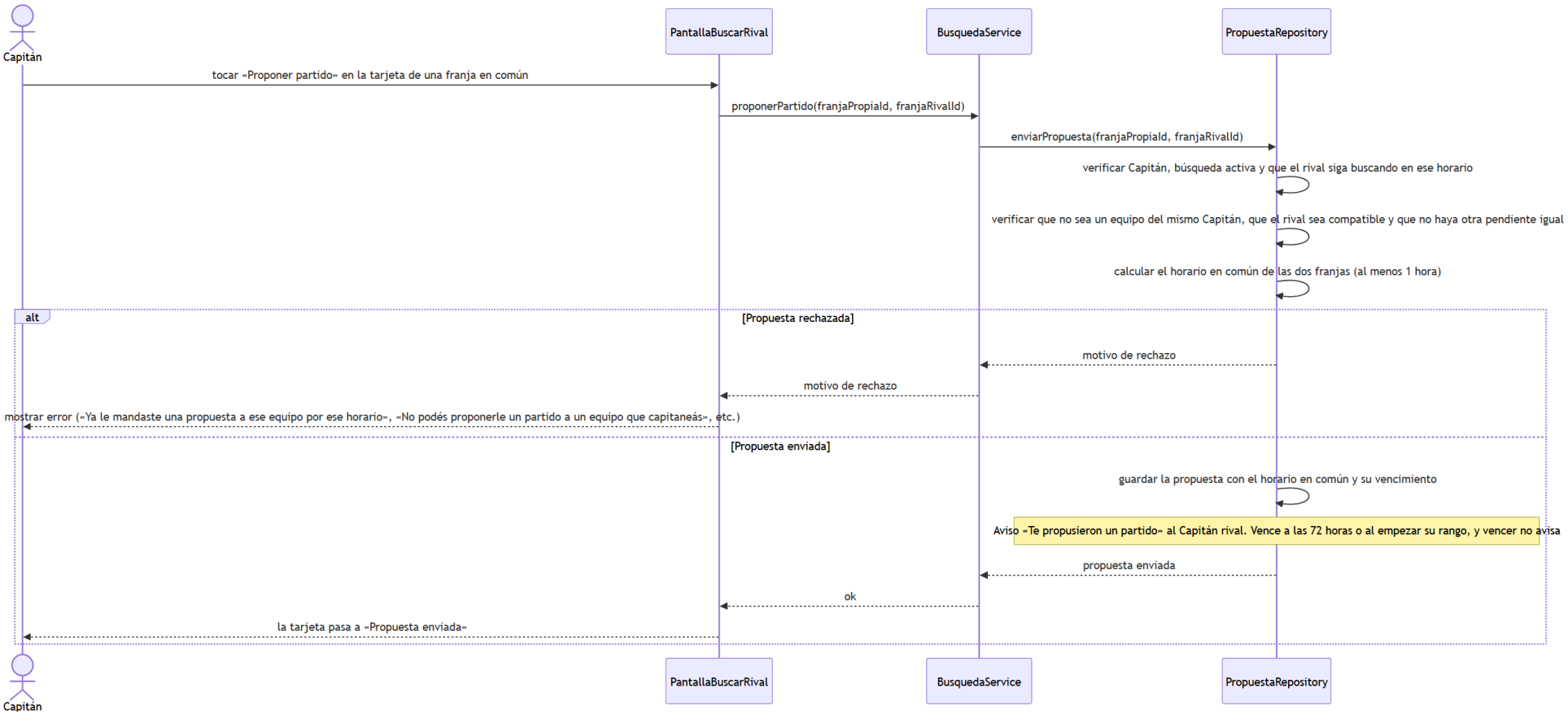
UC-D04 · Proponer un partido

- **Actor:** Capitán.
- **Requerimientos:** RF-D11, RF-D12. **Pantalla:** P2 · Buscar rival. **Diagrama:** S04.
- **Precondiciones:** ve al rival en los compatibles.
- **Estado:** Implementado (`send_proposal`).

Diagrama de secuencia · **S04 · UC-D04 — Proponer un partido**

Caso de uso: UC-D04 · Pantalla: P2 · Requerimientos: RF-D11, RF-D12

No hay formulario ni desplegables: el Capitán toca «Proponer partido» sobre una tarjeta de S03. La propuesta **no lleva mensaje** y **nadie elige un rango**.



Cómo se lee

- **Qué viaja:** solo los ids de las dos franjas. **El horario lo calcula la app**: es la parte compartida de las dos franjas; nadie elige un rango y no hay mensaje.
- **La base repite todas las reglas** (los ids vienen del cliente y no son de confiar): el Capitán, que la búsqueda siga activa, que el rival siga buscando en ese horario, que no sean equipos del mismo Capitán, que haya al menos 1 h en común, que sea compatible y que no haya una pendiente igual.

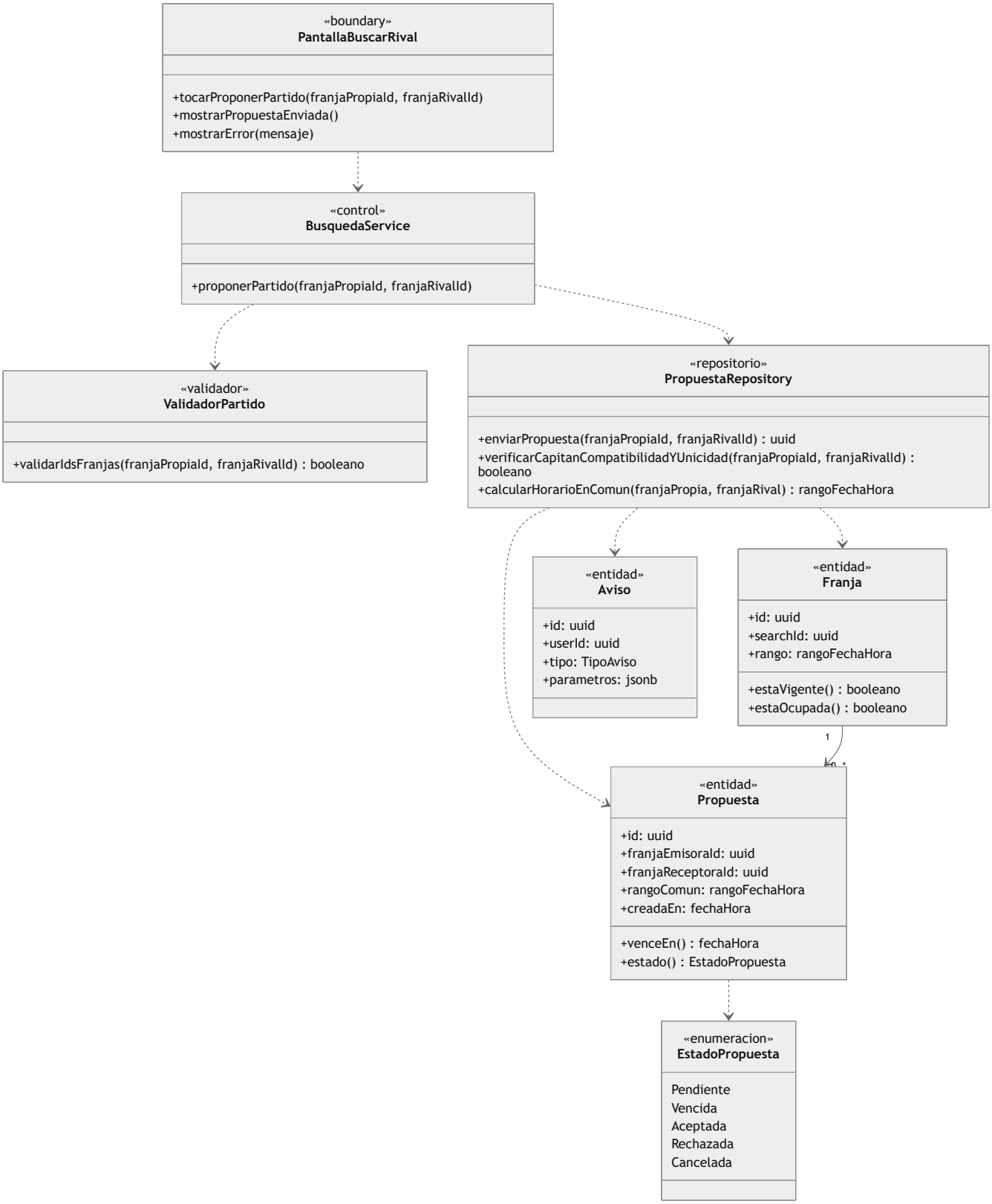
- **Sin límite** de propuestas pendientes por equipo; solo no se puede mandar otra al mismo rival por el mismo horario mientras la anterior esté pendiente.
- **Vence sola** a las 72 h o cuando empieza su rango (lo que ocurra primero) y **no avisa**: el estado se calcula al leer.
- **Una propuesta cerrada no impide volver a proponer.**

Correspondencia en el código

`sendProposal` (`src/app/partidos/actions.ts`) llama a la RPC `send_proposal(p_slot_id, p_rival_slot_id)`. Probado en `supabase/tests/proposals_test.sql`.

Diagrama de clases · **UC-D04 · Proponer un partido**

Caso de uso: UC-D04 · **Pantalla:** P2 · Buscar rival · **Secuencia:** S04



> Pantalla · **P2 · Buscar rival**

Es la misma pantalla que se describe, con su captura, en P2 · Buscar rival, debajo de UC-D01 · Publicar una búsqueda de rival. Este caso usa la parte que indica su ficha («Pantalla»).

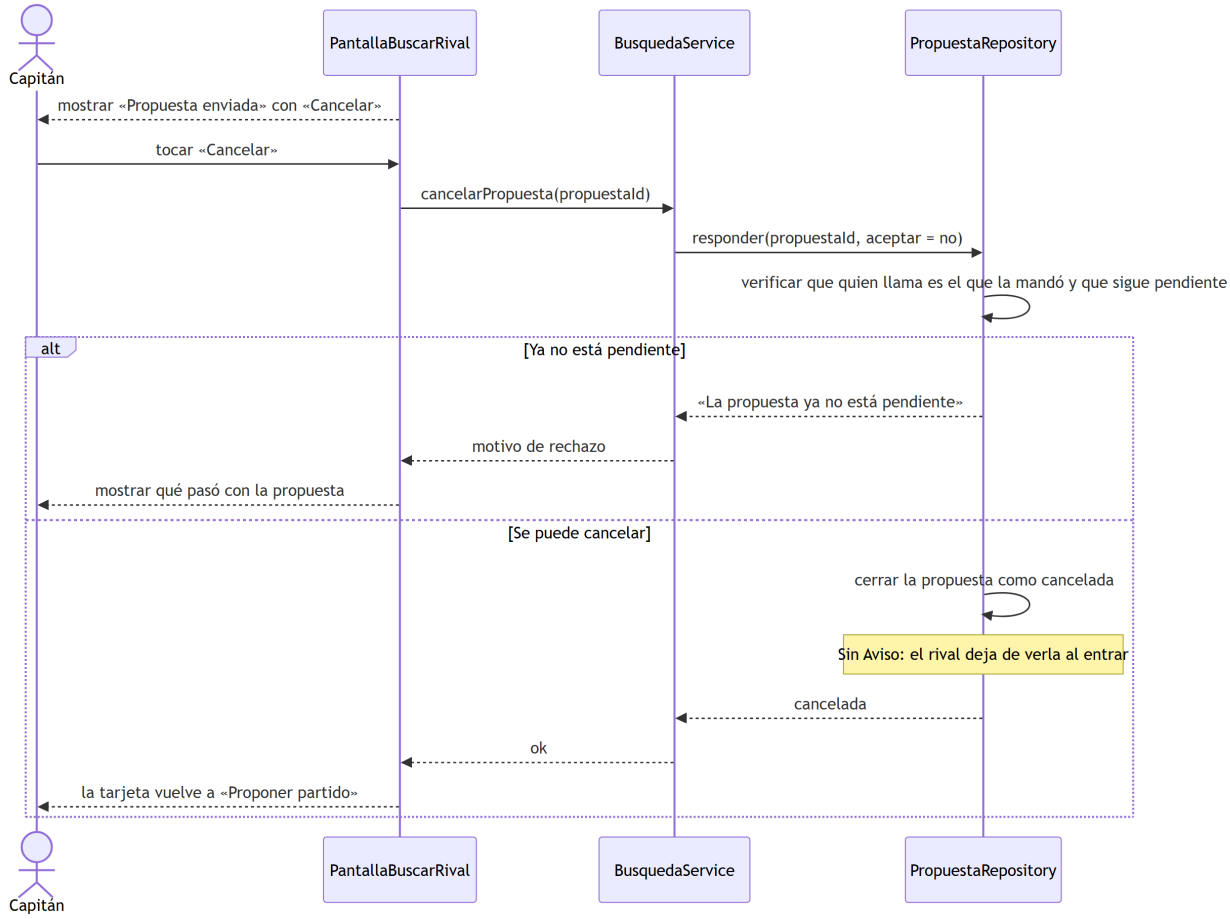
UC-D05 · Cancelar una propuesta enviada

- **Actor:** Capitán que la mandó.
- **Requerimientos:** RF-D13. **Pantalla:** P2 · Buscar rival. **Diagrama:** S05.
- **Precondiciones:** la propuesta está pendiente.
- **Estado:** Implementado (la misma función `respond_proposal` que rechazar; el sistema distingue por quién la llama).

> Diagrama de secuencia · **S05 · UC-D05 — Cancelar una propuesta enviada**

Caso de uso: UC-D05 · Pantalla: P2 · Requerimientos: RF-D13

No hay formulario ni desplegables.



Cómo se lee

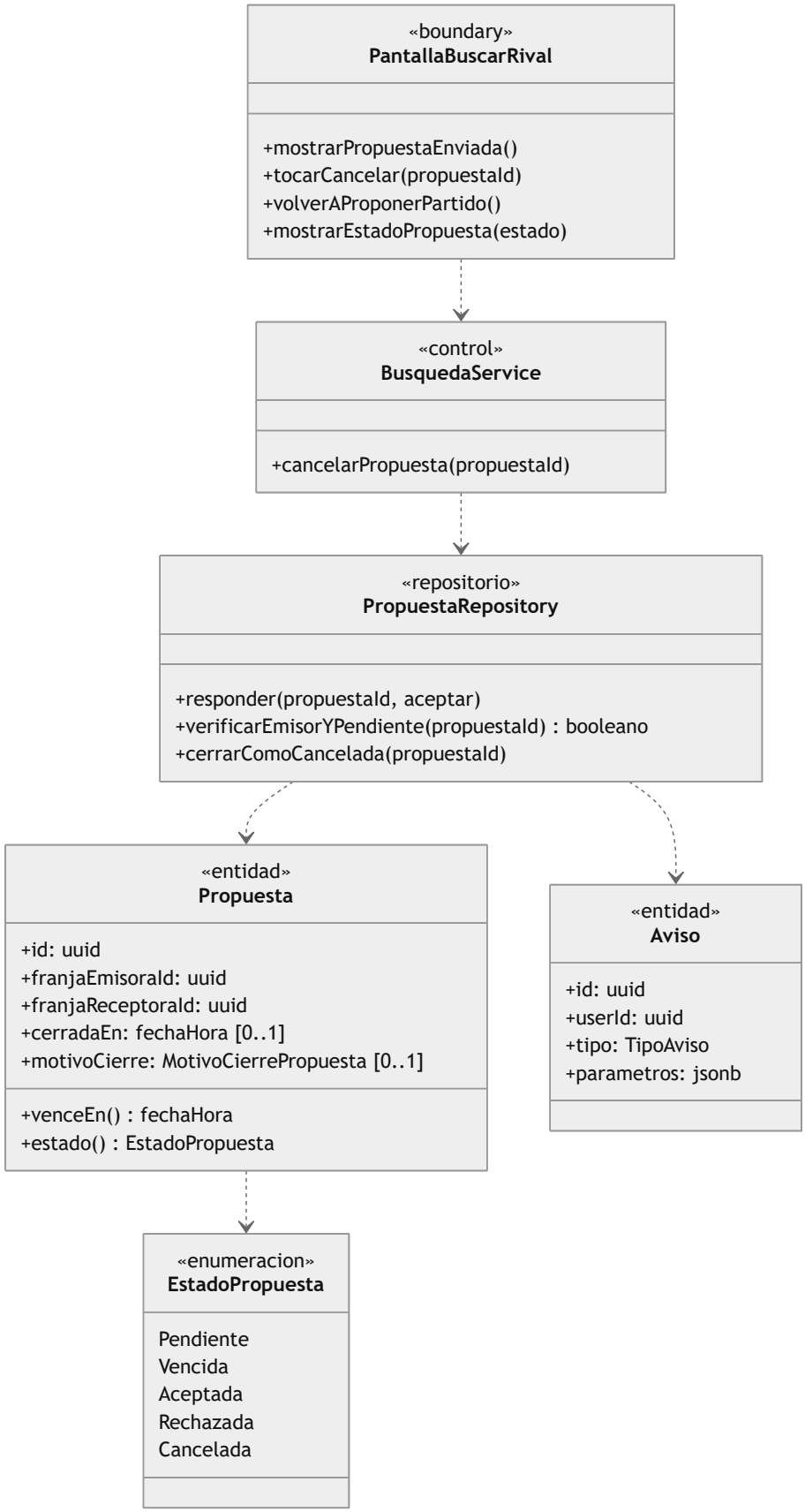
- **Cancelar no avisa.** Solo avisan las cancelaciones en cascada (sacar una franja, cancelar la búsqueda, disolver el equipo, el bloqueo por faltas). El rival simplemente deja de ver la propuesta al entrar a Partidos: **esa pantalla es la fuente de verdad**.
- **Se puede volver a proponer** el mismo horario.
- **Es la misma función que rechazar** (`respond_proposal`): el sistema distingue qué hace por **quién la llama**. El receptor que dice «no» rechaza (con Aviso); el emisor que dice «no» cancela (sin Aviso).

Correspondencia en el código

`declineProposal` (`src/app/partidos/actions.ts`) llama a `respond_proposal(p_proposal_id, p_accept = false)`.

Diagrama de clases · **UC-D05 · Cancelar una propuesta enviada**

Caso de uso: UC-D05 · **Pantalla:** P2 · Buscar rival · **Secuencia:** S05



> Pantalla · **P2 · Buscar rival**

Es la misma pantalla que se describe, con su captura, en P2 · Buscar rival, debajo de UC-D01 · Publicar una búsqueda de rival. Este caso usa la parte que indica su ficha («Pantalla»).

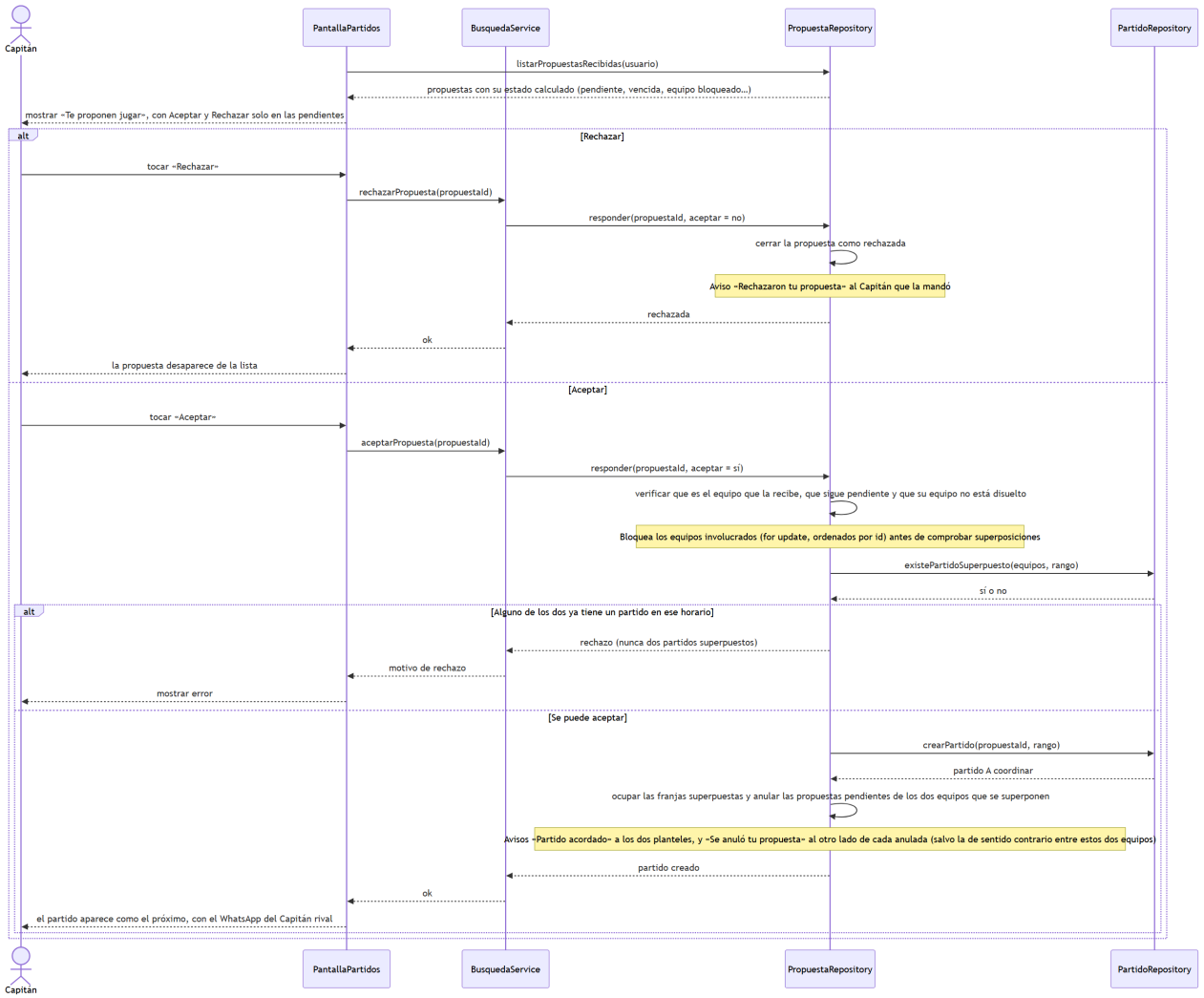
UC-D06 · Aceptar o rechazar una propuesta

- **Actor:** Capitán que la recibió.
- **Requerimientos:** RF-D14 a RF-D18. **Pantalla:** P1 · Partidos (sección «Te proponen jugar»). **Diagrama:** S06.
- **Precondiciones:** la propuesta está pendiente (no venció ni la cancelaron).
- **Postcondición (aceptar):** partido por jugarse; las horas que ocupa son siempre el rango de la propuesta.
- **Estado:** Implementado (`respond_proposal`).

Diagrama de secuencia · **S06 · UC-D06 — Aceptar o rechazar una propuesta**

Caso de uso: UC-D06 · Pantalla: P1 («Te proponen jugar») · Requerimientos: RF-D14 a RF-D18

No hay formulario ni desplegables: son dos botones (**Aceptar** y **Rechazar**) sobre cada propuesta recibida.



Cómo se lee

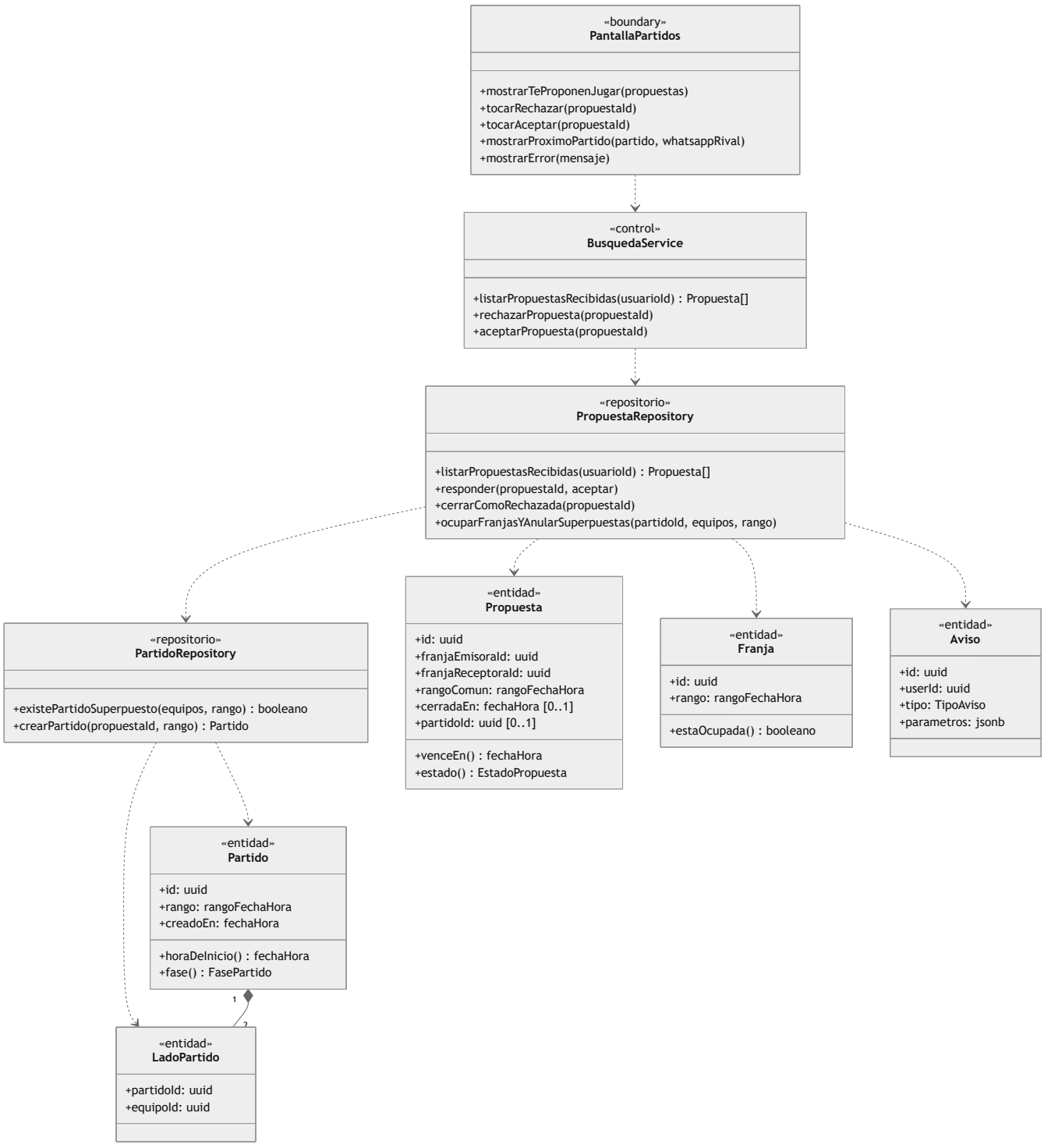
- **Qué nace al aceptar:** un **Partido** con el rango de la propuesta, en estado **A coordinar**. Las horas que ocupa son siempre ese rango. Cada Capitán ve el **WhatsApp del otro** (siempre el del Capitán actual) mientras el partido esté por jugarse.
- **Nunca dos partidos superpuestos.** Al aceptar, la base bloquea las filas de los equipos involucrados y comprueba si alguno ya tiene un partido en ese horario; si lo tiene, la aceptación se rechaza. Al nacer el partido, además, las franjas que se superponen quedan **ocupadas** (salen de los compatibles) y se **anulan** las propuestas pendientes que se superponen.
- **Equipo bloqueado por faltas:** la propuesta se ve cancelada, sin botones, con la nota «el equipo está bloqueado por faltas», hasta cuando vencería. Es la única forma de que el rival se entere: el bloqueo no avisa.
- **El rechazo sí avisa** («Rechazaron tu propuesta»); cancelar la propia no (S05).
- **Las propuestas vencidas** (72 h o rango empezado) se calculan al leer y no avisan.

Correspondencia en el código

`acceptProposal` y `declineProposal` (`src/app/partidos/actions.ts`) llaman a `respond_proposal(p_proposal_id, p_accept)`. Probado en `proposals_test.sql` y `matches_test.sql`.

Diagrama de clases · **UC-D06 · Aceptar o rechazar una propuesta**

Caso de uso: UC-D06 · **Pantalla:** P1 · Partidos (sección «Te proponen jugar») · **Secuencia:** S06



Pantalla · **P1 · Partidos**

Ruta: `/partidos` · **Casos de uso:** UC-D06 a UC-D14 · **Requerimientos:** RF-D14 a RF-D30 · **Diagramas:** S06 a S14

⏳ **Captura pendiente** (varias, una por estado).

Qué muestra, de arriba abajo:

1. **Te proponen jugar** (solo el Capitán): una tarjeta por propuesta recibida, «**Real Floresta** te propone jugar Sáb 7 · 20 a 22 h», con **Aceptar** y **Rechazar**. Una propuesta cancelada por el bloqueo del rival se ve sin botones, con la nota «el equipo está bloqueado por faltas».
2. **El próximo partido**, en grande: tres líneas (cuándo, contra quién, dónde o «Falta cancha y hora») y **una sola acción visible**:
   - desde 24 h antes: «**¿Van?**» con **Vamos** y **No vamos**;
   - si no: «Cargar cancha y hora» (abre el formulario) o el link para cambiarlas. El **WhatsApp del Capitán rival** es el ícono de arriba a la derecha. «Tirar el partido» (o «Tirar el partido (suma falta)») es un link chico al pie que abre el formulario del motivo.
3. **Contanos cómo salió**: los partidos esperando resultado, con el resultado como un **marcador** (los dos escudos y los goles en el medio, el equipo propio a la izquierda), «No vinieron» como link chico, y el vencimiento del plazo.
4. **Más adelante**: los otros partidos por jugarse.
5. **Historial · N partidos**, plegado: los cerrados, del más nuevo al más viejo, una línea cada uno con la fecha, el marcador con los dos nombres y «Ganamos», «Perdimos», «Empatamos», «Se cayó», «En conflicto» o «Falta». Va en primera persona, porque «Ganó» no dice quién. El detalle (quién lo tiró, las dos versiones) vive en la página del equipo.

Lo que pide una respuesta va abierto en la fila con un solo botón primario: confirmar o corregir cancha y hora, «¿Cómo salió?», confirmar el resultado del rival, «Fuimos», la caída de común acuerdo, **Calificar**. Lo que no pide nada va plegado detrás de un link («Cambiar cancha y hora», «Corregir el resultado», «Tirar el partido»).

Formularios de la pantalla:

Formulario	Campos	De dónde salen las opciones
Cargar cancha y hora	**Cancha** (texto, 1 a 120 caracteres); **hora** (desplegable)	La hora es un **desplegable de a 15 minutos dentro del rango del partido**, que calcula la app (`coordinationTimes`); no se guarda en la base
Tirar el partido / pedir la caída	**Motivo** (texto, 1 a 140 caracteres)	—
Resultado	Dos goles (0 a 99)	—
Calificar	**Estrellas** (1 a 5, botones de opción) y comentario (hasta 500 caracteres, privado)	Lista fija del código

Estados de la pantalla: *con partidos*; *vacío* («Todavía no hay partidos», que lleva a Buscar rival si es Capitán o a Mis equipos); *error* (banda con el motivo que devolvió la base); *Jugador* (todo visible, **sin botones**); *sin sesión* (redirige al login).

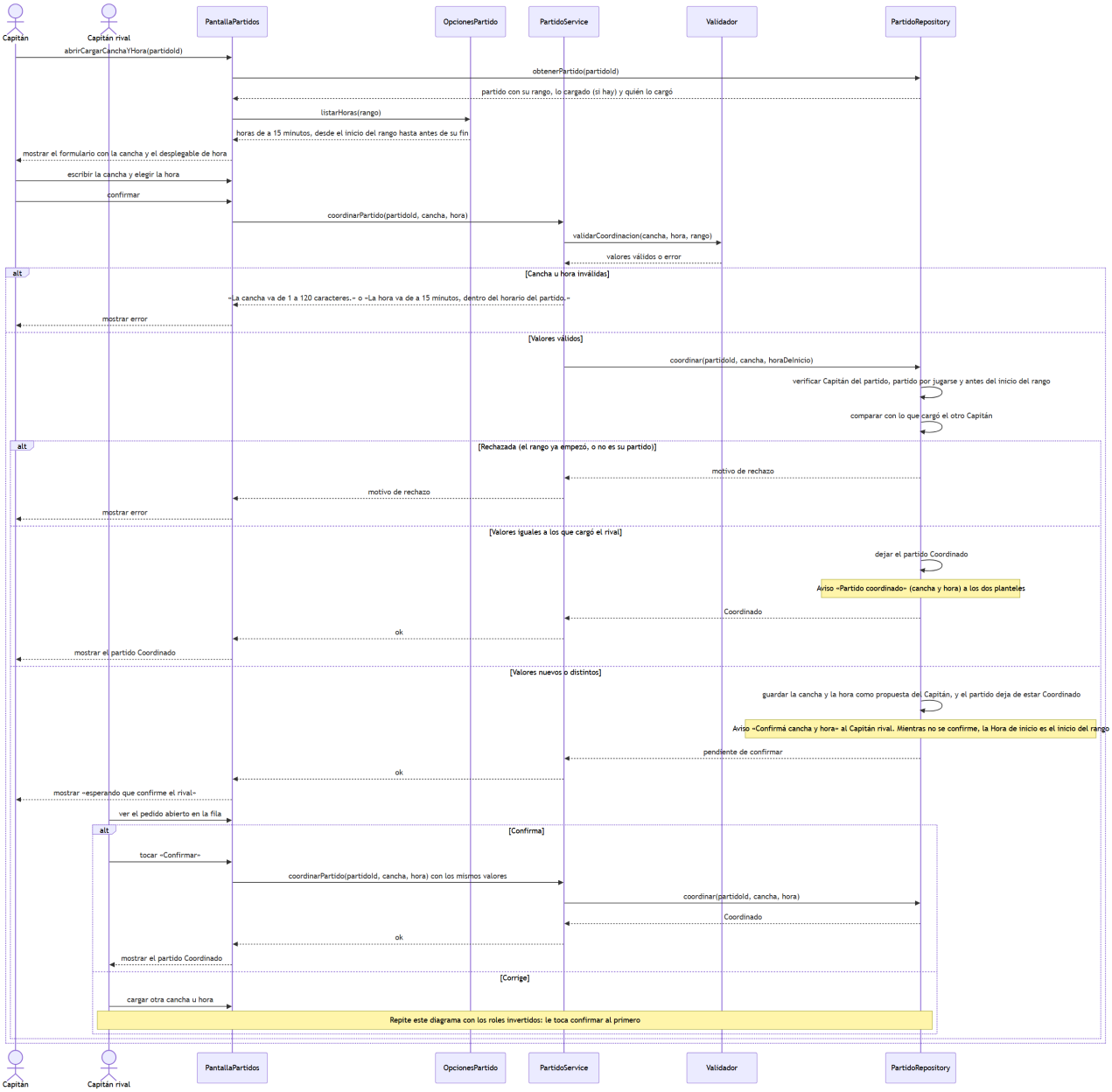
UC-D07 · Coordinar cancha y hora

- **Actor:** cualquiera de los dos Capitanes.
- **Requerimientos:** RF-D19, RF-D20. **Pantalla:** P1 · Partidos (el próximo partido y las filas). **Diagrama:** S07.
- **Precondiciones:** partido por jugarse y **antes del inicio del rango**.
- **Estado:** Implementado (`coordinate_match`).

Diagrama de secuencia · **S07 · UC-D07 — Coordinar cancha y hora**

Caso de uso: UC-D07 · Pantalla: P1 (el próximo partido y las filas) · Requerimientos: RF-D19, RF-D20

El formulario tiene **un desplegable: la hora** (de a 15 minutos) y un campo de texto, la **cancha**. Siguiendo la guía de formularios, la pantalla obtiene las opciones **antes** de que el Capitán elija. La hora **no sale de la base ni de una lista fija**: la calcula la app a partir del rango del partido (`OpcionesPartido`). Al enviar viajan solo los valores elegidos. La carga y la coordinación van en el mismo diagrama. Los Capitanes arreglan **por WhatsApp**; la app solo registra lo acordado.



Cómo se lee

- **Antes de elegir:** la pantalla trae el partido y calcula las horas posibles con `OpcionesPartido`. En un partido de **20 a 22**, las horas son de 20:00 a 21:45 (desde el inicio del rango incluido hasta el fin excluido).
- **Al enviar,** `coordinarPartido(...)` recibe solo la **cancha** y la **hora** elegidas. La cancha es texto libre (de 1 a 120 caracteres) y se muestra con «Ver en el mapa».

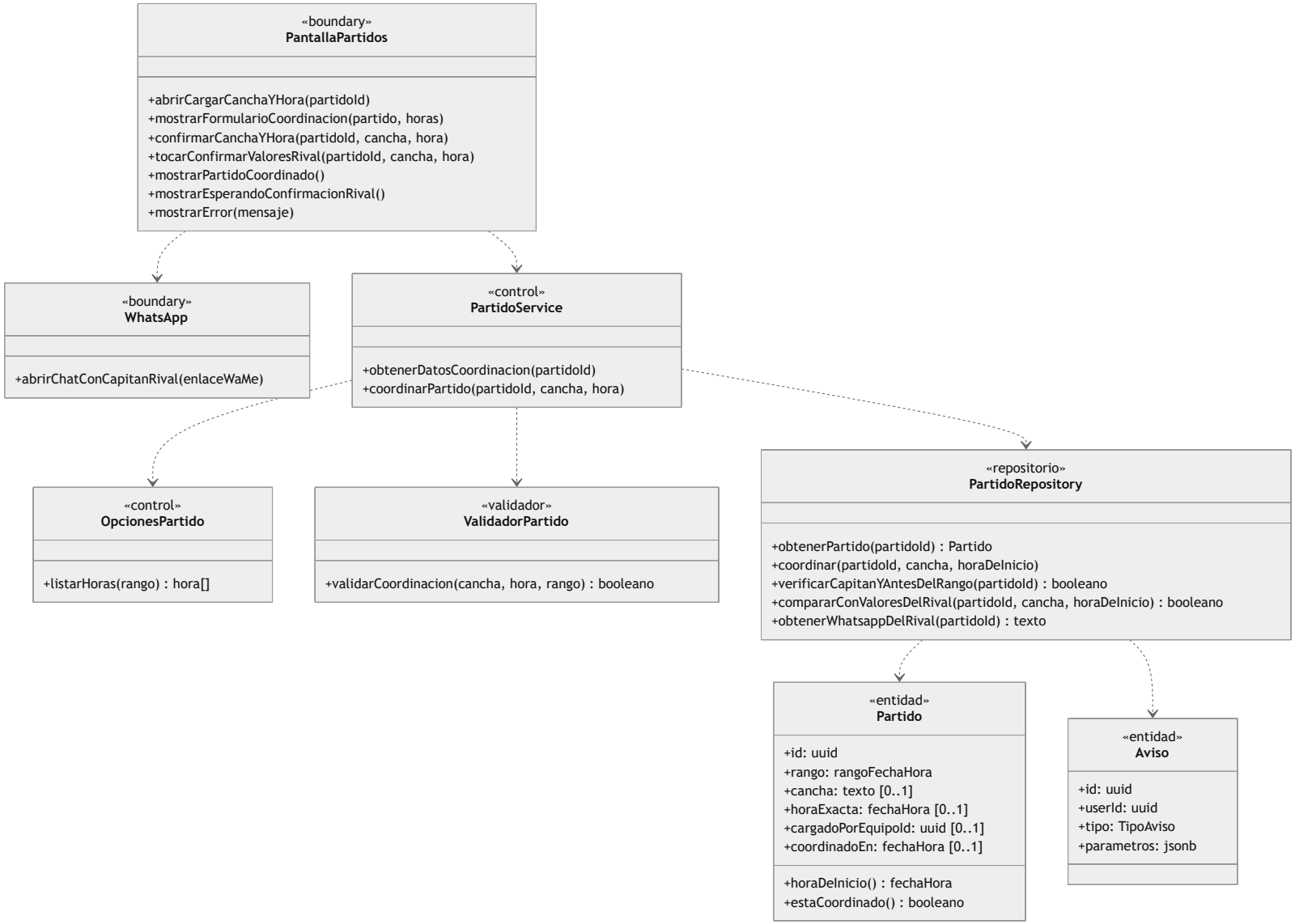
- **Cargar, confirmar y corregir son la misma operación.** Si el Capitán manda **los mismos valores** que cargó el otro, el partido queda **Coordinado**; si manda otros, es una corrección y le toca confirmar al primero.
- **Solo antes del inicio del rango.** Una vez que el rango empezó, no se carga, confirma ni corrige, **aunque la hora coordinada sea más tarde**: si no, corregir a las 20:30 un partido de 20 a 22 coordinado a las 21 lo des-coordinaría cuando ya habría empezado.
- **Cambiar después de Coordinado** es posible hasta el inicio del rango: el partido deja de estar Coordinado, le toca confirmar al otro y la Hora de inicio vuelve a ser el inicio del rango. Los «Vamos» ya dados no se borran. **Para cambiar de día,** se tira el partido y se propone de nuevo.
- **Coordinar es una ayuda, no un requisito:** un partido que llega a su hora sin coordinar sigue el flujo normal.

Correspondencia en el código

`coordinateMatch` (`src/app/partidos/actions.ts`): `parseCoordination` → RPC `coordinate_match(p_match_id, p_venue, p_start_at)`. Las horas salen de `coordinationTimes` (`src/utils/match-forms.ts`). Probado en `match_coordination_test.sql`.

Diagrama de clases · **UC-D07 · Coordinar cancha y hora**

Caso de uso: UC-D07 · **Pantalla:** P1 · Partidos (el próximo partido y las filas) · **Secuencia:** S07



> Pantalla · **P1 · Partidos**

Es la misma pantalla que se describe, con su captura, en P1 · Partidos, debajo de UC-D06 · Aceptar o rechazar una propuesta. Este caso usa la parte que indica su ficha («Pantalla»).

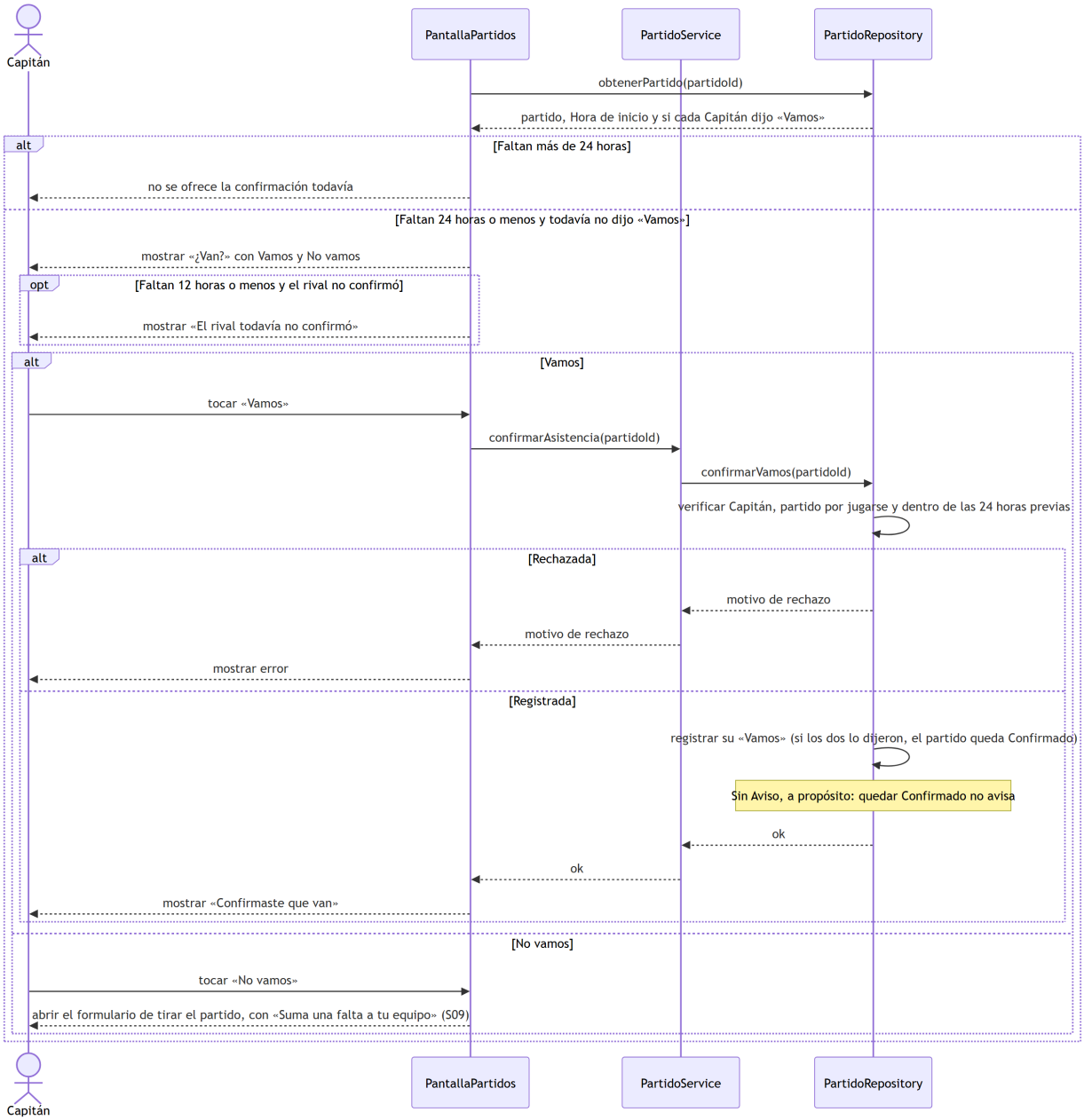
UC-D08 · Confirmar asistencia

- **Actor:** cada Capitán.
- **Requerimientos:** RF-D21. **Pantalla:** P1 · Partidos («¿Van?»). **Diagrama:** S08.
- **Precondiciones:** faltan 24 h o menos para la **Hora de inicio** y el partido está por jugarse.
- **Estado:** Implementado (`confirm_going`).

Diagrama de secuencia · **S08 · UC-D08 — Confirmar asistencia**

Caso de uso: UC-D08 · Pantalla: P1 («¿Van?») · Requerimientos: RF-D21

No hay formulario ni desplegables: son dos botones, **Vamos** y **No vamos**.



Cómo se lee

- **La ventana empieza 24 h antes de la Hora de inicio.** Antes, no se ofrece. Después, al entrar, cada Capitán que todavía no confirmó ve «¿Van?».
- **Confirmado no es una fase:** es un dato del partido por jugarse. Se alcanza cuando los dos dijeron «Vamos», y **no genera ningún Aviso**, a propósito.
- **Desde 12 h antes**, si el rival todavía no confirmó, la pantalla lo dice («El rival todavía no confirmó»); tampoco avisa: se ve al entrar.
- **Si nadie confirma, no pasa nada:** el partido sigue en pie. Confirmar es una ayuda, no un requisito.
- **«No vamos» no es otra función:** abre el formulario de tirar el partido (S09), que con menos de 24 h suma una falta. Por eso el formulario lo advierte.

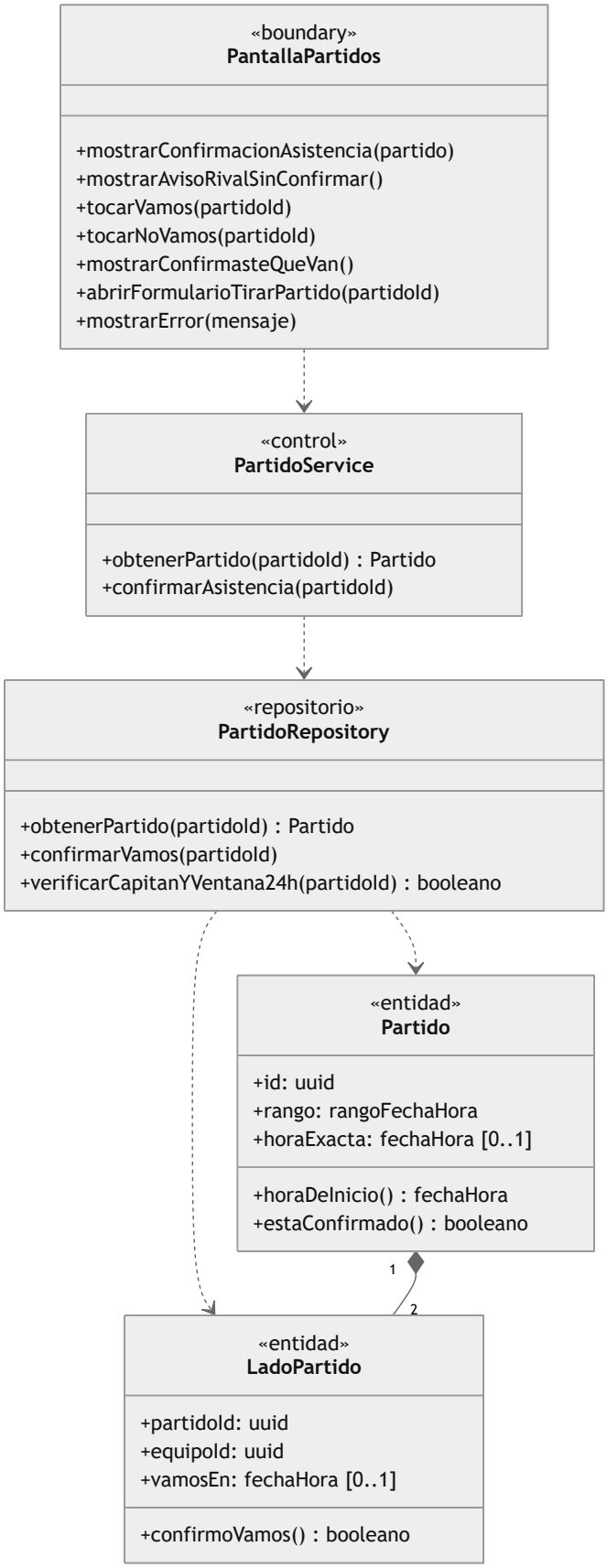
- **La Hora de inicio** es la hora exacta coordinada o, si no se coordinó, el inicio del rango.

Correspondencia en el código

`confirmGoing` (`src/app/partidos/actions.ts`) llama a la RPC `confirm_going(p_match_id)`. Probado en `match_coordination_test.sql`.

Diagrama de clases · **UC-D08 · Confirmar asistencia**

Caso de uso: UC-D08 · **Pantalla:** P1 · Partidos («¿Van?») · **Secuencia:** S08



> Pantalla · **P1 · Partidos**

Es la misma pantalla que se describe, con su captura, en P1 · Partidos, debajo de UC-D06 · Aceptar o rechazar una propuesta. Este caso usa la parte que indica su ficha («Pantalla»).

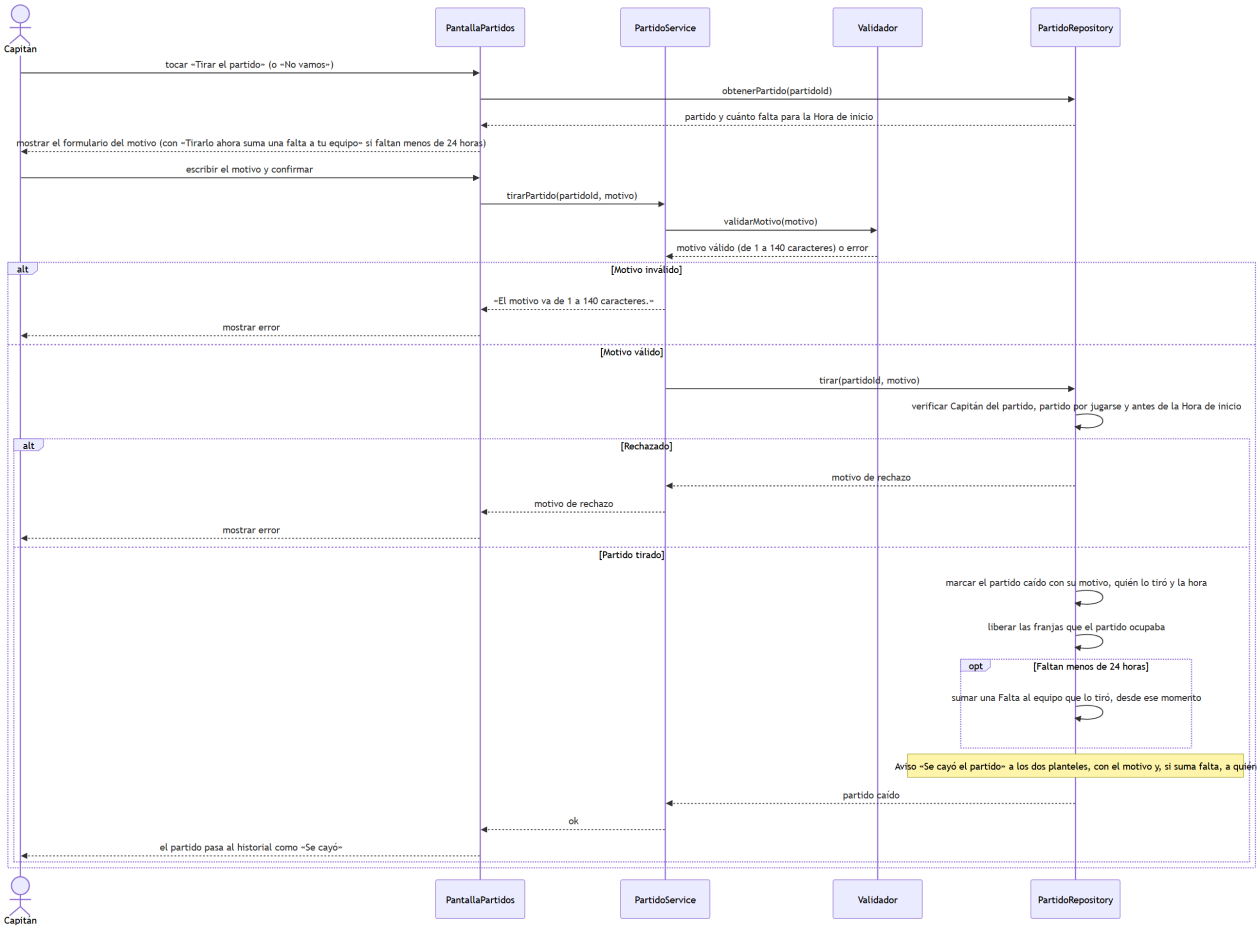
UC-D09 · Tirar el partido

- **Actor:** cualquiera de los dos Capitanes.
- **Requerimientos:** RF-D22, RF-D31. **Pantalla:** P1 · Partidos («Tirar el partido»). **Diagrama:** S09.
- **Precondiciones:** partido por jugarse, antes de la **Hora de inicio**.
- **Estado:** Implementado (`drop_match`).

Diagrama de secuencia · **S09 · UC-D09 — Tirar el partido**

Caso de uso: UC-D09 · Pantalla: P1 («Tirar el partido») · Requerimientos: RF-D22, RF-D31

El formulario tiene un único campo, el **motivo** (texto, sin desplegables). Si falta menos de 24 h, el formulario **advierte que suma una falta**.



Cómo se lee

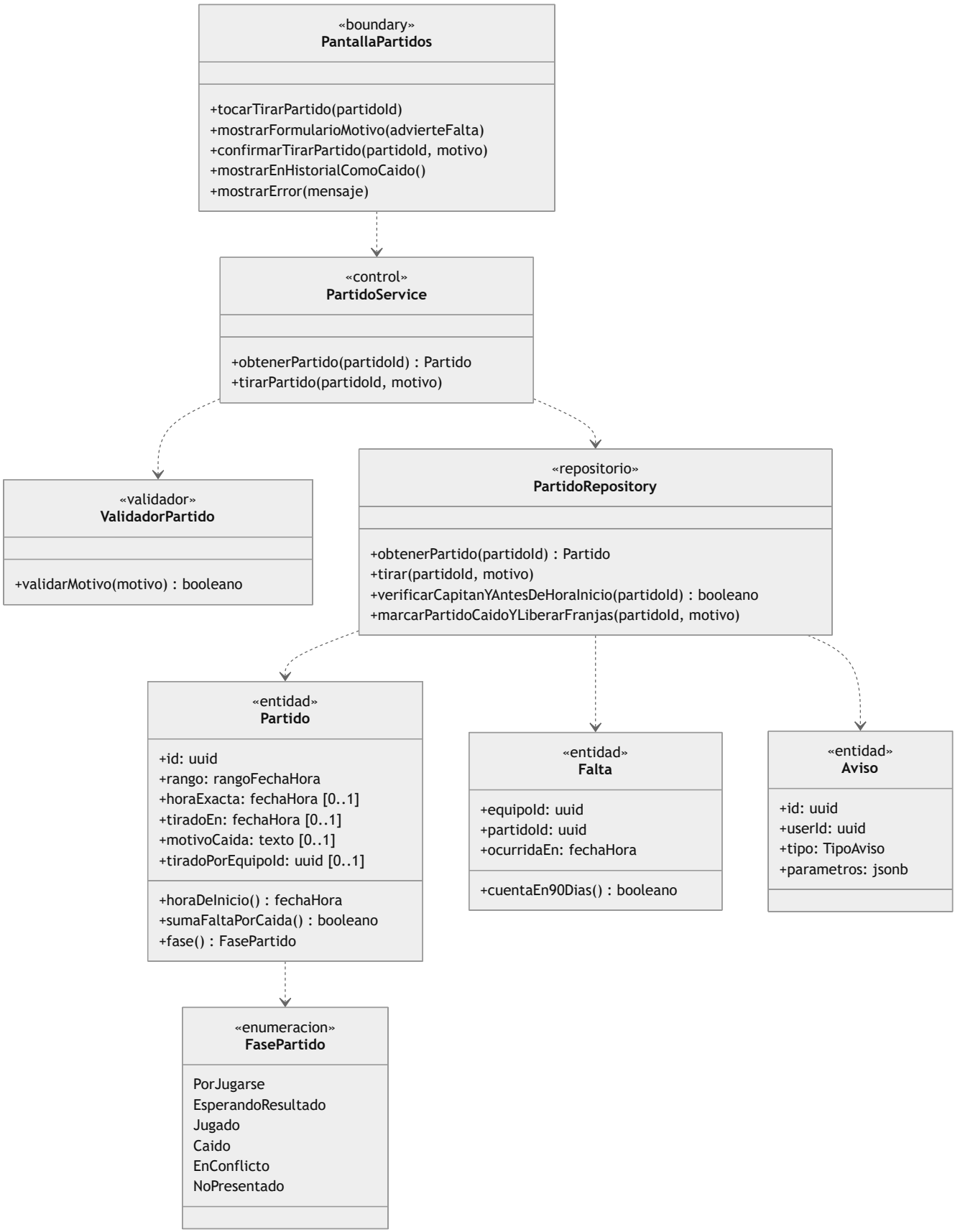
- **La falta la decide la fase, no el Capitán.** Con **menos de 24 h** para la Hora de inicio, tirar el partido suma una **Falta** al equipo que lo tiró, **desde el momento en que lo tiró**. Con **24 h justas**, todavía no. Con más de 24 h, no.
- **El motivo es obligatorio** (1 a 140 caracteres) y queda **visible en el historial público**; nadie lo modera (riesgo aceptado el 27/10/2026). «Equipo suspendido» lo escribe el sistema en el mismo campo, para los partidos que cae una Suspensión.
- **Qué pasa al caer:** el partido entra al historial del equipo **desde que se tira**, aunque su fecha no haya llegado; las franjas que ocupaba se liberan; y los dos planteles reciben el Aviso.
- **Con menos de 24 h hay una salida sin falta:** pedir la **caída de común acuerdo** (S10).
- **Solo antes de la Hora de inicio.** Después, el partido solo admite resultado o «no vinieron».
- **Las faltas cuentan 90 días** para el presentismo, y 3 faltas bloquean al equipo 30 días. El bloqueo no avisa.

Correspondencia en el código

`dropMatch` (`src/app/partidos/actions.ts`): `parseDropReason` → RPC `drop_match(p_match_id, p_reason)`. Las faltas salen de la vista `team_faults`. Probado en `match_drops_test.sql` y `team_faults_test.sql`.

Diagrama de clases · **UC-D09 · Tirar el partido**

Caso de uso: UC-D09 · **Pantalla:** P1 · Partidos («Tirar el partido») · **Secuencia:** S09



> Pantalla · **P1 · Partidos**

Es la misma pantalla que se describe, con su captura, en P1 · Partidos, debajo de UC-D06 · Aceptar o rechazar una propuesta. Este caso usa la parte que indica su ficha («Pantalla»).

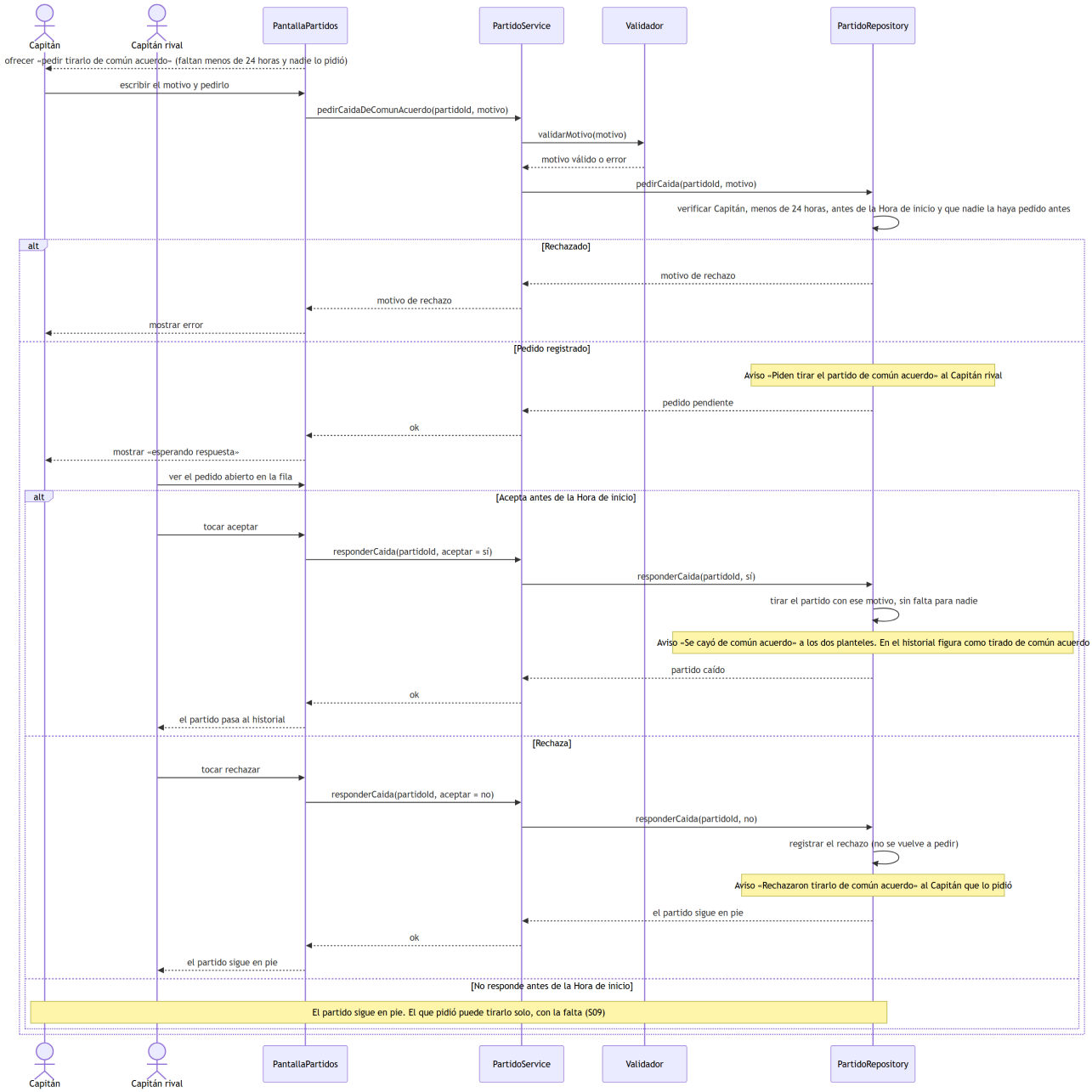
UC-D10 · Caída de común acuerdo

- **Actor:** el Capitán que la pide; el Capitán rival responde.
- **Requerimientos:** RF-D23, RF-D35. **Pantalla:** P1 · Partidos. **Diagrama:** S10.
- **Precondiciones:** faltan menos de 24 h (con más, tirarlo no suma falta y no se ofrece); nadie pidió una antes en este partido; antes de la hora de inicio.
- **Estado:** Implementado (`request_mutual_drop` , `answer_mutual_drop`).

Diagrama de secuencia · **S10 · UC-D10 — Caída de común acuerdo**

Caso de uso: UC-D10 · Pantalla: P1 · Requerimientos: RF-D23, RF-D35

El formulario es el del motivo (S09); no hay desplegables. El caso tiene dos actores: el Capitán que **pide** y el Capitán rival que **responde**.



Cómo se lee

- **Cuándo se ofrece:** solo con **menos de 24 h** para la Hora de inicio (con más, tirarlo no suma falta y no hace falta), antes de esa hora, y si **nadie pidió una antes** en este partido.
- **Si el rival acepta,** el partido se cae con ese motivo, **sin falta para nadie**, y queda en el historial como tirado de común acuerdo.
- **Si rechaza o no responde,** el partido sigue en pie y el que pidió todavía puede **tirarlo solo, con la falta** (S09). **No se vuelve a pedir:** ninguno de los dos.

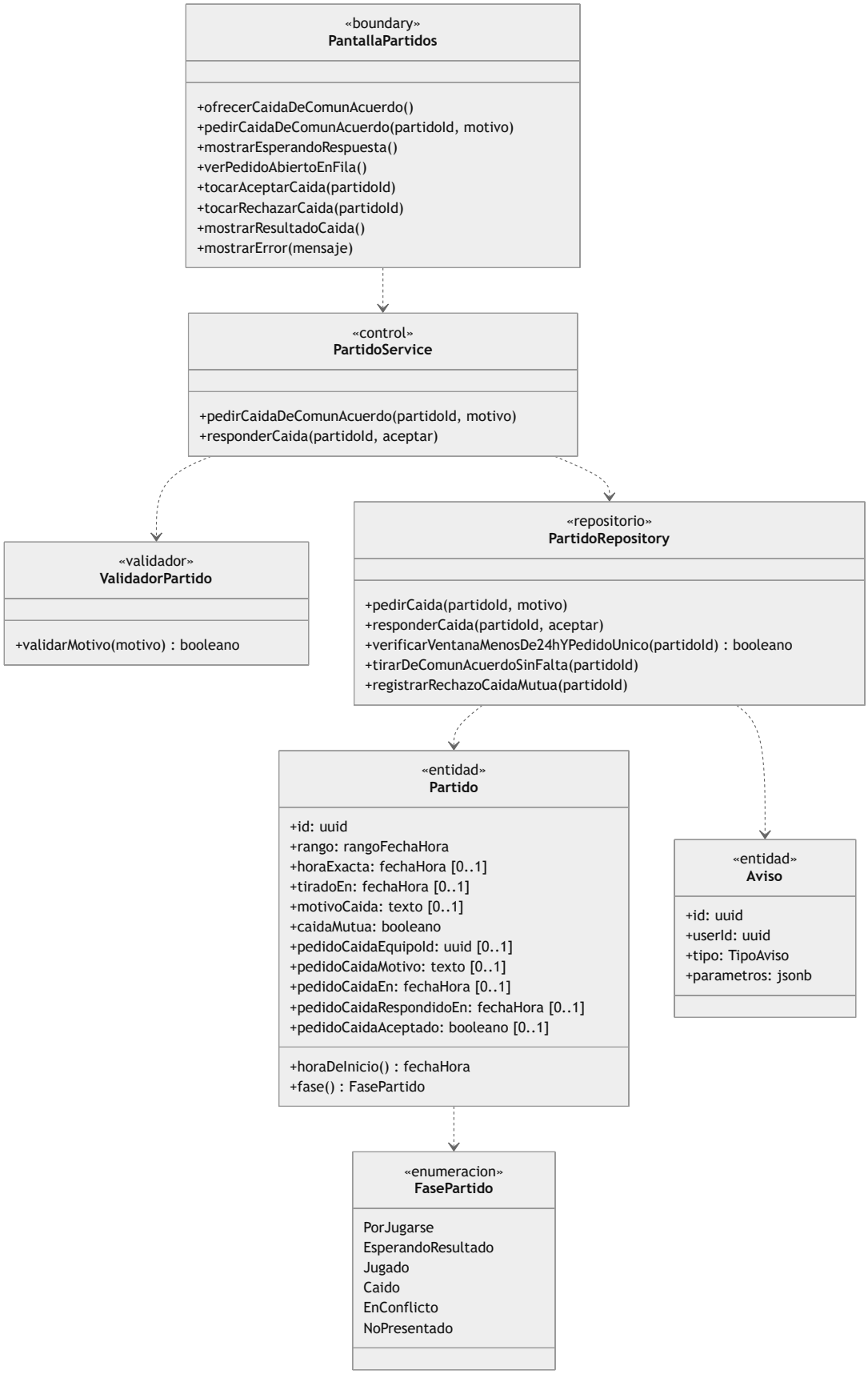
- **Un cambio de Capitán en el medio no lo traba:** el pedido es del equipo, y el Capitán nuevo responde el pedido pendiente (las búsquedas, propuestas y partidos son del equipo).
- **Los Avisos de este caso:** «Piden tirar el partido de común acuerdo» (al rival, pide acción), «Se cayó de común acuerdo» (a los dos planteles) y «Rechazaron tirarlo de común acuerdo» (al que pidió).

Correspondencia en el código

`requestMutualDrop` y `answerMutualDrop` (`src/app/partidos/actions.ts`) llaman a `request_mutual_drop(p_match_id, p_reason)` y `answer_mutual_drop(p_match_id, p_accept)`. Probado en `match_drops_test.sql`.

Diagrama de clases · **UC-D10 · Caída de común acuerdo**

Caso de uso: UC-D10 · **Pantalla:** P1 · Partidos · **Secuencia:** S10



> Pantalla · **P1 · Partidos**

Es la misma pantalla que se describe, con su captura, en P1 · Partidos, debajo de UC-D06 · Aceptar o rechazar una propuesta. Este caso usa la parte que indica su ficha («Pantalla»).

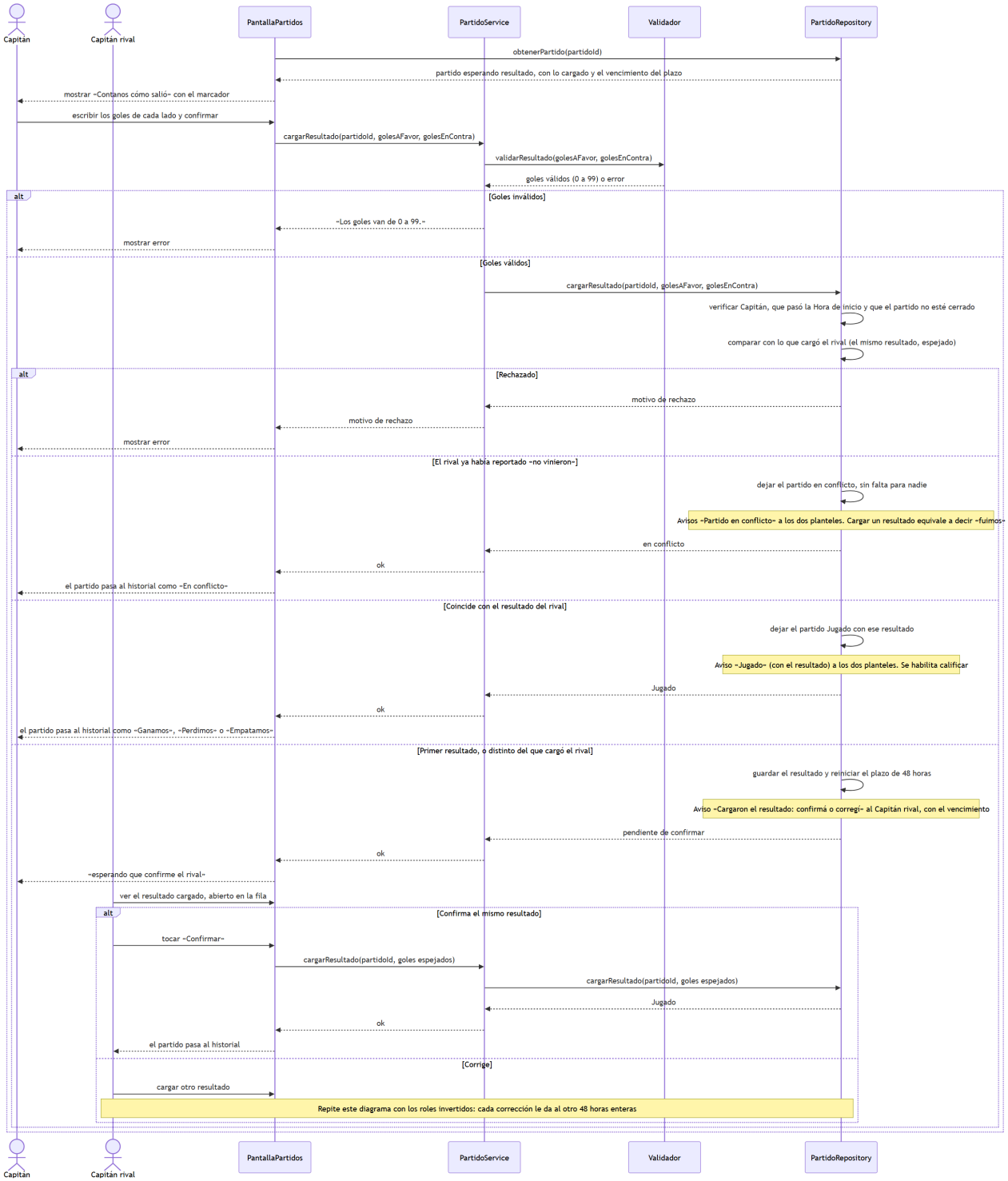
UC-D11 · Cargar el resultado

- **Actor:** cada Capitán.
- **Requerimientos:** RF-D24, RF-D25, RF-D27. **Pantalla:** P1 · Partidos («Contanos cómo salió»). **Diagrama:** S11.
- **Precondiciones:** pasó la **Hora de inicio**; el partido está **esperando resultado**.
- **Estado:** Implementado (`report_result`).

Diagrama de secuencia · **S11 · UC-D11 — Cargar el resultado**

Caso de uso: UC-D11 · Pantalla: P1 («Contanos cómo salió») · Requerimientos: RF-D24, RF-D25, RF-D27

El formulario es un **marcador** con dos goles (números de 0 a 99, el equipo propio a la izquierda): no hay desplegables. El caso tiene dos actores: quien **carga** y el Capitán rival, que **confirma o corrige**.



Cómo se lee

- **Una sola función para cargar, confirmar y corregir.** Si los goles coinciden (espejados) con los que cargó el otro, el partido queda **Jugado**; si no, es una carga o una corrección y le toca al otro.

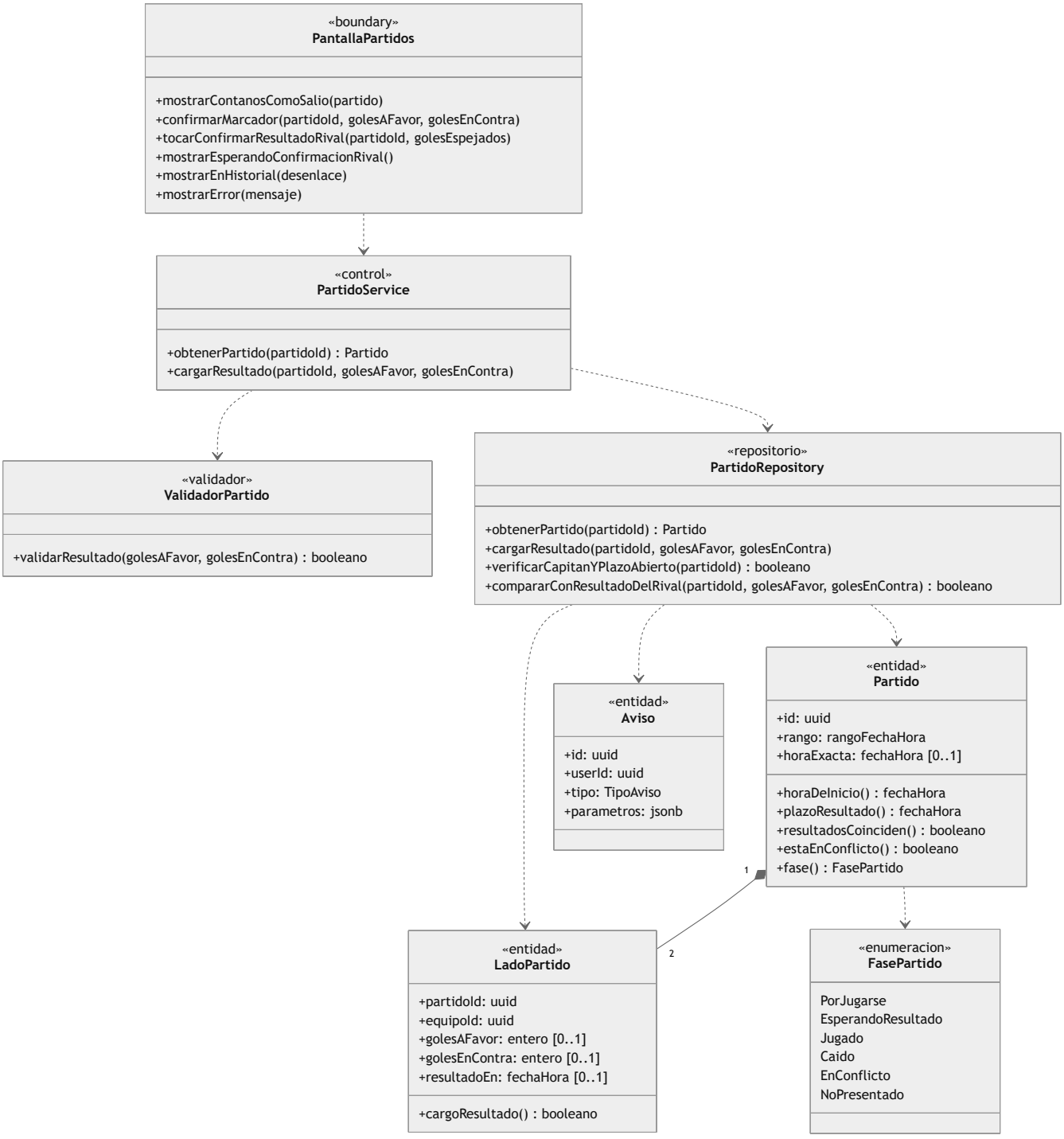
- **Los plazos de 48 h** se calculan **al leer**, contados desde la última carga de cualquiera de los dos (o, si nadie cargó, desde el fin del rango), y no avisan al vencer: si el rival no hace nada, el partido queda **Jugado con el resultado cargado**; con resultados distintos, cada corrección da 48 h enteras al otro, sin tope, y si siguen sin coincidir al vencer, **Jugado sin resultado**; si nadie carga, **Jugado sin resultado**, sin falta para nadie.
- **Cargar un resultado equivale a decir «fuimos».** Si el rival ya reportó «no vinieron» (S12), el partido queda **en conflicto**. Quien cargó un resultado ya no puede reportar «no vinieron».
- **Un partido cerrado no se corrige más:** Jugado, en conflicto o con falta.
- **Solo después de la Hora de inicio.** Antes, el partido solo admite tirarlo.
- **Calificar** se habilita al quedar Jugado (S13).

Correspondencia en el código

`reportResult` (`src/app/partidos/actions.ts`): `parseResult` → RPC `report_result(p_match_id, p_goals_for, p_goals_against)`. El marcador es el componente `Scoreboard`. Probado en `match_results_test.sql`.

Diagrama de clases · **UC-D11 · Cargar el resultado**

Caso de uso: UC-D11 · **Pantalla:** P1 · Partidos («Contanos cómo salió») · **Secuencia:** S11



Pantalla · **P1 · Partidos**

Es la misma pantalla que se describe, con su captura, en P1 · Partidos, debajo de UC-D06 · Aceptar o rechazar una propuesta. Este caso usa la parte que indica su ficha («Pantalla»).

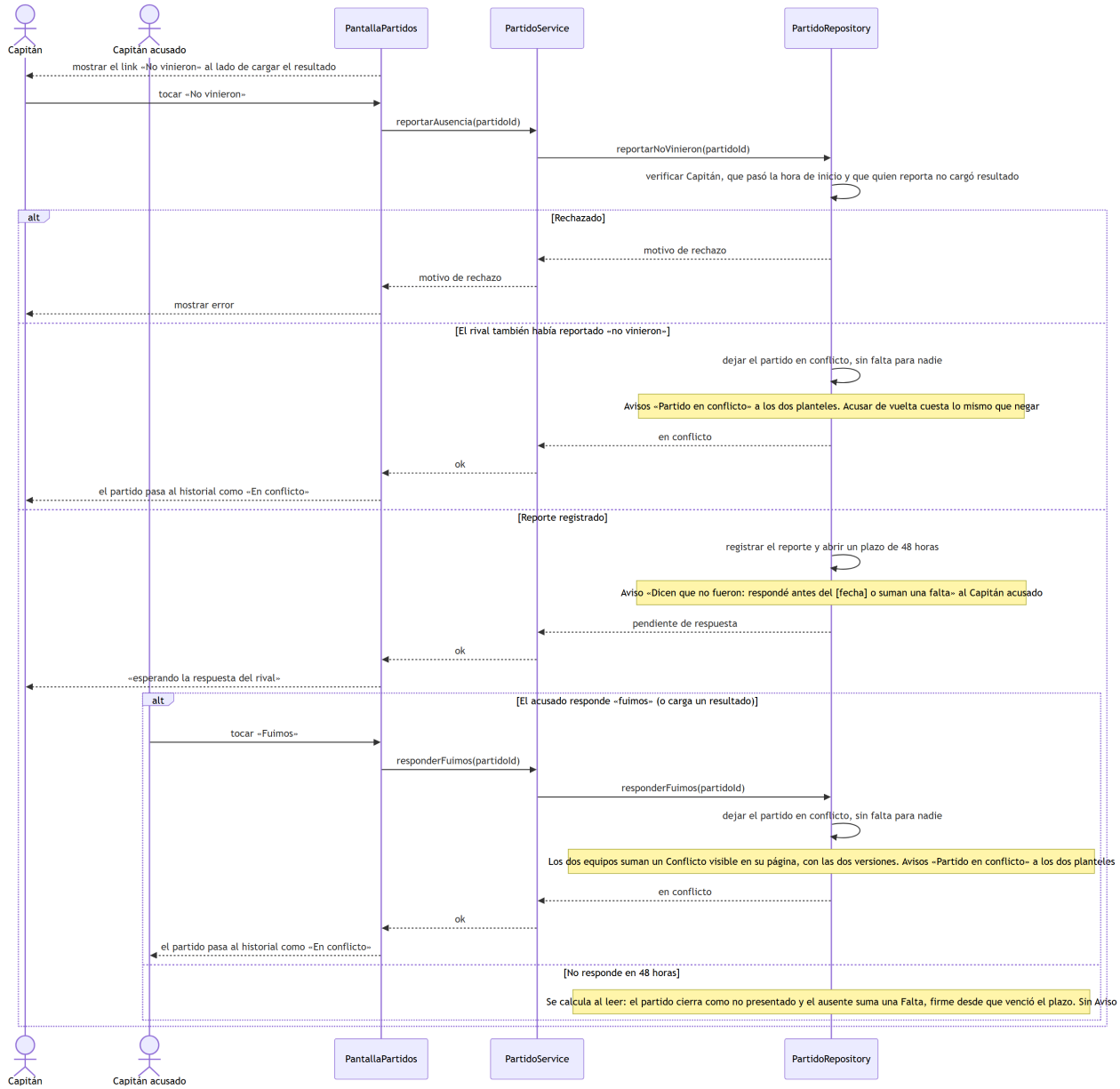
UC-D12 · Reportar que el rival no se presentó

- **Actor:** el Capitán que reporta; el Capitán acusado responde.
- **Requerimientos:** RF-D26, RF-D27, RF-D31, RF-D33. **Pantalla:** P1 · Partidos («No vinieron»). **Diagrama:** S12.
- **Precondiciones:** pasó la hora de inicio; el que reporta no cargó resultado.
- **Estado:** Implementado (`report_no_show` , `reply_went`).

Diagrama de secuencia · **S12 · UC-D12 — Reportar que el rival no se presentó**

Caso de uso: UC-D12 · Pantalla: P1 («No vinieron») · Requerimientos: RF-D26, RF-D27, RF-D31, RF-D33

No hay formulario ni desplegables: son botones. El caso tiene dos actores: el Capitán que **reporta** y el Capitán **acusado**, que responde. **Nadie arbitra.**



Cómo se lee

- **Cuándo se puede:** una vez pasada la hora de inicio, y solo si quien reporta **no cargó un resultado** (cargarlo equivale a decir «fuimos»). Quien reportó ya no carga resultado: solo corrige.
- **El plazo de 48 h** lo calcula la base al leer; vencerlo **no avisa**. Si el acusado no responde, el partido cierra como **no presentado** y el ausente suma una **Falta** (cuenta 90 días para el presentismo).
- **Conflicto:** si el acusado responde «fuimos» (o carga un resultado), o si **los dos** reportan «no vinieron» (en la pantalla, «Ellos tampoco vinieron»), el partido queda **en conflicto**: **sin falta para nadie**. Los dos equipos suman un **Conflicto**, que se muestra en su página con las dos versiones, pero **nunca penaliza**. Un partido en conflicto no se califica.

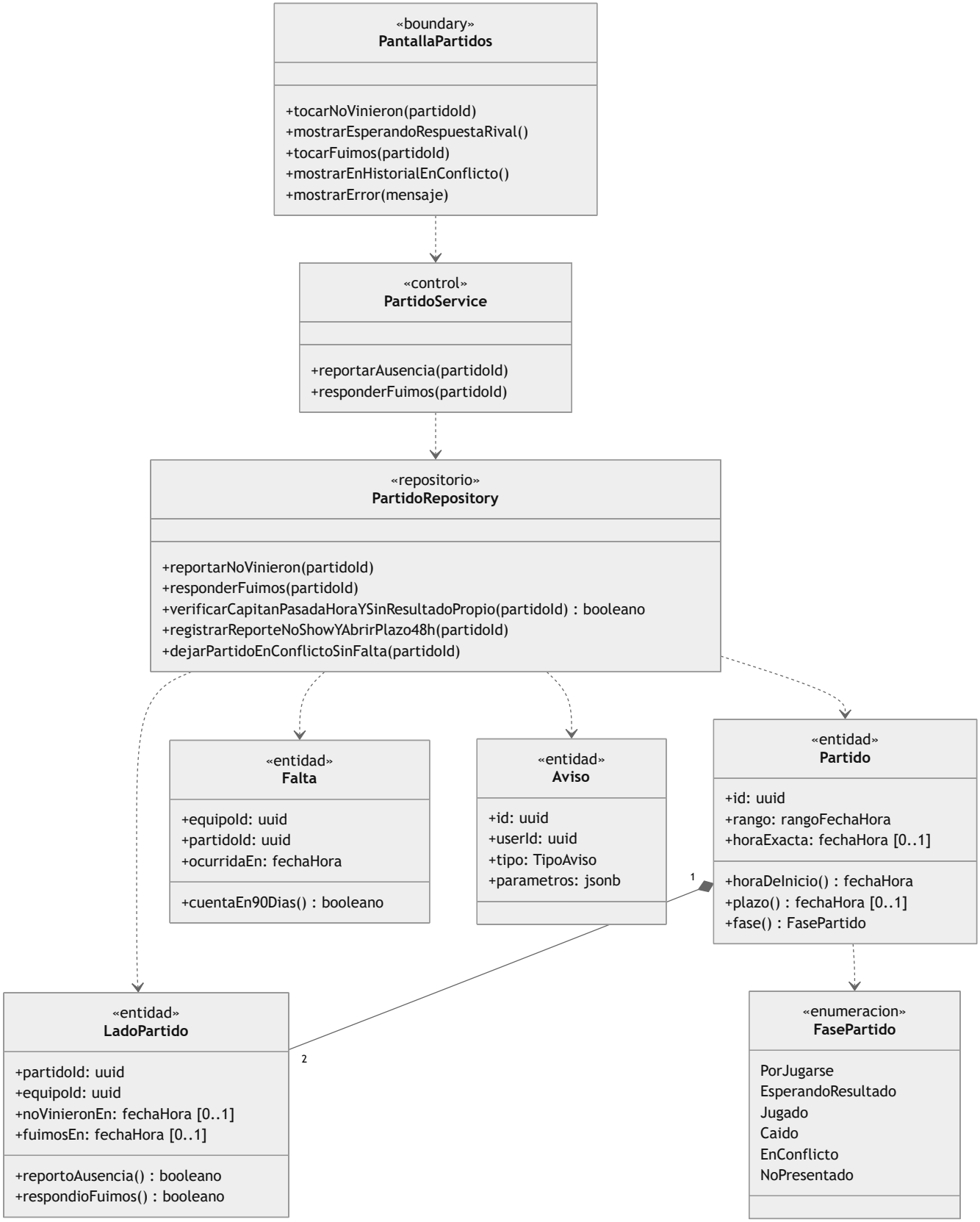
- **Nadie arbitra.** El administrador no interviene en partidos (ADR 0002): acusar de vuelta cuesta lo mismo que negar, y eso desalienta las acusaciones falsas.

Correspondencia en el código

`reportNoShow` y `replyWent` (`src/app/partidos/actions.ts`) llaman a `report_no_show(p_match_id)` y `reply_went(p_match_id)`. Probado en `match_results_test.sql`.

Diagrama de clases · **UC-D12 · Reportar que el rival no se presentó**

Caso de uso: UC-D12 · **Pantalla:** P1 · Partidos («No vinieron») · **Secuencia:** S12



> Pantalla · **P1 · Partidos**

Es la misma pantalla que se describe, con su captura, en P1 · Partidos, debajo de UC-D06 · Aceptar o rechazar una propuesta. Este caso usa la parte que indica su ficha («Pantalla»).

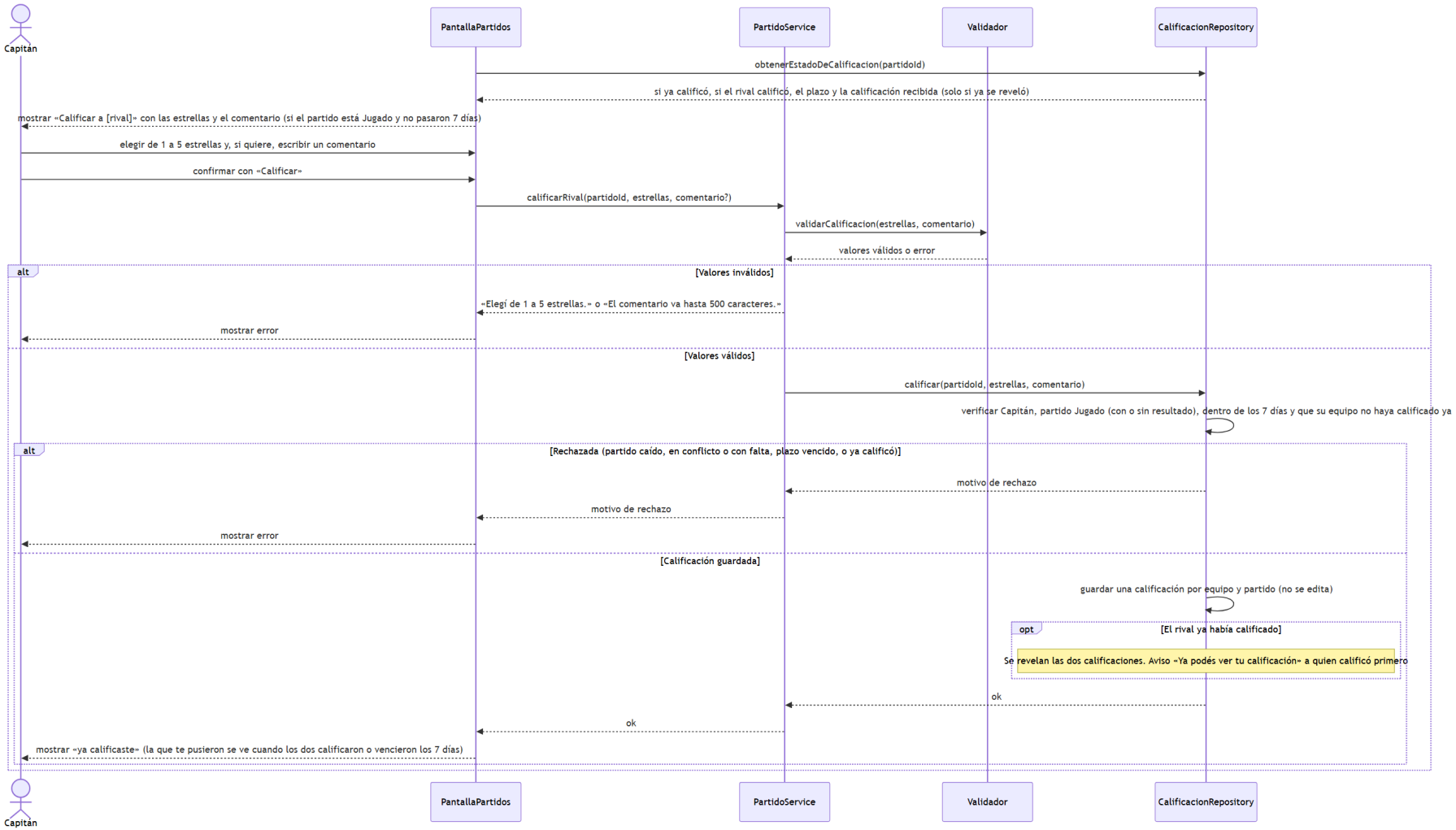
UC-D13 · Calificar al rival

- **Actor:** Capitán.
- **Requerimientos:** RF-D28, RF-D29. **Pantalla:** P1 · Partidos («Calificar a …»). **Diagrama:** S13.
- **Precondiciones:** el partido está **Jugado** (con o sin resultado) y no pasaron 7 días desde que quedó Jugado.
- **Estado:** Implementado (`rate_rival`).

Diagrama de secuencia · **S13 · UC-D13 — Calificar al rival**

Caso de uso: UC-D13 · Pantalla: P1 («Calificar a …») · Requerimientos: RF-D28, RF-D29

El formulario tiene **una elección entre cinco opciones** (de 1 a 5 estrellas) y un comentario de texto. Las estrellas son una **lista fija del código** (botones de opción), así que no hay repositorio que consultar. Al enviar viajan solo los valores elegidos.



Cómo se lee

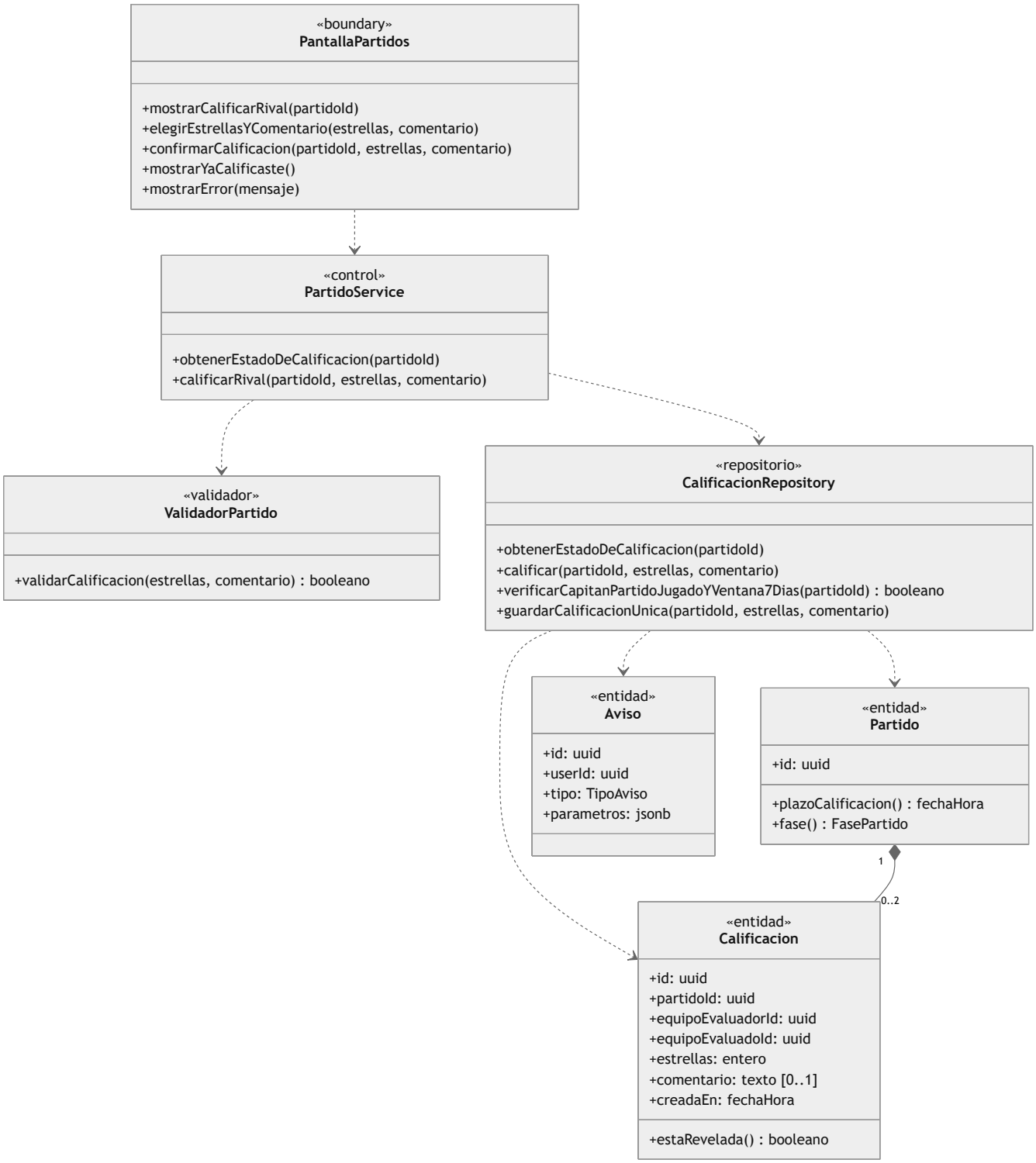
- **Cuándo se puede:** solo en un partido **Jugado** (con o sin resultado) y hasta **7 días** después de que quedó Jugado. Un partido caído, en conflicto o con falta **no se califica**. Un equipo **disuelto** se sigue calificando hasta que venzan los 7 días.
- **Una calificación por equipo y partido, y no se edita.** El Capitán califica al rival; 1 a 5 estrellas y un comentario opcional de hasta 500 caracteres.
- **El comentario es privado:** solo lo lee el administrador al revisar un equipo reportado; la tabla de calificaciones no se lee por la API.
- **Revelación cruzada, para que nadie califique en represalia.** Cada Capitán ve la calificación que le pusieron recién cuando **los dos calificaron** o **pasaron los 7 días**. Cuando la segunda calificación revela las dos, el que calificó primero recibe «Ya podés ver tu calificación»; revelar por vencimiento no avisa.
- **El promedio público** (las estrellas de la página del equipo) cuenta **solo las calificaciones reveladas**.

Correspondencia en el código

`rateRival` (`src/app/partidos/actions.ts`): `parseRating` → RPC `rate_rival(p_match_id, p_stars, p_comment)`. Probado en `ratings_test.sql`.

Diagrama de clases · **UC-D13 · Calificar al rival**

Caso de uso: UC-D13 · **Pantalla:** P1 · Partidos («Calificar a …») · **Secuencia:** S13



Pantalla · **P1 · Partidos**

Es la misma pantalla que se describe, con su captura, en P1 · Partidos, debajo de UC-D06 · Aceptar o rechazar una propuesta. Este caso usa la parte que indica su ficha («Pantalla»).

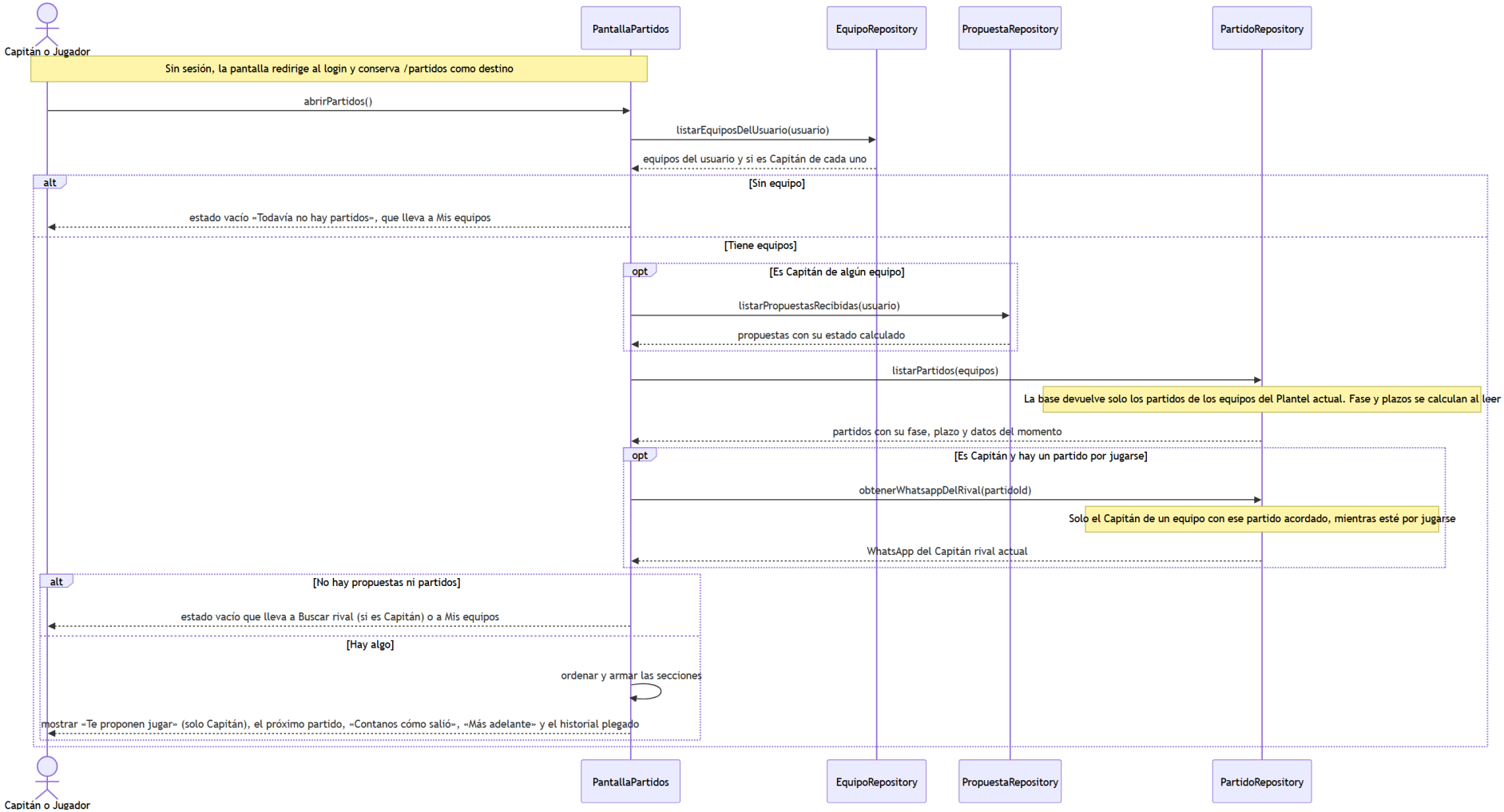
UC-D14 · Ver mis partidos

- **Actor:** Capitán o Jugador.
- **Requerimientos:** RF-D01, RF-D30. **Pantalla:** P1 · Partidos. **Diagrama:** S14.
- **Estado:** Implementado.

Diagrama de secuencia · **S14 · UC-D14 — Ver mis partidos**

Caso de uso: UC-D14 · Pantalla: P1 · Requerimientos: RF-D01, RF-D30

Es una consulta: no hay formulario ni desplegables. El actor es el Capitán o un Jugador del Plantel.



Cómo se lee

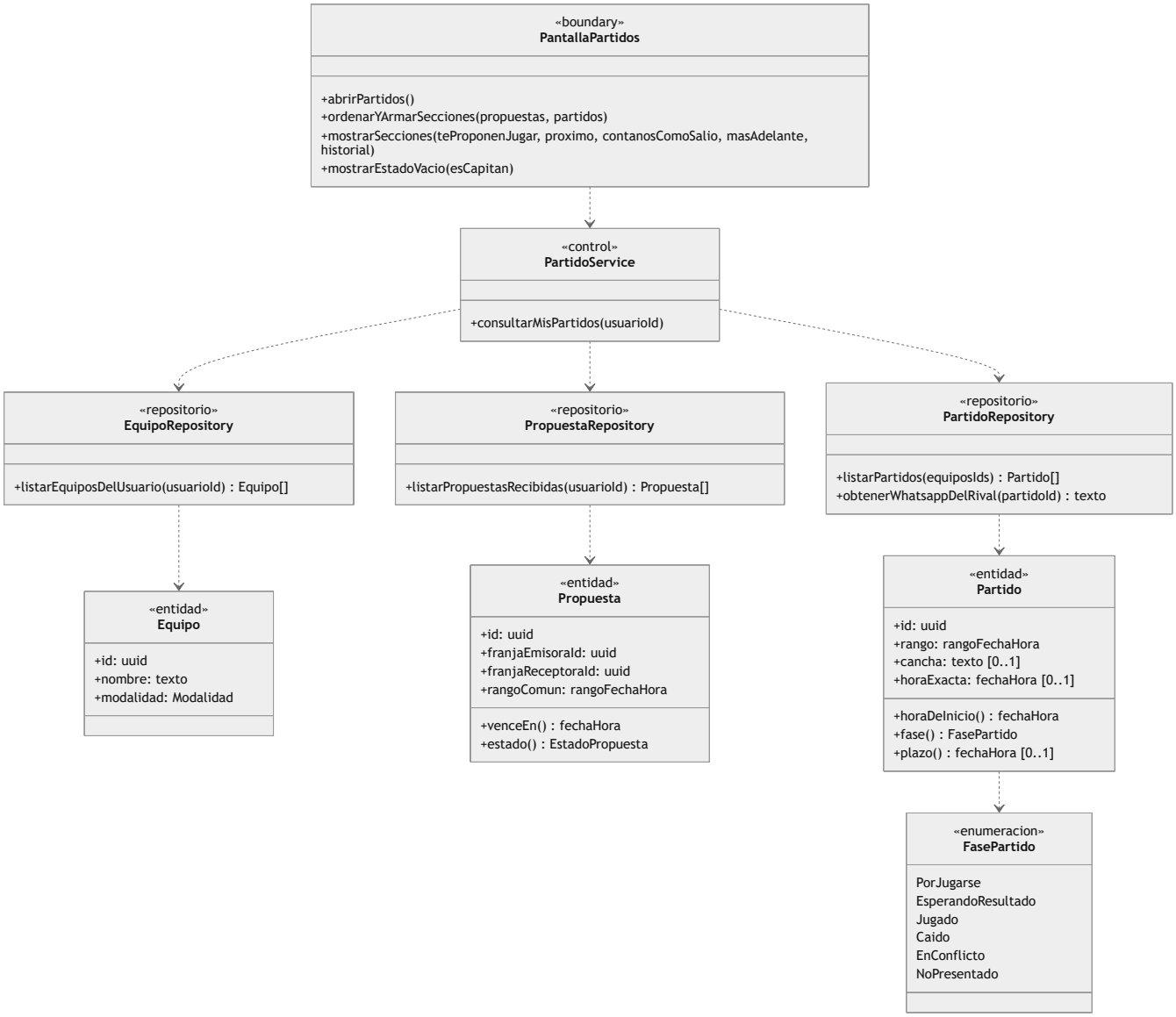
- **Es la fuente de verdad:** todo lo pendiente se ve acá, **haya o no Aviso**. El Aviso es un puntero; esta pantalla tiene el estado actual.
- **El Jugador ve todo, sin botones.** Las propuestas las ve solo el Capitán. El **WhatsApp del Capitán rival** lo ve solo el Capitán, con un partido acordado y mientras esté por jugarse: lo entrega una función de la base que verifica eso.
- **Visibilidad por Plantel actual:** quien sale del Plantel deja de ver los partidos del equipo; el Capitán anterior, si queda en el Plantel, sigue viendo los partidos pero deja de ver las propuestas.
- **Muestra los partidos de todos los equipos del usuario juntos;** el nombre del equipo propio aparece solo si el usuario está en más de uno.
- **Cada fila muestra lo que pide respuesta abierto, con un solo botón primario;** lo que no pide nada, plegado detrás de un link.
- **El historial** (los cerrados: «Ganamos», «Perdimos», «Empatamos», «Se cayó», «En conflicto», «Falta») está plegado y va en primera persona. El detalle público (quién lo tiró, las dos versiones) vive en la página del equipo, **sin cancha ni hora**.
- **La fase y los plazos se calculan al leer** con la hora actual; no hay tareas programadas.

Correspondencia en el código

`app/partidos/page.tsx` lee los partidos, las propuestas y los equipos, y arma las secciones con `utils/match-display.ts`; la fase la calcula la función `match_phase`. El WhatsApp sale de `rival_captain_whatsapp(p_match_id)`.

Diagrama de clases · **UC-D14 · Ver mis partidos**

Caso de uso: UC-D14 · **Pantalla:** P1 · Partidos · **Secuencia:** S14



Pantalla · **P1 · Partidos**

Es la misma pantalla que se describe, con su captura, en P1 · Partidos, debajo de UC-D06 · Aceptar o rechazar una propuesta. Este caso usa la parte que indica su ficha («Pantalla»).

Parte 3 · Anexos y trazabilidad

Guía de pantallas: estado de las capturas y mapa

Versión del **04/11/2026**. Cada pantalla de Partidos, con su ruta, los casos, los requerimientos, lo que muestra y la captura. Cada una va debajo del caso de uso que la usa por primera vez; si otro caso usa la misma pantalla, remite a esa descripción.

Estado de las capturas

Marca	Significa
✅ **Vigente**	Sacada de `main` con la UI actual.
⏳ **Pendiente**	Todavía no hay captura.

Hoy no hay ninguna captura de este módulo. Las dos pantallas exigen sesión de un Capitán o de un Jugador (sin ella, el control de acceso redirige a Iniciar sesión) y la base de desarrollo no respondía. Para sacarlas hacen falta las **cuentas QA** del equipo (ver diseno-partidos.md) y el script de escenarios `scripts/qa/partidos.ts` para armar los estados que dependen de plazos. Al final está la lista con los pasos.

Mapa de pantallas

Pantalla	Ruta	Casos de uso	Sesión
P1 · Partidos	`/partidos`	UC-D06 a UC-D14	Obligatoria
P2 · Buscar rival	`/partidos/buscar`	UC-D01 a UC-D05	Obligatoria (solo Capitanes con equipo)

Las dos son **pestañas** de un mismo módulo («Partidos» y «Buscar rival»), que son links: cada vista es una URL compartible. La pestaña «Partidos» muestra un punto lima cuando hay algo que responder. En el celular no hay título «Partidos» ni «Buscar rival»: las pestañas son el encabezado. **Las dos muestran lo de todos los equipos del usuario juntos**; el nombre del equipo propio aparece solo cuando el usuario está en más de uno.

Capturas pendientes

Convención: PNG en `docs/partidos/pantallas/`, ventana de unos 1280 px de ancho (y ~390 px para el celular, donde se piensa esta pantalla), con **datos de prueba**. Hacen falta las cuentas QA (Capitán de un equipo, un Jugador de ese equipo, un Capitán rival y un usuario ajeno) y `scripts/qa/partidos.ts` para armar los casos con plazos.

Archivo	Pantalla y estado	Cómo conseguirlo
`buscar-sin-busqueda.png`	P2 con el formulario de la primera franja	Un Capitán sin búsqueda activa.
`buscar-buscando.png`	P2 «Buscando rival» con franjas	Con una búsqueda activa y dos horarios.
`buscar-compatibles.png`	P2 con rivales compatibles	Con un rival (por ejemplo, «Los Rivales de Prueba») que publicó un horario que se pisa.
`buscar-propuesta-enviada.png`	P2 con «Propuesta enviada»	Después de proponer.
`buscar-bloqueado.png`	P2 con el equipo bloqueado	Con el escenario `bloqueado`.

Archivo	Pantalla y estado	Cómo conseguirlo
`partidos-propuesta-recibida.png`	P1 con «Te proponen jugar»	El Capitán rival con una propuesta pendiente.
`partidos-proximo.png`	P1 con el próximo partido	Un partido acordado, sin coordinar y coordinado.
`partidos-van.png`	P1 con «¿Van?»	Un partido a menos de 24 h.
`partidos-resultado.png`	P1 con «Contanos cómo salió»	Un partido que ya empezó.
`partidos-historial.png`	P1 con el historial desplegado	Con el escenario `historial`.
`partidos-jugador.png`	P1 vista del Jugador, sin botones	La cuenta de un Jugador del plantel.
`partidos-vacio.png`	P1 vacío	Una cuenta sin equipo.

Diagramas de secuencia: criterios, participantes y convenciones

Versión del **04/11/2026**. Un diagrama por caso de uso, en Mermaid (GitHub los dibuja directo). Cada uno va debajo de su caso de uso, antes de su pantalla.

Diagrama	Caso de uso	Pantalla	Estado
S01 · Publicar una búsqueda	UC-D01	P2	Parcial (Suspensión)
S02 · Editar o cancelar la búsqueda	UC-D02	P2	Implementado
S03 · Ver rivales compatibles	UC-D03	P2	Implementado
S04 · Proponer un partido	UC-D04	P2	Implementado
S05 · Cancelar una propuesta	UC-D05	P2	Implementado
S06 · Aceptar o rechazar una propuesta	UC-D06	P1	Implementado
S07 · Coordinar cancha y hora	UC-D07	P1	Implementado
S08 · Confirmar asistencia	UC-D08	P1	Implementado
S09 · Tirar el partido	UC-D09	P1	Implementado
S10 · Caída de común acuerdo	UC-D10	P1	Implementado
S11 · Cargar el resultado	UC-D11	P1	Implementado
S12 · Reportar que no se presentó	UC-D12	P1	Implementado
S13 · Calificar al rival	UC-D13	P1	Implementado
S14 · Ver mis partidos	UC-D14	P1	Implementado

Los PNG están dibujados sobre fondo blanco, para pegarlos en un informe. La fuente de cada diagrama es el bloque Mermaid de su archivo: si se cambia uno, hay que regenerar su PNG.

Relación con los diagramas generales. `docs/diagramas-de-secuencia/diagramas-secuencia.md` cubre solo ocho casos de Partidos (D12 a D19, que corresponden a S01, S03, S04, S06, S05, S07, S08 y S09), con participantes genéricos. Los de este módulo los **detallan** y suman los que faltaban: editar o cancelar la búsqueda, caída de común acuerdo, resultado, «no vinieron», calificar y mis partidos. Los generales no muestran la carga de las opciones de los formularios.

Criterios aplicados

Son los de la guía para diagramas con formularios y desplegables.

1. **Un caso de uso por diagrama.** La carga de las opciones de un formulario y la acción que sigue van **en el mismo diagrama**.
2. **Si el caso empieza con un formulario con opciones**, el diagrama muestra a la pantalla obteniendo las opciones **antes** de que el Capitán elija. Pasa en **publicar la búsqueda** (S01: equipo, modalidad y turno) y en **coordinar cancha y hora** (S07: la hora).
3. **Un objeto por responsabilidad o fuente de datos.** El equipo se consulta en la base (`EquipoRepository`); la modalidad y el turno son listas fijas del código (`OpcionesBusqueda`); la hora de a 15 minutos la calcula la app a partir del rango del partido (`OpcionesPartido`). No se inventan repositorios para listas que no se guardan.
4. **Al enviar**, el método del caso de uso recibe **solo los valores elegidos** (`equipoId`, `modalidad`, `franjas`; `cancha`, `hora`), no las listas.
5. **La validación se repite al guardar, y la que manda es la de la base.** TypeScript valida para dar un mensaje claro; **cada función de Postgres repite todas las reglas**. Los ids que llegan del cliente no son de confiar.
6. **Cada diagrama muestra solo los participantes que intervienen.**

Participantes: responsabilidad y correspondencia en el repo

Son responsabilidades conceptuales. No existen como clases TypeScript con esos nombres: la app usa Server Actions, utilidades puras y funciones de Postgres.

Participante	Responsabilidad	Correspondencia en el repo
Capitán, CapitanRival	La persona que inicia el caso y, cuando hace falta, su contraparte.	La sesión y la marca de Capitán en `team_members`.
PantallaBuscarRival	La pantalla `/partidos/buscar`.	`app/partidos/buscar/page.tsx`, con `SlotPicker.tsx`.
PantallaPartidos	La pantalla `/partidos`, con sus filas y formularios.	`app/partidos/page.tsx` y `components/partidos/` (`MatchRow`, `NextMatch`, `MatchActions`, `Scoreboard`).
OpcionesBusqueda	Entrega las modalidades y los turnos: **listas fijas del código**.	`SPORT_MODALITIES` (`utils/modality.ts`) y los turnos de `SlotPicker.tsx`.
OpcionesPartido	Entrega las horas posibles de un partido: de a 15 min dentro de su rango, **calculadas por la app**.	`coordinationTimes` (`utils/match-forms.ts`).
BusquedaService	Coordina la búsqueda y las propuestas.	Las Server Actions `saveRivalSearch`, `sendProposal`, `declineProposal` y `acceptProposal` (`app/partidos/actions.ts`).
PartidoService	Coordina las acciones sobre un partido.	Las Server Actions `coordinateMatch`, `confirmGoing`, `dropMatch`, `requestMutualDrop`, `answerMutualDrop`, `reportResult`, `reportNoShow`, `replyWent` y `rateRival`.
Validador	Valida entradas antes de actuar (la base las repite).	`parseRivalSearchForm` (`utils/rival-search-form.ts`) y `parseCoordination`, `parseDropReason`, `parseResult`, `parseRating` (`utils/match-forms.ts`).
EquipoRepository	Entrega los equipos que capitanea el usuario.	Tablas `teams` y `team_members`.
BusquedaRepository	Lee y escribe búsquedas y franjas, y entrega los compatibles.	`rival_searches`, `search_slots`, `save_rival_search`, `compatible_rivals`.

Participante	Responsabilidad	Correspondencia en el repo
PropuestaRepository	Lee y escribe propuestas.	`proposals`, `send_proposal`, `respond_proposal`, `cancel_proposals`.
PartidoRepository	Lee y escribe partidos y sus lados.	`matches`, `match_sides` y las funciones `coordinate_match`, `confirm_going`, `drop_match`, `request_mutual_drop`, `answer_mutual_drop`, `report_result`, `report_no_show`, `reply_went`; fases y plazos con `match_phase` y `match_deadline`.
CalificacionRepository	Lee y escribe calificaciones.	`ratings` y `rate_rival`.

Convenciones de dibujo

- Flecha continua (`->>`): una llamada. Flecha punteada (`-->>`): su respuesta.
- Un mensaje de un participante a sí mismo es un paso interno (por ejemplo, una verificación dentro de una función de Postgres).
- Una nota sobre un participante marca una regla que **hace cumplir la base** o un paso que genera un **Aviso**.
- Entre paréntesis y en minúscula van los **valores que viajan**; entre comillas «así», los mensajes que ve la persona.
- `opt` es un paso opcional; `alt` / `else`, caminos que se excluyen.

Trazabilidad y pendientes

Documentación completa del módulo D (Partidos: buscar rival) de Torneos Amateur BA, versión del **04/11/2026**. En este documento, sus piezas son la Parte 1 (requerimientos), la Parte 2 (cada caso de uso con su diagrama y su pantalla) y los anexos de esta parte.

Fuentes, en orden de autoridad: decisiones-casos-de-uso.md → diseno-partidos.md → ADR 0002 → casos-de-uso.md → CONTEXT.md → código y migraciones.

> **No confundir los prefijos.** Los `RF-D##` de este módulo son los del módulo D (Partidos). La especificación vieja de Equipos (28/10/2026) usaba `RF-D##` para los suyos cuando llamaba «módulo D» a Equipos; esos hoy son los `RF-C##` de Equipos.

Matriz de trazabilidad

Caso de uso	Requerimientos	Pantalla	Diagrama	Estado
UC-D01 Publicar una búsqueda	RF-D02 a D06, D32	P2 `/partidos/buscar`	S01	**Parcial** (Suspensión)
UC-D02 Editar o cancelar la búsqueda	RF-D07, D08	P2	S02	Implementado
UC-D03 Ver rivales compatibles	RF-D09, D10, D31	P2	S03	Implementado
UC-D04 Proponer un partido	RF-D11, D12	P2	S04	Implementado
UC-D05 Cancelar una propuesta	RF-D13	P2	S05	Implementado
UC-D06 Aceptar o rechazar una propuesta	RF-D14 a D18	P1 `/partidos`	S06	Implementado
UC-D07 Coordinar cancha y hora	RF-D19, D20	P1	S07	Implementado
UC-D08 Confirmar asistencia	RF-D21	P1	S08	Implementado
UC-D09 Tirar el partido	RF-D22, D31	P1	S09	Implementado

Caso de uso	Requerimientos	Pantalla	Diagrama	Estado
UC-D10 Caída de común acuerdo	RF-D23, D35	P1	S10	Implementado
UC-D11 Cargar el resultado	RF-D24, D25, D27	P1	S11	Implementado
UC-D12 Reportar que no se presentó	RF-D26, D27, D31, D33	P1	S12	Implementado
UC-D13 Calificar al rival	RF-D28, D29	P1	S13	Implementado
UC-D14 Ver mis partidos	RF-D01, D30	P1	S14	Implementado

Equivalencia con el PDF original

Vigente	PDF original
UC-D01 Publicar una búsqueda	UC-13
UC-D03 Ver rivales compatibles	UC-14
UC-D04 Proponer un partido	UC-15
UC-D06 Aceptar o rechazar una propuesta	UC-16
UC-D05 Cancelar una propuesta	UC-17
UC-D07 Coordinar cancha y hora	UC-18
UC-D08 Confirmar asistencia	UC-19
UC-D09 Tirar el partido	UC-20
UC-D02, D10 a D14	Sin correspondencia exacta en los documentos del repo (los diagramas generales dejan UC-21 a UC-24 «para otra tanda»)

Hallazgos y pendientes

El módulo **está casi completo**: los 14 casos están implementados. Las observaciones son pocas:

1. **«Copiar horarios de la última búsqueda» no existe.** `decisiones-casos-de-uso.md` dice que reemplaza a la grilla de disponibilidad eliminada, pero no hay nada en el código y `casos-de-uso.md` no lo menciona. Hay que decidir si se programa o si se quita esa frase de las decisiones.
2. **La Suspensión no se verifica** en `save_rival_search` ni en las demás funciones de Partidos (UC-D01 la pide como precondición). Depende de UC-F05.
3. **La calificación no tiene lector.** Los comentarios de calificación los debe leer el administrador al revisar un equipo reportado (UC-F04), pero la tabla `ratings` no la lee nadie por la API. Falta definir cómo accede el backoffice.
4. **El motivo de tirar un partido no se modera** y queda visible en el historial público (riesgo aceptado el 27/10/2026).
5. **Los diagramas generales** (D12 a D19) cubren 8 de los 14 casos y no muestran la carga de las opciones del formulario; los de este módulo sí, y suman los casos que faltaban.
6. **Capturas: no hay ninguna.** Las dos pantallas exigen sesión (y la base de desarrollo no respondía). Para sacarlas hacen falta las cuentas QA y el script de escenarios `scripts/qa/partidos.ts` (que arma los casos con plazos); la lista, con los pasos, está en 03-pantallas.md.
7. **Tests:** es el módulo mejor probado: pgTAP para cada frontera de fase y plazo, cada función y cada policy o vista (`rival_searches` , `proposals` , `matches` , `match_coordination` , `match_drops` , `match_results` ,

`ratings`, `team_faults`, `team_history`, `compatible_rivals`, `notices`), y pruebas unitarias de la validación y el texto.

8. **Si se cambia un diagrama,** hay que regenerar su PNG (el bloque Mermaid de cada archivo es la fuente).

Diagramas de clases: trazabilidad de las clases

Clase	Estereotipo	Implementación en el repo	Estado
`PantallaBuscarRival` / `PantallaPartidos`	`«boundary»`	`src/app/partidos/buscar/page.tsx`, `src/app/partidos/page.tsx` y formularios de acción en `src/components/matches/`	Implementadas
`WhatsApp`	`«boundary»`	`whatsappLink` (`src/utils/whatsapp.ts`)	Implementado
`BusquedaService` / `PartidoService` / `OpcionesBusqueda` / `OpcionesPartido`	`«control»`	Server Actions en `src/app/partidos/actions.ts` y `src/app/partidos/buscar/actions.ts`, `SPORT_MODALITIES` (`src/utils/modality.ts`), `TIME_SLOTS` y horas de inicio (`src/utils/matches.ts`)	Implementados
`ValidadorPartido`	`«validador»`	`parseSearchSlots`, `parseProposalTarget`, `parseCoordinateForm`, `parseDropReason`, `parseResultForm`, `parseRatingForm` (`src/utils/matches.ts`)	Implementado
`EquipoRepository`	`«repositorio»`	Consultas de pertenencia y capitanías en `public.teams` y `public.team_members`	Implementado
`BusquedaRepository` / `PropuestaRepository`	`«repositorio»`	Tablas `public.rival_searches`, `search_slots`, `proposals`; funciones `save_rival_search`, `compatible_rivals`, `send_proposal`, `respond_proposal`, `proposal_status` (`20261028000003..06`)	Implementados
`PartidoRepository`	`«repositorio»`	Tablas `public.matches`, `match_sides`, `team_faults`; funciones `coordinate_match`, `confirm_going`, `drop_match`, `request_mutual_drop`, `answer_mutual_drop`, `report_result`, `report_no_show`, `reply_went`, `match_phase`, `match_deadline`, `rival_captain_whatsapp` (`20261028000006..09`)	Implementado
`CalificacionRepository`	`«repositorio»`	Tabla `public.ratings`; funciones `rate_rival`, `my_match_rating_status` (`20261028000009_match_results.sql`)	Implementado
`BusquedaDeRival` / `Franja` / `Propuesta` / `Partido` / `LadoPartido` / `Calificacion` / `Falta`	`«entidad»`	Esquema `20261028000003_rival_searches.sql` a `20261028000017_team_public_history.sql`	Implementadas

Módulo E · Organizador

Primero los requerimientos del módulo completo. Después, el modelo de dominio y, para cada caso de uso, su diagrama de secuencia, su diagrama de clases y su pantalla. Al final, los anexos y la trazabilidad.

Parte 1 · Requerimientos

Versión del **04/11/2026**. Es la primera parte del módulo: sigue la sección de casos de uso, donde cada caso lleva debajo su diagrama de secuencia y su pantalla. El README tiene la matriz que los une.

Fuentes, en orden de autoridad: decisiones-casos-de-uso.md, casos-de-uso.md (módulo E, 03/11/2026), el glosario CONTEXT.md y el código y las migraciones del repo al día de hoy.

1. Alcance

El módulo permite a un **Organizador publicar torneos**, **gestionarlos** desde su panel («Mis torneos»), mantenerlos al día («Sigue vigente»), **editarlos**, **cargar y dar de baja inscripciones** y **dar de baja** un torneo. Cualquier usuario con sesión puede publicar; al hacerlo pasa a ser Organizador. Todo torneo tiene **exactamente un Organizador**.

Límite de producto: la app **nunca inscribe ni cobra**. La inscripción se arregla por fuera (el capitán le escribe al organizador por WhatsApp o Instagram) y **la carga el organizador** en su panel. El capitán no «se anota» desde la app y no confirma la vinculación.

Fuera del módulo:

Tema	Dónde está
Ver el catálogo y la ficha pública, y contactar al organizador	Módulo B — Catálogo de torneos
Resolver un reporte de un torneo, marcarlo Observado o darlo de baja como administrador	Módulo F — Administración
Crear la cuenta y el perfil (nombre y foto del organizador)	Módulo A — Cuenta y acceso
Los Equipos que se vinculan a una inscripción	Módulo C — Equipos

2. Actores

Actor	Quién es	Qué hace en este módulo
Usuario	Persona con sesión	Publica un torneo y pasa a ser Organizador.
Organizador	Usuario que publicó al menos un torneo	Gestiona sus torneos desde el **Panel de organizador**. No tiene ningún poder de administración.
Administrador	Integrante del equipo de Torneos BA	Puede dar de baja un torneo por un reporte (UC-F03); no gestiona torneos ajenos desde este panel.
Sistema	La base de datos y la app	Calcula el Estado, la antigüedad y los cupos al leer, y reparte los Avisos de baja.

3. Requerimientos funcionales

Estado: `Implementado` está en `main`. `Parcial` tiene una parte sin cumplir, que se indica. `Pendiente` es alcance vigente sin programar.

Publicar y editar

ID	Requerimiento	Casos de uso	Estado
RF-E01	**Cualquier usuario con sesión** publica un torneo y pasa a ser Organizador. El organizador (`user_id`) sale de la sesión, **nunca del formulario**. Un usuario con **Suspensión** no puede publicar.	UC-E01	**Parcial**: la Suspensión no se verifica todavía (módulo F pendiente)
RF-E02	**Obligatorios:** nombre, modalidad (Fútbol 5, 7, 8 u 11), Zona, cierre de inscripción, inicio y fin, dirección de la sede y **al menos un contacto** (WhatsApp o Instagram). **Opcionales:** precio, flyer, cupo, descripción y reglamento.	UC-E01, UC-E03	Implementado
RF-E03	Las fechas cumplen **cierre ≤ inicio ≤ fin**; si no, el formulario no guarda. La fecha de fin es obligatoria: sin ella el torneo nunca llegaría a «Finalizado».	UC-E01, UC-E03	Implementado
RF-E04	**Contactos normalizados:** el WhatsApp se guarda solo con dígitos, en formato argentino de celular (`549` + código de área sin 0 + número sin 15); el Instagram se guarda solo como usuario (acepta `@usuario` o la URL del perfil).	UC-E01, UC-E03	Implementado
RF-E05	**Precio:** vacío es «Consultar» (null) y 0 es «Gratis». **Cupo:** entero de al menos 1.	UC-E01, UC-E03	Implementado
RF-E06	**Flyer** opcional, JPG, PNG o WebP de hasta **3 MB**. Se sube **después del alta**, con el id del torneo, para no dejar archivos huérfanos. Si la subida falla, el torneo **ya está publicado**: se avisa en la ficha y se reintenta desde Datos.	UC-E01, UC-E03	Implementado
RF-E07	El torneo **nace verificado** y aparece en el catálogo con su Estado calculado.	UC-E01	Implementado
RF-E08	**Todo torneo tiene exactamente un Organizador.** No existe el «torneo copiado» sin organizador (descartado el 07/10/2026). La **carga manual** de torneos de Instagram la hace alguien del equipo con su propia cuenta, como organizador común, y pone el contacto del organizador real.	UC-E01	Implementado
RF-E09	**Editar** cualquier campo del formulario (incluido reemplazar el flyer) con las mismas validaciones del alta. Solo el dueño, y solo las columnas del formulario. **Editar no cuenta como verificar.** Al guardar, vuelve a la pantalla de gestión.	UC-E03	Implementado
RF-E10	Al editar un torneo **con equipos anotados**, el organizador **confirma** el cambio («Hay N equipos anotados, ¿confirmás el cambio?»). Sin equipos no se pregunta.	UC-E03	Implementado (no figura en `casos-de-uso.md`)
RF-E11	**Subir el cupo** es la forma de admitir más inscripciones cuando está completo.	UC-E03, UC-E06	Implementado

Panel y gestión

ID	Requerimiento	Casos de uso	Estado
RF-E12	**Panel «Mis torneos»**, en tres niveles: (1) la lista, con una fila por torneo (nombre, Zona, modalidad, cupo, Estado y antigüedad de verificación) y dos acciones, «Sigue vigente» y «Gestionar»; (2) la **pantalla de gestión**; (3) la ficha pública (para el dueño, con la barra «Este torneo es tuyo · Gestionar»).	UC-E02	Implementado
RF-E13	**Orden del panel:** primero los que llevan 30 días o más sin verificar, del más viejo al más nuevo; después el resto; y al final los **Finalizados**, sin antigüedad ni botón de verificar.	UC-E02	Implementado
RF-E14	**Pantalla de gestión:** cabecera con los números clave (cupo o inscriptos, cierre de inscripción, fechas y última confirmación), «Ver ficha pública» y «Sigue vigente»; pestañas **Datos** e **Inscripciones**, cada una con su URL (`?tab=`); y «Dar de baja» al final, aparte.	UC-E02	Implementado

ID	Requerimiento	Casos de uso	Estado
RF-E15	Un torneo **ajeno** da **404**, igual que uno inexistente: no se confirma que exista un torneo con ese id que no es tuyo.	UC-E02	Implementado

Verificar y dar de baja

ID	Requerimiento	Casos de uso	Estado
RF-E16	**«Sigue vigente»** reinicia la **antigüedad de verificación**: el torneo vuelve a mostrarse sin ámbar ni rojo. Solo el dueño, y solo por la función de la base, que es la única que escribe la fecha.	UC-E04	Implementado
RF-E17	«Sigue vigente» **no se ofrece** en un torneo **Finalizado** ni en uno **ya confirmado hoy** (menos de 24 h): el botón sale apagado.	UC-E04	Implementado (la segunda regla no figura en `casos-de-uso.md`)
RF-E18	«Sigue vigente» **no saca la marca Observado**: solo la saca el administrador.	UC-E04	Regla vigente; la marca todavía no existe (módulo F)
RF-E19	**Dar de baja** un torneo Abierto, Cerrado o En curso. **Nunca un Finalizado:** los finalizados son la trayectoria del organizador.	UC-E05	Implementado
RF-E20	La baja **siempre pide confirmación** con el nombre del torneo y la cantidad de equipos anotados (aunque sea 0): no se puede deshacer.	UC-E05	Implementado
RF-E21	La baja **retira el torneo**: desaparece del catálogo, su ficha devuelve 404 y se borra el flyer de Storage.	UC-E05	Implementado
RF-E22	La baja **avisa a los planteles**: cada jugador del Plantel de cada equipo con Inscripción vinculada recibe **un único Aviso** (aunque esté en dos equipos anotados), con link a la página del equipo: «Se dio de baja el torneo 'X', en el que estaba anotado tu equipo. Hablá con el organizador por lo que hayas pagado.».	UC-E05	Implementado

Inscripciones

ID	Requerimiento	Casos de uso	Estado
RF-E23	**Dos tipos de inscripción:** la **vinculada** (se busca un equipo de la app por nombre y se lo selecciona; el resultado muestra siempre la Zona, porque los nombres se repiten) y la **por nombre** (se escribe el nombre de un equipo que no está en la app, de 2 a 50 caracteres).	UC-E06	Implementado
RF-E24	**Ambas cuentan para el cupo** y aparecen por nombre en la ficha pública. **Solo la vinculada** genera Avisos y figura en la historia del equipo.	UC-E06	Implementado
RF-E25	Un equipo de la app tiene **como mucho una inscripción vinculada por torneo**.	UC-E06	Implementado
RF-E26	**Cupo completo:** se rechaza y se sugiere subir el cupo desde Datos.	UC-E06	Implementado
RF-E27	Un **Equipo disuelto** no recibe inscripciones.	UC-E06	Implementado
RF-E28	Un torneo **Cerrado** o **En curso** sí admite inscripciones (el organizador sigue acomodando la lista); un **Finalizado** no cambia sus inscripciones.	UC-E06, UC-E07	Implementado
RF-E29	El **capitán no confirma** la vinculación. El organizador **no ve el WhatsApp** del capitán: ya lo tiene, porque el capitán le escribió.	UC-E06	Implementado

ID	Requerimiento	Casos de uso	Estado
RF-E30	Si el cupo se completa, el torneo pasa a **Cerrado**.	UC-E06	Implementado (lo calcula la base)
RF-E31	**Dar de baja una inscripción:** el equipo deja de figurar y libera su lugar. Si el torneo estaba Cerrado por cupo y no pasó el cierre de inscripción, vuelve a **Abierto**.	UC-E07	Implementado

4. Requerimientos no funcionales

ID	Requisito	Evidencia
RNF-E01	**Solo el dueño gestiona su torneo, y lo decide la base** (RLS), no la pantalla. Crear exige `auth.uid() = user_id`; editar y dar de baja, ser el dueño.	`20260930000001_tournaments_owner_only_rls.sql`, `tournaments_rls_test.sql`
RNF-E02	**Permisos por columna.** El organizador solo puede escribir las columnas del formulario (más `user_id` al crear): no puede escribir `verified_at` ni el Estado.	`20260930000004_tournaments_update_form_columns.sql`, `tournaments_update_test.sql`, `tournaments_form_test.sql`
RNF-E03	**Las transiciones van por funciones de Postgres, no por `update` directo:** `verify_tournament` (la única que escribe `verified_at`), `withdraw_tournament` (la única que borra torneos), `add_inscription`, `add_linked_inscription` y `remove_inscription` (las únicas que escriben en `inscriptions`).	`20260930000005_tournaments_verify.sql`, `20260930000006_tournaments_withdraw.sql`, `20261007000000_inscriptions.sql`, `20261014000004_linked_inscriptions.sql`
RNF-E04	**Los ids que llegan del cliente no son de confiar.** Que el torneo sea del que llama, que no esté Finalizado y que haya cupo lo controla la función en la base.	Comentarios y acciones de `mis-torneos/`
RNF-E05	**Estado, cupos y antigüedad se calculan al leer**, sin tareas programadas.	`20260930000002_tournaments_computed_status.sql`
RNF-E06	**Mensajes claros por cada error de la base**, según su SQLSTATE: cupo completo (`23514`), torneo Finalizado (`22023`), torneo ajeno o inexistente (`42501`), equipo inexistente (`P0002`), disuelto (`55000`) o ya anotado (`23505`).	`inscriptions.ts` (`inscriptionErrorMessage`)
RNF-E07	**Funciona sin JavaScript** en las acciones que son formularios: «Sigue vigente», dar de baja una inscripción, anotar un equipo vinculado y el buscador de equipos (un formulario GET con `?q=`).	`VerifyButton`, `TeamSearch`
RNF-E08	**Un alta que falla no deja archivos huérfanos:** el flyer se sube después del alta; en la baja se borra después de que la base la confirma.	`crear-torneo/actions.ts`, `mis-torneos/actions.ts`
RNF-E09	**El flyer lo valida la app y lo repite Storage** (tipo y 3 MB): el bucket es la última defensa.	`tournament-flyer.ts`, `20260930000007_tournament_flyers.sql`, `tournament_flyers_test.sql`

ID	Requisito	Evidencia
RNF-E10	**El formulario no se vacía ante un error:** devuelve el mensaje y lo ya cargado (salvo el archivo, que un navegador no deja precargar).	`TournamentFormState`
RNF-E11	**Las pestañas son links** (`?tab=`): cada una tiene su URL compartible y la búsqueda de equipos convive en la misma URL.	`tournament-manage.ts`
RNF-E12	**Lógica pura verificable:** `parseTournamentForm` , `normalizeInstagram` , `parseFlyer` , `parseTeamName` , `parseTeamQuery` , `verificationAge` , `sortForPanel` , `withdrawConfirmation` , con sus `*.test.ts` (`pnpm test`).	`src/utils/`

5. Sin decisión y diferencias con `casos-de-uso.md`

Tema	Situación
Suspensión al publicar	`casos-de-uso.md` pone «sin Suspensión» como precondición de UC-E01, pero la Suspensión no existe todavía (UC-F05). Hoy cualquier usuario con sesión publica.
«Sigue vigente» el mismo día	El código apaga el botón si se confirmó hace menos de 24 h (`verifyBlock`). La regla no figura en `casos-de-uso.md` ni en las decisiones. Hay que decidir si se documenta.
Confirmar al editar con equipos anotados	El código pide confirmación cuando hay equipos anotados. Tampoco figura en las decisiones.
Marca Observado	«Sigue vigente» no la saca, pero la marca todavía no existe: depende de UC-F03.
Nombre del producto en los mensajes	La interfaz dice «Torneos Scraper»; la documentación, «Torneos BA».

6. Reglas de referencia

- **Estado del torneo:** Abierto (no pasó el cierre y hay lugar), Cerrado (pasó el cierre sin empezar, o se completó el cupo), En curso (entre inicio y fin), Finalizado (pasó el fin; sale del catálogo y no se puede dar de baja).
- **Antigüedad de verificación:** días completos desde la última verificación; 30 días, ámbar; 60 días, rojo con «Sin confirmar». Crear verifica; editar no.
- **Términos:** se dice «baja» (nunca «eliminar»); «Inscripción vinculada» e «Inscripción por nombre»; «Zona» (nunca «barrio» ni «localidad»).

Parte 2 · Modelo de dominio y casos de uso, con sus diagramas y su pantalla

Versión del **04/11/2026**. Detalla los casos de uso del módulo E de casos-de-uso.md (UC-E01 a UC-E07) con los requerimientos que cubre cada uno (01-requerimientos.md), la pantalla que lo representa (03-pantallas.md) y su diagrama de secuencia (04-diagramas-secuencia).

Cómo leerlo. Cada caso expresa una meta y tiene **un actor iniciador**. **Estado** dice si está en `main`. Los mensajes entre comillas son los que ve la persona.

Mapa de casos de uso

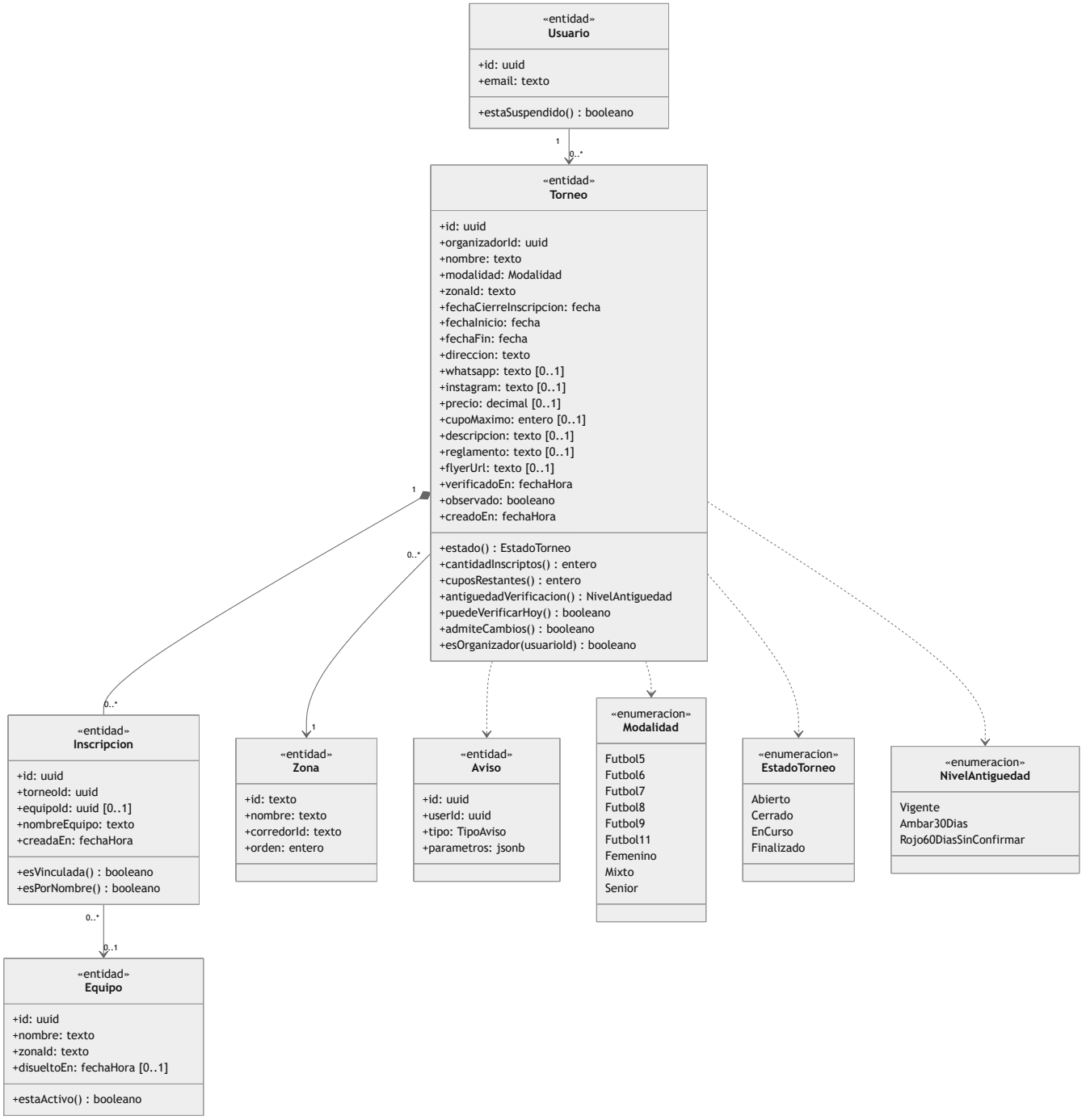
Caso	Meta del actor	Actor	Estado	ID en el PDF original
UC-E01	Publicar un torneo	Usuario	**Parcial** (Suspensión)	SPEC-E01
UC-E02	Ver mis torneos	Organizador	Implementado	SPEC-E02
UC-E03	Editar un torneo	Organizador	Implementado	SPEC-E01
UC-E04	Verificar que el torneo sigue vigente	Organizador	Implementado	SPEC-E02
UC-E05	Dar de baja un torneo	Organizador	Implementado	SPEC-E02
UC-E06	Cargar una inscripción	Organizador	Implementado	SPEC-E03
UC-E07	Dar de baja una inscripción	Organizador	Implementado	SPEC-E03

El PDF original agrupaba publicar y editar en SPEC-E01, y consultar, verificar y dar de baja en SPEC-E02; los diagramas generales de `diagramas-secuencia.md` ya los separan (D23 a D28). El flujo del PDF donde el capitán se anotaba desde la app se retiró. La carga manual por el equipo no es un caso aparte: es UC-E01 con una cuenta propia.

Qué no es un caso de uso acá: cambiar de pestaña en la pantalla de gestión (`?tab=`) o abrir «Ver ficha pública» no toman ninguna decisión.

Diagrama de clases · **Modelo de dominio del módulo**

Vista estructural de las entidades, relaciones de composición/asociación y enumeraciones que comparten los casos de uso `UC-E01` a `UC-E07`.



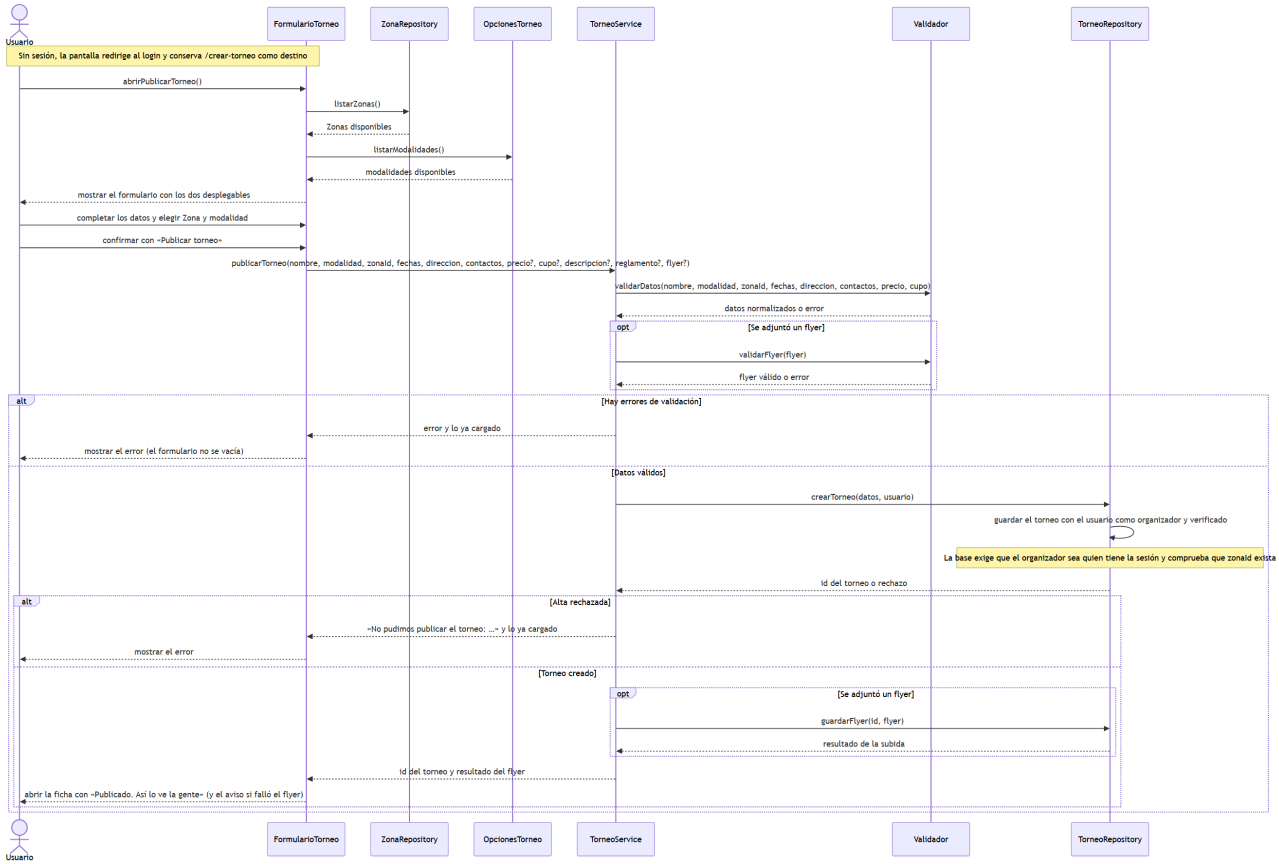
UC-E01 · Publicar un torneo

- **Actor:** Usuario (pasa a ser **Organizador** al publicar).
- **Requerimientos:** RF-E01 a RF-E08. **Pantalla:** P2 · Publicar torneo (`/crear-torneo`). **Diagrama:** S01.
- **Precondiciones:** sesión; sin **Suspensión** (regla vigente; hoy no se verifica).
- **Postcondición:** el torneo tiene exactamente un organizador; el usuario ve «Mis torneos» en su menú.
- **Estado: Parcial** (`src/app/crear-torneo`): todo, salvo la verificación de la Suspensión.

Diagrama de secuencia · **S01 · UC-E01 — Publicar un torneo**

Caso de uso: UC-E01 · Pantalla: P2 · Requerimientos: RF-E01 a RF-E08

El formulario tiene **dos desplegables**: la **Zona** y la **modalidad**. Siguiendo la guía de formularios, la pantalla obtiene las opciones de cada uno **antes** de que la persona elija: las Zonas, que se consultan en la base, de `ZonaRepository`; las modalidades, que son una lista fija del código, de `OpcionesTorneo`. Al enviar viajan solo los valores elegidos (`zonaId`, `modalidad`). La carga de las opciones y la publicación van en el mismo diagrama.



Cómo se lee

- **Antes de elegir:** la pantalla obtiene las Zonas de `ZonaRepository` y las modalidades de `OpcionesTorneo`, y recién entonces muestra el formulario. Al enviar viajan solo los **valores elegidos**.
- **El organizador sale de la sesión, nunca del formulario.** `crearTorneo(datos, usuario)` recibe el usuario de la sesión, y la base lo exige otra vez (`auth.uid() = user_id`). Cualquier campo de más que mande el cliente no se guarda: el organizador solo puede escribir las columnas del formulario.
- **Se valida de nuevo al guardar.** `Validador` revisa nombre, modalidad, Zona, fechas (**cierre ≤ inicio ≤ fin**), dirección, contactos (al menos uno), precio y cupo aunque el formulario tenga sus restricciones; y la base comprueba que la Zona exista.
- **El torneo nace verificado** y se muestra en el catálogo con su Estado calculado.
- **El flyer se sube después del alta**, no dentro de ella: el archivo se llama como el id del torneo, que antes no existe, y así un alta que falla no deja un archivo huérfano. Si lo que falla es la subida, el torneo **ya está publicado**: la ficha avisa «No pudimos subir el flyer» y se reintenta desde la pestaña Datos.
- **Suspensión (pendiente):** un usuario suspendido no puede publicar. Todavía no se verifica porque la Suspensión no está implementada (UC-F05).

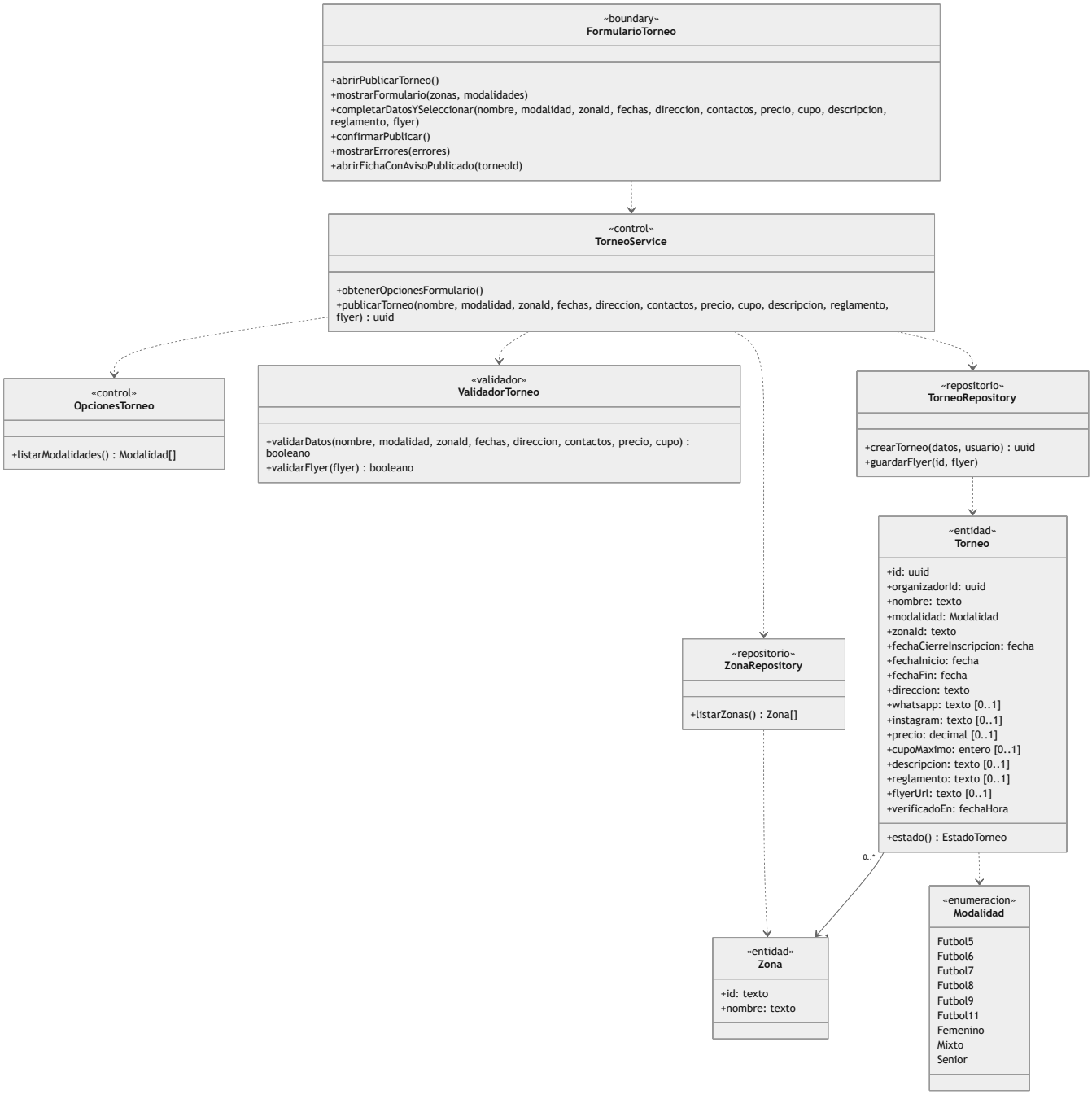
- **Si el formulario falla, no se vacía:** devuelve el error y lo ya cargado (salvo el archivo, que un navegador no deja precargar).

Correspondencia en el código

`createTournament` (`src/app/crear-torneo/actions.ts`): `parseTournamentForm` → `parseFlyer` → `insert` en `tournaments` con `user_id` de la sesión → `uploadFlyer` → redirige a `/torneos/{id}?publicado=1`. Las Zonas salen de `zones` mediante `getZoneGroups()`; las modalidades, de `SPORT_MODALITIES`.

Diagrama de clases · **UC-E01 · Publicar un torneo**

Caso de uso: UC-E01 · **Pantalla:** P2 · Publicar torneo (`/crear-torneo`) · **Secuencia:** S01



Pantalla · **P2 · Publicar torneo**

Ruta: `/crear-torneo` · **Caso de uso:** UC-E01 · **Requerimientos:** RF-E01 a RF-E08 · **Diagrama:** S01

> ⏳ **Captura pendiente.**

Qué muestra. «← Mis torneos», el título «Publicar torneo» y un formulario que es **el mismo de la edición** (pestaña Datos de P3):

Campo	Tipo	De dónde salen las opciones	Obligatorio
Nombre	texto (placeholder «Copa Amistad Sabatina»)	—	Sí
Modalidad	desplegable	Lista **fija del código** (`SPORT_MODALITIES` : Fútbol 5, 7, 8 y 11)	Sí
Zona	desplegable, agrupado por corredor	Se **consulta en la base** (`zones` , vía `getZoneGroups`)	Sí
Dirección de la sede	texto («Calle y altura de la cancha. En la ficha se abre en el mapa.»)	—	Sí
WhatsApp	teléfono («Con código de área y sin el 15.»)	—	Uno de los dos contactos
Instagram	texto («Solo el usuario.»)	—	Uno de los dos contactos
Cierre de inscripción	fecha	—	Sí
Inicio	fecha	—	Sí
Fin	fecha	—	Sí
Precio (opcional)	número («Vacío: se muestra “Consultar”. 0: “Gratis”.»)	—	No
Cupo (opcional)	entero ≥ 1 («Cantidad máxima de equipos.»)	—	No
Imagen (flyer)	archivo JPG, PNG o WebP, hasta 3 MB	—	No
Descripción	texto largo («Premios, horarios, tercer tiempo…»)	—	No
Reglamento	texto largo	—	No

El botón es «Publicar torneo» (con «Publicando…» mientras trabaja).

Estados: *normal*; *error* (banda con el motivo, **conservando lo cargado**; el archivo del flyer no se puede precargar); *sin sesión* (redirige a `/login?next=/crear-torneo`); y, tras publicar, la **ficha pública** con «Publicado. Así lo ve la gente» (y, si falló la subida, «No pudimos subir el flyer…»).

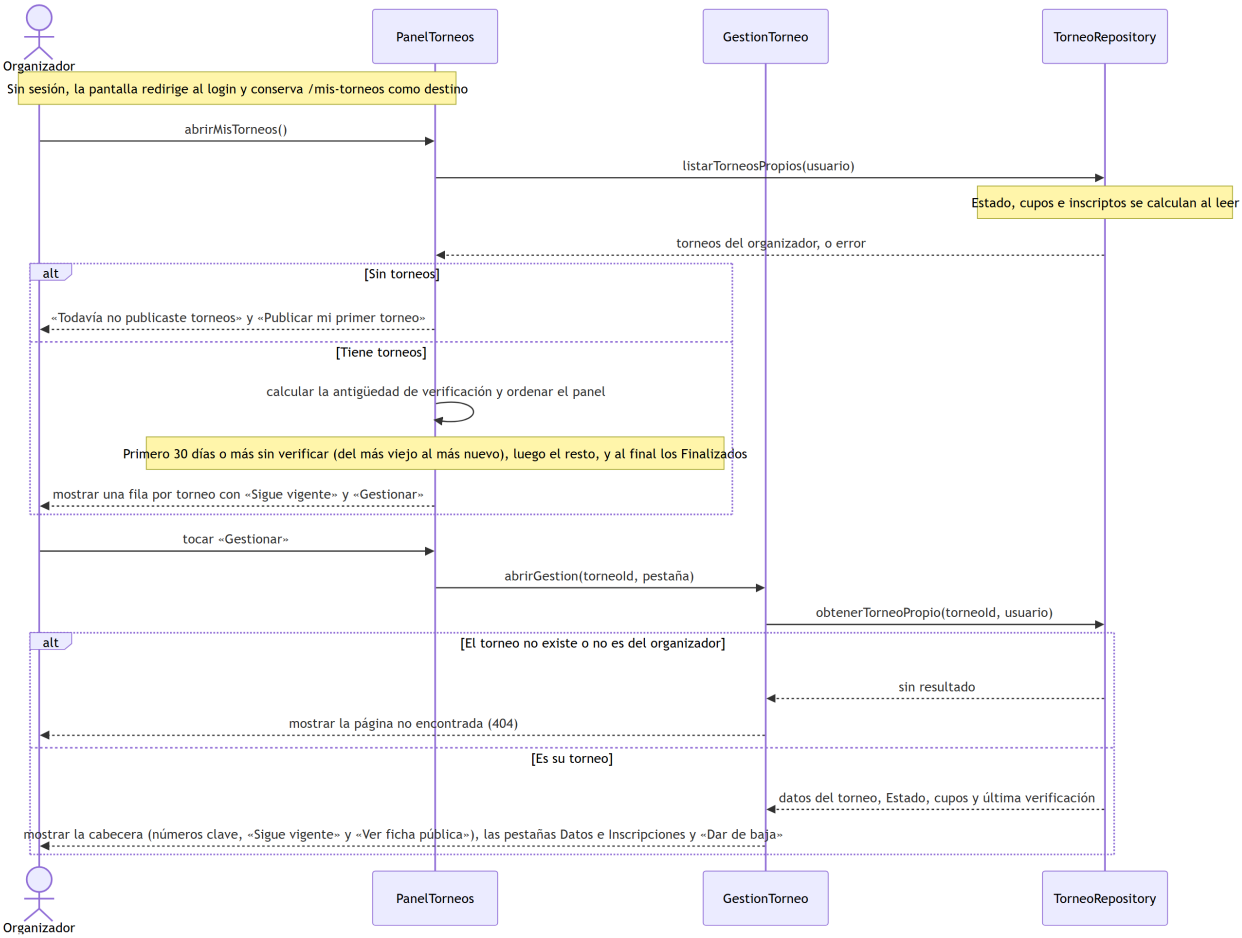
UC-E02 · Ver mis torneos (Panel de organizador)

- **Actor:** Organizador.
- **Requerimientos:** RF-E12 a RF-E15. **Pantallas:** P1 · Mis torneos y P3 · Gestión del torneo. **Diagrama:** S02.
- **Precondiciones:** sesión.
- **Niveles del panel:** (1) esta lista; (2) la **pantalla de gestión** de cada torneo, con cabecera (números clave, «Ver ficha pública», «Sigue vigente»), pestañas **Datos** e **Inscripciones** y «Dar de baja» al final, aparte; (3) la ficha pública (UC-B03 A3).
- **Estado:** Implementado (`src/app/mis-torneos`).

Diagrama de secuencia · **S02 · UC-E02 — Ver mis torneos (Panel de organizador)**

Caso de uso: UC-E02 · Pantallas: P1 y P3 · Requerimientos: RF-E12 a RF-E15

Es una consulta: no hay formulario ni desplegables.



Cómo se lee

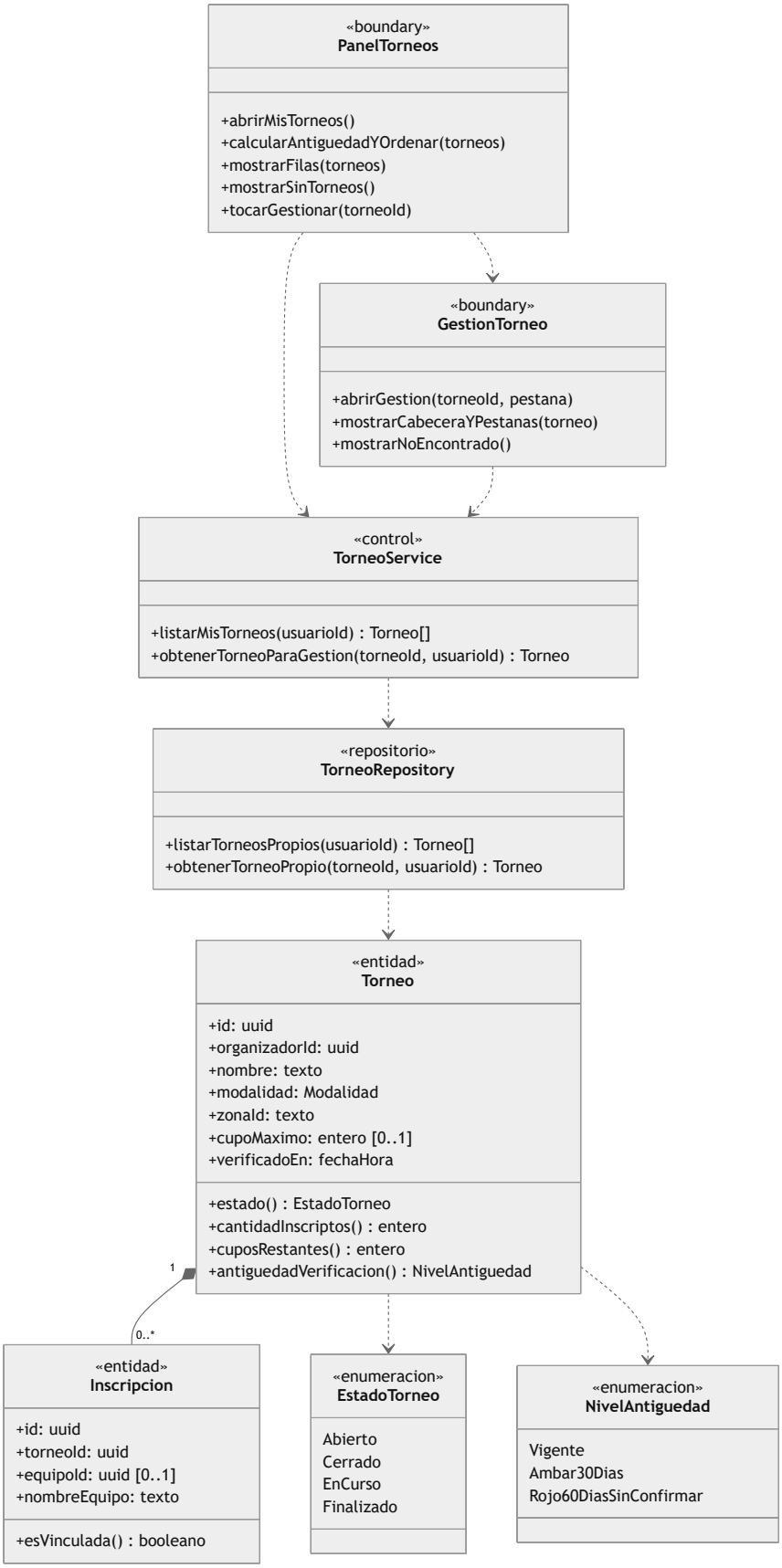
- **Una fila por torneo** con nombre, Zona, modalidad, cupo, Estado y antigüedad, y solo dos acciones: «Sigue vigente» y «Gestionar». Editar, inscripciones y baja viven **solo en la pantalla de gestión**.
- **El orden pone primero lo que pide atención:** los torneos con 30 días o más sin verificar, del más viejo al más nuevo. Los **Finalizados** van al final, sin antigüedad ni botón de verificar.
- **Un torneo ajeno da 404, igual que uno inexistente:** no se confirma que exista un torneo con ese id que no es tuyo. Un id que no es un uuid hace fallar la consulta y cae en el mismo 404.
- **Tres niveles:** la lista, la gestión y la ficha pública (que, para el dueño, agrega la barra «Este torneo es tuyo · Gestionar»).
- La pestaña vive en `?tab=` (Datos o Inscripciones): cada una tiene su URL, y cualquier valor desconocido abre Datos.

Correspondencia en el código

`MisTorneosPage` (`src/app/mis-torneos/page.tsx`) lee los torneos del usuario y usa `sortForPanel`; `GestionTorneoPage` (`src/app/mis-torneos/[id]/page.tsx`) filtra por `id` y `user_id` y llama a `notFound()` si no hay resultado.

Diagrama de clases · **UC-E02 · Ver mis torneos (Panel de organizador)**

Caso de uso: UC-E02 · **Pantallas:** P1 · Mis torneos y P3 · Gestión del torneo · **Secuencia:** S02



> Pantalla · **P1 · Mis torneos**

Ruta: `/mis-torneos` · **Casos de uso:** UC-E02, UC-E04 · **Requerimientos:** RF-E12, RF-E13, RF-E16, RF-E17 · **Diagramas:** S02, S04

> ⏳ **Captura pendiente.**

Qué muestra. «← Volver al catálogo», el título «Mis torneos» con una explicación breve («Tocá "Sigue vigente" para confirmar que un torneo está en pie: a los 30 días sin confirmar se marca en ámbar y a los 60, en rojo como "sin confirmar"») y el botón **«Publicar torneo»**. Debajo, **una fila por torneo** con su Estado, su antigüedad de verificación, el nombre, la Zona, la modalidad y el cupo, y dos acciones: **«Sigue vigente»** y **«Gestionar»**.

Orden: primero los torneos con 30 días o más sin verificar (del más viejo al más nuevo), después el resto, y al final los Finalizados (sin antigüedad ni botón de verificar).

Estados: *con torneos*; *sin torneos* («Todavía no publicaste torneos» y «Publicar mi primer torneo»); *«Sigue vigente» apagado* (torneo Finalizado o ya confirmado hoy); *sin sesión* (redirige a `/login?next=/mis-torneos`, y desde «Publicalo» abre Crear cuenta con el aviso).

> Pantalla · **P3 · Gestión del torneo**

Ruta: `/mis-torneos/{id}` · **Casos de uso:** UC-E02 a UC-E07 · **Requerimientos:** RF-E09 a RF-E31 · **Diagramas:** S03, S04, S05, S06, S07

> ⏳ **Capturas pendientes** de la cabecera y de las dos pestañas.

Cabecera (siempre visible). «← Mis torneos»; el nombre del torneo y su Estado; cuatro **números clave**: **Cupo** (o Inscriptos, si no hay cupo), **Cierre de inscripción**, **Fechas** y **Última confirmación** (con su antigüedad, o «Finalizó: ya no se confirma»); y dos botones: **«Sigue vigente»** y **«Ver ficha pública»**.

Pestañas (links, cada una con su URL): **Datos** e **Inscripciones**. Al final de la pantalla, aparte, **«Dar de baja»**.

Pestaña Datos. El formulario de P2 con los valores actuales y el botón de guardar; si hay equipos anotados, pide confirmación («Hay N equipos anotados, ¿confirmás el cambio?»). Guardar vuelve a esta pantalla.

Pestaña Inscripciones.

- La sección **«Anotar un equipo»**, con la indicación «Arreglá la inscripción con el capitán por WhatsApp y cargala acá buscando su equipo en la app». Tiene un **buscador de equipos de la app** («Buscar un equipo de la app», botón «Buscar»), que es un formulario GET: la búsqueda vive en `?q=`. Cada resultado muestra el nombre del equipo y su **Zona**, y un botón **«Anotar»**; si ya está anotado, dice «Ya anotado». Muestra hasta 10 equipos activos.
- Debajo, **«¿El equipo no está en la app? Anotalo solo con el nombre»**: un campo «Nombre del equipo» y el botón «Anotar equipo» (con «Anotando…» mientras trabaja).
- La sección **«Equipos anotados»**, con cada equipo (los vinculados son un link a su página, con su Zona) y «Dar de baja» en cada uno. Sin inscriptos: «Todavía no hay equipos anotados.»

«Dar de baja» (el torneo) abre una confirmación con el nombre y la cantidad de equipos anotados: «¿Dar de baja "X" con N equipos anotados? No se puede deshacer.»

Estados: *normal*; *cupo completo* (error al anotar: «Subí el cupo desde la pestaña "Datos"…»); *torneo Finalizado* (sin «Sigue vigente» activo ni baja); *torneo ajeno o inexistente* (404); *sin sesión* (redirige a `/login` conservando la pestaña).

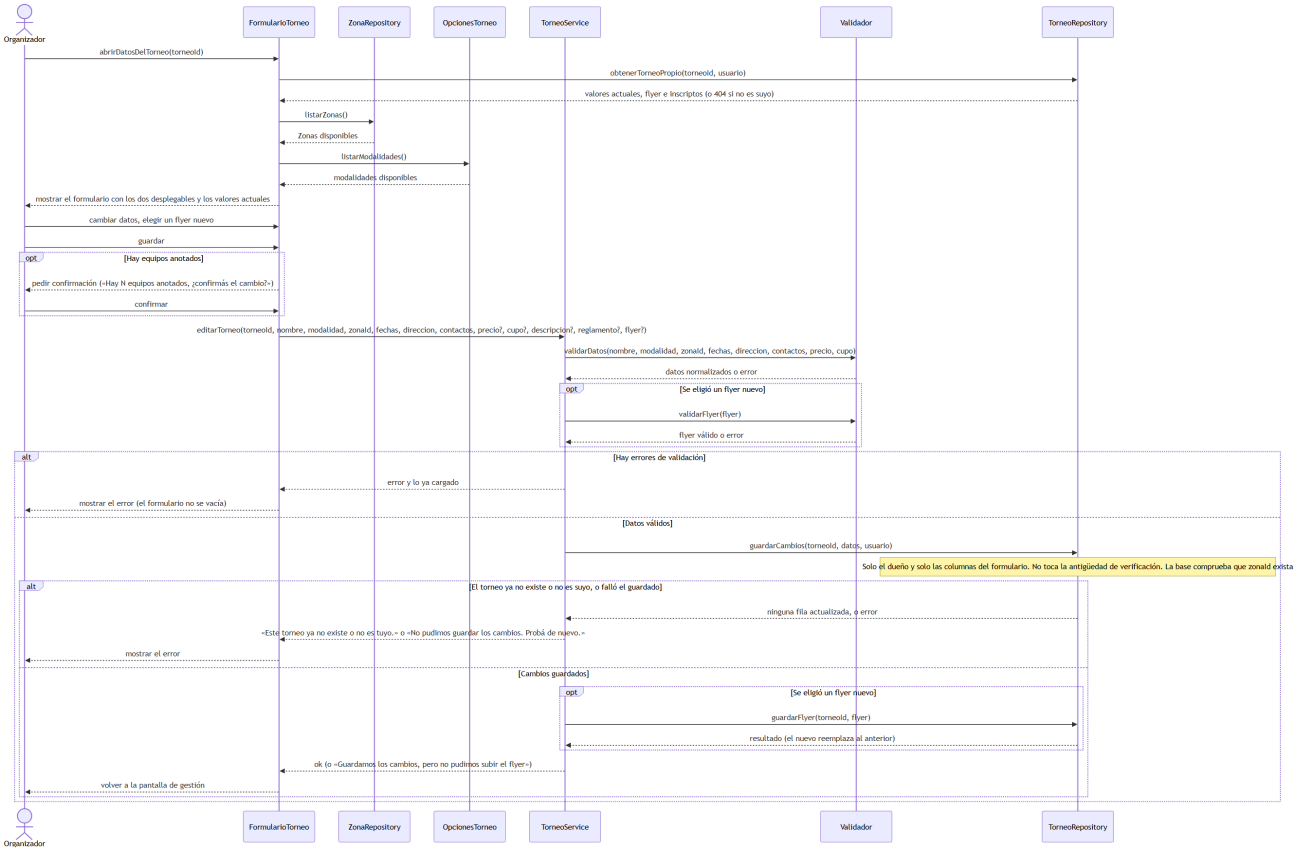
UC-E03 · Editar un torneo

- **Actor:** Organizador del torneo.
- **Requerimientos:** RF-E02 a RF-E06, RF-E09 a RF-E11. **Pantalla:** P3 · Gestión del torneo (pestaña Datos). **Diagrama:** S03.
- **Precondiciones:** es el dueño.
- **Estado:** Implementado (`src/app/mis-torneos/[id]/editar`).

Diagrama de secuencia · **S03 · UC-E03 — Editar un torneo**

Caso de uso: UC-E03 · Pantalla: P3 (pestaña Datos) · Requerimientos: RF-E02 a RF-E06, RF-E09 a RF-E11

Es el mismo formulario que S01, con los dos desplegables (**Zona** y **modalidad**): sigue el mismo criterio. La carga de las opciones y el guardado van en el mismo diagrama. La diferencia: el formulario llega con los **valores actuales**, se **confirma** si hay equipos anotados y la base solo deja editar al dueño.



Cómo se lee

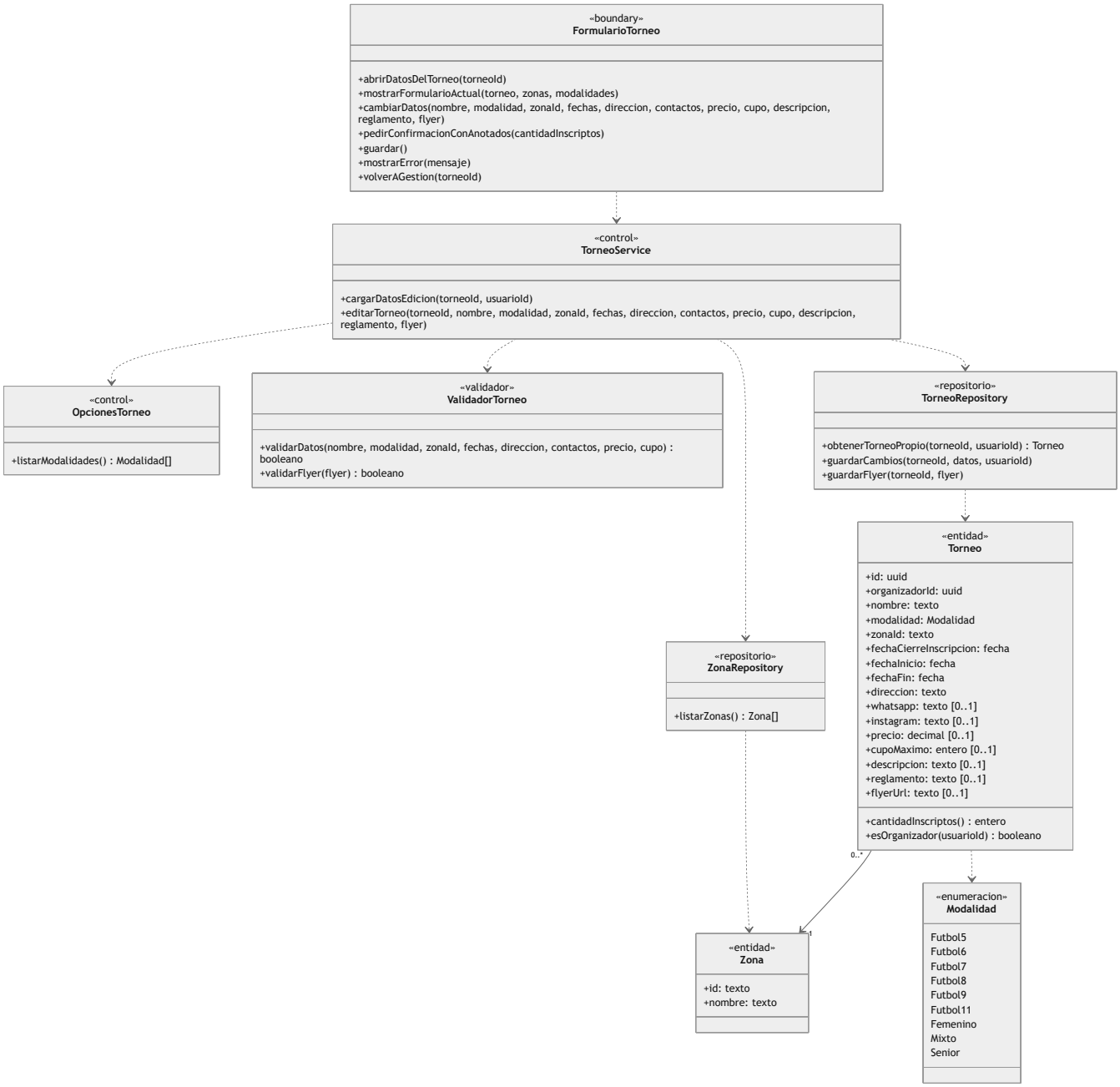
- **Mismo patrón que publicar:** las Zonas de `ZonaRepository`, las modalidades de `OpcionesTorneo`, y al enviar solo los **valores elegidos**. Los valores actuales vienen de `obtenerTorneoPropio`; el WhatsApp se muestra como lo cargó la persona (sin el `549`).
- **Quién puede editar lo decide la base**, no el código: la policy de actualización exige ser el dueño y los permisos por columna dejan escribir solo las del formulario. Con un torneo ajeno o dado de baja la actualización **no da error: afecta cero filas**, y por eso el mensaje «Este torneo ya no existe o no es tuyo».
- **Se valida de nuevo al guardar**, con las mismas reglas del alta.
- **Editar no cuenta como verificar:** `verified_at` no se toca (la persona no puede escribirlo, solo `verify_tournament`). Reiniciar la antigüedad es «Sigue vigente» (S04).
- **Confirmación con equipos anotados:** si hay inscriptos, el cambio los afecta (fechas, cupo, sede), así que se pide confirmar. Sin equipos, no se pregunta.
- **Subir el cupo** es la forma de admitir más inscripciones cuando está completo.
- **El flyer:** dejarlo vacío conserva el actual; uno nuevo **reemplaza** al anterior (mismo objeto). Si la subida falla, los cambios ya están guardados y se avisa.

Correspondencia en el código

`updateTournament` (`src/app/mis-torneos/[id]/editar/actions.ts`): `parseTournamentForm` → `parseFlyer` → `update` filtrado por `id` y `user_id` → `uploadFlyer` → redirige a `/mis-torneos/{id}?tab=datos` . La confirmación sale de `editConfirmation` (`utils/tournament-form.ts`).

Diagrama de clases · **UC-E03 · Editar un torneo**

Caso de uso: UC-E03 · **Pantalla:** P3 · Gestión del torneo (pestaña Datos) · **Secuencia:** S03



Pantalla · **P3 · Gestión del torneo**

Es la misma pantalla que se describe, con su captura, en P3 · Gestión del torneo, debajo de UC-E02 · Ver mis torneos (Panel de organizador). Este caso usa la parte que indica su ficha («Pantalla»).

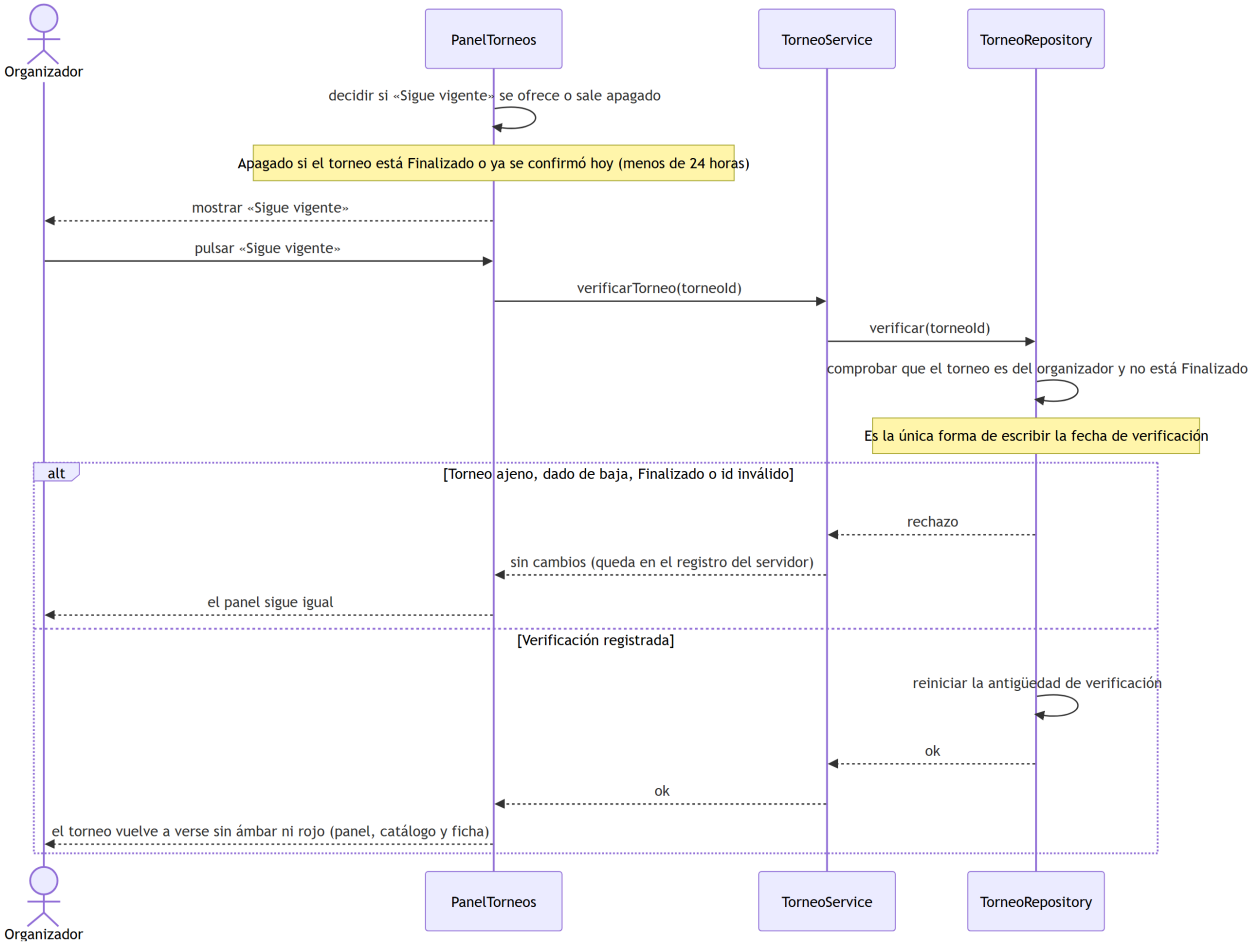
UC-E04 · Verificar que el torneo sigue vigente

- **Actor:** Organizador del torneo.
- **Requerimientos:** RF-E16 a RF-E18. **Pantallas:** P1 y P3 (botón «Sigue vigente»). **Diagrama:** S04.
- **Precondiciones:** el torneo no está Finalizado.
- **Estado:** Implementado (`verifyTournament` , función `verify_tournament`).

Diagrama de secuencia · **S04 · UC-E04 — Verificar que el torneo sigue vigente**

Caso de uso: UC-E04 · Pantallas: P1 y P3 (botón «Sigue vigente») · Requerimientos: RF-E16 a RF-E18

No hay desplegables ni datos que cargar: es un solo botón, que es un formulario con el id del torneo en un campo oculto (funciona sin JavaScript).



Cómo se lee

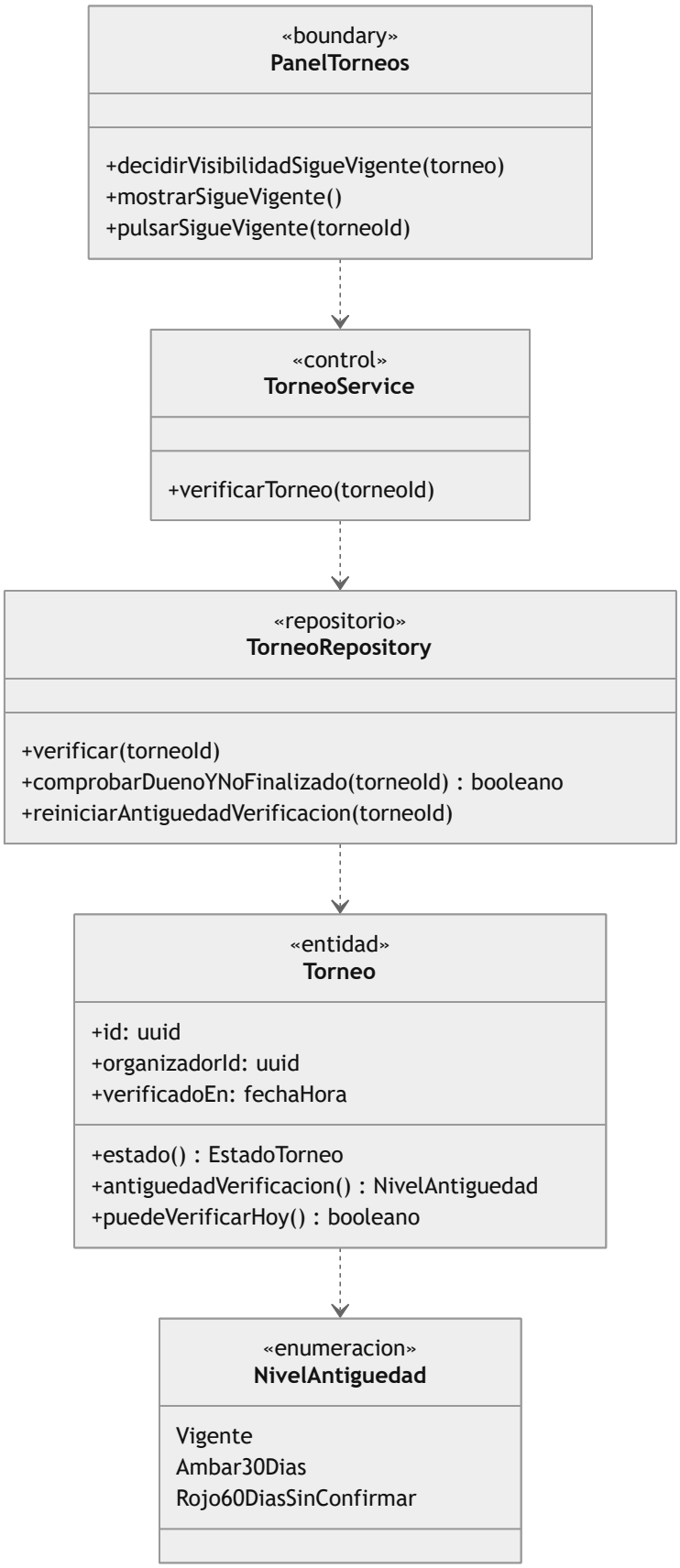
- **Qué hace:** reinicia la **antigüedad de verificación** (días completos desde la última confirmación). A los 30 días el torneo se ve en ámbar y a los 60 en rojo con «Sin confirmar»; «Sigue vigente» lo devuelve a «Verificado hoy».
- **El botón se apaga en dos casos:** un torneo **Finalizado** (ya no tiene que confirmar que sigue en pie) y uno **ya confirmado hoy** (la regla del código; no figura en `casos-de-uso.md`).
- **Solo la función de la base escribe la fecha.** El organizador no puede actualizar `verified_at` directamente ni con una edición: por eso **editar no cuenta como verificar**.
- **Si la base rechaza, no se muestra error:** solo puede pasar con un torneo ajeno, dado de baja, Finalizado o un id inventado, y el panel nunca le ofrece esos casos al dueño. Alcanza con el registro del servidor.
- **No saca la marca Observado** (que pone el administrador por reportes y solo él saca); la marca todavía no existe.

Correspondencia en el código

`verifyTournament` (`src/app/mis-torneos/actions.ts`) llama a la RPC `verify_tournament`; `VerifyButton` decide si el botón sale apagado con `verifyBlock` (`utils/tournament-verification.ts`).

Diagrama de clases · **UC-E04 · Verificar que el torneo sigue vigente**

Caso de uso: UC-E04 · **Pantallas:** P1 y P3 (botón «Sigue vigente») · **Secuencia:** S04



> Pantalla · **P1 · Mis torneos**

Es la misma pantalla que se describe, con su captura, en P1 · Mis torneos, debajo de UC-E02 · Ver mis torneos (Panel de organizador). Este caso usa la parte que indica su ficha («Pantalla»).

> Pantalla · **P3 · Gestión del torneo**

Es la misma pantalla que se describe, con su captura, en P3 · Gestión del torneo, debajo de UC-E02 · Ver mis torneos (Panel de organizador). Este caso usa la parte que indica su ficha («Pantalla»).

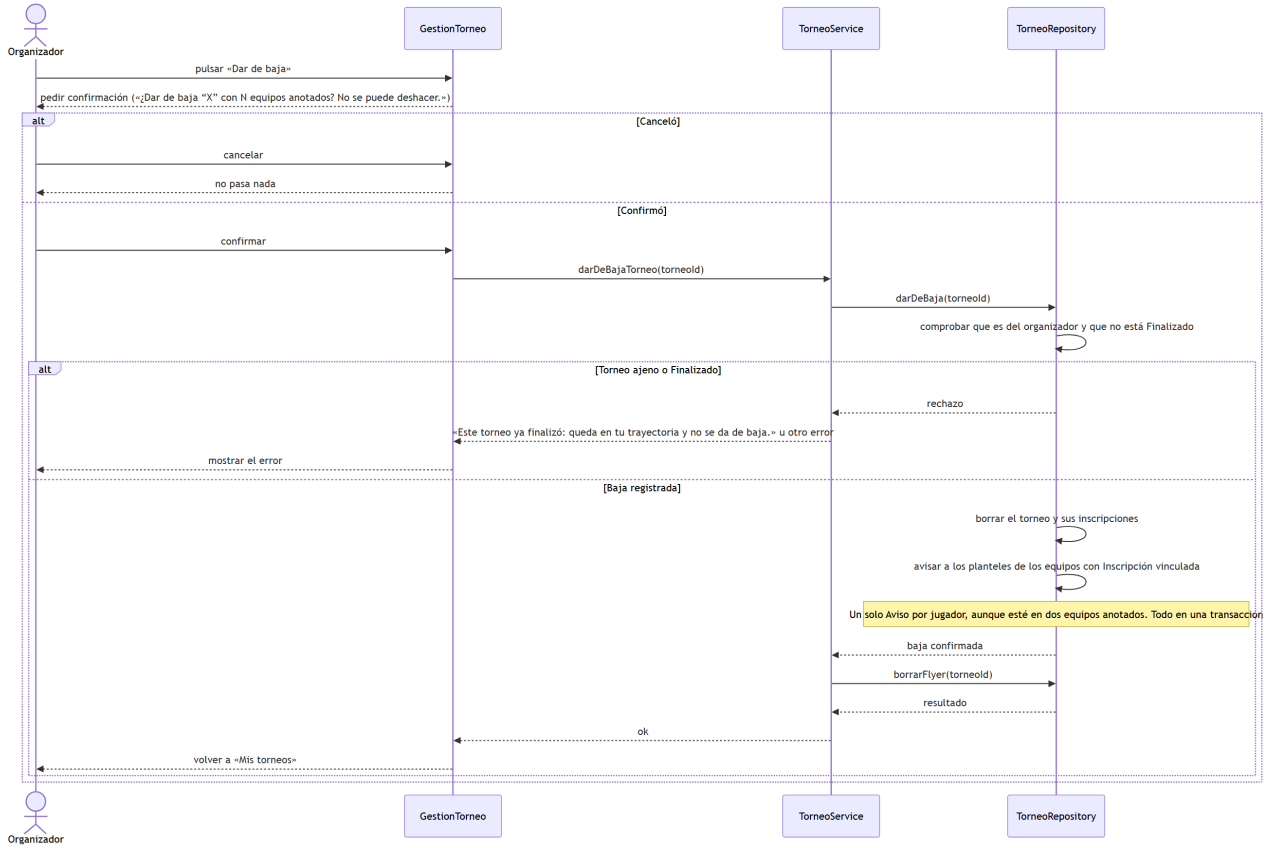
UC-E05 · Dar de baja un torneo

- **Actor:** Organizador del torneo (o Administrador, UC-F03).
- **Requerimientos:** RF-E19 a RF-E22. **Pantalla:** P3 · Gestión del torneo («Dar de baja», al final). **Diagrama:** S05.
- **Precondiciones:** el torneo está Abierto, Cerrado o En curso. **Nunca Finalizado:** los finalizados son la trayectoria del organizador.
- **Aviso a los planteles:** «Se dio de baja el torneo 'X', en el que estaba anotado tu equipo. Hablá con el organizador por lo que hayas pagado.» Solo las inscripciones **vinculadas** avisan; las por nombre no tienen plantel al que avisar.
- **Postcondición:** el torneo no existe; los equipos vinculados dejan de tenerlo en su historia (la baja se lleva sus inscripciones).
- **Estado:** Implementado (`withdrawTournament` , función `withdraw_tournament`).

Diagrama de secuencia · **S05 · UC-E05 — Dar de baja un torneo**

Caso de uso: UC-E05 · Pantalla: P3 («Dar de baja», al final) · Requerimientos: RF-E19 a RF-E22

No hay desplegables. La baja siempre pide una confirmación con el nombre y la cantidad de equipos anotados.



Cómo se lee

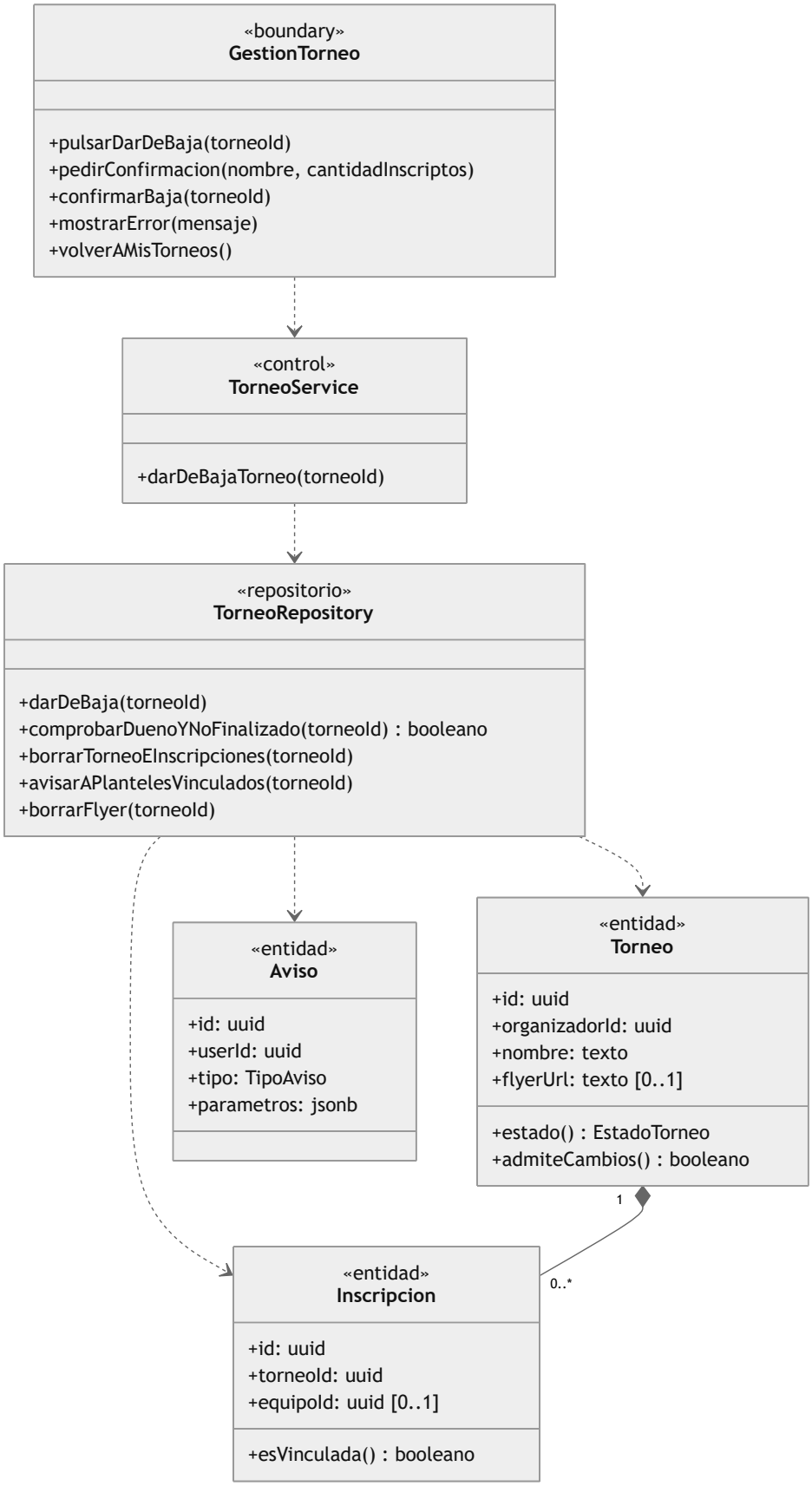
- **Confirmación siempre**, haya o no equipos anotados: la baja no tiene vuelta atrás. A diferencia de editar, que solo confirma si hay inscriptos.
- **Nunca un Finalizado:** los torneos finalizados son la trayectoria del organizador y se ven en su página pública. Aunque el botón no se ofrezca, el Estado puede cambiar entre que se carga la página y se pulsa (por ejemplo, el torneo termina a medianoche): la base lo vuelve a comprobar y el mensaje se muestra.
- **Qué desaparece:** el torneo sale del catálogo, su ficha devuelve 404 y se borra el flyer de Storage.
- **El flyer se borra después de la baja.** Postgres no puede borrar archivos, así que se hace por la API de Storage una vez que la base confirmó. Así, un torneo que no pudo darse de baja no se queda sin flyer.
- **Aviso a los planteles:** cada jugador del Plantel (el del momento de la baja) de cada equipo con **Inscripción vinculada** recibe **un único Aviso**, con link a la página del equipo, porque la ficha ya devuelve 404: «Se dio de baja el torneo 'X', en el que estaba anotado tu equipo. Hablá con el organizador por lo que hayas pagado.» Las inscripciones por nombre no tienen plantel al que avisar.
- **Quién más puede:** el administrador, por un reporte (UC-F03), con las mismas consecuencias.

Correspondencia en el código

`withdrawTournament` (`src/app/mis-torneos/actions.ts`) llama a la RPC `withdraw_tournament` (versión de `20261028000016_withdraw_tournament_one_notice.sql`) y después a `removeFlyer` ; `WithdrawButton` pide la confirmación con `withdrawConfirmation` (`utils/tournament-withdraw.ts`). Probado en `supabase/tests/tournaments_withdraw_test.sql` .

Diagrama de clases · **UC-E05 · Dar de baja un torneo**

Caso de uso: UC-E05 · **Pantalla:** P3 · Gestión del torneo («Dar de baja», al final) · **Secuencia:** S05



> Pantalla · **P3 · Gestión del torneo**

Es la misma pantalla que se describe, con su captura, en P3 · Gestión del torneo, debajo de UC-E02 · Ver mis torneos (Panel de organizador). Este caso usa la parte que indica su ficha («Pantalla»).

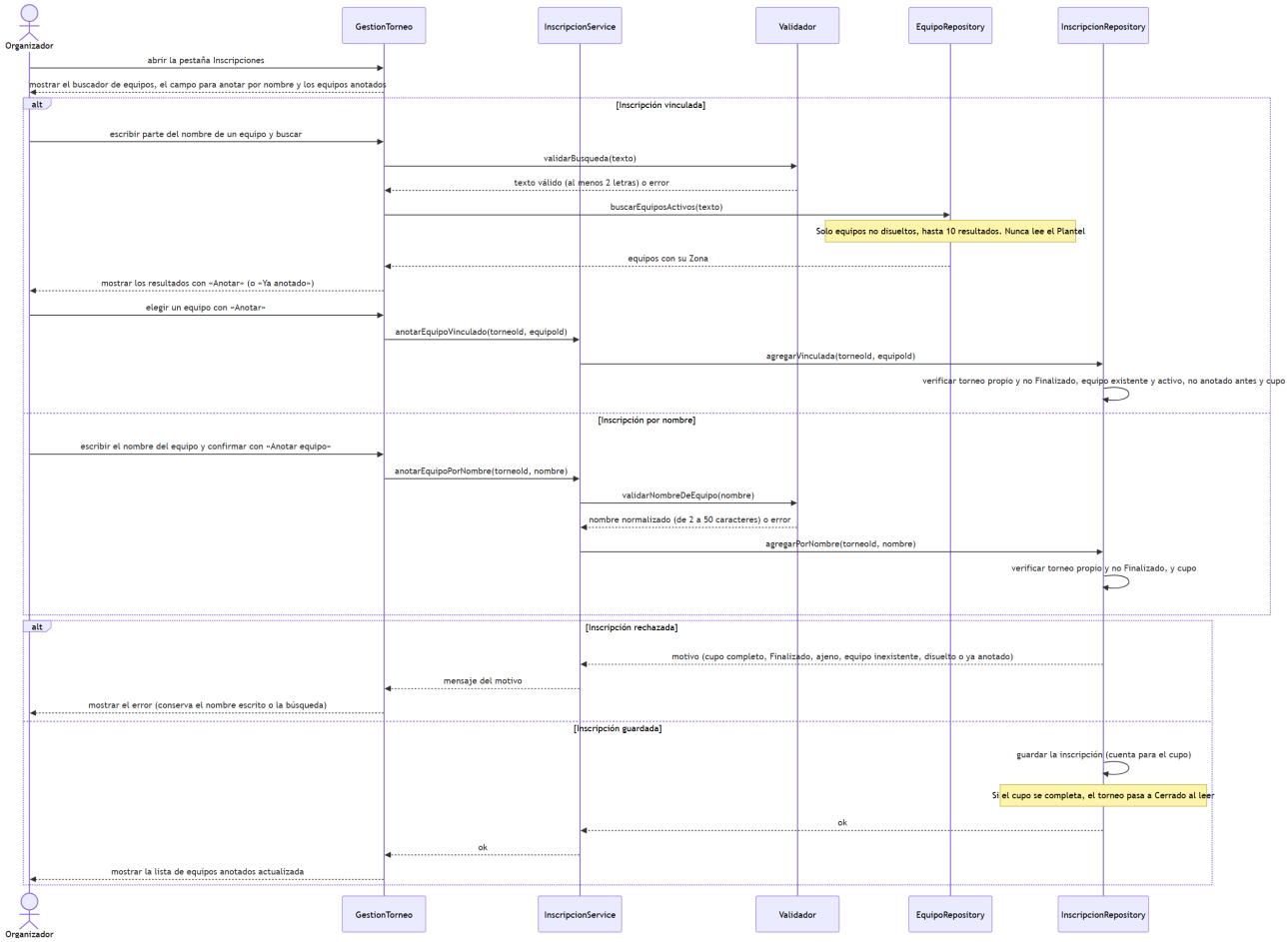
UC-E06 · Cargar una inscripción

- **Actor:** Organizador del torneo.
- **Requerimientos:** RF-E23 a RF-E30. **Pantalla:** P3 · Gestión del torneo (pestaña Inscripciones). **Diagrama:** S06.
- **Precondiciones:** ya arregló todo con el capitán **por fuera de la app**. El torneo no está Finalizado. Hay lugar en el cupo.
- **Postcondición:** si el cupo se completó, el torneo pasa a **Cerrado**.
- **Estado:** Implementado (`addInscription` , `addLinkedInscription` , `TeamSearch`).

Diagrama de secuencia · **S06 · UC-E06 — Cargar una inscripción**

Caso de uso: UC-E06 · Pantalla: P3 (pestaña Inscripciones) · Requerimientos: RF-E23 a RF-E30

No hay desplegables: el equipo vinculado se elige de **resultados de una búsqueda por nombre** (un campo de texto), no de una lista fija. Los resultados salen de la base, así que los provee `EquipoRepository`. Los dos tipos de inscripción son **un solo caso de uso** y van en el mismo diagrama.



Cómo se lee

- **La inscripción ya se arregló afuera.** El capitán le escribió al organizador por WhatsApp o Instagram (UC-B04); acá el organizador solo **registra** lo acordado. El capitán **no confirma** la vinculación y el organizador **no ve el WhatsApp** del capitán.
- **Dos tipos.** La **vinculada** busca un equipo de la app (el resultado muestra la **Zona**, porque los nombres se repiten) y solo ella genera Avisos y figura en la historia del equipo. La **por nombre** es para un equipo que no está en la app. **Las dos cuentan para el cupo** y aparecen por nombre en la ficha pública.
- **Se valida de nuevo al guardar.** Que el resultado aparezca en la pantalla no garantiza que el equipo siga activo o libre: las reglas las comprueba la función de la base, que es la única que escribe en las inscripciones. El `torneoId` y el `equipoId` vienen del cliente y no son de confiar.
- **Los rechazos, por su código:** cupo completo («Subí el cupo desde la pestaña "Datos"…»), torneo Finalizado, torneo ajeno o inexistente, equipo inexistente, **equipo disuelto** (no recibe inscripciones) y **equipo ya anotado** (como mucho una vinculada por equipo y torneo). Un error desconocido da un mensaje genérico.
- **Torneo Cerrado o En curso:** sí admite inscripciones; el organizador sigue acomodando la lista. Un Finalizado no.

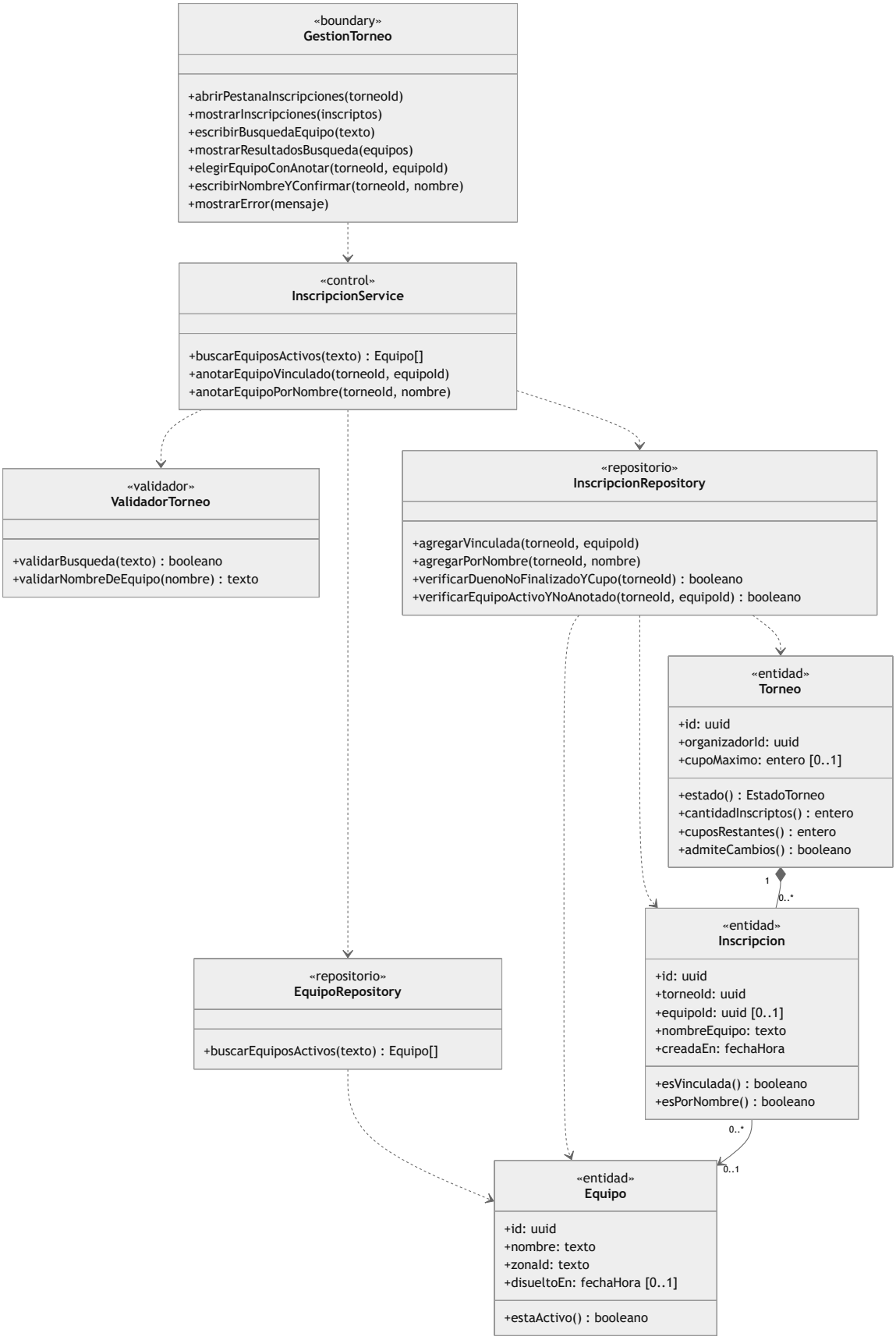
- **Si falla una vinculada,** la pantalla vuelve a la pestaña **conservando la búsqueda**, para que el error se vea al lado de lo que se eligió. Con el equipo anotado, la búsqueda cumplió y vuelve a la lista limpia.
- **Funciona sin JavaScript:** el buscador es un formulario GET (`?q=`) y «Anotar» es un formulario con los ids en campos ocultos.

Correspondencia en el código

`addInscription` y `addLinkedInscription` (`src/app/mis-torneos/[id]/inscripciones/actions.ts`) llaman a las RPC `add_inscription` y `add_linked_inscription` ; el buscador es `TeamSearch` (con `parseTeamQuery`) y el nombre se valida con `parseTeamName` (`utils/inscriptions.ts`). Probado en `inscriptions_test.sql` y `linked_inscriptions_test.sql` .

Diagrama de clases · **UC-E06 · Cargar una inscripción**

Caso de uso: UC-E06 · **Pantalla:** P3 · Gestión del torneo (pestaña Inscripciones) · **Secuencia:** S06



> Pantalla · **P3 · Gestión del torneo**

Es la misma pantalla que se describe, con su captura, en P3 · Gestión del torneo, debajo de UC-E02 · Ver mis torneos (Panel de organizador). Este caso usa la parte que indica su ficha («Pantalla»).

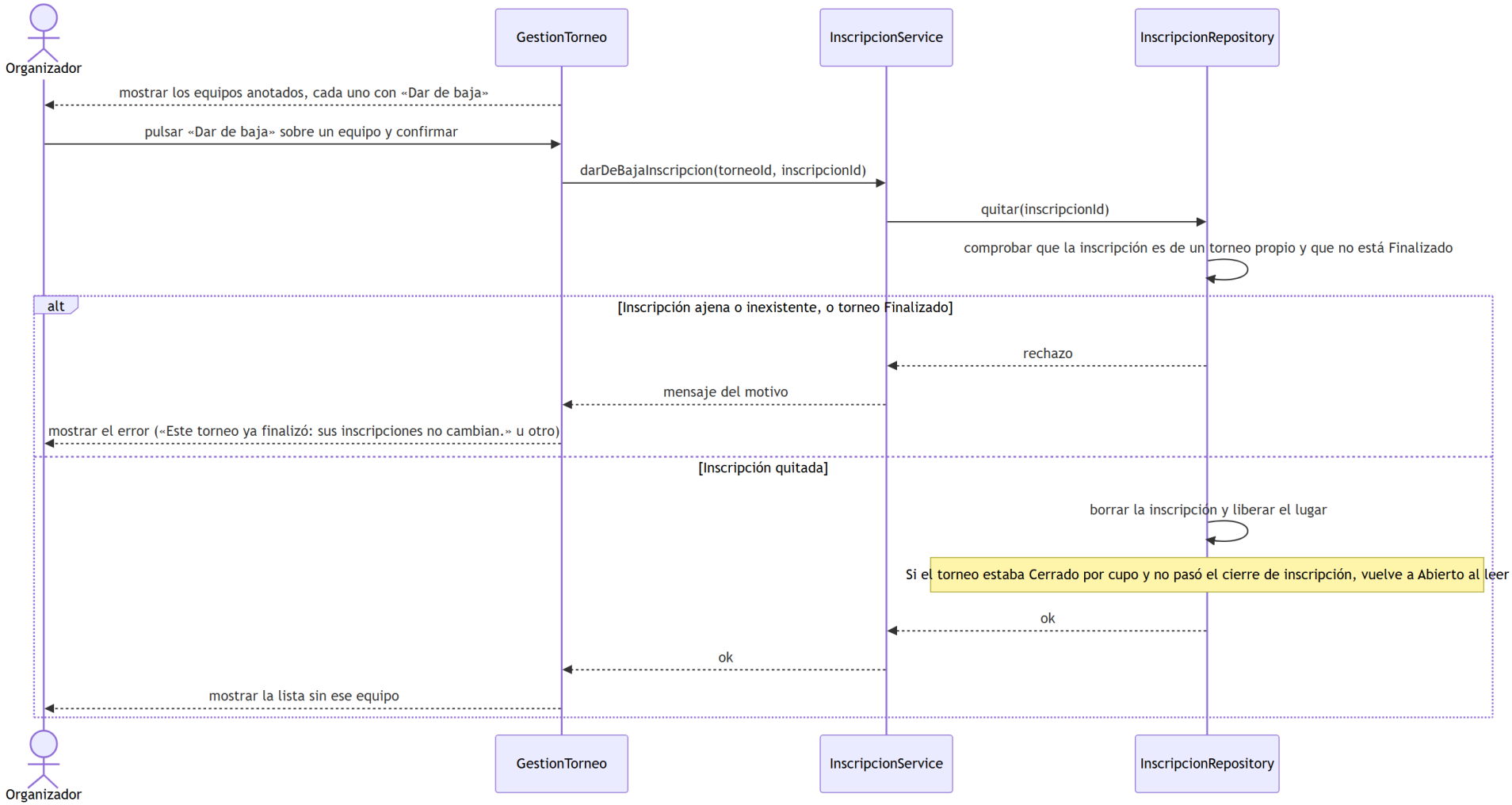
UC-E07 · Dar de baja una inscripción

- **Actor:** Organizador del torneo.
- **Requerimientos:** RF-E28, RF-E31. **Pantalla:** P3 · Gestión del torneo (pestaña Inscripciones). **Diagrama:** S07.
- **Precondiciones:** el torneo no está Finalizado.
- **Postcondición:** si el torneo estaba Cerrado por cupo y no pasó el cierre de inscripción, vuelve a **Abierto**.
- **Estado:** Implementado (`removeInscription` , función `remove_inscription`).

Diagrama de secuencia · **S07 · UC-E07 — Dar de baja una inscripción**

Caso de uso: UC-E07 · Pantalla: P3 (pestaña Inscripciones) · Requerimientos: RF-E28, RF-E31

No hay desplegables. Es un botón en cada equipo anotado, que es un formulario con los ids en campos ocultos (funciona sin JavaScript).



Cómo se lee

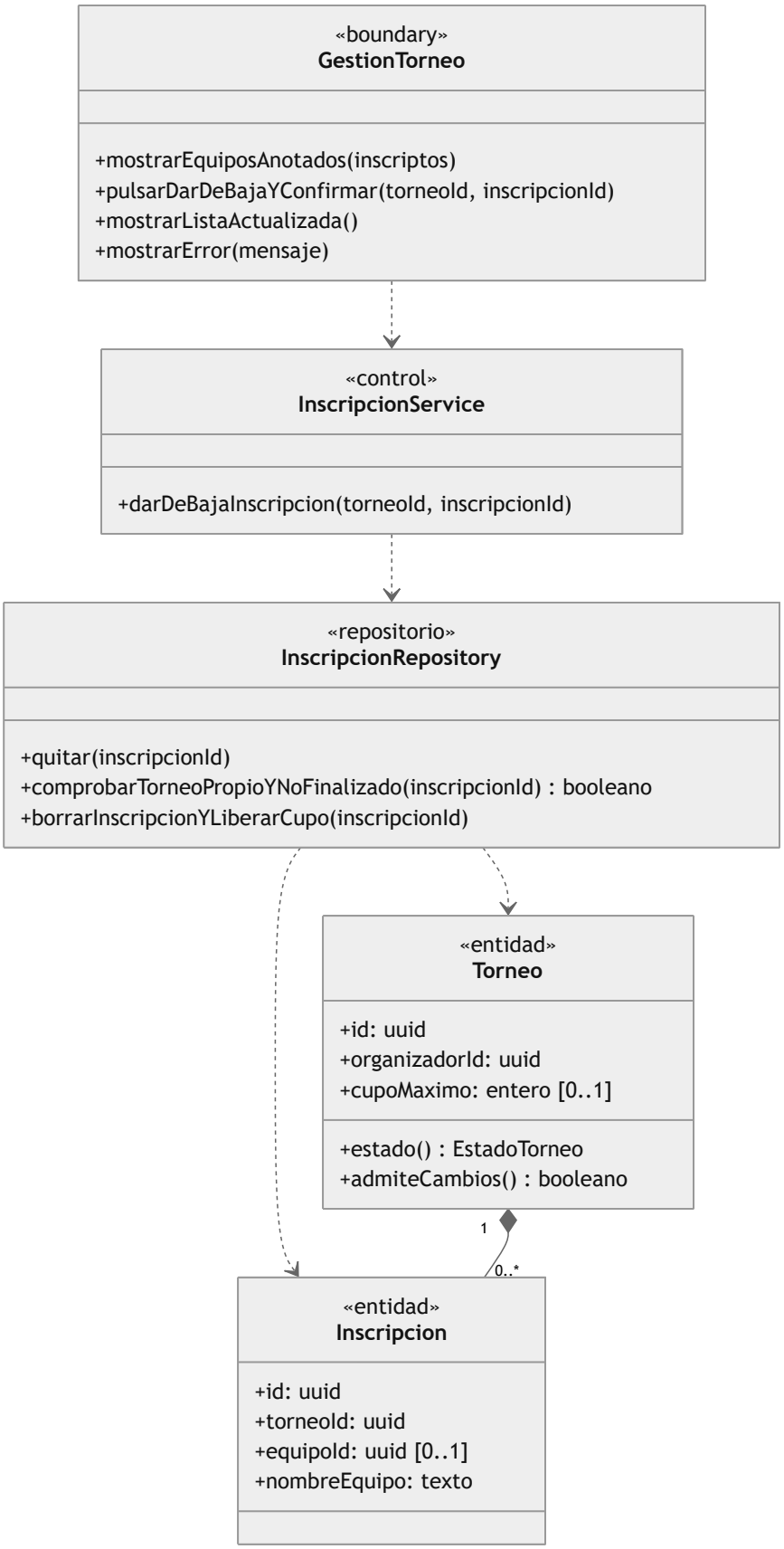
- **Libera el lugar:** el equipo deja de figurar en el torneo (y en su ficha pública) y el cupo vuelve a tener un lugar. Si el torneo estaba **Cerrado por cupo** y todavía no pasó el cierre de inscripción, **vuelve a Abierto**: el Estado se calcula al leer.
- **Un torneo Finalizado no cambia sus inscripciones.** Aunque el botón no se ofrezca, el torneo puede terminar entre que se cargó la página y se pulsó: la base lo vuelve a comprobar.
- **Los ids vienen del cliente y no son de confiar:** que la inscripción sea de un torneo del organizador lo decide la función de la base.
- **No avisa a nadie.** Dar de baja una inscripción no genera Avisos: solo la baja del torneo avisa a los planteles.

Correspondencia en el código

`removeInscription` (`src/app/mis-torneos/[id]/inscripciones/actions.ts`) llama a la RPC `remove_inscription` y, si falla, vuelve a la pestaña con el código del error en la URL.

Diagrama de clases · **UC-E07 · Dar de baja una inscripción**

Caso de uso: UC-E07 · **Pantalla:** P3 · Gestión del torneo (pestaña Inscripciones) · **Secuencia:** S07



> Pantalla · **P3 · Gestión del torneo**

Es la misma pantalla que se describe, con su captura, en P3 · Gestión del torneo, debajo de UC-E02 · Ver mis torneos (Panel de organizador). Este caso usa la parte que indica su ficha («Pantalla»).

Parte 3 · Anexos y trazabilidad

Guía de pantallas: estado de las capturas y mapa

Versión del **04/11/2026**. Cada pantalla del Panel de organizador, con su ruta, el caso, los requerimientos, lo que muestra y la captura. Cada una va debajo del caso de uso que la usa por primera vez; si otro caso usa la misma pantalla, remite a esa descripción.

Estado de las capturas

Marca	Significa
✅ **Vigente**	Sacada de `main` con la UI actual.
⏳ **Pendiente**	Todavía no hay captura.

Hoy no hay ninguna captura de este módulo. Todas sus pantallas exigen una sesión (y la base de desarrollo, que no respondía): sin sesión, `/crear-torneo` y `/mis-torneos` redirigen a Iniciar sesión. Al final está la lista de capturas con los pasos para sacarlas.

Mapa de pantallas

Pantalla	Ruta	Casos de uso	Sesión
P1 · Mis torneos	`/mis-torneos`	UC-E02, UC-E04	Obligatoria
P2 · Publicar torneo	`/crear-torneo`	UC-E01	Obligatoria
P3 · Gestión del torneo	`/mis-torneos/{id}` (pestañas `?tab=datos` e `?tab=inscripciones`)	UC-E02 a UC-E07	Dueño del torneo

`/mis-torneos/{id}/editar` y `/mis-torneos/{id}/inscripciones` ya no son pantallas: redirigen a la pestaña correspondiente de P3. Los tres paneles son **el Panel de organizador en tres niveles**: la lista (P1), la gestión (P3) y la ficha pública (la del módulo B, con la barra «Este torneo es tuyo · Gestionar»).

Capturas pendientes

Convención: PNG en `docs/organizador/pantallas/`, ventana de unos 1280 px de ancho, con datos de prueba. **Hace falta una cuenta con sesión, la base de desarrollo disponible y al menos dos torneos** (uno vigente y uno finalizado).

Archivo	Pantalla y estado	Cómo conseguirlo
`mis-torneos.png`	P1 con torneos	Un organizador con torneos de distintas antigüedades (incluido uno con 30 días o más sin verificar, un Finalizado y uno «ya confirmado hoy»).
`mis-torneos-vacio.png`	P1 sin torneos	Una cuenta que nunca publicó.
`publicar-torneo.png`	P2 vacío	Abrir `/crear-torneo` con sesión.
`publicar-torneo-error.png`	P2 con un error	Enviar con el cierre de inscripción posterior al inicio.
`publicar-torneo-zona.png`	P2 con el desplegable de Zona abierto	Abrir el desplegable «Zona» para ver los corredores.

Archivo	Pantalla y estado	Cómo conseguirlo
`gestion-datos.png`	P3, pestaña Datos	Abrir «Gestionar» de un torneo.
`gestion-inscripciones.png`	P3, pestaña Inscripciones	Con equipos anotados (vinculados y por nombre) y con una búsqueda de equipo con resultados.
`gestion-cupo-completo.png`	P3 con el error de cupo completo	Intentar anotar con el cupo lleno.
`gestion-baja.png`	La confirmación de «Dar de baja»	Pulsar «Dar de baja» (sin aceptar).
`gestion-finalizado.png`	P3 de un torneo Finalizado	Para mostrar «Finalizó: ya no se confirma».

Diagramas de secuencia: criterios, participantes y convenciones

Versión del **04/11/2026**. Un diagrama por caso de uso, en Mermaid (GitHub los dibuja directo). Cada uno va debajo de su caso de uso, antes de su pantalla.

Diagrama	Caso de uso	Pantalla	Estado
S01 · Publicar un torneo	UC-E01	P2	Parcial (Suspensión)
S02 · Ver mis torneos	UC-E02	P1	Implementado
S03 · Editar un torneo	UC-E03	P3	Implementado
S04 · Verificar un torneo	UC-E04	P1 y P3	Implementado
S05 · Dar de baja un torneo	UC-E05	P3	Implementado
S06 · Cargar una inscripción	UC-E06	P3	Implementado
S07 · Dar de baja una inscripción	UC-E07	P3	Implementado

Los PNG están dibujados sobre fondo blanco, para pegarlos en un informe. La fuente de cada diagrama es el bloque Mermaid de su archivo: si se cambia uno, hay que regenerar su PNG.

Relación con los diagramas generales. `docs/diagramas-de-secuencia/diagramas-secuencia.md` tiene diagramas de todo el proyecto con participantes genéricos. Los de este módulo los **detallan**: D23 a D28 corresponden a S01, S03, S02, S04, S05 y S06 (en ese orden: publicar, editar, consultar, verificar, dar de baja y cargar inscripción); S07 no tiene equivalente. Los generales no muestran la carga de las Zonas y las modalidades de los formularios.

Criterios aplicados

Son los de la guía para diagramas con formularios y desplegables.

1. **Un caso de uso por diagrama.** La carga de las opciones de un formulario y el alta o la edición van **en el mismo diagrama**.
2. **Si el caso empieza con un formulario con desplegables**, el diagrama muestra al actor abriendo la pantalla y a la pantalla pidiendo las opciones de cada desplegable **antes** de que el actor elija. Pasa en **publicar** (S01) y **editar** (S03): comparten el formulario, con la **Zona** y la **modalidad**.
3. **Un objeto por responsabilidad o fuente de datos**, no uno por desplegable. Las Zonas se consultan en la base: `ZonaRepository`. La modalidad es una lista fija del código: `OpcionesTorneo`. No se inventa un repositorio para las modalidades.

4. **Al enviar**, el método del caso de uso recibe **solo los valores elegidos** (`zonaId` , `modalidad`), no las listas.
5. **La validación se repite al guardar**: una opción mostrada no garantiza que siga siendo válida (la Zona pudo dejar de existir, o el pedido pudo armarse a mano). Y **quién es el dueño, el Estado y el cupo los comprueba la base** en cada acción.
6. **Cada diagrama muestra solo los participantes que intervienen.**

Participantes: responsabilidad y correspondencia en el repo

Son responsabilidades conceptuales. No existen como clases TypeScript con esos nombres.

Participante	Responsabilidad	Correspondencia en el repo
Usuario, Organizador	La persona que inicia el caso.	La sesión; Organizador es quien publicó un torneo.
FormularioTorneo	El formulario de torneo (alta y edición).	`components/TournamentForm.tsx` , en `crear-torneo/page.tsx` y `mis-torneos/[id]/DatosTab.tsx` .
PanelTorneos	La lista «Mis torneos».	`mis-torneos/page.tsx` , con `VerifyButton` .
GestionTorneo	La pantalla de gestión con sus pestañas.	`mis-torneos/[id]/page.tsx` , `DatosTab.tsx` , `InscripcionesTab.tsx` , `TeamSearch.tsx` , `WithdrawButton.tsx` .
ZonaRepository	Entrega la lista cerrada de Zonas, **consultada en la base**.	`getZoneGroups()` en `ZoneSelect.tsx` : lee `zones` con corredores y regiones.
OpcionesTorneo	Entrega las modalidades: **lista fija del código**.	`SPORT_MODALITIES` (`utils/modality.ts`).
TorneoService	Coordina los casos de uso del torneo.	Las Server Actions de `crear-torneo/actions.ts` , `mis-torneos/actions.ts` y `mis-torneos/[id]/editar/actions.ts` .
InscripcionService	Coordina cargar y dar de baja inscripciones.	`mis-torneos/[id]/inscripciones/actions.ts` .
Validador	Valida entradas antes de escribir.	`parseTournamentForm` , `parseFlyer` , `parseTeamName` , `parseTeamQuery` (`utils/`).
TorneoRepository	Lee y escribe torneos y su flyer, y ejecuta las transiciones (verificar, dar de baja).	Tabla `tournaments` con RLS, las funciones `verify_tournament` y `withdraw_tournament` , y Storage (bucket `flyers` , `utils/supabase/flyer.ts`).
InscripcionRepository	Lee y escribe las inscripciones.	Tabla `inscriptions` (solo por las funciones `add_inscription` , `add_linked_inscription` y `remove_inscription`).
EquipoRepository	Busca equipos activos por nombre.	Tabla `teams` (lectura pública).

Convenciones de dibujo

- Flecha continua (`->>`): una llamada. Flecha punteada (`-->>`): su respuesta.
- Un mensaje de un participante a sí mismo es un paso interno (por ejemplo, una verificación dentro de una función de Postgres).
- Una nota sobre un participante marca una regla que **hace cumplir la base**.
- Entre paréntesis y en minúscula van los **valores que viajan**; entre comillas «así», los mensajes que ve la persona.
- `opt` es un paso opcional; `alt` / `else` , caminos que se excluyen.

Trazabilidad y pendientes

Documentación completa del módulo E (Organizador) de Torneos Amateur BA, versión del **04/11/2026**. En este documento, sus piezas son la Parte 1 (requerimientos), la Parte 2 (cada caso de uso con su diagrama y su pantalla) y los anexos de esta parte.

Fuentes, en orden de autoridad: decisiones-casos-de-uso.md → casos-de-uso.md → CONTEXT.md → código y migraciones.

Matriz de trazabilidad

Caso de uso	Requerimientos	Pantalla	Diagrama	Estado
UC-E01 Publicar un torneo	RF-E01 a E08	P2 `/crear-torneo`	S01	**Parcial** (Suspensión)
UC-E02 Ver mis torneos	RF-E12 a E15	P1 `/mis-torneos` , P3	S02	Implementado
UC-E03 Editar un torneo	RF-E02 a E06, E09 a E11	P3 pestaña Datos	S03	Implementado
UC-E04 Verificar que sigue vigente	RF-E16 a E18	P1 y P3	S04	Implementado
UC-E05 Dar de baja un torneo	RF-E19 a E22	P3	S05	Implementado
UC-E06 Cargar una inscripción	RF-E23 a E30	P3 pestaña Inscripciones	S06	Implementado
UC-E07 Dar de baja una inscripción	RF-E28, E31	P3 pestaña Inscripciones	S07	Implementado

Equivalencia con el PDF original

Vigente	PDF original
UC-E01 Publicar un torneo	SPEC-E01
UC-E02 Ver mis torneos	SPEC-E02
UC-E03 Editar un torneo	SPEC-E01
UC-E04 Verificar que sigue vigente	SPEC-E02
UC-E05 Dar de baja un torneo	SPEC-E02
UC-E06 Cargar una inscripción	SPEC-E03 (el flujo donde el capitán se anotaba desde la app se retiró)
UC-E07 Dar de baja una inscripción	SPEC-E03

Hallazgos y pendientes

Este es **el módulo más completo**: todos los casos están implementados. Las observaciones son menores.

1. **La Suspensión no se verifica al publicar** (UC-E01). `casos-de-uso.md` pone «sin Suspensión» como precondición; la Suspensión no existe todavía (UC-F05). Por eso el caso es **Parcial**.
2. **Dos reglas del código no están documentadas en `casos-de-uso.md` ni en las decisiones:** «Sigue vigente» se apaga si el torneo se confirmó hace menos de 24 h, y editar un torneo con equipos anotados pide confirmación. Hay que decidir si se documentan como reglas del producto (los dejé en los requerimientos como implementados).
3. **La marca Observado todavía no existe**, así que «Sigue vigente no la saca» es una regla sin efecto hasta que se implemente UC-F03.
4. **El formulario de torneo y el de equipo siguen el mismo patrón** (Zona de la base, modalidad de una lista del código): los diagramas S01 y S03 aplican la guía de desplegables.

5. **Los diagramas generales (D23 a D28)** no muestran la carga de las Zonas y las modalidades; los de este módulo sí.
6. **Capturas: no hay ninguna.** Las tres pantallas exigen sesión y la base de desarrollo no respondía. La lista, con los pasos, está en 03-pantallas.md.
7. **Tests de base:** hay pgTAP para RLS, estado, formulario, actualización, verificación, baja, zona, flyers, inscripciones e inscripciones vinculadas (`tournaments_*_test.sql` , `inscriptions_test.sql` , `linked_inscriptions_test.sql` , `tournament_flyers_test.sql`).
8. **Si se cambia un diagrama,** hay que regenerar su PNG (el bloque Mermaid de cada archivo es la fuente).

Diagramas de clases: trazabilidad de las clases

Clase	Estereotipo	Implementación en el repo	Estado
`FormularioTorneo`	`«boundary»`	`src/components/TournamentForm.tsx` en `src/app/crear-torneo/page.tsx` y `src/app/mis-torneos/[id]/DatosTab.tsx`	Implementada
`PanelTorneos`	`«boundary»`	`src/app/mis-torneos/page.tsx` + `src/components/VerifyButton.tsx`	Implementada
`GestionTorneo`	`«boundary»`	`src/app/mis-torneos/[id]/page.tsx` , `DatosTab.tsx` , `InscripcionesTab.tsx` , `TeamSearch.tsx` , `WithdrawButton.tsx`	Implementada
`TorneoService` / `InscripcionService` / `OpcionesTorneo`	`«control»`	Server Actions en `src/app/crear-torneo/actions.ts` , `src/app/mis-torneos/actions.ts` , `src/app/mis-torneos/[id]/editar/actions.ts` , `inscripciones/actions.ts` y `SPORT_MODALITIES` (`src/utils/modality.ts`)	Implementados
`ValidadorTorneo`	`«validador»`	`parseTournamentForm` , `parseFlyer` (`src/utils/tournament-form.ts`), `parseTeamName` , `parseTeamQuery` (`src/utils/inscriptions.ts`)	Implementado
`ZonaRepository`	`«repositorio»`	`getZoneGroups()` en `src/components/ZoneSelect.tsx` (`public.zones` , `corridors` , `regions`)	Implementado
`TorneoRepository`	`«repositorio»`	Tabla `public.tournaments` con RLS, funciones `verify_tournament` y `withdraw_tournament` (`20261028000016`), bucket `flyers` (`src/utils/supabase/flyer.ts`)	Implementado
`InscripcionRepository`	`«repositorio»`	Tabla `public.inscriptions` , funciones `add_inscription` , `add_linked_inscription` , `remove_inscription` (`20261001000000_inscriptions.sql` , `20261028000002_teams.sql`)	Implementado
`EquipoRepository`	`«repositorio»`	Tabla `public.teams` (búsqueda de equipos activos por nombre)	Implementado
`Torneo` / `Inscripcion` / `Equipo` / `Zona` / `Aviso`	`«entidad»`	`tournaments` , `inscriptions` , `teams` , `zones` , `notices`	Implementadas

Módulo F · Administración

Primero los requerimientos del módulo completo. Después, el modelo de dominio y, para cada caso de uso, su diagrama de secuencia, su diagrama de clases y su pantalla. Al final, los anexos y la trazabilidad.

Parte 1 · Requerimientos

Versión del **04/11/2026**. Es la primera parte del módulo: sigue la sección de casos de uso, donde cada caso lleva debajo su diagrama de secuencia y su pantalla. El README tiene la matriz que los une.

Fuentes, en orden de autoridad: decisiones-casos-de-uso.md, casos-de-uso.md (módulo F, 03/11/2026), acceso-administrador.md (etapa del 14/10/2026), el glosario CONTEXT.md y el código y las migraciones del repo al día de hoy.

1. Alcance

El módulo es el **backoffice del equipo de Torneos BA**: el **Administrador** entra por un acceso propio, ve un resumen de la plataforma y —cuando se implementen las etapas siguientes— **modera**: resuelve reportes de torneos y de equipos, suspende cuentas y busca usuarios.

Hoy está implementada solo la primera etapa: el acceso diferenciado, el marco del backoffice (sidebar), el resumen con contadores reales y la pantalla «Seguridad y acceso». **La moderación (UC-F03 a UC-F06) está definida pero sin programar:** no hay tablas, funciones ni pantallas de reportes, suspensiones ni registro de acciones.

Límites de producto:

- El Administrador **no arbitra partidos**: los conflictos se resuelven solos, sin árbitro (ADR 0002). Por eso no hay ninguna pantalla de partidos en el backoffice.
- **Administrador y Organizador son roles distintos**: el organizador no tiene ningún poder de administración.
- **El rol nunca se asigna desde la app.**

Fuera del módulo:

Tema	Dónde está
Crear una cuenta y el acceso de las personas comunes	Módulo A — Cuenta y acceso
Generar los reportes (botón «Reportar»)	Módulos B (torneos) y C (equipos), ambos pendientes
Qué hace la Suspensión sobre los partidos y las búsquedas	Módulos C y D (la Suspensión los cancela)

2. Actores

Actor	Quién es	Qué hace en este módulo
Administrador	Integrante del equipo de Torneos BA con una fila en `admins`	Entra al backoffice, mira el resumen y (a futuro) modera.
Usuario	Cualquier persona con cuenta	Si intenta entrar al backoffice sin el rol, recibe acceso denegado y **conserva su sesión normal**.
Sistema	La app y la base de datos	Verifica el rol en el servidor y calcula los contadores.

3. Requerimientos funcionales

Estado: `Implementado` está en `main`. `Pendiente` es alcance vigente sin programar.

Acceso y rol

ID	Requerimiento	Casos de uso	Estado
RF-F01	**Entrada diferenciada:** `/admin/login` es un formulario público (email y contraseña), **sin registro ni invitado**. Usa la **misma cuenta de Supabase** que el sitio y después consulta `is_admin()` con esa sesión.	UC-F01	Implementado
RF-F02	**El rol se asigna a mano:** una persona autorizada inserta el usuario en `public.admins` desde el SQL Editor. No hay registro de administradores ni selección de rol en ningún formulario; no se usan `user_metadata`, un campo del perfil ni nada que venga del cliente.	UC-F01	Implementado
RF-F03	**Solo el rol habilita el acceso.** Credenciales inválidas, cuenta sin rol, error al verificar el rol o un resultado que no sea exactamente `true` **no habilitan** el backoffice. Cada caso tiene su mensaje: «No pudimos iniciar sesión. Revisá tu email y contraseña.», «Tu cuenta no tiene acceso al Panel de administración.» y «No pudimos verificar el acceso de administrador. Probá de nuevo.».	UC-F01	Implementado
RF-F04	Una **cuenta común** que intenta entrar conserva su sesión normal en el sitio, sin adquirir ningún permiso.	UC-F01	Implementado
RF-F05	**Toda ruta `/admin/…` exige sesión y rol**, salvo el login exacto. Sin sesión, se redirige a `/admin/login` conservando el destino interno (con sus parámetros); con sesión sin rol, a `/admin/login` con «acceso denegado». El destino de regreso se limita al backoffice y excluye el propio login, para evitar bucles.	UC-F01	Implementado
RF-F06	**El rol se verifica en dos capas:** el control de acceso de la ruta (`proxy`) y, además, cada página y el layout del backoffice (`requireAdmin()`), que **no se fían** de la primera.	UC-F01	Implementado
RF-F07	Con una sesión de administrador ya abierta, `/admin/login` no muestra el formulario: lleva al destino.	UC-F01	Implementado

Marco y resumen

ID	Requerimiento	Casos de uso	Estado
RF-F08	El backoffice tiene **su propio marco**: sidebar propia, **sin la barra ni el footer del catálogo**. Arriba, «Volver al sitio»; al pie, la cuenta y «Cerrar sesión» (solo el dispositivo actual). En el celular, la navegación se despliega y se cierra al elegir una sección o con Escape.	UC-F02	Implementado
RF-F09	El **resumen** muestra los **contadores reales** de torneos, equipos y usuarios. Una consulta fallida o sin conteo muestra «No disponible», **nunca un cero inventado**, y un aviso «No pudimos cargar todos los contadores».	UC-F02	Implementado
RF-F10	**Seguridad y acceso** muestra la cuenta actual, que su rol fue verificado en el servidor, que el rol se asigna a mano en la base y los controles de esta etapa.	UC-F02	Implementado
RF-F11	Las secciones **Usuarios, Torneos y equipos, Reportes, Registro de actividad, Soporte, Avisos y Configuración** figuran en la sidebar como **pendientes, sin enlaces ni acciones ficticias**.	UC-F02	Implementado

Moderación

ID	Requerimiento	Casos de uso	Estado
RF-F12	**Resolver un reporte de torneo** con una de tres resoluciones: descartar, marcar el torneo como **Observado**, o darlo de baja (con las mismas consecuencias que la baja del organizador, incluidos los Avisos a los planteles vinculados).	UC-F03	**Pendiente**
RF-F13	La marca **Observado** se ve en el catálogo y la ficha, y **solo el administrador la saca**: «Sigue vigente» del organizador no la toca.	UC-F03	**Pendiente**
RF-F14	**Resolver un reporte de equipo:** el administrador revisa la página del equipo y los comentarios privados de calificación que recibió, y elige **descartar** o **suspender al Capitán**, que responde por la conducta del Plantel.	UC-F04	**Pendiente**
RF-F15	**Suspender una cuenta** por 7 días, 30 días o indefinida, con un **motivo**. El usuario recibe un Aviso con el motivo y un email de contacto; no hay apelación dentro de la app.	UC-F05	**Pendiente**
RF-F16	**Efectos de la Suspensión:** la persona puede entrar y mirar, pero no publicar torneos, crear equipos, buscar rival ni recibir la capitanía. Sus torneos siguen visibles y los puede editar y verificar. Sus búsquedas se cancelan y las propuestas de su equipo se anulan, con aviso. Sus partidos futuros **se caen** con el motivo «Equipo suspendido», sin falta para nadie. Los partidos pendientes de resultado los puede terminar de cargar. Su equipo no aparece entre los compatibles, su página sigue visible y **no puede cambiar de capitán** mientras dure.	UC-F05	**Pendiente** (hoy la base solo contempla que un suspendido no reciba la capitanía, y ni eso lo verifica)
RF-F17	La **Suspensión no es el Bloqueo por faltas**: la Suspensión recae sobre una cuenta y la decide el administrador; el Bloqueo por faltas recae sobre un equipo y es automático.	UC-F05	Regla vigente
RF-F18	**Registro de acciones:** cada resolución del administrador (reporte descartado, torneo observado o dado de baja, cuenta suspendida) queda registrada con **quién, cuándo y qué decidió**.	UC-F03 a UC-F05	**Pendiente**
RF-F19	**Buscar usuarios** por nombre o email y abrir la cuenta para ver sus equipos, torneos y reportes, y suspenderla si corresponde.	UC-F06	**Pendiente**

4. Requerimientos no funcionales

ID	Requisito	Evidencia
RNF-F01	**La tabla `admins` no se puede leer ni escribir desde la API.** Tiene RLS sin policies; el rol se consulta solo con `is_admin()`, una función `security definer` que devuelve un booleano sin exponer los ids de los demás administradores.	`20260928000002_profiles.sql`, `20261007000001_is_admin_rpc.sql`
RNF-F02	**`is_admin()` la ejecutan solo usuarios con sesión.** Revocar `anon` no alcanza, porque cada rol conserva el permiso de `PUBLIC`: se revocan los dos.	`20261013000001_is_admin_permissions.sql`, `is_admin_test.sql`
RNF-F03	**El rol nunca se escribe desde el cliente** ni se decide con datos del formulario.	`authenticateAdmin` (`utils/supabase/admin-login.ts`)
RNF-F04	**Dos capas de autorización**, ninguna sustituida por la presentación: el marco del backoffice (`AppFrame`) decide solo la presentación, no la autorización del servidor, de las acciones futuras ni las policies de RLS.	`proxy.ts`, `utils/admin.ts`, `AppFrame.tsx`
RNF-F05	**El destino de regreso es siempre interno** y, en el acceso de administrador, limitado al backoffice.	`admin-access.ts` y `admin-access.test.ts`

ID	Requisito	Evidencia
RNF-F06	**Al cliente solo llega la `anon key`.** La `service_role key` nunca sale de un contexto de servidor controlado.	`src/utils/supabase/`
RNF-F07	**Las cookies renovadas no se pierden al redirigir** desde el control de acceso.	`proxy.ts` (`redirectWithCookies`)
RNF-F08	**Datos honestos:** una consulta fallida muestra «No disponible», nunca un valor inventado. Y no hay módulos simulados: lo pendiente figura como pendiente.	`admin/(backoffice)/page.tsx`
RNF-F09	**Accesibilidad.** Campos con etiqueta y `autoComplete` , errores anunciados (`role="alert"`), sidebar con `aria-label` , `aria-current` en la sección actual y menú móvil con `aria-expanded` , cerrable con Escape.	`admin/login/page.tsx` , `AdminSidebar.tsx`
RNF-F10	**Pruebas:** la lógica del acceso (credenciales, sesión recién autenticada, cuenta sin rol, fallo de la verificación, destinos) y los permisos de `is_admin()` están cubiertos.	`admin-access.test.ts` , `is_admin_test.sql`
RNF-F11	**Cada etapa nueva se define antes de implementarse.** MFA, gestión de sesiones, permisos granulares, controles sobre usuarios, auditoría, soporte y parámetros de negocio requieren decidir su regla antes de crear tablas, permisos o acciones.	acceso-administrador.md

5. Sin decisión

Hay que resolver estas preguntas antes de implementar las etapas pendientes. Al decidir cada una, agregar la regla a decisiones-casos-de-uso.md.

Tema	Pregunta
Email de contacto de la Suspensión	El Aviso de suspensión «incluye un email de contacto». Falta definir cuál es.
Buscar usuarios por email (UC-F06)	El email vive en la autenticación (`auth.users`), no en `profiles` , que es lo que se lee desde la app. Buscarlo exige una función de servidor con permisos especiales; hay que definir cómo hacerlo sin exponer la `service_role key` ni los emails de todos.
Reportes repetidos	Si una misma persona puede reportar más de una vez al mismo equipo (para los torneos es una sola vez por usuario y torneo).
Comentarios de calificación	El administrador los lee al revisar un equipo reportado; falta definir cómo accede la pantalla, porque los comentarios son privados y la tabla de calificaciones no es de lectura pública.
Cuando vence una Suspensión	Qué pasa al vencer con lo que la Suspensión canceló: los partidos caídos por «Equipo suspendido» no se reactivan solos, y las búsquedas hay que volver a publicarlas. Falta confirmar que es lo esperado.
Varios administradores	No hay permisos granulares ni registro de quién es cada administrador más allá de la fila en `admins` .

6. Reglas de referencia

- **Administrador:** integrante del equipo de Torneos BA; el rol se asigna a mano en la base, nunca desde la app.
- **Suspensión:** sanción sobre una cuenta, por 7 días, 30 días o indefinida; impide crear o publicar cosas nuevas pero no entrar ni mirar. No confundir con el **Bloqueo por faltas**.
- **Observado:** marca que el administrador pone a un torneo reportado; la ven catálogo y ficha, y solo él la saca.
- **Términos:** «baja» (nunca «eliminar»); «Equipo disuelto»; el partido «se cae», nunca «se suspende».

Parte 2 · Modelo de dominio y casos de uso, con sus diagramas y su pantalla

Versión del **04/11/2026**. Detalla los casos de uso del módulo F de casos-de-uso.md (UC-F01 a UC-F06) con los requerimientos que cubre cada uno (01-requerimientos.md), la pantalla que lo representa (03-pantallas.md) y su diagrama de secuencia (04-diagramas-secuencia).

Cómo leerlo. Cada caso expresa una meta y tiene **un actor iniciador**. **Estado** dice si está en `main`. Los casos pendientes son alcance vigente: solo faltan programarse. Para ellos, el diagrama es un **diseño objetivo** basado en las reglas escritas.

Mapa de casos de uso

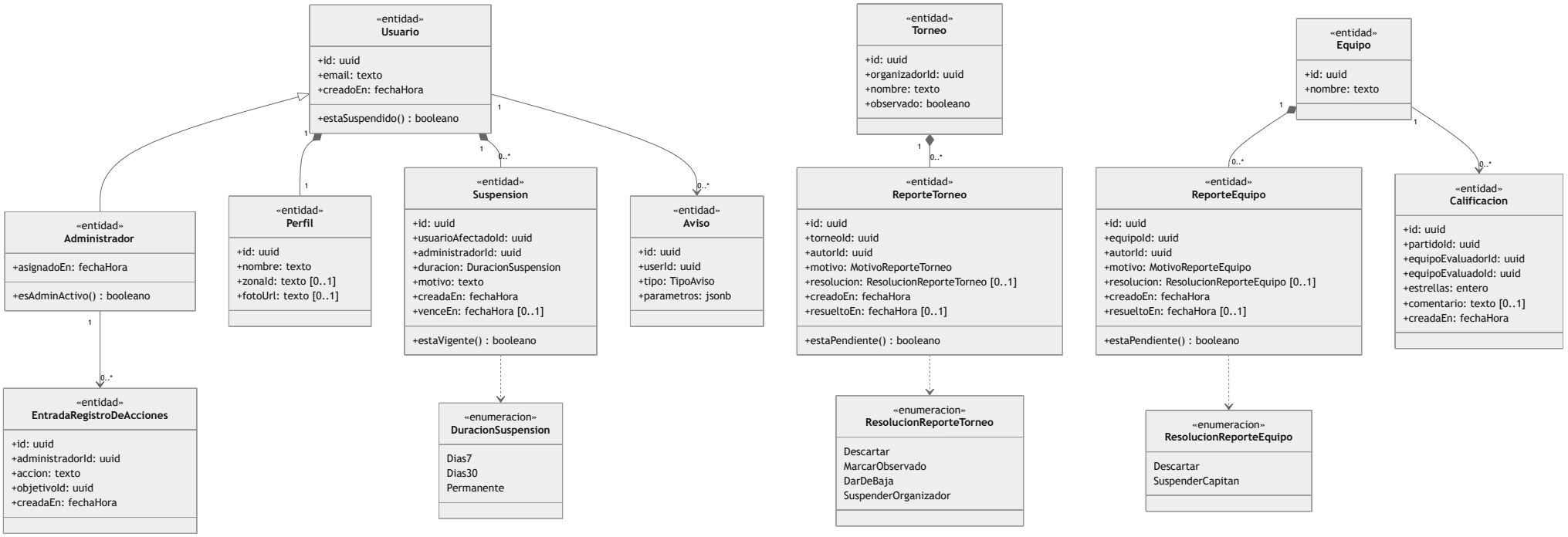
Caso	Meta del actor	Actor	Estado	ID en el PDF original
UC-F01	Acceder al Panel de administración	Administrador	Implementado	CU-F01
UC-F02	Consultar el tablero	Administrador	**Parcial** (resumen y seguridad)	—
UC-F03	Resolver un reporte de torneo	Administrador	**Pendiente**	CU-F03 y CU-F06 (registrar la resolución)
UC-F04	Resolver un reporte de equipo	Administrador	**Pendiente**	—
UC-F05	Suspender una cuenta	Administrador	**Pendiente**	CU-F05
UC-F06	Buscar usuarios	Administrador	**Pendiente**	—

La numeración de los casos del PDF original (CU-F01 a CU-F06) no coincide uno a uno con la vigente: el CU-F04 (arbitrar partidos) se retiró porque el administrador no arbitra partidos, y el tablero, el reporte de equipos y la búsqueda de usuarios son casos nuevos. Los diagramas generales de `diagramas-secuencia.md` D30 a D33 cubren acceder, resolver un reporte, suspender y registrar la resolución.

Qué no es un caso de uso acá: abrir «Seguridad y acceso», desplegar la sidebar en el celular o «Volver al sitio» no toman ninguna decisión; van en pantallas.

Diagrama de clases · **Modelo de dominio del módulo**

Vista estructural de las entidades, relaciones de herencia/composición y enumeraciones que comparten los casos de uso `UC-F01` a `UC-F06`.



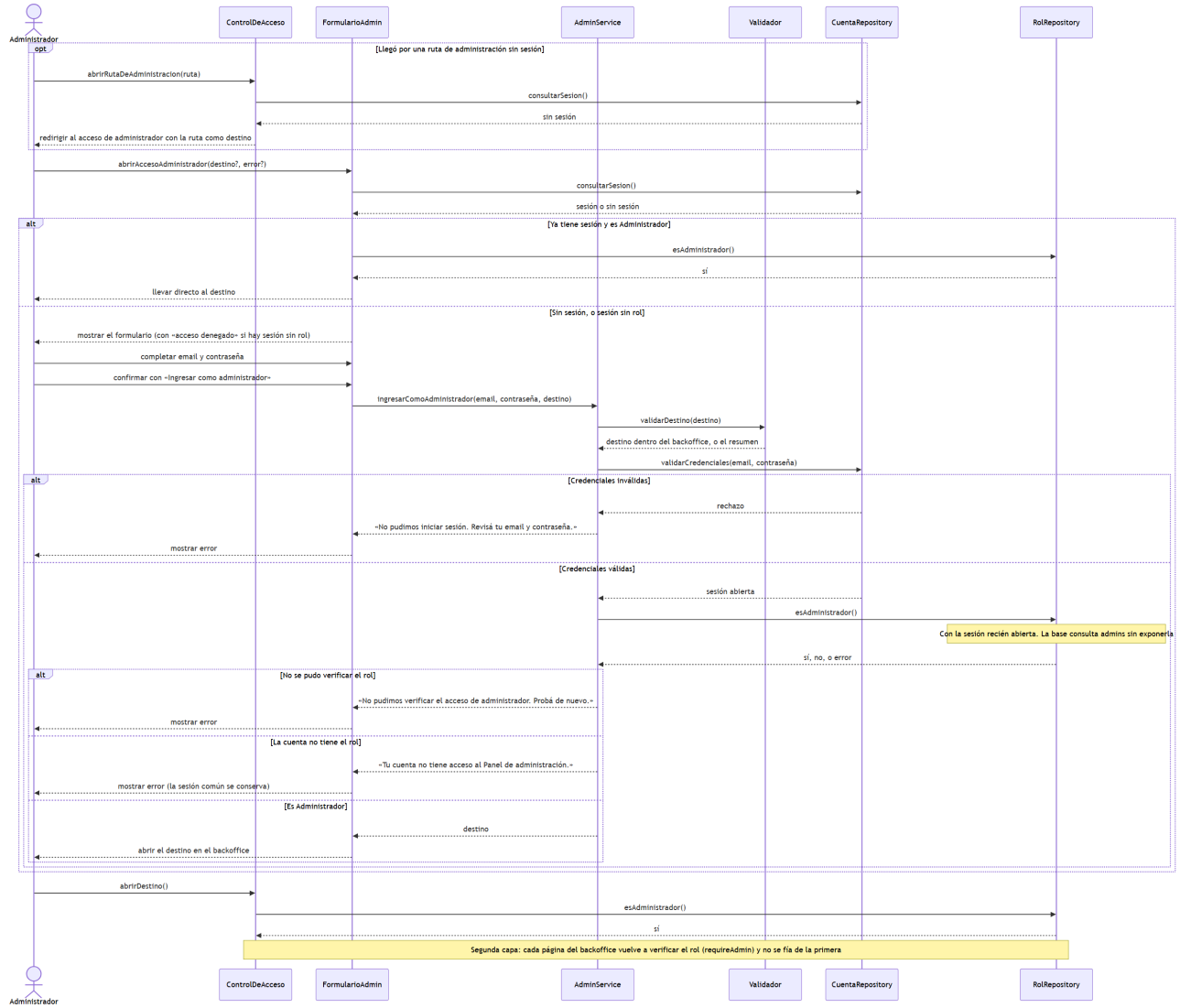
UC-F01 · Acceder al Panel de administración

- **Actor:** Administrador.
- **Requerimientos:** RF-F01 a RF-F07. **Pantalla:** P1 · Acceso de administrador (`/admin/login`). **Diagrama:** S01.
- **Precondiciones:** tiene una cuenta y el rol asignado en la base.
- **Postcondición:** sesión de administrador abierta en ese dispositivo. No existe una segunda sesión de administrador: es la misma cuenta.
- **Estado:** Implementado (`src/app/admin/login` , `src/utils/admin.ts` , `src/proxy.ts`).

Diagrama de secuencia · **S01 · UC-F01 — Acceder al Panel de administración**

Caso de uso: UC-F01 · Pantalla: P1 · Requerimientos: RF-F01 a RF-F07

El formulario tiene solo email y contraseña: no hay desplegables. El diagrama incluye las **dos capas** que verifican el rol y el caso en que la persona llega redirigida.



Cómo se lee

- **El rol no lo decide el formulario.** Ningún campo del formulario puede conceder el rol: se consulta `is_admin()` **con la sesión recién abierta**. El rol se asigna a mano, insertando el usuario en `admins` desde el SQL Editor; nunca desde la app, el perfil ni `user_metadata`.
- **Solo un `true` exacto habilita.** Credenciales inválidas, cuenta sin rol, error de la verificación o un resultado que no sea booleano **no habilitan** el backoffice. Cada caso tiene su mensaje.
- **Una cuenta común conserva su sesión normal**: el error de «acceso denegado» no la cierra, y la pantalla le ofrece seguir en el catálogo.
- **Dos capas de verificación.** `ControlDeAcceso` redirige las rutas `/admin/…` sin sesión o sin rol; pero cada página y el layout del backoffice vuelven a verificar, porque una petición directa o una página en caché podría saltearse la primera. La capa real es la segunda.
- **El destino es interno y limitado al backoffice.** Un destino externo, o el propio `/admin/login`, se reemplaza por `/admin`: así no hay bucles ni redirecciones abiertas. Al redirigir, se conservan las cookies de sesión

renovadas.

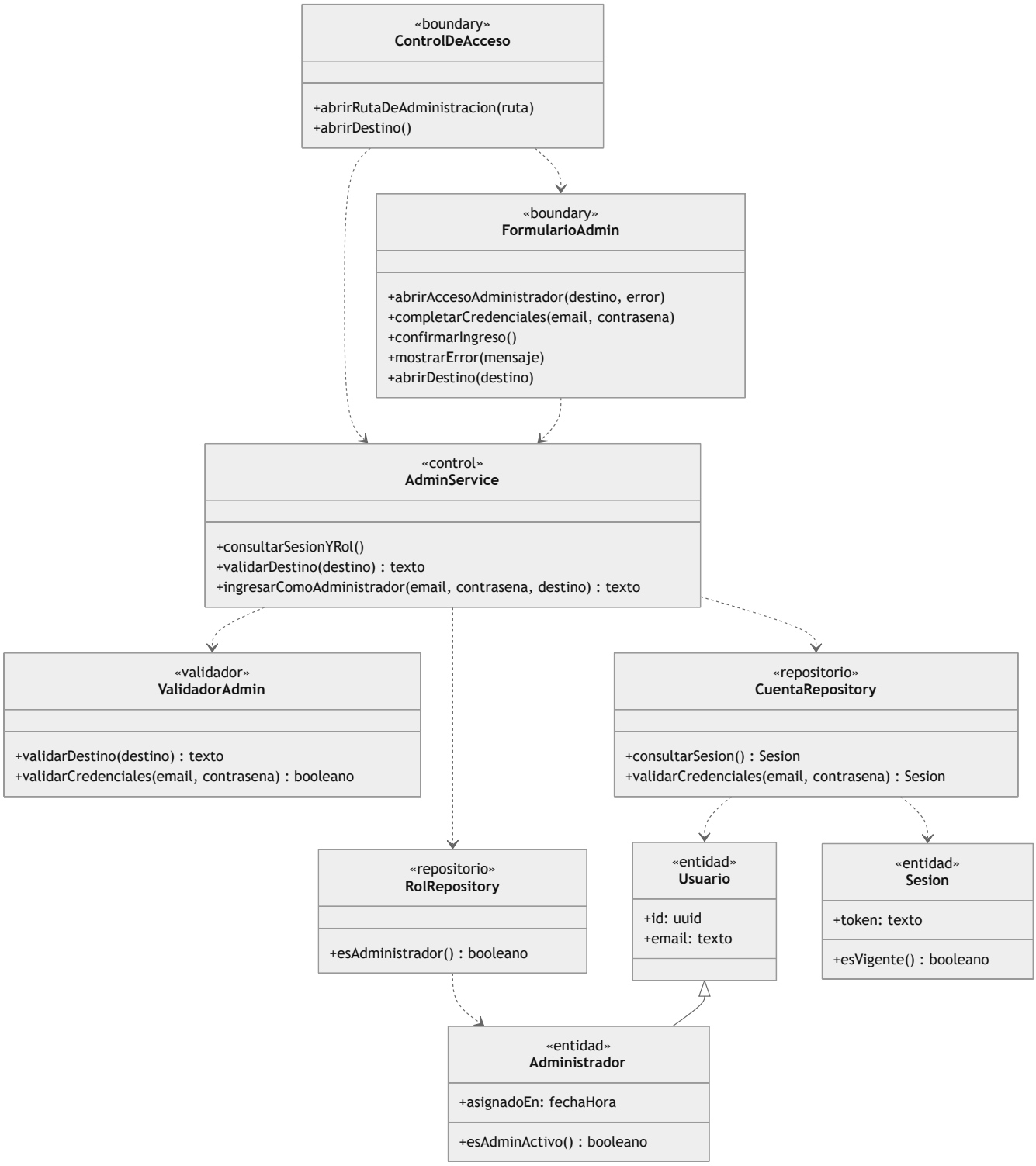
- **No existe una segunda sesión de administrador:** es la misma cuenta que usa el sitio.

Correspondencia en el código

`loginAsAdmin` (`src/app/admin/login/actions.ts`) → `authenticateAdmin` (`src/utils/supabase/admin-login.ts`): `signInWithPassword` y después la RPC `is_admin`. El control de acceso es `src/proxy.ts` y la segunda capa es `requireAdmin()` (`src/utils/admin.ts`).

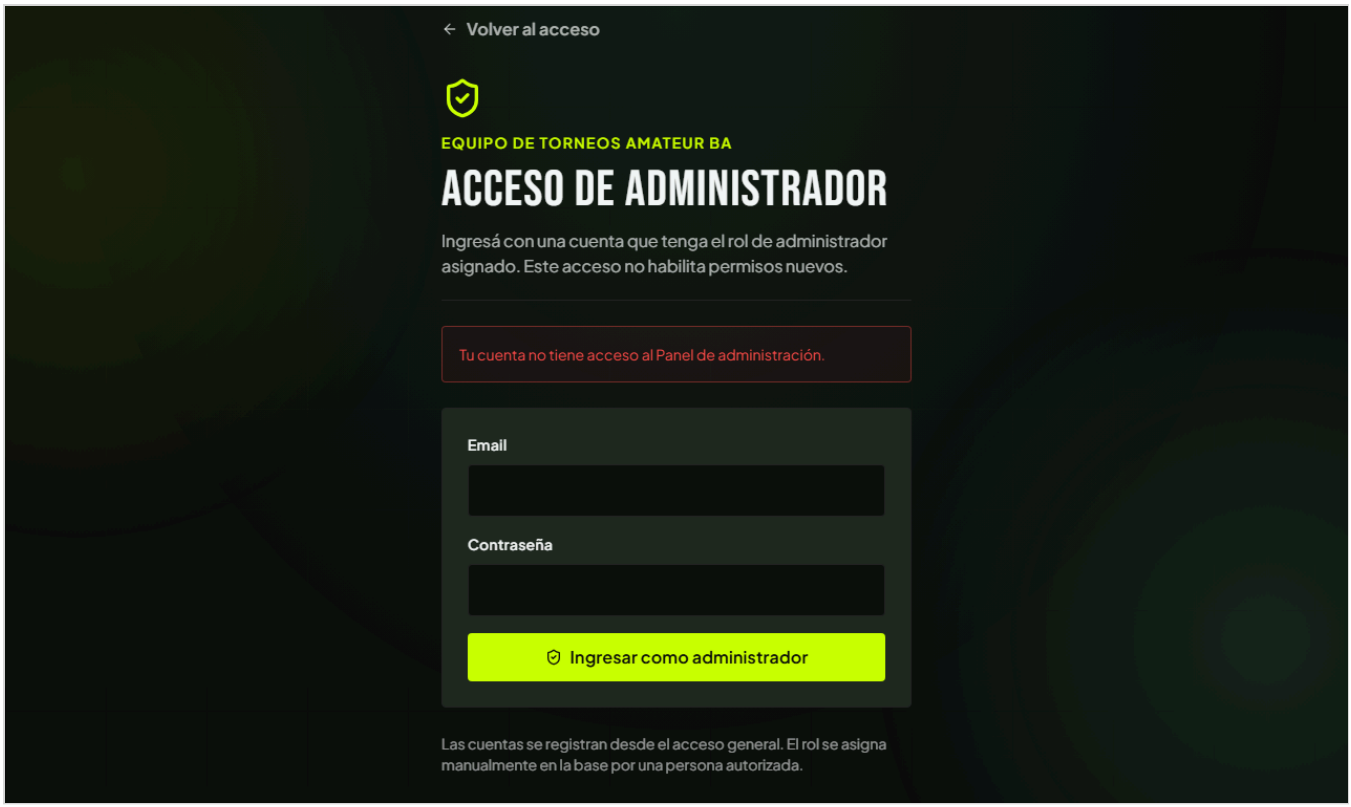
Diagrama de clases · **UC-F01 · Acceder al Panel de administración**

Caso de uso: UC-F01 · **Pantalla:** P1 · Acceso de administrador (`/admin/login`) · **Secuencia:** S01



Pantalla · **P1 · Acceso de administrador**

Ruta: `/admin/login` · **Caso de uso:** UC-F01 · **Requerimientos:** RF-F01 a RF-F07 · **Diagrama:** S01



*Captura ✅ vigente, con el error de **acceso denegado** (`/admin/login?error=denied`). Es la misma pantalla sin el aviso rojo cuando se abre normalmente (`pantallas/acceso-administrador.png`).*

Qué muestra. Sin la barra del sitio: el link «← Volver al acceso» (a `/login`), el ícono de escudo, «Equipo de Torneos Amateur BA», el título **«Acceso de administrador»** y la bajada «Ingresá con una cuenta que tenga el rol de administrador asignado. Este acceso no habilita permisos nuevos». El formulario:

Campo	Tipo	Obligatorio
Email	email	Sí
Contraseña	contraseña	Sí

El botón es «Ingresar como administrador» (con «Verificando acceso…» mientras trabaja). Al pie: «Las cuentas se registran desde el acceso general. El rol se asigna manualmente en la base por una persona autorizada». No hay desplegables, ni registro, ni «entrar como invitado».

Estados de la pantalla

Estado	Qué ve la persona
Normal	El formulario vacío.
Credenciales inválidas	Banda roja: «No pudimos iniciar sesión. Revisá tu email y contraseña.»
Cuenta sin rol	Banda roja: «Tu cuenta no tiene acceso al Panel de administración.» ✅ (captura) y, si tiene sesión, el link «Continuar en el catálogo con mi cuenta».
No se pudo verificar el rol	Banda roja: «No pudimos verificar el acceso de administrador. Probá de nuevo.»
Ya es administrador con sesión	No se ve la pantalla: se lo lleva al destino.

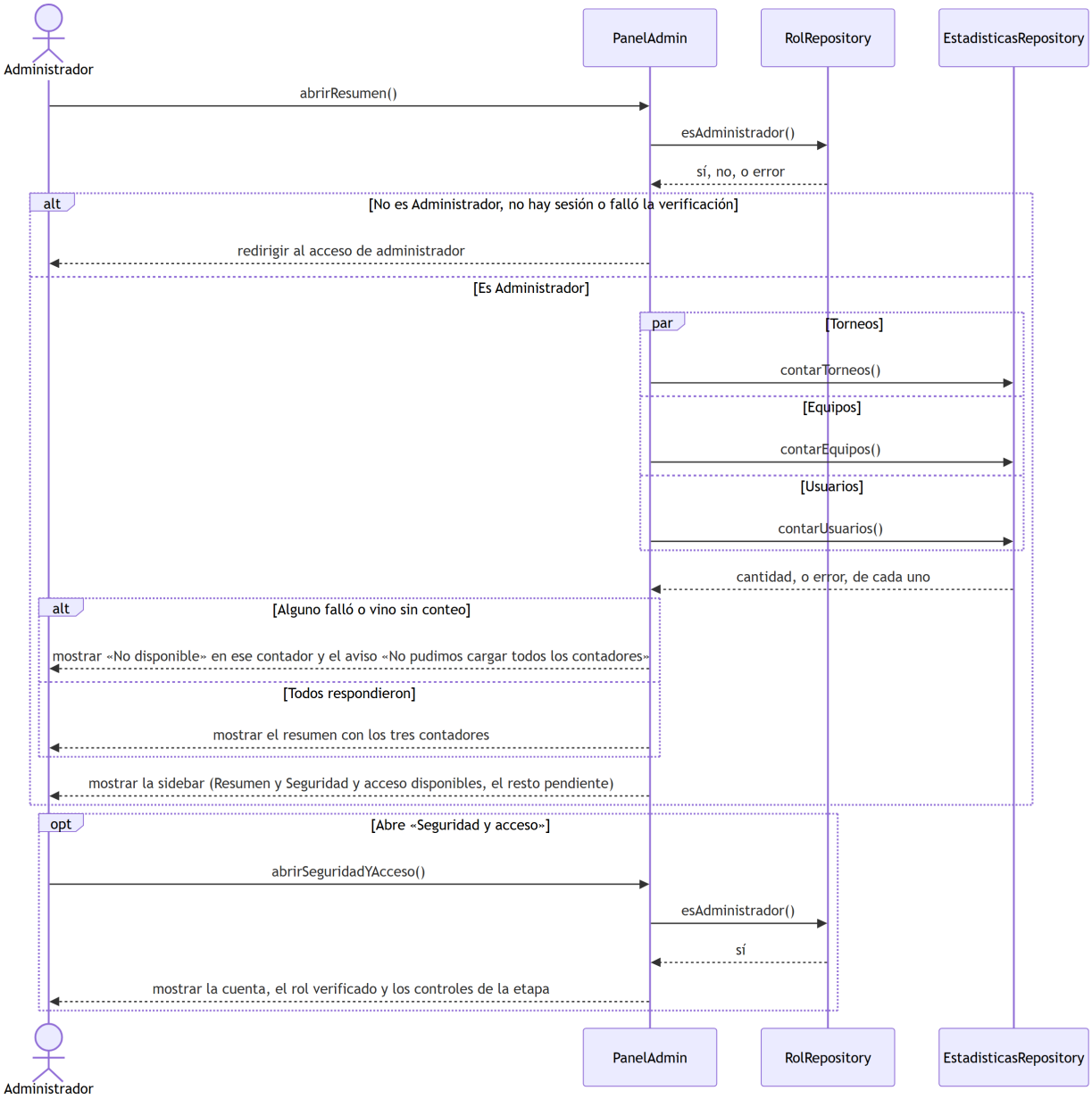
UC-F02 · Consultar el tablero

- **Actor:** Administrador.
- **Requerimientos:** RF-F08 a RF-F11. **Pantallas:** P2 · Resumen, P3 · Seguridad y acceso y P4 · Sidebar. **Diagrama:** S02.
- **Precondiciones:** es Administrador y tiene la sesión abierta.
- **Postcondición:** el administrador conoce el estado general de la plataforma.
- **Estado: Parcial.** Implementado el resumen y «Seguridad y acceso». Las demás secciones son pendientes.

Diagrama de secuencia · **S02 · UC-F02 — Consultar el tablero**

Caso de uso: UC-F02 · Pantallas: P2, P3 y P4 · Requerimientos: RF-F08 a RF-F11

Es una consulta: no hay formulario ni desplegables.



Cómo se lee

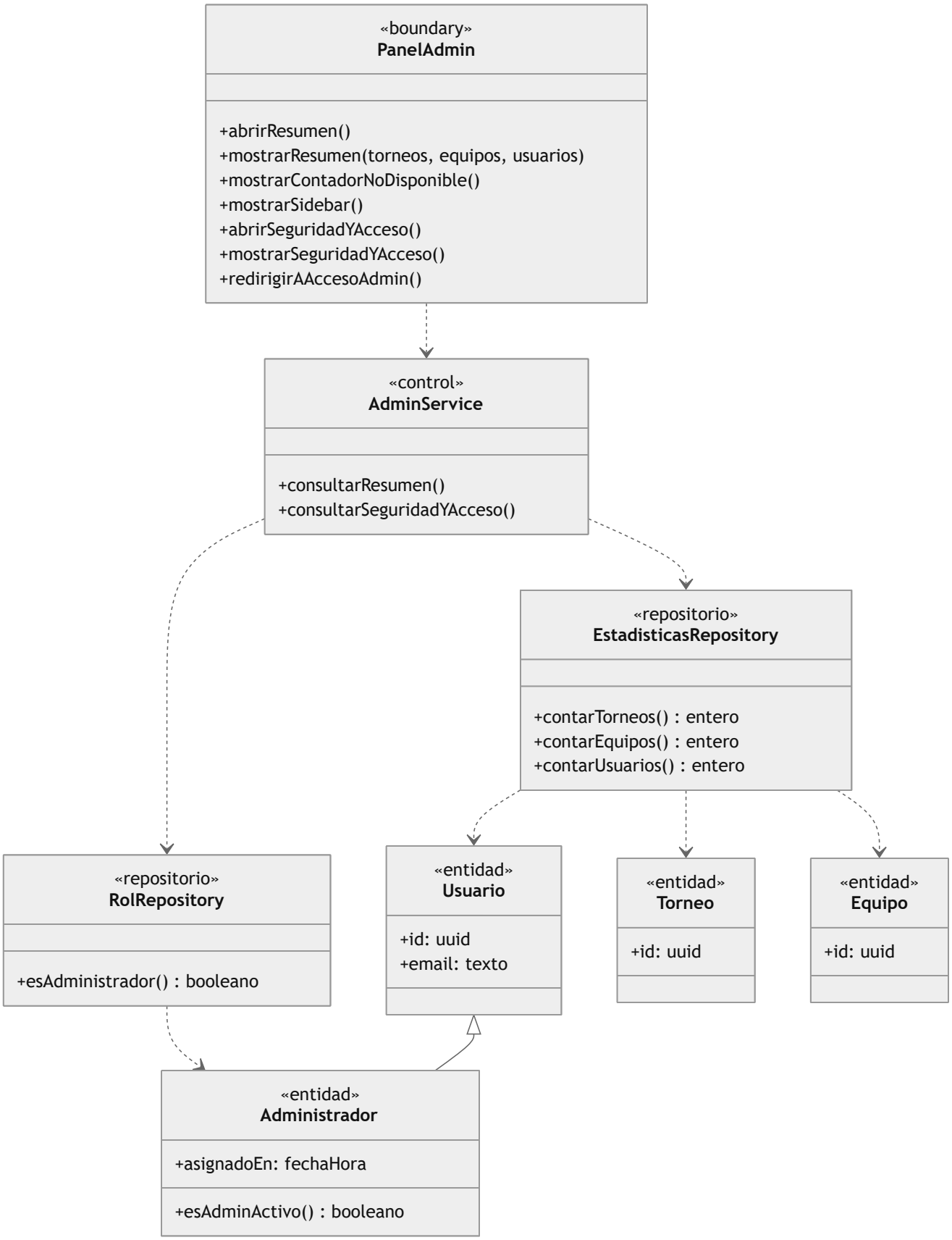
- **El rol se verifica en cada pantalla**, no solo al entrar: el resumen y «Seguridad y acceso» llaman a `esAdministrador()` antes de mostrar nada. Si falla, se vuelve al acceso.
- **Los contadores son reales y se piden en paralelo.** Una consulta fallida muestra **«No disponible»**, **nunca un cero inventado**, y el aviso lo dice. Los contadores salen de las tablas públicas `tournaments`, `teams` y `profiles`; «Usuarios» es la cantidad de perfiles.
- **No hay datos simulados.** Las secciones Usuarios, Torneos y equipos, Reportes, Registro de actividad, Soporte, Avisos y Configuración figuran en la sidebar como pendientes, **sin enlaces**. Se retiró un placeholder «Sin reportes pendientes» porque el módulo de reportes no existe.
- «Volver al sitio» y «Cerrar sesión» (solo este dispositivo) están en la sidebar; son navegación, no casos aparte.

Correspondencia en el código

`AdminPage` (`src/app/admin/(backoffice)/page.tsx`) llama a `requireAdmin()` y hace tres `select('id', { count: 'exact', head: true })` con `Promise.all`; `AdminSidebar` (`src/components/AdminSidebar.tsx`) define las secciones.

Diagrama de clases · **UC-F02 · Consultar el tablero**

Caso de uso: UC-F02 · **Pantallas:** P2 · Resumen, P3 · Seguridad y acceso y P4 · Sidebar · **Secuencia:** S02



Pantalla · **P2 · Resumen de la plataforma**

Ruta: `/admin` · **Caso de uso:** UC-F02 · **Requerimientos:** RF-F08, RF-F09, RF-F11 · **Diagrama:** S02

> ⏳ **Captura pendiente:** requiere una cuenta con el rol de administrador.

Qué muestra. El título «Resumen de la plataforma» y «Panel de administración · Torneos Amateur BA»; tres tarjetas con los **contadores reales** de **Torneos**, **Equipos** y **Usuarios**; y dos bloques: **«Seguridad y acceso»** (con el link «Ver acceso de mi cuenta») y **«Próximas etapas»** (explica que Usuarios, contenido, reportes, actividad, soporte, avisos y configuración se implementarán por etapas).

Estados: *normal*; *contador no disponible* (la tarjeta dice «No disponible» y arriba aparece el aviso ámbar «No pudimos cargar todos los contadores…»). Nunca se muestra un cero inventado.

Pantalla · **P3 · Seguridad y acceso**

Ruta: `/admin/seguridad` · **Caso de uso:** UC-F02 · **Requerimientos:** RF-F10 · **Diagrama:** S02

> ⏳ **Captura pendiente.**

Qué muestra. «Seguridad y acceso» con dos secciones: **«Acceso confirmado»** (cuenta actual, rol «Administrador, verificado en el servidor» y «Asignación del rol: manual en la base; no se puede cambiar desde el perfil ni el formulario de acceso») y **«Controles de esta etapa»** (comprobación de sesión y rol antes de mostrar datos, acceso denegado cuando falla la comprobación, cerrar sesión solo en este dispositivo). Cierra aclarando que MFA, gestión de sesiones, permisos granulares, suspensiones y registro de actividad requieren sus propias etapas: **no es una auditoría de seguridad completa**.

Pantalla · **P4 · Sidebar de administración**

Ruta: todas las de `/admin` · **Caso de uso:** UC-F02 · **Requerimientos:** RF-F08, RF-F11

> ⏳ **Captura pendiente**, en escritorio y en celular.

Qué muestra. En escritorio, una columna fija a la izquierda; en el celular, una barra superior con un botón que despliega la navegación.

1. **Encabezado:** el nombre del panel (link a `/admin`) y, debajo, «← Volver al sitio» (al catálogo), visible también en el celular sin abrir el menú.
2. **Secciones disponibles: Resumen** (`/admin`) y **Seguridad y acceso** (`/admin/seguridad`); la actual se marca con `aria-current`.
3. **Secciones pendientes** (sin enlace y rotuladas como pendientes): Usuarios, Torneos y equipos, Reportes, Registro de actividad, Soporte, Avisos y Configuración.
4. **Al pie:** la cuenta y «Cerrar sesión» (solo este dispositivo).

El menú del celular se cierra al elegir una sección o con Escape.

UC-F03 · Resolver un reporte de torneo

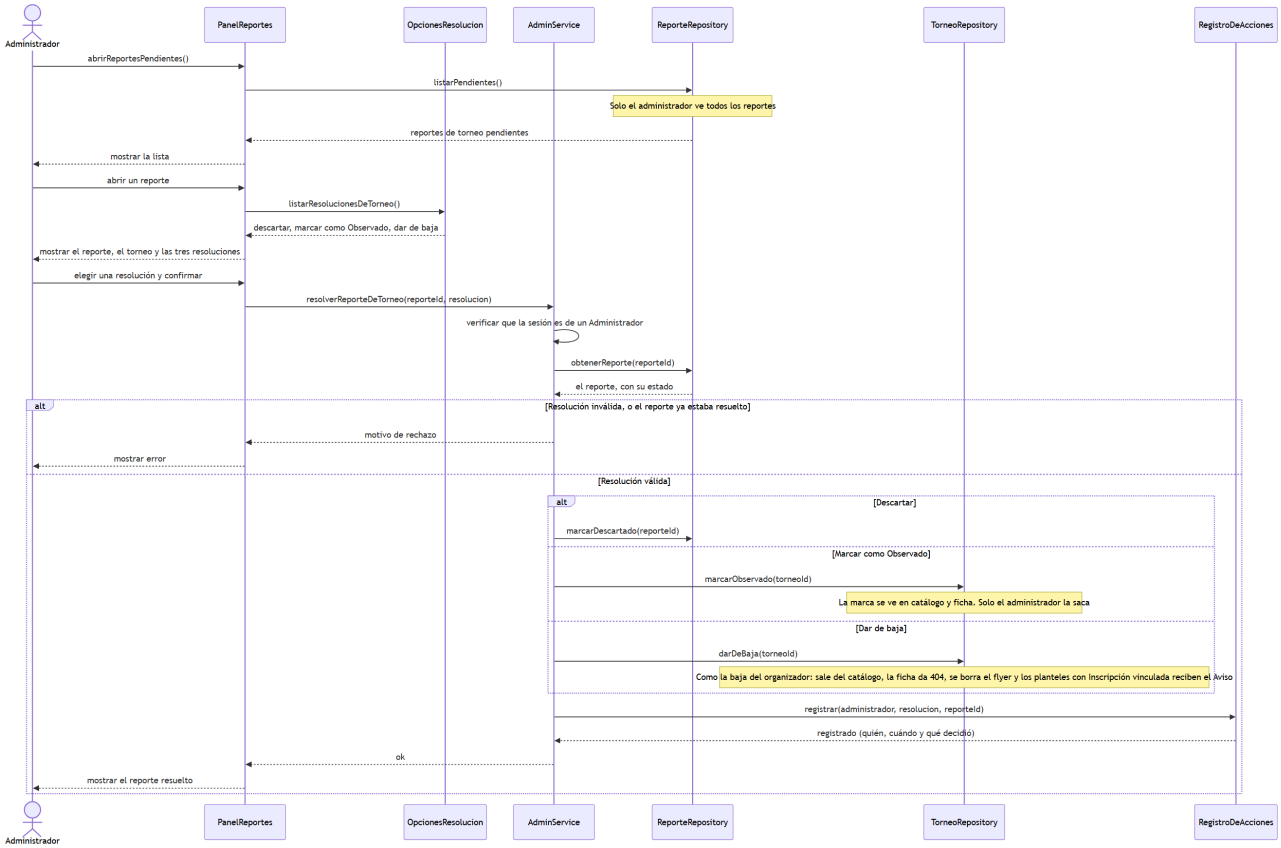
- **Actor:** Administrador.
- **Requerimientos:** RF-F12, RF-F13, RF-F18. **Pantalla:** no existe todavía. **Diagrama:** S03 (diseño objetivo).
- **Precondiciones:** hay un **Reporte** pendiente (UC-B06).
- **Postcondición:** la resolución queda en el **registro de acciones** (quién, cuándo, qué decidió).
- **Estado: Pendiente.**

Diagrama de secuencia · **S03 · UC-F03 — Resolver un reporte de torneo**

Caso de uso: UC-F03 · Pantalla: no existe todavía · Requerimientos: RF-F12, RF-F13, RF-F18

> **Diseño objetivo, pendiente de implementación.** Hoy no hay tabla de reportes, ni pantalla, ni registro de acciones. Los participantes `PanelReportes`, `OpcionesResolucion`, `AdminService`, `ReporteRepository` y `RegistroDeAcciones` no existen todavía y sus nombres son provisorios.

El caso termina con una **elección entre tres resoluciones**. Siguiendo la guía de formularios, la pantalla obtiene la lista **antes** de que el administrador elija. Son una **lista fija del código**, así que las provee un proveedor de opciones y no un repositorio. Elegir y aplicar van en el mismo diagrama.

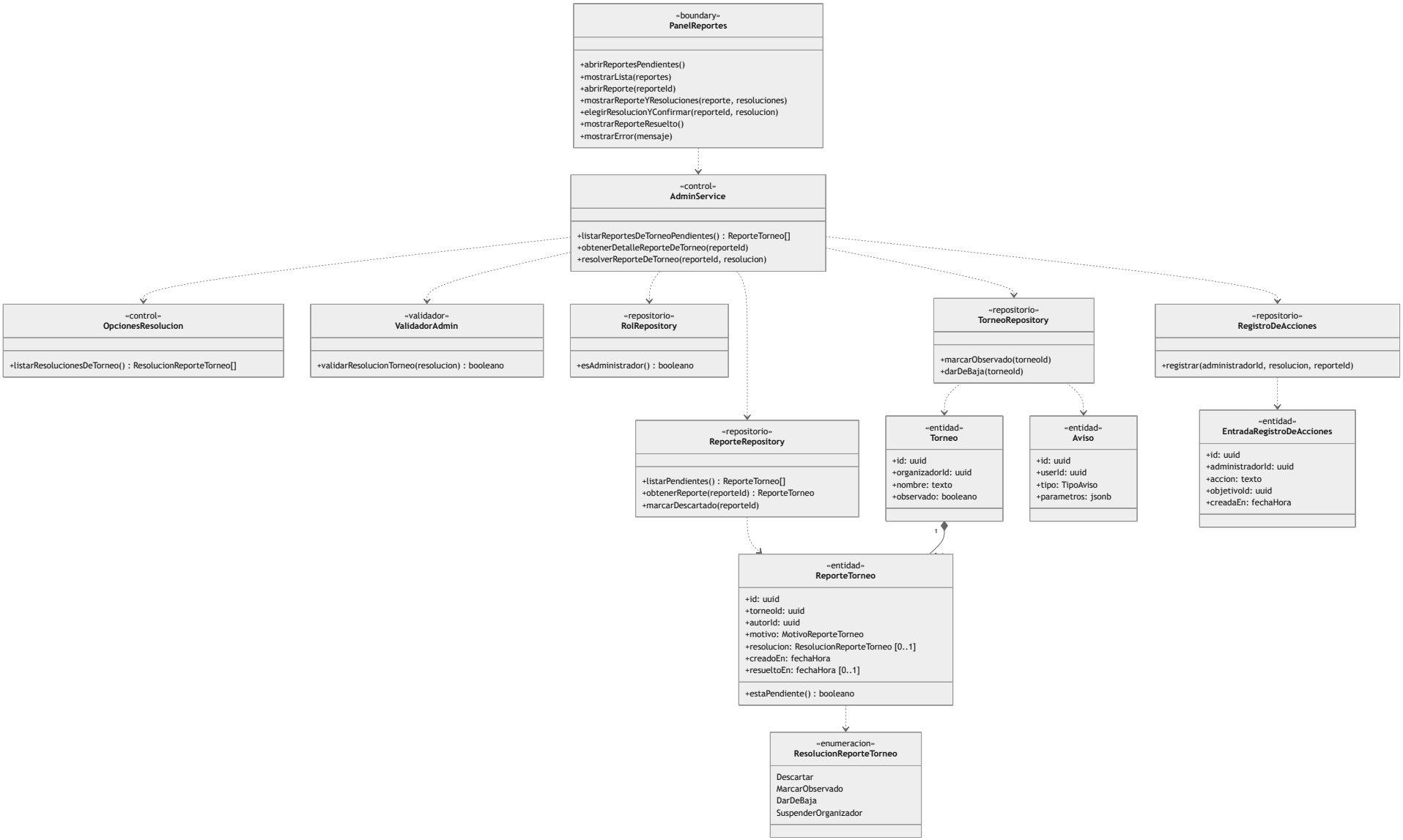


Cómo se lee

- **Antes de elegir:** la pantalla trae las tres resoluciones de `OpcionesResolucion`; al elegir, viaja solo el valor elegido (`resolucion`).
- **Se valida de nuevo al aplicar.** Que el reporte aparezca pendiente en la pantalla no garantiza que lo siga estando (otra pestaña pudo resolverlo) ni que la resolución sea una de las tres. Y **el rol se verifica en el servidor** en cada acción: la pantalla no autoriza.
- **Observado vs. «Sigue vigente»:** el organizador reinicia la antigüedad de verificación con «Sigue vigente», pero **no** puede sacar la marca Observado.
- **La baja por reporte es la misma que la del organizador** (UC-E05): mismas consecuencias, incluido el Aviso a los planteles vinculados. Un torneo Finalizado no se da de baja: es la trayectoria del organizador.
- **Queda registrado** quién resolvió, cuándo y qué decidió (RF-F18). Los diagramas generales llaman a este paso «Registrar resolución administrativa» (D33).
- **Sin decidir:** el detalle del registro de acciones (qué datos guarda exactamente y quién lo puede leer). Solo está definido que registra quién, cuándo y qué decidió.

Diagrama de clases · **UC-F03 · Resolver un reporte de torneo**

Caso de uso: UC-F03 · **Pantalla:** no existe todavía · **Secuencia:** S03 (diseño objetivo)



> Pantalla · **Todavía no existe**

Este caso de uso no tiene pantalla en la app todavía; el caso y el diagrama describen el diseño objetivo.

UC-F04 · Resolver un reporte de equipo

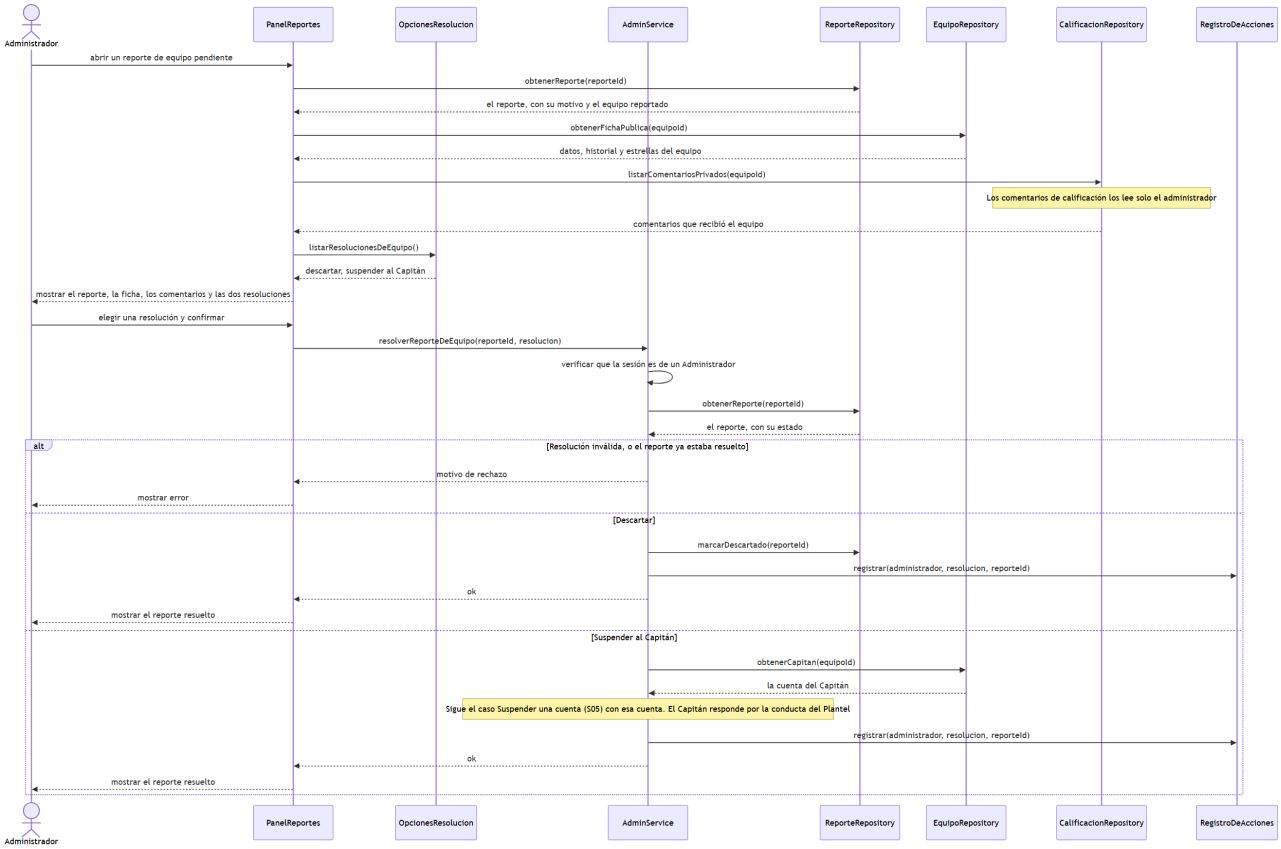
- **Actor:** Administrador.
- **Requerimientos:** RF-F14, RF-F18. **Pantalla:** no existe todavía. **Diagrama:** S04 (diseño objetivo).
- **Precondiciones:** hay un reporte pendiente sobre un equipo (UC-C10).
- **Postcondición:** queda en el registro de acciones.
- **Estado: Pendiente.**

> Diagrama de secuencia · **S04 · UC-F04 — Resolver un reporte de equipo**

Caso de uso: UC-F04 · Pantalla: no existe todavía · Requerimientos: RF-F14, RF-F18

> **Diseño objetivo, pendiente de implementación.** Hoy no hay tabla de reportes de equipos, ni pantalla, ni registro de acciones. Los participantes `PanelReportes`, `OpcionesResolucion`, `AdminService`, `ReporteRepository`, `CalificacionRepository` y `RegistroDeAcciones` no existen todavía y sus nombres son provisorios.

El caso termina con una elección entre **dos resoluciones**, una **lista fija del código** que provee un proveedor de opciones, cargada **antes** de elegir. Suspender al Capitán es el caso siguiente (S05).



Cómo se lee

- **Qué revisa el administrador antes de decidir:** la página del equipo (datos, historial y estrellas) y los **comentarios privados de calificación** que recibió. Esos comentarios solo los lee el administrador.
- **La Suspensión recae sobre el Capitán**, no sobre los jugadores: él responde por la conducta del Plantel.
- **El rol y el estado del reporte se vuelven a verificar al aplicar la resolución.** Una opción mostrada no garantiza que siga siendo válida.
- **El reporte lo origina un Capitán o un jugador** de un equipo que jugó al menos un partido contra el reportado (UC-C10).
- **Sin decidir:** cómo accede la pantalla al Capitán del equipo (el Plantel es privado) y a los comentarios de calificación (la tabla no es de lectura pública). Habrá que resolverlo con una función de servidor que verifique el rol de administrador.

Diagrama de clases · **UC-F04 · Resolver un reporte de equipo**

Caso de uso: UC-F04 · **Pantalla:** no existe todavía · **Secuencia:** S04 (diseño objetivo)



Pantalla · **Todavía no existe**

Este caso de uso no tiene pantalla en la app todavía; el caso y el diagrama describen el diseño objetivo.

UC-F05 · Suspender una cuenta

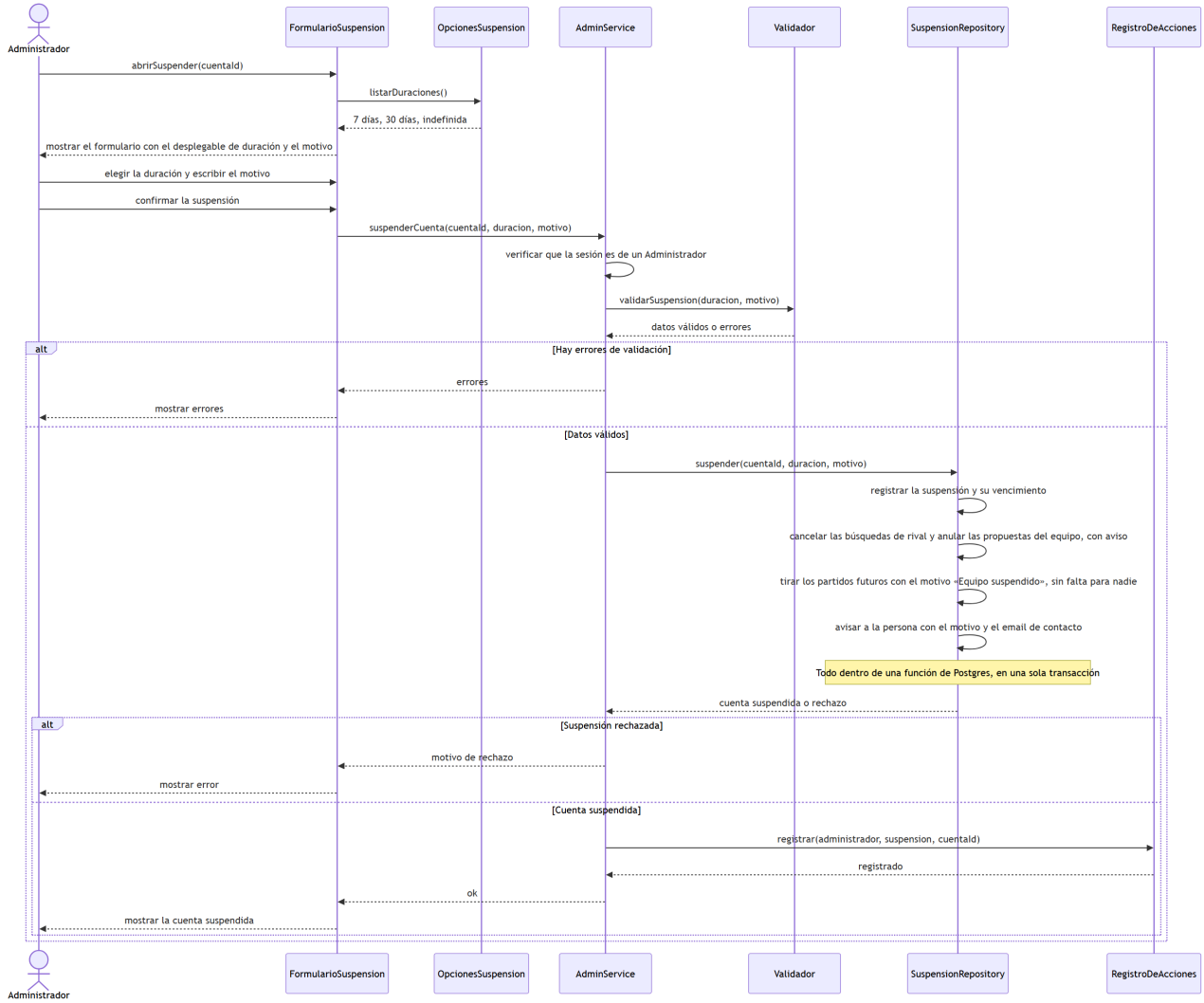
- **Actor:** Administrador.
- **Requerimientos:** RF-F15 a RF-F18. **Pantalla:** no existe todavía. **Diagrama:** S05 (diseño objetivo).
- **Precondiciones:** la cuenta existe (se llega desde un reporte, UC-F04, o desde la búsqueda, UC-F06).
- **Efectos de la Suspensión:**
  - Puede entrar y mirar, pero **no** publicar torneos, crear equipos, buscar rival ni recibir la capitanía.
  - Sus torneos siguen visibles y los puede editar y verificar.
  - Sus búsquedas de rival se cancelan y las propuestas pendientes de su equipo se anulan, con aviso.
  - Sus partidos futuros **se caen** con el motivo «Equipo suspendido» (lo escribe el sistema), sin falta para nadie.
  - Los partidos pendientes de resultado los puede terminar de cargar.
  - Su equipo no aparece entre los compatibles, su página sigue visible y **no puede cambiar de capitán** mientras dure.
- **No confundir** con el **Bloqueo por faltas**, que es automático, recae sobre el equipo y no lo decide nadie.
- **Postcondición:** la cuenta queda suspendida; la acción queda en el registro.
- **Estado: Pendiente.** Hoy la base solo contempla que un suspendido no reciba la capitanía, y ni eso se verifica todavía (hay un comentario «Pendiente cuando exista la suspensión» en `transfer_captaincy`).

Diagrama de secuencia · **S05 · UC-F05 — Suspender una cuenta**

Caso de uso: UC-F05 · Pantalla: no existe todavía · Requerimientos: RF-F15 a RF-F18

> **Diseño objetivo, pendiente de implementación.** Hoy la base no tiene Suspensión: solo hay un comentario «Pendiente cuando exista la suspensión» en `transfer_captaincy`. Los participantes `FormularioSuspension`, `OpcionesSuspension`, `AdminService`, `Validador`, `SuspensionRepository` y `RegistroDeAcciones` no existen todavía y sus nombres son provisorios.

El formulario tiene **un desplegable: la duración** (7 días, 30 días o indefinida) y un **motivo** de texto. La duración es una **lista fija del código**, así que la provee un proveedor de opciones y no un repositorio, cargada **antes** de que el administrador elija. Al enviar viajan solo los valores elegidos. La carga y la suspensión van en el mismo diagrama.



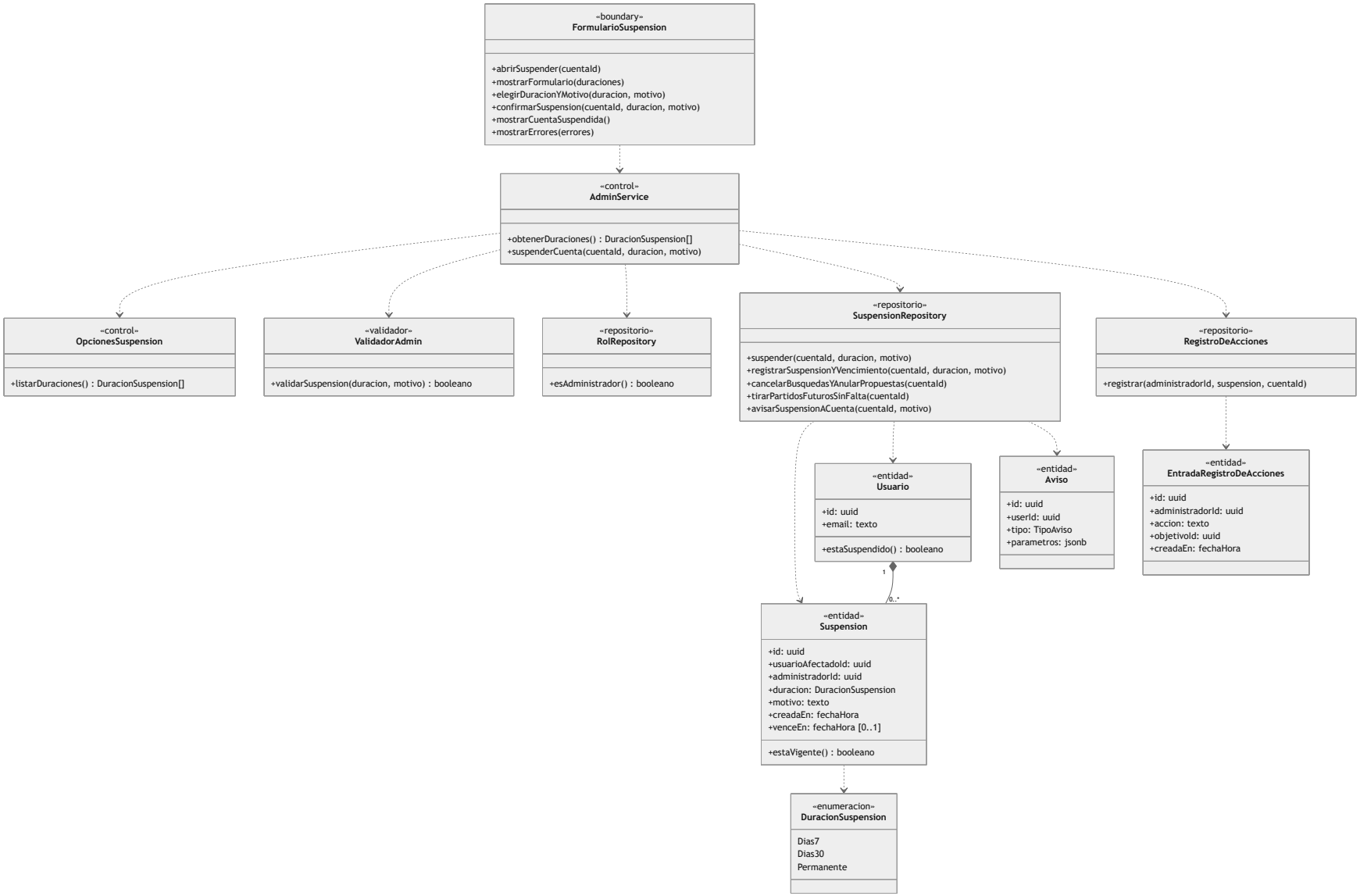
Cómo se lee

- **Antes de elegir:** el formulario trae las duraciones de `OpcionesSuspension`; al enviar viaja solo el valor elegido (`duracion`).
- **Se valida de nuevo al guardar**, y el rol de administrador se verifica **en el servidor** en cada acción.
- **Qué impide la Suspensión:** la persona puede **entrar y mirar**, pero no publicar torneos, crear equipos, buscar rival ni recibir la capitanía. Sus torneos siguen visibles y los puede editar y verificar; los partidos pendientes de resultado los puede terminar de cargar.

- **Qué cancela o cae:** sus búsquedas de rival (con sus propuestas pendientes, con aviso) y sus partidos futuros, que **se caen** con el motivo «Equipo suspendido» (lo escribe el sistema), **sin falta para nadie**. Su equipo deja de aparecer entre los compatibles, su página sigue visible y **no puede cambiar de capitán** mientras dure.
- **No hay apelación dentro de la app:** el Aviso incluye el motivo y un email de contacto.
- **No es el Bloqueo por faltas**, que es automático, recae sobre el equipo y no lo decide nadie.
- **Sin decidir:** cuál es el email de contacto del Aviso y qué reglas tiene el motivo (largo máximo, si es obligatorio); y qué pasa con lo cancelado al vencer la Suspensión.

Diagrama de clases · **UC-F05 · Suspender una cuenta**

Caso de uso: UC-F05 · **Pantalla:** no existe todavía · **Secuencia:** S05 (diseño objetivo)



Pantalla · **Todavía no existe**

Este caso de uso no tiene pantalla en la app todavía; el caso y el diagrama describen el diseño objetivo.

UC-F06 · Buscar usuarios

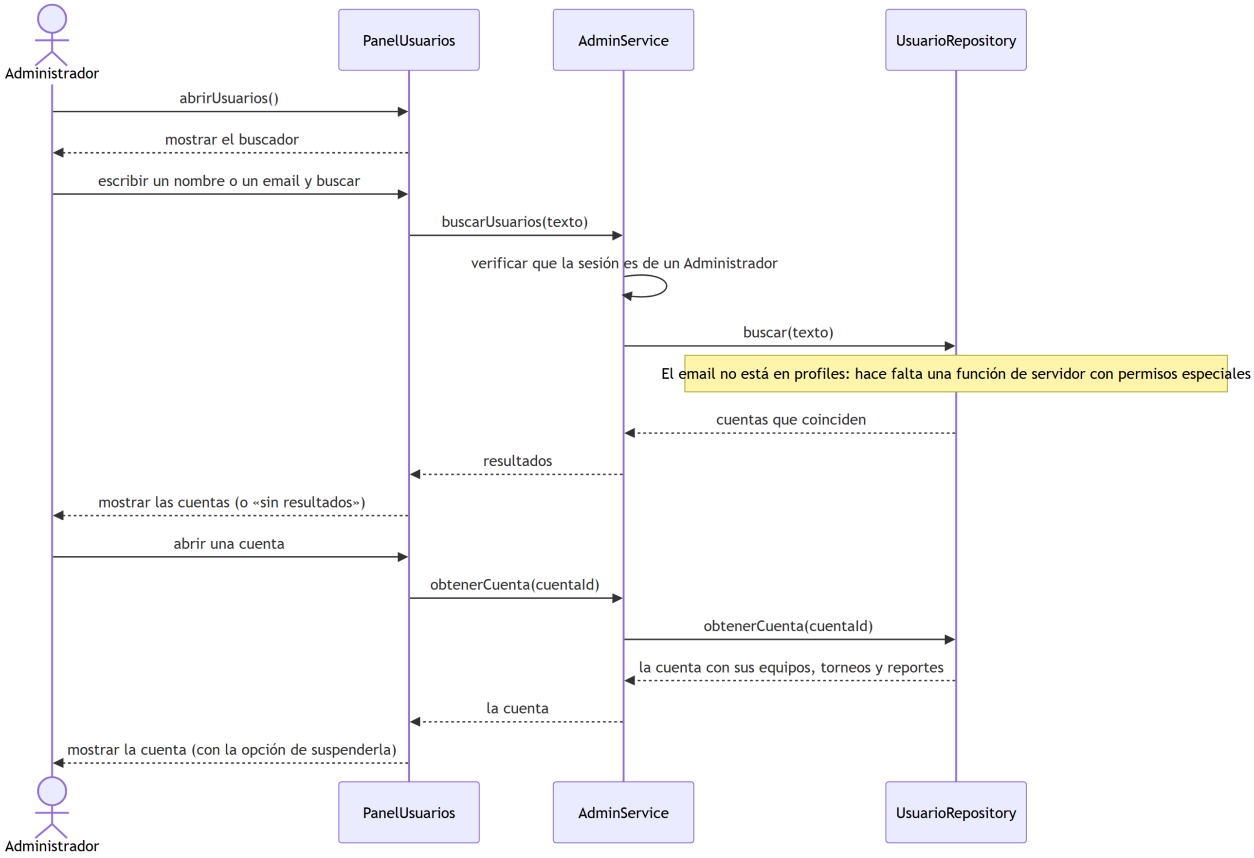
- **Actor:** Administrador.
- **Requerimientos:** RF-F19. **Pantalla:** no existe todavía. **Diagrama:** S06 (diseño objetivo).
- **Estado: Pendiente.** Buscar por email exige resolver cómo leerlo sin exponer datos (ver «Sin decisión»).

> Diagrama de secuencia · **S06 · UC-F06 — Buscar usuarios**

Caso de uso: UC-F06 · Pantalla: no existe todavía · Requerimientos: RF-F19

> **Diseño objetivo, pendiente de implementación.** Hoy no hay ninguna pantalla ni función de búsqueda de usuarios. Los participantes `PanelUsuarios`, `AdminService` y `UsuarioRepository` no existen todavía y sus nombres son provisorios.

El formulario es un campo de texto (nombre o email): no hay desplegables ni listas que cargar.

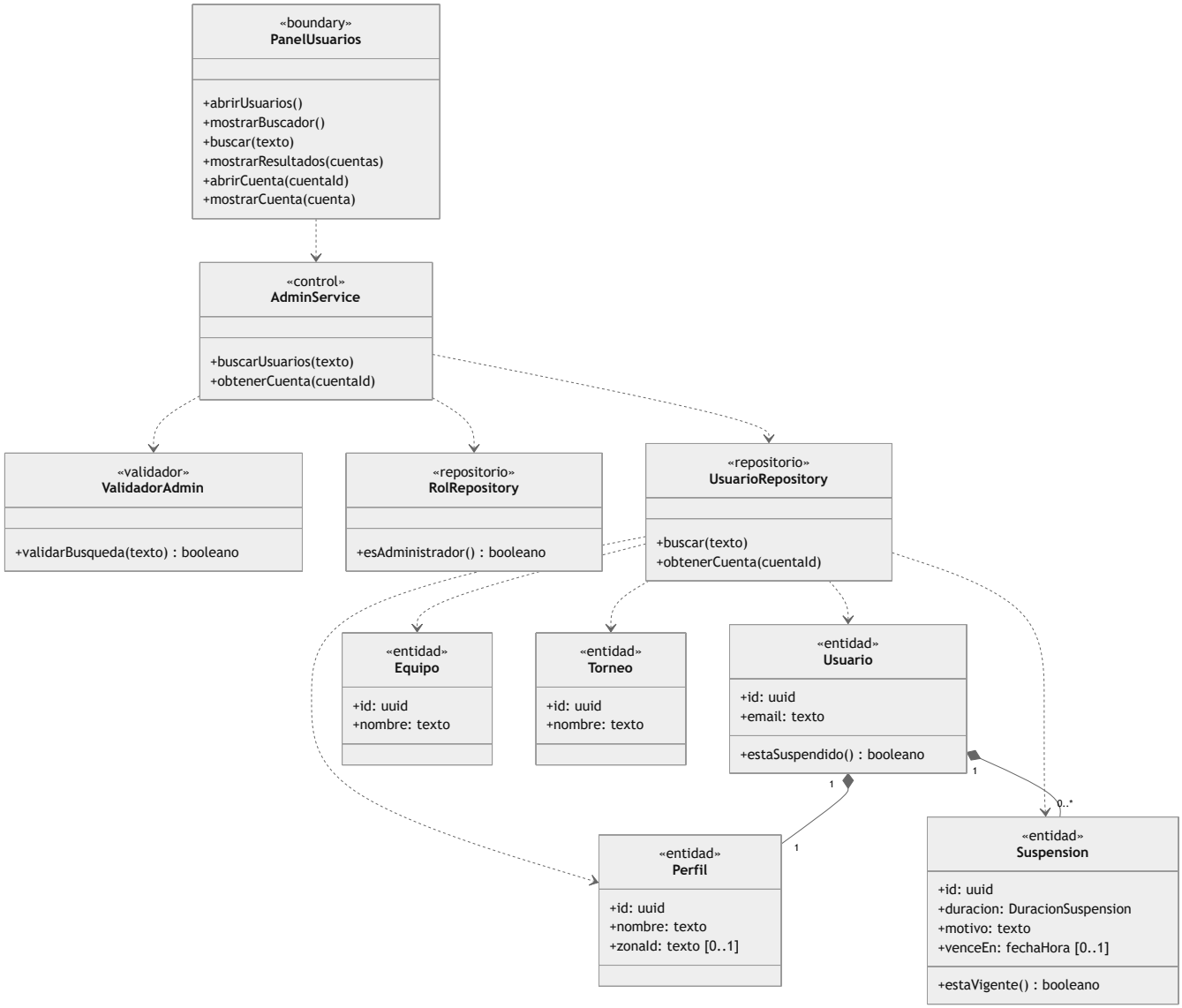


Cómo se lee

- **El rol se verifica en el servidor** en cada búsqueda: la pantalla no autoriza.
- **Qué se ve de una cuenta:** sus equipos, sus torneos y sus reportes; y desde ahí se la puede **suspender** (S05).
- **Sin decidir, y es lo más delicado del caso:** el **email** vive en la autenticación (`auth.users`), que no se lee desde la app. Buscar por email exige una función de servidor que **verifique el rol de administrador** y no exponga los emails de todos; y la `service_role key` nunca debe salir de un contexto de servidor controlado.

Diagrama de clases · **UC-F06 · Buscar usuarios**

Caso de uso: UC-F06 · **Pantalla:** no existe todavía · **Secuencia:** S06 (diseño objetivo)



Pantalla · **Todavía no existe**

Este caso de uso no tiene pantalla en la app todavía; el caso y el diagrama describen el diseño objetivo.

Parte 3 · Anexos y trazabilidad

Guía de pantallas: estado de las capturas y mapa

Versión del **04/11/2026**. Cada pantalla del backoffice que existe hoy, con su ruta, el caso, los requerimientos, lo que muestra y la captura. Cada una va debajo del caso de uso que la usa por primera vez; si otro caso usa la misma pantalla, remite a esa descripción.

Estado de las capturas

Marca	Significa
✅ **Vigente**	Sacada el 04/11/2026 de `main`, con la UI actual, a 1280 px de ancho.
⏳ **Pendiente**	Todavía no hay captura. Todas las pantallas del backoffice salvo el acceso requieren una cuenta con el rol de administrador.

Las capturas están en `pantallas/`.

Lo que no tiene pantalla: los casos de moderación (UC-F03 a UC-F06: reportes, suspensión, búsqueda de usuarios) **no tienen pantalla porque no están implementados**. Sus diagramas son un diseño objetivo.

Mapa de pantallas

Pantalla	Ruta	Caso de uso	Sesión
P1 · Acceso de administrador	`/admin/login`	UC-F01	No (es el formulario público)
P2 · Resumen de la plataforma	`/admin`	UC-F02	Administrador
P3 · Seguridad y acceso	`/admin/seguridad`	UC-F02	Administrador
P4 · Sidebar de administración	todas las de `/admin`	UC-F02	Administrador

Todas las rutas `/admin/…` están protegidas (sesión y rol), **salvo** `/admin/login`. A diferencia del resto de la app, **el backoffice no lleva la barra superior ni el footer del catálogo**: tiene su propio marco.

Capturas pendientes

Convención: PNG en `docs/administracion/pantallas/`, ventana de unos 1280 px de ancho (y ~390 px para el celular), con datos de prueba. **Hace falta una cuenta con el rol de administrador** (se asigna insertando su id en `admins`, ver acceso-administrador.md).

Archivo	Pantalla y estado	Cómo conseguirlo
`resumen.png`	P2 con los tres contadores	Entrar como administrador.
`resumen-no-disponible.png`	P2 con un contador «No disponible»	Solo reproducible con una falla de la base; opcional.
`seguridad-y-acceso.png`	P3	Abrir «Ver acceso de mi cuenta».
`sidebar-escritorio.png`	P4 en escritorio	Con la sidebar visible, mostrando las secciones pendientes.
`sidebar-celular.png`	P4 en el celular, desplegada	Ancho ~390 px, abrir el menú.

Archivo	Pantalla y estado	Cómo conseguirlo
`acceso-administrador-credenciales.png`	P1 con «No pudimos iniciar sesión…»	Ingresar una contraseña incorrecta.

Diagramas de secuencia: criterios, participantes y convenciones

Versión del **04/11/2026**. Un diagrama por caso de uso, en Mermaid (GitHub los dibuja directo). Cada uno va debajo de su caso de uso, antes de su pantalla.

Diagrama	Caso de uso	Pantalla	Estado
S01 · Acceder al panel	UC-F01	P1	Implementado
S02 · Consultar el tablero	UC-F02	P2, P3, P4	Parcial
S03 · Resolver un reporte de torneo	UC-F03	—	**Pendiente** (diseño objetivo)
S04 · Resolver un reporte de equipo	UC-F04	—	**Pendiente** (diseño objetivo)
S05 · Suspender una cuenta	UC-F05	—	**Pendiente** (diseño objetivo)
S06 · Buscar usuarios	UC-F06	—	**Pendiente** (diseño objetivo)

Los PNG están dibujados sobre fondo blanco, para pegarlos en un informe. La fuente de cada diagrama es el bloque Mermaid de su archivo: si se cambia uno, hay que regenerar su PNG.

Relación con los diagramas generales. `docs/diagramas-de-secuencia/diagramas-secuencia.md` tiene diagramas de todo el proyecto con participantes genéricos. Los de este módulo los **detallan**: D30 (acceder a administración) corresponde a S01, D31 (resolver reporte) a S03 y S04, D32 (suspender cuenta) a S05, y D33 (registrar la resolución) está dentro de S03 a S05, en el paso del registro de acciones.

> **Diseño objetivo.** S03 a S06 dibujan casos que **todavía no están programados**: dibujan lo que dicen las reglas escritas. Los participantes marcados como «no existen todavía» tienen nombres provisorios.

Criterios aplicados

Son los de la guía para diagramas con formularios y desplegables.

1. **Un caso de uso por diagrama.** Cargar las opciones de una elección y la acción que sigue van **en el mismo diagrama**.
2. **Si el caso empieza con una elección entre opciones**, el diagrama muestra a la pantalla pidiendo las opciones **antes** de que el actor elija. Acá: las resoluciones posibles de un reporte (S03 y S04) y las duraciones de una Suspensión (S05).
3. **Un objeto por responsabilidad o fuente de datos.** Esas listas son **fijas del código** (no se guardan en la base), así que las provee un **proveedor de opciones** y no un repositorio.
4. **Al enviar**, se pasan **solo los valores elegidos** (`resolucion` , `duracion` , `motivo`), nunca las listas.
5. **La validación se repite al guardar.** Una opción mostrada no garantiza que siga siendo válida: el reporte pudo resolverse en otra pestaña, o el pedido pudo armarse a mano. Y **el rol se vuelve a verificar en el servidor en cada acción**: la pantalla no autoriza.
6. **Cada diagrama muestra solo los participantes que intervienen.**

Participantes: responsabilidad y correspondencia en el repo

Son responsabilidades conceptuales. No existen como clases TypeScript con esos nombres.

Participante	Responsabilidad	Correspondencia en el repo
Administrador, Usuario	La persona que inicia el caso. El Usuario es una cuenta común.	La sesión, y una fila en `admins` para el Administrador.
ControlDeAcceso	Primera capa: decide si una ruta `/admin/…` exige sesión y rol, y redirige.	`src/proxy.ts`.
FormularioAdmin	La pantalla `/admin/login`.	`admin/login/page.tsx`.
PanelAdmin	El resumen, «Seguridad y acceso» y la sidebar.	`admin/(backoffice)/page.tsx`, `seguridad/page.tsx`, `AdminSidebar.tsx`.
AdminService	Coordina el acceso y las acciones del administrador.	La Server Action `loginAsAdmin` (`admin/login/actions.ts`) y `authenticateAdmin` (`utils/supabase/admin-login.ts`). Para la moderación, no existe todavía.
Validador	Valida entradas antes de actuar.	`adminDestination` (`utils/admin-access.ts`).
CuentaRepository	Abre y consulta la sesión.	Supabase Auth (`signInWithPassword`, `getUser`).
RolRepository	Responde si la sesión actual es de un administrador.	La función `is_admin()` (`security definer`) sobre la tabla `admins`, que no se puede leer directo. Se usa desde `proxy.ts` y `requireAdmin()` (`utils/admin.ts`).
EstadisticasRepository	Cuenta torneos, equipos y usuarios.	Consultas `count` sobre `tournaments`, `teams` y `profiles`.
OpcionesResolucion, OpcionesSuspension	Entregan las resoluciones de un reporte y las duraciones de una Suspensión: **listas fijas del código**.	**No existen todavía.**
PanelReportes, FormularioSuspension, PanelUsuarios	Las pantallas de moderación.	**No existen todavía.**
ReporteRepository, TorneoRepository, EquipoRepository, CalificacionRepository, SuspensionRepository, UsuarioRepository, RegistroDeAcciones	Los datos de la moderación.	Pendientes. `TorneoRepository` y `EquipoRepository` sí existen como tablas `tournaments` y `teams`; el resto **no existe todavía**.

Convenciones de dibujo

- Flecha continua (`->>`): una llamada. Flecha punteada (`-->>`): su respuesta.
- Un mensaje de un participante a sí mismo es un paso interno.
- Una nota sobre un participante marca una regla que **hace cumplir la base**, un paso pendiente o una diferencia con la documentación.
- Entre paréntesis y en minúscula van los **valores que viajan**; entre comillas «así», los mensajes que ve la persona.
- `opt` es un paso opcional; `alt` / `else`, caminos que se excluyen; `par`, pasos en paralelo.

Trazabilidad y pendientes

Documentación completa del módulo F (Administración) de Torneos Amateur BA, versión del **04/11/2026**. En este documento, sus piezas son la Parte 1 (requerimientos), la Parte 2 (cada caso de uso con su diagrama y su pantalla) y los anexos de esta parte.

Fuentes, en orden de autoridad: decisiones-casos-de-uso.md → casos-de-uso.md → acceso-administrador.md → CONTEXT.md → código y migraciones.

Matriz de trazabilidad

Caso de uso	Requerimientos	Pantalla	Diagrama	Estado
UC-F01 Acceder al panel	RF-F01 a F07	P1 `/admin/login`	S01	Implementado
UC-F02 Consultar el tablero	RF-F08 a F11	P2, P3, P4	S02	**Parcial**
UC-F03 Resolver reporte de torneo	RF-F12, F13, F18	—	S03	**Pendiente**
UC-F04 Resolver reporte de equipo	RF-F14, F18	—	S04	**Pendiente**
UC-F05 Suspender una cuenta	RF-F15 a F18	—	S05	**Pendiente**
UC-F06 Buscar usuarios	RF-F19	—	S06	**Pendiente**

Equivalencia con el PDF original

Vigente	PDF original
UC-F01 Acceder al panel	CU-F01
UC-F02 Consultar el tablero	— (caso nuevo)
UC-F03 Resolver reporte de torneo	CU-F03 (y CU-F06, registrar la resolución)
UC-F04 Resolver reporte de equipo	— (caso nuevo)
UC-F05 Suspender una cuenta	CU-F05
UC-F06 Buscar usuarios	— (caso nuevo)
—	CU-F04, arbitrar partidos: se retiró (el administrador no arbitra partidos)

La correspondencia del CU-F02 del PDF no está clara en los documentos del repo: figura como «para otra tanda» en los diagramas generales.

Hallazgos y pendientes

1. **El módulo está a un tercio.** Está hecho el **acceso diferenciado** (con sus dos capas de verificación del rol), el marco del backoffice y el resumen. **No existe nada de la moderación:** no hay tablas de reportes, de suspensiones ni de registro de acciones, y los botones «Reportar» de torneos y equipos tampoco existen. Los cuatro casos pendientes dependen de eso.
2. **Los reportes son la dependencia crítica.** Sin UC-B06 (reportar un torneo) y UC-C10 (reportar un equipo) el administrador no tiene qué resolver. Conviene programarlos antes que la pantalla de reportes del backoffice.
3. **La Suspensión toca otros módulos.** Cancela búsquedas, anula propuestas y tira partidos futuros (módulos C y D) y hoy la base no verifica ni siquiera que un suspendido no reciba la capitanía. Hay que decidir qué hace falta antes de programarla.

4. **Decisiones abiertas** (detalle): el email de contacto del Aviso de suspensión; cómo buscar usuarios por email sin exponer datos ni la `service_role key`; cómo accede el administrador a los comentarios privados de calificación y al Capitán de un equipo; y qué pasa con lo cancelado cuando vence una Suspensión.
5. **Un mensaje de error en español, no como en el acceso general.** El acceso de administrador dice «No pudimos iniciar sesión. Revisá tu email y contraseña.»; el acceso general dice «Invalid Credentials» en inglés.
6. **Capturas:** hay dos vigentes del acceso (normal y con «acceso denegado»). Las demás requieren una cuenta con el rol de administrador: la lista está en 03-pantallas.md.
7. **Tests:** `is_admin_test.sql` y `admin-access.test.ts` cubren el acceso. Cada etapa de moderación deberá llegar con sus pruebas (la regla del proyecto: toda policy de RLS nueva viene con su test).
8. **Si se cambia un diagrama,** hay que regenerar su PNG (el bloque Mermaid de cada archivo es la fuente).

Diagramas de clases: trazabilidad de las clases

Clase	Estereotipo	Implementación en el repo	Estado
`ControlDeAcceso`	`«boundary»`	`src/proxy.ts` + `requireAdminSession` (`src/utils/supabase/admin.ts`)	Implementado
`FormularioAdmin`	`«boundary»`	`src/app/admin/login/page.tsx`	Implementada
`PanelAdmin`	`«boundary»`	`src/app/admin/(panel)/layout.tsx`, `src/app/admin/(panel)/page.tsx`, `src/app/admin/(panel)/seguridad/page.tsx`	Implementada (parcial en métricas nuevas)
`PanelReportes` / `FormularioSuspension` / `PanelUsuarios`	`«boundary»`	— (previstos en `03-pantallas.md`)	**Pendiente**
`AdminService` / `OpcionesResolucion` / `OpcionesSuspension`	`«control»`	`adminLogin` (`src/app/admin/login/actions.ts`) + consultas en `src/app/admin/(panel)/page.tsx` + listas fijas de resoluciones y duraciones	Parcial (faltan UC-F03 a UC-F06)
`ValidadorAdmin`	`«validador»`	`safeAdminNextPath` (`src/utils/safe-next-path.ts`) + validadores de moderación	Parcial
`CuentaRepository` / `RolRepository`	`«repositorio»`	Supabase Auth + función `public.is_admin()` y tabla `public.admins` (`20260928000002_profiles.sql`)	Implementados
`EstadisticasRepository`	`«repositorio»`	`countTable` en `src/app/admin/(panel)/page.tsx` (`tournaments`, `teams`, `profiles`)	Implementado
`ReporteRepository` / `SuspensionRepository` / `UsuarioRepository` / `RegistroDeAcciones`	`«repositorio»`	— (tablas de reportes, suspensiones y auditoría)	**Pendiente**
`Usuario` / `Administrador` / `Perfil` / `Torneo` / `Equipo` / `Calificacion` / `Aviso`	`«entidad»`	`auth.users`, `public.admins`, `profiles`, `tournaments`, `teams`, `ratings`, `notices`	Implementadas
`ReporteTorneo` / `ReporteEquipo` / `Suspension` / `EntradaRegistroDeAcciones`	`«entidad»`	— (esquema de moderación y auditoría)	**Pendiente**